Ch3. 업무자동화 관련 패키지

업무자동화에 사용하는 대표적인 API와 라이브러리들에 대해서 배워봅시다.

3-1. Excel - OpenPyxl 소개

파이썬에서 엑셀 업무 자동화에 자주 쓰이는 openpyxl 라이브러리에 대해 알아봅시다. openpyxl은 엑셀의 데이터를 읽거나 쓰고 제어할 수 있는 대표적인 라이브러리입니다. 해당 챕터는 openpyxl의 모든 부분에 대해서 설명하진 않습니다. 업무 자동화에 자주 쓰이는 내용과 저자가 실제 자동화 프로그램에 활용하고 있는 기초적인 내용에 대해 학습합니다. 책에서 다루지 않는 내용에 대해서는 openpyxl 공식 홈페이지를 참고하시기 바랍니다.

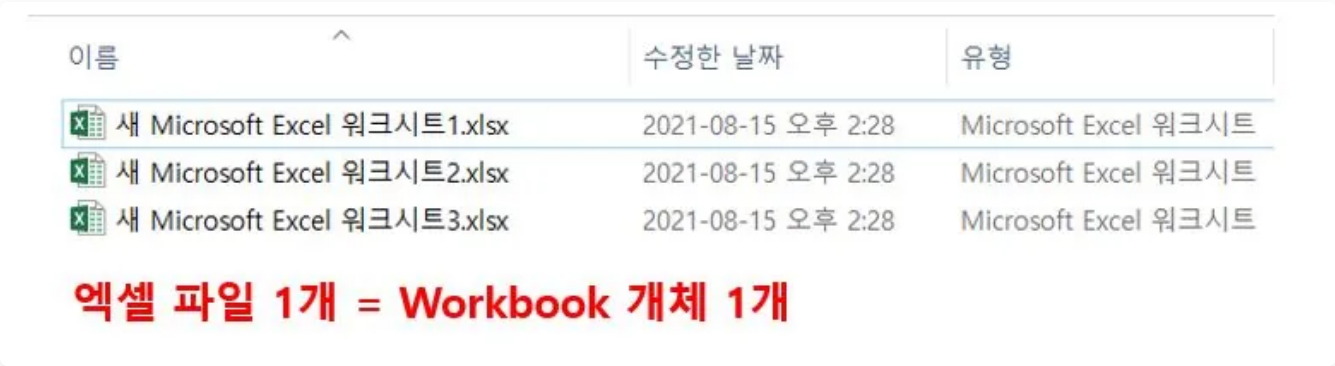
Openpyxl 공식 페이지 : https://openpyxl.readthedocs.io/en/stable/tutorial.html

1) 엑셀의 구성요소

엑셀 자동화에 사용하는 OpenPyxl 사용법을 설명하기 전에 엑셀의 구성요소인 Workbook, Sheet, Cell, Range에 대한 개념을 알면 코드를 이해하는데 도움이 됩니다. OpenPyxl 뿐만 아니라 엑셀을 다룰 수 있는 라이브러리라면 아래 내용은 공통적으로 알아야 할 항목입니다. 먼저 엑셀의 기본적인 구성 요소에 대해 설명해보겠습니다.

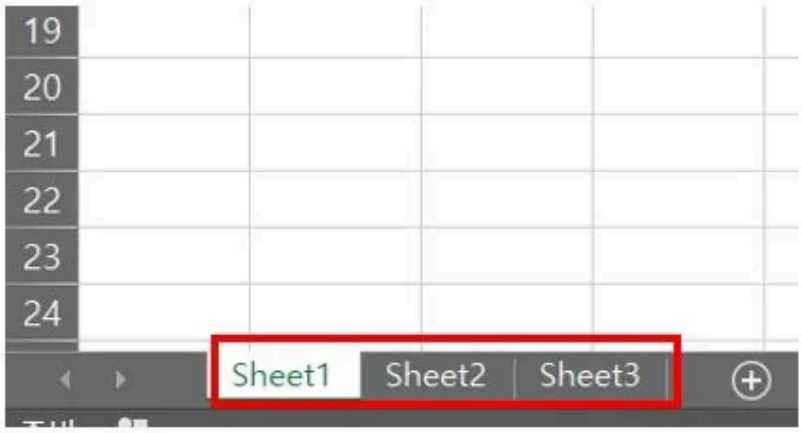
1. Workbook

엑셀에는 Workbook이라는 오브젝트(개체) 개념이 있습니다. 간단히 설명하자면 우리가 흔히 실행하는 엑셀 확장자 파일(.xls, .xlsx, .xlsm 등)을 Workbook 개체라고 합니다. 예를 들어 'text.xlsx'라는 파일이 있을 때 이 것을 Workbook 개체 1개라고 말할 수 있습니다. 파이썬 openpyxl에서는 엑셀 파일을 생성하거나 특정 엑셀 파일에 접근할 때, Workbook 객체를 생성하는 과정을 무조건 거쳐야 합니다.



2. WorkSheet (=Sheet)

엑셀을 많이 사용해보신 분이라면 알겠지만 엑셀 파일 1개에는 여러가지 Sheet가 존재합니다. 데이터를 입력할 수 있는 Cell이 모여진 공간을 엑셀에서는 Sheet라고 부르며, 1개의 Workbook 안에는 여러개의 Sheet를 가질 수 있습니다. 여러개의 Sheet가 있을 수 있기 때문에, openpyxl 모듈에는 특정 Sheet에 접근하기 위해 이를 설정하는 코드를 작성해야 합니다.



Workbook 개체 1개에는 여러 개의 Sheet가 있을 수 있음.

3. Cell

아래 이미지와 같이 엑셀 Sheet에서 데이터를 쓸 수 있는 공간 1개(1칸)을 Cell이라고 합니다. 아래 4)에서 설명하겠지만 Range와 비슷한 개념이기도 합니다. 엑셀에서는 특정 위치의 데이터를 읽어오거나 쓰고 싶을 때, 위치 값을 Cell의 Row와 Column 값을 통해 접근합니다. Row는 말그대로 몇번째 행인가를 의미하는 값이며, Column은 몇번째 열인지를 나타냅니다. 아래 이미지는 1개의 Cell에 대해서 나타냈지만 실제로는 여러 개의 Cell을 한번에 설정할 수도 있습니다.

Cells(1,1)

	A	B	C	D
1	날짜	내용	지출	
2	2021-06-08	식비	₩10,000	
3	2021-06-08	식비	₩20,000	
4	2021-06-08	식비	₩5,000	
5	2021-06-08	사무용품	₩4,000	
6	2021-06-08	비품	₩2,000	
7	2021-06-08	유류비	₩1,000	
8	2021-06-08	기타	₩5,000	
9				
10				

Cells(3,3)

	A	B	C	D
1	날짜	내용	지출	
2	2021-06-08	식비	₩10,000	
3	2021-06-08	식비	₩20,000	
4	2021-06-08	식비	₩5,000	
5	2021-06-08	사무용품	₩4,000	
6	2021-06-08	비품	₩2,000	
7	2021-06-08	유류비	₩1,000	
8	2021-06-08	기타	₩5,000	
9				
10				

4. Range

엑셀에서는 각 Cell을 지칭하는 Index를 제공합니다. 예를들면 1행1열의 위치는 "A1", 1행2열의 위치는 "B1"과 같은 index입니다. Range를 사용하면 사용자가 직관적으로 Index를 통해 각 셀에 접근할 수 있습니다. Cell을 통해 접근할 때는 row, column에 대한 숫자값을 이용해야하는 점과는 차이가 있습니다. 아래 이미지를 보며 이해해보도록 합시다.

	A	B	C	D
1	날짜	내용	지출	
2	2021-06-08	식비	₩10,000	
3	2021-06-08	식비	₩20,000	
4	2021-06-08	식비	₩5,000	
5	2021-06-08	사무용품	₩4,000	
6	2021-06-08	비품	₩2,000	
7	2021-06-08	유류비	₩1,000	
8	2021-06-08	기타	₩5,000	
9				
10				

2) WorkBook, WorkSheet 객체 설정하기

이번 파트에서는 openpyxl을 통해 Workbook을 생성하고, Sheet 를 설정하는 방법에 대해 알아보겠습니다. openpyxl을 통해 엑셀 파일을 제어하려면 무조건 거쳐야 하는 과정입니다.

1. openpyxl 설치 및 import 하기

openpyxl을 설치하고 모듈을 import 하는 방법을 알아보겠습니다.

1) openpyxl 설치

openpyxl은 파이썬의 내장 모듈이 아니므로 별도 설치 과정을 거쳐야 합니다. 사용하고 있는 개발 환경에 따라 터미널 창에 아래 명령어를 입력하여 openpyxl 모듈을 설치하도록 합시다.

```
#python
pip install openpyxl
```

```
#Anaconda
conda install openpyxl
```

2) import 하기

```
import openpyxl as op
```

앞으로 사용하는 코드는 위와 같이 import 했다고 가정하고 설명합니다. openpyxl을 코드 상에서는 'op'로 줄여 표현할 것입니다.

2. Workbook 객체 생성하기

Workbook을 생성한다는 의미는 사용자가 접근해야 할 엑셀 파일을 설정하겠다는 의미입니다. Workbook을 생성하는 경우는 2가지 케이스가 있습니다. 새로운 엑셀 파일을 만들거나, 원래 있는 파일에 접근하는 경우입니다.

1) 새로운 엑셀 파일을 만들 경우

새로운 엑셀 파일을 만드는 경우는 어렵지 않습니다. 아래와 같이 코드를 입력하면 새로운 Workbook 객체를 파이썬에서 생성합니다. 아직 저장한 상태는 아니므로, 실제 파일은 없는 상태입니다.

<코드>

```
import  openpyxl  as  op  #openpyxl 모듈 import

wb = op.Workbook() #새로운 Workbook 객체 생성

print(wb) #객체를 출력해보기
```

<결과>

```
<openpyxl.workbook.workbook.Workbook object at 0×0000020CB5E0A0C8>
```

wb를 출력해보면 openpyxl의 Workbook object라는 내용을 확인할 수 있습니다. 설명을 위해 출력을 해본 것이므로, 위 결과값이 무엇을 의미하는지는 몰라도 무관합니다.

생성한 Workbook 객체를 실제 파일로 저장하기 위해선 .save( )라는 함수를 사용하면 됩니다. 아래 코드는 위에서 생성한 Workbook 객체인 'wb'를 'openpyxl_test.xlsx' 라는 파일명으로 저장합니다.

```
wb.save("openpyxl_test.xlsx")
```

저장 할 위치를 정해주고 싶다면 아래와 같이 절대 경로(or 상대경로)로 입력하면 됩니다.

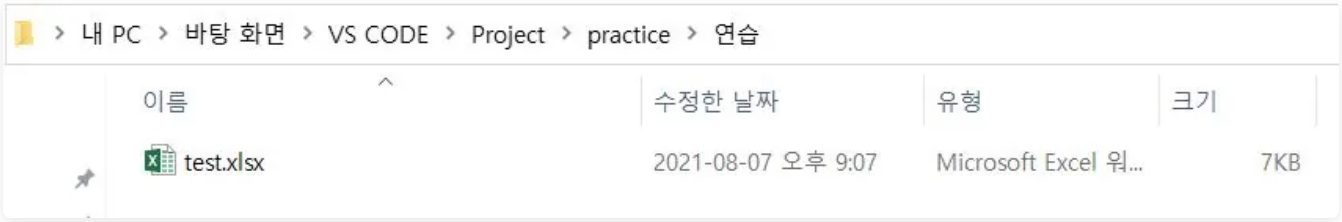
```
wb.save(r"C:\Users\Desktop\VS CODE\openpyxl_test.xlsx")
```

> *경로 안에 있는 r 표시는 unicode 에러 발생시 사용합니다. 파이썬에서는 역 슬래쉬를 인식하지 못하여 디코더 에러가 발생합니다. 따라서 경로를 표현할 때 슬래쉬(/)를 사용하거나, 역 슬래쉬를 사용해야 할 경우 문자열 앞에 r을 표기하여 줍니다.*

2) 기존에 만들어져 있는 파일로 객체를 생성하는 경우

이번에는 새로운 엑셀 파일을 생성하는 것이 아닌 기존에 만들어져 있는 엑셀파일을 Workbook 객체로 생성해보겠습니다. 실제 업무 자동화에서는 기존 엑셀 파일을 가공하는 경우가 많기 때문에 2)의 경우를 더 많이 사용하기도 합니다.

기존 엑셀 파일을 Workbook 객체로 생성하고 싶다면, .load_workbook( ) 함수를 사용하면 됩니다. 아래 경로에 있는 'test.xlsx'파일을 Workbook 객체로 생성해보겠습니다.



먼저 위 파일이 있는 path를 변수를 통해 지정해보겠습니다. path라는 변수명을 가진 문자열이며, 위 주석에서 설명한 부분이지만 unicode 에러를 방지하기 위해 문자열 앞에 "r"을 표기하였습니다.

```
path = r"C:\Users\Desktop\VS CODE\Project\practice\연습"
wb = op.load_workbook(path + "/test.xlsx")
```

그리고 객체가 제대로 생성되었는지 확인하기 위해 print 함수를 통해 wb를 다시 출력해보겠습니다.

```
<openpyxl.workbook.workbook.Workbook object at 0×00000233DBA2F108>
```

wb를 출력해보면 Workbook 객체가 제대로 생성된 것을 확인할 수 있습니다. "2) 원래 있는 파일로 객체 생성하기"과 "1) 새로운 엑셀파일 만드기"는 WorkBook 객체를 생성하는 방법이 다를 뿐, .save( ) 함수를 사용하는 방법은 동일합니다.

3. WorkSheet 설정하기

2번 과정까지는 Workbook 객체를 새로 생성하거나 이미 만들어진 엑셀 파일에 접근할 것인지에 대한 내용이었습니다.

다음은, 엑셀 파일 내부의 어떤 Sheet에 접근할 것인지 Sheet를 설정해야 합니다. 3-1-1) 엑셀의 구성요소 파트에서 엑셀 파일 1개에는 여러개 Sheet가 있을 수 있음을 설명했습니다.

Sheet를 설정하는 방법은 Sheet를 새로 생성하여 객체를 설정하거나 기존 만들어진 Sheet로 설정하는 방법이 있습니다.

1) Sheet 객체 생성1 - 새로운 Sheet 만들기

Workbook과 마찬가지로 openpyxl은 Sheet를 새로 생성하는 기능도 제공합니다. 단 Sheet를 생성할 때 주의해야할 점은 해당 엑셀 파일의 객체(Workbook) 변수를 통해 접근해야 한다는 점입니다. 아래 코드 예시를 보며 이해해봅시다.

<코드>

```
import  openpyxl  as  op  #openpyxl 모듈 import

wb = op.Workbook() #새로운 워크북 객체 생성

ws = wb.create_sheet("new_sheet1") #wb 객체를 통해 새로운 시트 생성(시트명 : new_sheet1)

print(ws) #ws 객체 출력해보기

wb.save("test_result.xlsx") #해당 워크북(엑셀파일) 저장하기
```

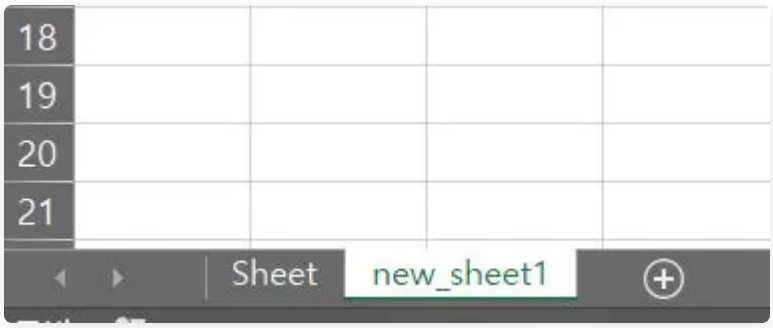
<결과>

```
<Worksheet "new_sheet1">
```

위 코드에서 새로운 시트를 생성하기 위해 wb.create_sheet( ) 라는 함수를 사용했습니다. 괄호 안의 내용은 생성할 Sheet명입니다. 주의해야할 점에서 앞에서 설명드렸다시피 Workbook 객체를 통해 생성해야 하기 때문에, "wb"라는 객체 변수로 Sheet를 생성했습니다.

만약 기존에 만들어진 엑셀 파일에 새로운 Sheet를 생성한다면 wb=op.load_workbook( ) 함수를 통해 Workbook 객체를 생성하시면 됩니다. (위 내용 중2-2) 참고)

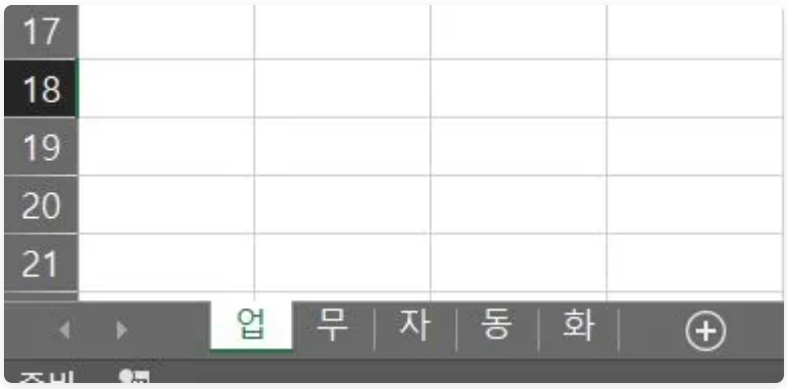
ws를 출력해보면 Worksheet "new_sheet1" 라는 결과를 확인할 수 있습니다. 실제 저장 된 엑셀 파일에서 Sheet가 생성되었는지 확인해보도록 하겠습니다.



저장 된 엑셀 파일을 열어보면 "new_sheet1" 이라는 시트가 생성되었습니다. 하지만, 위 코드에서 생성하지 않았던 "Sheet"라는 시트도 보이네요. 이 것은 엑셀에서 Workbook 객체를 생성하면 기본적으로 Sheet 1개를 생성하기 때문입니다.

2) Sheet 객체 생성2 - 기존 만들어져 있는 Sheet에 접근하기

새로운 Sheet를 생성하는 것이 아닌 기존 만들어져 있는 Sheet에 접근하려면 어떻게 할까요?? 설명을 드리기 위해 "test.xlsx" 라는 엑셀 파일에 아래 5개의 Sheet를 임의로 생성하였습니다.(업,무,자,동,화)



기존에 만들어져 있는 Sheet에 접근하기 위한 방법은 2가지가 있습니다. 순서대로 알아보겠습니다.

첫번째 방법은 "활성화되어있는 Sheet"에 접근하는 방법입니다. "활성화 되어있는 Sheet"란 엑셀 파일에서 현재 선택되어 있는 Sheet입니다. 위 이미지의 경우 "업" 시트가 되겠네요. 활성화되어있는 Sheet의 경우 .active란 코드로 Sheet를 설정할 수 있습니다.

```
ws = wb.active #활성화되어있는 시트 설정
print(ws)
```

위 코드의 결과를 보면 아래와 같습니다.

```
<Worksheet "업">
```

두번째 방법은 시트명으로 접근하는 방법입니다. wb["시트명"]으로 Sheet를 설정할 수 있습니다.

```
ws = wb["무"] #무 Sheet로 설정하기
print(ws)
```

결과를 보면 아래와 같습니다.

```
<Worksheet "무">
```

위에서 설정한 Sheet에 대한 변수 ws를 통해 내부 Cell의 데이터를 읽거나 쓸 수 있습니다. 이 내용은 "3-1-3) Cell 데이터 읽기/쓰기" 파트에서 다룰 예정입니다.

3) Workbook 객체의 모든 Sheet명 출력

이번에는 엑셀 파일의 모든 Sheet를 출력해보겠습니다. .sheetnames( )라는 함수를 통해 엑셀 파일의 내부 Sheet 목록을 모두 출력할 수 있습니다. 내부 Sheet를 파이썬의 자료 형 중 리스트로 저장하기 때문에 for문과 같이 반복문을 통해 접근하는게 가능해집니다.

해당 함수는 엑셀 업무 자동화에서 자주 사용되는 부분이니 기억해두시면 좋습니다. 위 엑셀 파일 시트 5개(업,무,자,동,화)를 리스트로 저장하여 출력하는 코드를 작성해보겠습니다.

<코드>

```
import  openpyxl  as  op  #openpyxl 모듈 import
wb = op.load_workbook(r"C:\UsersDesktop\VS CODE\Project\practice\연습\test.xlsx")
ws_list = wb.sheetnames  #해당 Workbook의 시트 목록을 리스트로 저장
print(ws_list) #리스트 출력
```

<결과>

```
['업', '무', '자', '동', '화']
```

위 코드를 응용하여 for문을 작성하고 각 Sheet 객체를 생성하여 출력하는 코드를 작성해보겠습니다.

<코드>

```
import  openpyxl  as  op  #openpyxl 모듈 import

wb = op.load_workbook(r"C:\UsersDesktop\VS CODE\Project\practice\연습\test.xlsx")

ws_list = wb.sheetnames  #해당 Workbook의 시트 목록을 리스트로 저장

for  sht  in  ws_list:
    ws = wb[sht] #Sheet 객체 생성
    print(ws) #객체 출력
```

〈결과〉

```
<Worksheet "업">
<Worksheet "무">
<Worksheet "자">
<Worksheet "동">
<Worksheet "화">
```

for 문에 따라 리스트의 각 Sheet명을 이용하여 객체를 생성하여 출력해보았습니다. 위 코드 블럭은 엑셀 자동화에 자주 쓰이는 부분이니 기억해두도록 합시다.

3) Cell 데이터 읽기/쓰기/삭제

이번 파트에서는 엑셀 각 Sheet에 데이터를 입력하거나 읽어오는 부분에 대해 정리해보겠습니다. 이번 파트 내용을 이해하기 위해서는 "3-1-1) 엑셀의 구성요소"와 "3-1-2) Workbook, Sheet 설정하기" 파트에 대한 내용이 먼저 선행되어야 합니다.

엑셀 Sheet의 Cell에 접근하는 내용에 대해서는 아래 5개 부분으로 나누어 설명 드리겠습니다.

1. Cell data 읽기

2. Cell data 쓰기

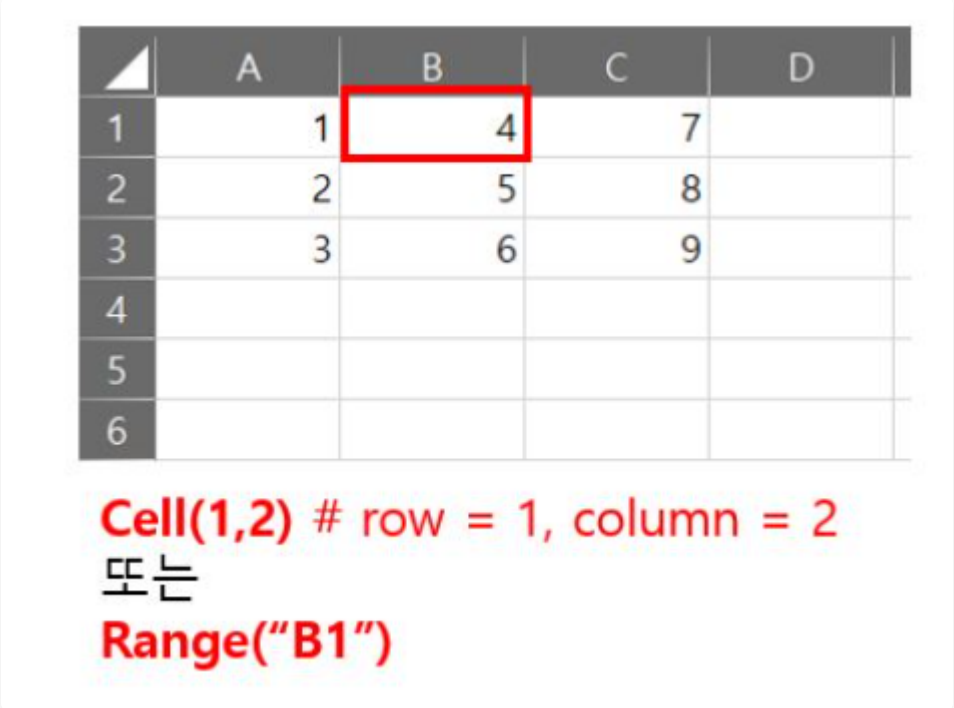
3. Cell data 삭제하기

4 .rows, .columns 속성 이해하기

5. 엑셀 함수 쓰기(max_row, max_column 속성 이해)

1. Cell의 Data 읽기

Cell의 Data를 읽어오는 방법을 학습하기 전에 먼저 3-1-1)에서 다뤘던 엑셀의 구성요소에 대해 간단하게 복습해보도록 합시다. 아래 이미지는 이전에 공부했던 내용을 요약한 것입니다.



위 이미지에서 숫자 4를 가리키는 표현법은 2가지입니다. Cell 속성을 사용하려면 행,열에 대한 위치값을 각각 알면 됩니다. 숫자 4를 기준으로 Cell(1,2)라고 표현할 수 있습니다. Cell( ) 내부의 숫자는 행 row=1(행), column=2(열)을 의미합니다. 만약, 엑셀의 고유 인덱스인 Range로 표기하려면 "B1"이라고 표기할 수 있습니다.

그럼 위에서 숫자 4를 파이썬으로 읽어오기 위해선 어떻게 코드를 작성해야 할까요? 아래 예시 코드를 보며 이해해보도록 합시다. 이전에 배웠던 Sheet 설정 객체 변수 ws를 통해 읽어올 값 위치에 접근할 수 있습니다.

엑셀 파일은 이전 파트에서 진행했던 'test.xlsx' 파일을 그대로 활용합니다. 해당 파일에는 (업,무,자,동,화) 5개 Sheet가 있었던 것을 인지하고 아래 내용을 진행하겠습니다. <코드>

```
import  openpyxl  as  op  #openpyxl 모듈 import

wb = op.load_workbook(r"test.xlsx") #워크북 객체 생성(파일명 : test.xlsx)
ws = wb.active  #활성화 되어있는 시트 설정(시트명 : "업")

#방법 1 : Sheet의 Cell 속성 사용하기
data1 = ws.cell(row=1, column=2).value

#방법 2 : 엑셀 인덱스(Range) 사용하기
data2 = ws["B1"].value

#위 결과 출력해보기
print("cell(1,2) : ", data1)
print('Range("B1"):', data2)
```

<결과>

```
cell(1,2) :  4
Range("B1"): 4
```

숫자 4를 openpyxl 코드로 읽어오는 방법은 2가지입니다. 첫번째는, Cell 속성을 사용하는 것입니다. 코드에서 보시다시피 .cell 함수를 통해 row값과 column 값을 입력해주면 해당 위치의 값을 읽어올 수 있습니다.

두번째는, 엑셀 고유 인덱스를 통해 직접 접근하는 것입니다. ws["위치인덱스"]와 같이 입력해주면 해당 위치의 값을 읽어올 수 있습니다.

각 표현법의 뒤에 .value를 붙여줘야 실제값을 읽어올 수 있다는 점은 주의해야 합니다. .value를 붙이지 않으면 각 Cell 위치에 대한 정보값을 튜플로 읽어옵니다. 직접 .value를 빼고 출력시켜보면 확인하실 수 있습니다. 아래 코드 및 결과를 보겠습니다.

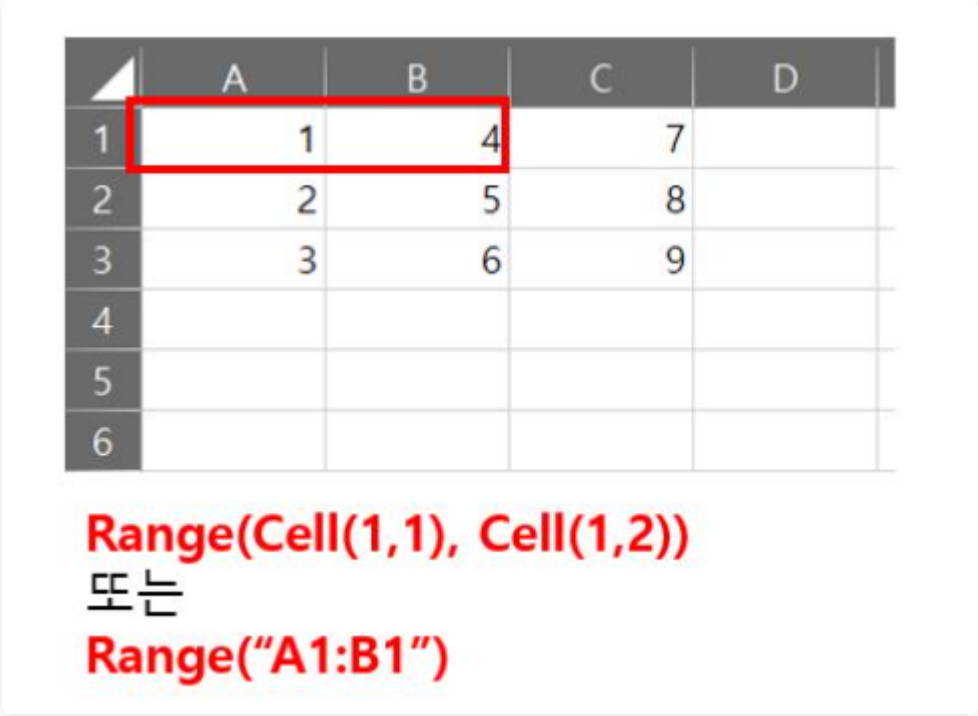
<코드>

```
#Value 빼고 출력해보기
data1 = ws.cell(row=1, column=2)
data2 = ws["B1"]
print(data1)
print(data2)
```

<결과>

```
<Cell '업'.B1> #data1 출력
<Cell '업'.B1> #data2 출력
```

.value를 삭제하고 출력한 결과 [Cell '업' .B1] 과 같은 형태로 저장되는 것을 확인할 수 있습니다. 해당 결과는 "Cell의 속성을 가지고 있고 '업'이라는 Sheet의 B1 위치의 값을 가리키고 있다"라는 뜻으로 이해하시면 됩니다. 그래서 결과를 보시면 알겠지만 위 2가지 코드의 결과는 동일합니다. (표현법이 다를 뿐 동일 위치이므로)



그럼 만약, 위 이미지 처럼 영역으로 된 부분의 data를 읽어오려면 어떻게 할까요? 아래 예시 코드를 통해 이해해보겠습니다.

먼저 rng라는 변수를 통해 범위를 설정하고 그대로 출력해보겠습니다.

<코드>

```
rng = ws["A1:B1"] #A1:B1 범위 저장
print("Range(a1:b1) : ", rng) #출력값은 튜플
```

<결과>

```
Range(a1:b1) :  ((<Cell '업'.A1>, <Cell '업'.B1>),)
```

결과에서 보시는대로 해당 영역의 위치 정보가 튜플로 저장됐습니다. 이번엔 조금 차이를 둬서 위 이미지의 데이터가 입력된 영역인 "A1:C3"으로 입력해보겠습니다.

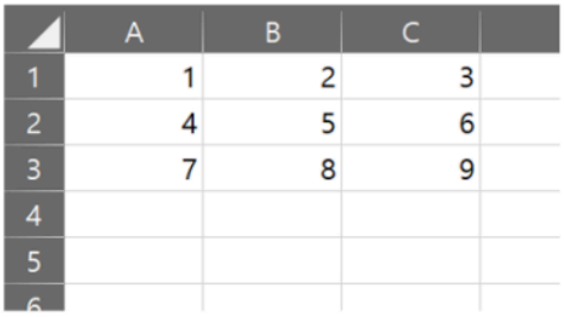
<코드>

```
rng = ws["A1:C3"] #A1:C3 범위 저장(총 9개 Cell)
print(rng) #결과 출력(튜플)
```

<결과>

```
((<Cell '업'.A1>, <Cell '업'.B1>, <Cell '업'.C1>), (<Cell '업'.A2>, <Cell '업'.B2>, <Cell '업'.
C2>), (<Cell '업'.A3>, <Cell '업'.B3>, <Cell '업'.C3>))
```

ws["A1:C3"]에 대한 위치 정보를 확인해보니 튜플 속의 튜플인 이차원 튜플로 각 위치 정보가 표기되어 저장됩니다. 위 결과를 보면 이해가 어려울 수 있으니 간단히 도식화 해보도록 하겠습니다.



	1	2	3	4	5	6	7	8	9
엑셀 Index	A1	B1	C1	A2	B2	C2	A3	B3	C3
행(Row)	1	1	1	2	2	2	3	3	3
열(Column)	1	2	3	1	2	3	1	2	3
Python(Tuple)	[0][0]	[0][1]	[0][2]	[1][0]	[1][1]	[1][2]	[2][0]	[2][1]	[2][2]

위 표에서 색깔이 다른 부분이 각각 하나의 튜플 요소라고 생각하시면 됩니다. 따라서 Python 튜플을 인덱스로 접근하기 위해서는 위 표를 이해하시면 됩니다.(튜플, 리스트 등 자료형은 인덱스가 0부터입니다.)

따라서, 위 튜플에 저장 된 위치정보를 .value 를 통해 값을 출력하려면 이중 for문을 사용하셔야 합니다. 튜플이 2차원 배열이기 때문입니다. 이중 for문을 통해 출력해보는 코드를 아래처럼 작성해보겠습니다.

<코드>

```
rng = ws["A1:C3"] #A1:C3 범위 저장(총 9개 Cell)

for  rng_data  in  rng: #튜플의 첫번째 차원 : 색깔이 다른 영역 data
    for  cell_data  in  rng_data: #튜플의 두번째 차원을 위한 for문 : 1개의 Cell 요소

    print(cell_data.value) #각 Cell 요소의 값 출력
```

<결과>

```
1
2
3
4
5
6
7
8
9
```

2. Cell에 데이터 쓰기

지금까지 공부했던 "1. Cell의 데이터 읽기"에서 Cell의 각 위치에 접근하는 방법을 이해하셨다면 데이터를 쓰는 방법은 어렵지 않습니다. 이번엔 "test.xlsx" 파일의 "무" Sheet에 접근하여 Cell에 데이터를 쓰는 방법을 이해해보도록 합시다.

아래 코드를 작성해보고 결과 엑셀 파일을 확인해보겠습니다.

<코드>

```
import  openpyxl  as  op  #openpyxl 모듈 import

wb = op.load_workbook(r"test.xlsx") #Workbook 객체 생성
ws = wb["무"] #WorkSheet 객체 생성("무" Sheet)

#"B1" Cell에 입력하기
ws.cell(row=1, column=2).value = "입력테스트1"

#"C1" Cell에 입력하기
ws["C1"].value = "입력테스트2"

wb.save("result.xlsx") #엑셀 파일 저장(파일명 : result.xlsx)
```

<결과>

	A	B	C	D
1		입력테스트1	입력테스트2	
2				
3				
4				
5				

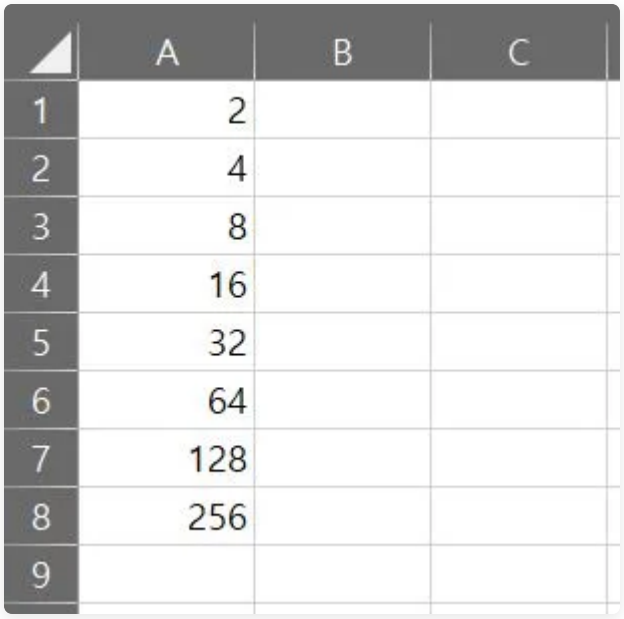
쓰는 방법은 읽는 방법과 크게 다르지 않습니다. ws 객체 변수의 Cell 속성이나 "C1"과 같은 고유 인덱스를 통해 원하는 위치에 값을 입력해주면 됩니다. 엑셀 데이터를 읽을 때와 마찬가지로 주의해야할 점은 .value 코드를 꼭 작성해주셔야 오류가 발생하지 않습니다.

이번엔 위 쓰기 내용을 조금 더 응용하여 임의의 숫자 리스트를 작성하고 일렬로 세워 입력하는 코드를 작성해보겠습니다.

<코드>

```
datalist = [2,4,8,16,32,64,128,256] #임의의 숫자 리스트 정의

i=1  # 행값을 바꾸기 위한 인덱스 정의

for  data  in  datalist:
    ws.cell(row = i, column=1).value = data  #A열(Column=1)에 행을 바꾸면서 입력
    i=i+1  #행값 증가

wb.save("result.xlsx")
```

<결과>



	A	B	C
1	2		
2	4		
3	8		
4	16		
5	32		
6	64		
7	128		
8	256		
9			

3. Cell data 삭제하기

Cell의 data를 삭제 하는 방법은 간단합니다. 대표적인 방법은 3가지를 소개하겠습니다.

1) cell.value 값을 공백으로 설정하기

저희는 "2. Cell에 data 쓰기" 파트에서 원하는 Cell에 data를 쓰는 방법을 배웠습니다. Data를 삭제하는 방법은 입력 값을 공백으로 두는 것입니다. 아래 예시를 확인해보겠습니다.

	A	B	C	D
1	1	4	7	
2	2	5	8	
3	3	6	9	
4				
5				

위 엑셀 파일에서 짝수 data만 공백으로 처리하는 코드를 작성해보겠습니다. 위에서 다뤘던 내용과 같이 "A1:C9" 영역을 튜플로 저장해서 조건문으로 처리해보겠습니다.

<코드>

```
import  openpyxl  as  op

#Workbook 객체 생성
wb = op.load_workbook("./Project/16. openpyxl 차트, 피벗/test data.xlsx")

#'test'라는 이름을 가진 Sheet를 객체로 생성
ws = wb['test']

#ws에서 데이터범위 설정
rng = ws["A1:C3"] #A1:C3 범위 저장(Cell 9개)

#튜플에 대한 for 문
for  row_data  in  rng:
    for  data  in  row_data:
        #해당 data가 2로 나눈 나머지가 0이면 공백처리
        if (data.value % 2) == 0:
            data.value = ""

#Workbook 객체 저장
wb.save("./Project/16. openpyxl 차트, 피벗/delete_result.xlsx")
```

<결과>

	A	B	C	D
1	1		7	
2		5		
3	3		9	
4				
5				
6				

결과 이미지에서 보듯이 짝수만 삭제된 것을 확인할 수 있습니다. 코드내용은 위 "1. Cell Data 읽기"에서 다뤘던 내용이므로 주석을 보며 다시 이해해보도록 합시다.

2) delete_rows, delete_cols 사용하기

1)에서 다뤘던 내용은 내용을 삭제하는 방법이지만 어떻게 보면 공백 ""을 입력하는 과정으로 "2. Cell Data 쓰기"와 크게 다를 점이 없습니다.

Openpyxl에서는 delete_rows, delete_cols 함수를 제공합니다. 이 함수들은 원하는 행이나 열을 설정한 범위 기준으로 삭제해줍니다. 1)과 다른 점은 Cell data만 삭제하는 것이 아닌 행, 열을 제거한다는 것입니다.

아래 예시를 보며 이해해보겠습니다. 먼저 사용법은 아래와 같습니다.

```
"Worksheet 객체명".delete_rows(첫행 값, 첫행부터 몇개까지?)
"Worksheet 객체명".delete_cols(첫열 값, 첫열부터 몇개까지?)
```

	A	B	C	D
1	1	4	7	
2	2	5	8	
3	3	6	9	
4	10	11	12	
5				
6				
7				

위 엑셀 파일을 활용하여 코드 테스트를 진행해보겠습니다. 먼저 행을 기준으로 삭제하는 방법입니다. 행을 기준으로 삭제할 때는 delete_rows 함수를 사용합니다.

주의해야 할 점은 괄호 안의 숫자 값입니다. delete_rows(1,2)의 의미는 1행부터 2행까지가 아니라 1행부터 시작해서 2개까지 라는 의미입니다.

<코드>

```
import openpyxl as op

#Workbook 객체 생성
wb = op.load_workbook("./Project/16. openpyxl 차트, 피벗/delete_test.xlsx")
#활성화 된 Sheet를 객체로 생성
ws = wb.active

#1행부터 2개행까지 행을 삭제한다.
ws.delete_rows(1,2)

#Workbook 객체 저장
wb.save("./Project/16. openpyxl 차트, 피벗/delete_result.xlsx")
```

<결과>

	A	B	C	D
1	3	6	9	
2	10	11	12	
3				
4				

열을 기준으로 삭제하는 delete_cols도 사용법은 동일합니다. 마찬가지로 괄호 (2,1)의 의미는 2열부터 1개 열까지라는 의미입니다. 따라서 2열만 삭제됩니다.

<코드>

```
ws = wb.active

ws.delete_cols(2,1)

wb.save("./Project/16. openpyxl 차트, 피벗/delete_result.xlsx")
```

<결과>

	A	B	C	D
1	1	7		
2	2	8		
3	3	9		
4	10	12		
5				
6				

3) 시트를 통째로 삭제하고 다시 생성하기

한 시트의 데이터를 모두 지우기 위해서는 1), 2)와 같이 공백값을 입력하거나 데이터가 있는 행,열을 모두 삭제하는 방법이 있습니다.

추가로 한가지 팁을 드리자면, 이번 방법과 같이 시트를 통째로 삭제하고 다시 생성하는 방법도 있습니다.(초기화)

.remove 함수로 시트를 삭제했고, .create_shet 함수로 시트를 똑같은 이름으로 다시 생성했습니다. 시트를 생성하거나 조작하는 방법은 "3-1-8) WorkSheet 조작하기" 파트에서도 다루니 참고하시면 됩니다.

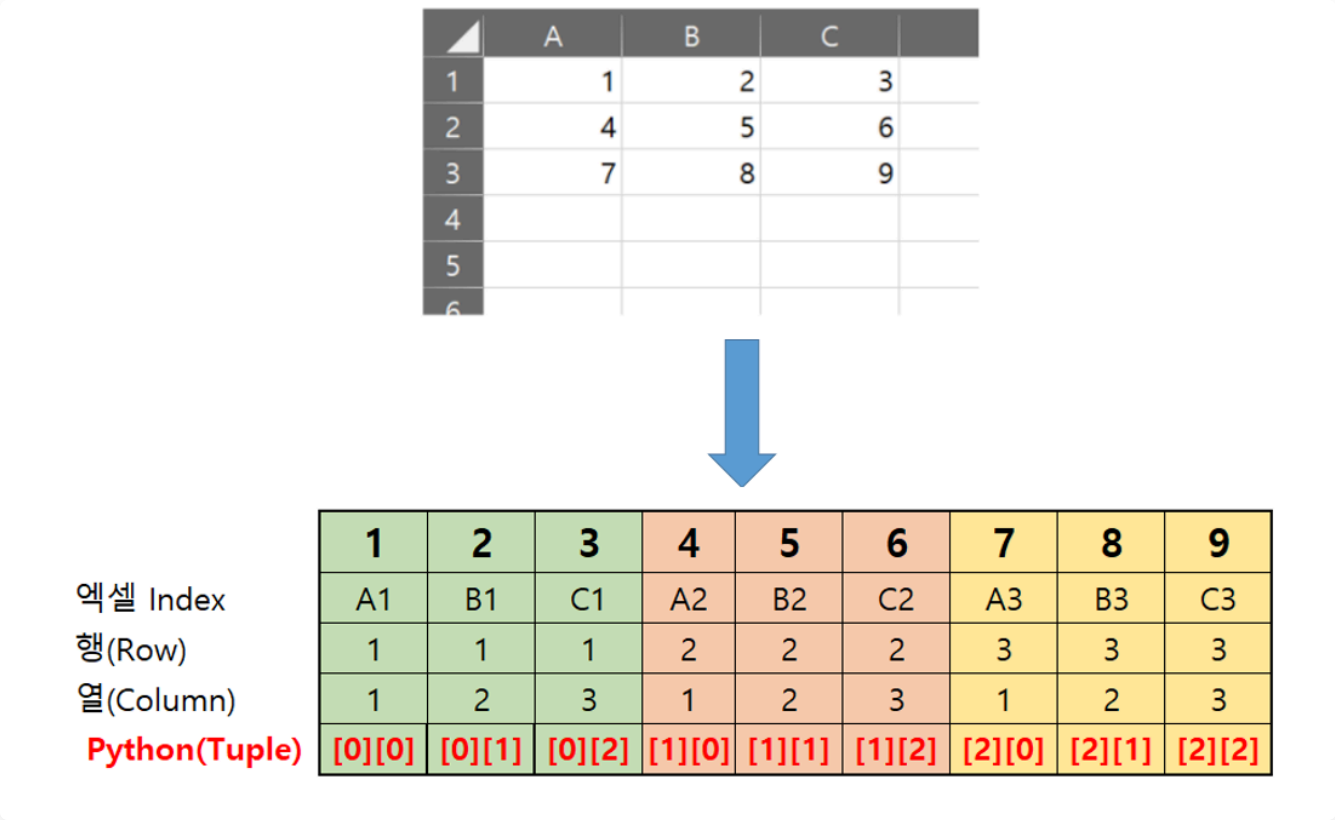
<코드>

```
import openpyxl as op

#Workbook 객체 생성
wb = op.load_workbook("./Project/16. openpyxl 차트, 피벗/delete_test.xlsx")
#'test'라는 이름을 가진 Sheet를 객체로 생성
ws = wb['test']
#test 시트 삭제
wb.remove(ws)
#test 시트 재생성
wb.create_sheet('test')

wb.save("./Project/16. openpyxl 차트, 피벗/delete_result.xlsx")
```

기본적인 읽기/쓰기 방법에 대해서 알아보았습니다. 다음 파트에서는 Sheet의 속성 중 .rows와 .columns라는 내용에 대해 알아보겠습니다.

4) rows, columns 속성 이해하기

openpyxl의 Sheet 객체(WorkSheet)에는 .rows와 .columns라는 속성이 있습니다. 이 속성들을 설명하기 전에 이전 편에서 설명했던 숫자 배열 이미지 및 코드를 그대로 가져오겠습니다.



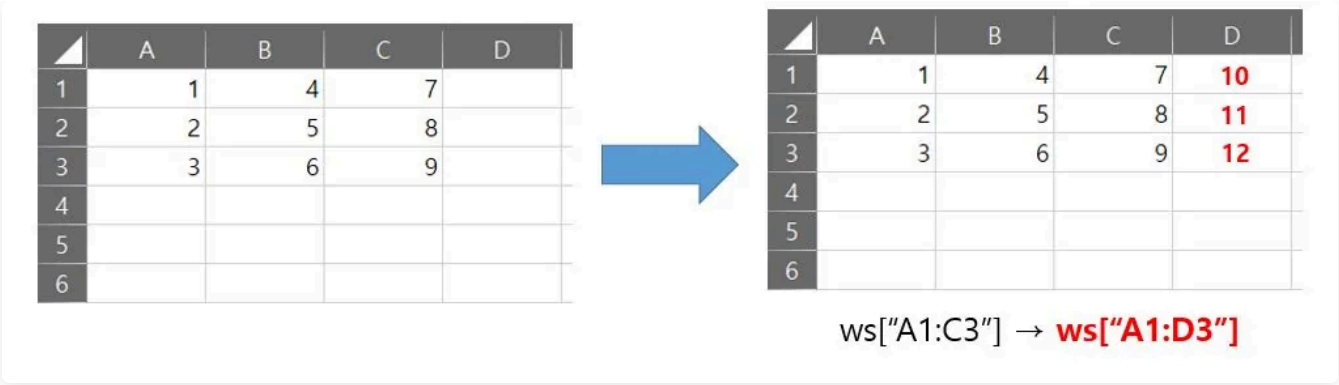
<코드>

```
rng = ws["A1:C3"] #A1:C3 범위 저장(총 9개 Cell)
print(rng) #결과 출력(튜플)
```

<결과>

```
((<Cell '업'.A1>, <Cell '업'.B1>, <Cell '업'.C1>), (<Cell '업'.A2>, <Cell '업'.B2>, <Cell '업'.
C2>), (<Cell '업'.A3>, <Cell '업'.B3>, <Cell '업'.C3>))
```

위에서 엑셀 다중 영역을 설정하기 위해 ws["A1:C3"]을 통해 각 Cell의 위치 정보를 rng 라는 변수에 저장했었습니다. 그런데 만약, 아래와 같이 데이터를 추가로 입력했다고 가정해봅시다.



"D"열에 숫자 [10,11,12]를 추가로 입력하였습니다. 만약, 엑셀 Cell에 입력 된 모든 데이터를 읽어오는 코드를 작성하였다면 위에서 작성했던 코드는 영역 설정 부분을 수정해야 합니다. 이미지와 같이 "A1:C3"를 "A1:D3"로 변경해야 합니다. 또, 데이터가 추가로 입력되면 다시 영역을 수정해야 할 것입니다.

만약, 데이터가 새로 입력되어도 영역을 자동으로 설정해주는 코드가 있다면 업무 자동화에서 좀 더 유용하지 않을까요? 이런 문제를 해결해주는 것이 Sheet의 속성인 .rows와 .columns입니다.

아래 예시를 통해 두 속성을 이해해보도록 합시다.

	A	B	C	D
1	1	4	7	
2	2	5	8	
3	3	6	9	
4				
5				

먼저 위 이미지를 기준으로 WorkSheet 객체를 생성하고 .rows와 .columns를 출력해보도록 하겠습니다.

1. rows 출력하기

<코드>

```
import  openpyxl  as  op  #openpyxl 모듈 import

wb = op.load_workbook("test.xlsx") #Workbook 객체 생성
ws = wb["업"] #"업"시트 객체 생성

print("#rows 출력")
for  row_rng  in  ws.rows:
    print(row_rng) #각 행에 대한 1차원 배열 출력(위치정보)
```

<결과>

```
#rows 출력
(<Cell '업'.A1>, <Cell '업'.B1>, <Cell '업'.C1>)
(<Cell '업'.A2>, <Cell '업'.B2>, <Cell '업'.C2>)
(<Cell '업'.A3>, <Cell '업'.B3>, <Cell '업'.C3>)
```

ws["A1:C3"]와 결과가 동일해보이지만 자세히 보시면 다릅니다. ws["A1:C3"]는 2차원 배열(튜플)이었고 이번 ws.rows는 각각의 1차원 배열이 출력되었습니다. 위 결과는 1차원 배열이 3번 출력 된 결과입니다. 각 1차원 배열은 행(row)의 위치 정보를 담고 있습니다.

2. columns 출력하기

<코드>

```
print("#columns 출력")
for  col_rng  in  ws.columns:
    print(col_rng)
```

<결과>

```
#columns 출력
(<Cell '업'.A1>, <Cell '업'.A2>, <Cell '업'.A3>)
(<Cell '업'.B1>, <Cell '업'.B2>, <Cell '업'.B3>)
(<Cell '업'.C1>, <Cell '업'.C2>, <Cell '업'.C3>)
```

ws.columns을 출력한 결과를 확인해봅시다. ws.rows와 다른 점은 각 1차원 배열(튜플)의 결과가 열의 정보를 담고있다는 겁니다. 위에서 다뤘던 .rows와 .columns의 차이를 도식화해보면 아래와 같습니다.

ws.rows

1	4	7	2	5	8	3	6	9
A1	B1	C1	A2	B2	C2	A3	B3	C3
[0]	[1]	[2]	[0]	[1]	[2]	[0]	[1]	[2]

엑셀 Index

튜플 Index

ws.columns

1	2	3	4	5	6	7	8	9
A1	A2	A3	B1	B2	B3	C1	C2	C3
[0]	[1]	[2]	[0]	[1]	[2]	[0]	[1]	[2]

엑셀 Index

튜플 Index

※ 각 색의 영역은 1차원 배열이므로 튜플 Index도 1차원

> *ws.rows 또는 ws.columns 은 파이썬 이터레이터(iterator)입니다. 이 책의 영역에서 벗어나 이터레이터에 대해 자세한 설명은 하지 않겠으나, 내부 값을 출력하기 위해선 반복문(for)를 사용해야 한다는 것을 기억해주시면 됩니다.*

3. 데이터 추가 후, rows 출력해보기

그럼 위 엑셀 파일에서 "D"열에 [10,11,12,13]를 입력하고 1) ws.rows 코드를 적용하여 다시 출력해보도록 하겠습니다. 열별로 차이를 두기 위해 "D"열에는 4개 데이터를 입력하였습니다.

	A	B	C	D	E
1	1	4	7	10	
2	2	5	8	11	
3	3	6	9	12	
4				13	
5					

그리고 "1) rows 출력하기"의 코드를 그대로 적용해보겠습니다.

<결과>

```
#rows 출력
(<Cell '업'.A1>, <Cell '업'.B1>, <Cell '업'.C1>, <Cell '업'.D1>)
(<Cell '업'.A2>, <Cell '업'.B2>, <Cell '업'.C2>, <Cell '업'.D2>)
(<Cell '업'.A3>, <Cell '업'.B3>, <Cell '업'.C3>, <Cell '업'.D3>)
(<Cell '업'.A4>, <Cell '업'.B4>, <Cell '업'.C4>, <Cell '업'.D4>)
```

마지막 행이 자동으로 추가가 되었습니다. 예시처럼 .rows와 .columns 속성은 입력 된 데이터에 따라 자동으로 위치를 인식하여 결과를 반환하여 줍니다.

위 결과값을 .value 속성을 활용하여 실제값을 출력해봅시다.

<코드>

```
print("#rows 출력")
for  row_rng  in  ws.rows:
    for  cell  in  row_rng: #각 행에 대한 1차원 배열 for문
        print(cell.value)
```

<결과>

```
#rows 출력
1
4
7
10
2
5
8
11
3
6
9
12
None
None
None
13
```

결과에 None이 포함되어 있습니다. 위 엑셀 양식을 보시면 A4, B4,C4가 빈칸인 것을 확인할 수 있죠? D4에 데이터가 입력됨으로써 A4:C4 영역도 같이 저장되었기 때문에 그렇습니다. 만약 None을 빼고 출력하고 싶다면 If문을 사용하시면 됩니다.

아래 코드로 변경하여 실행시켜 결과를 확인해보시기 바랍니다.

<코드>

```
print("#rows 출력")
for  row_rng  in  ws.rows:

    for  cell  in  row_rng: #각 행에 대한 1차원 배열 for문
        if  cell.value ≠ None: #None인경우 출력 제외
            print(cell.value)
```

5) 엑셀 함수 쓰기 - max_row, max_column 속성의 이해

openpyxl을 통해 엑셀 함수를 사용할 수 있을까요? 해당 파트는 openpyxl을 통해 엑셀 함수를 적용하는 법을 공부합니다. 정확히는, 엑셀 함수를 Cell에 쓰는 방법입니다.

먼저 아래 가상의 예시를 만들어보았습니다. 지출내역을 정리한 엑셀 파일입니다.

1. Cell에 엑셀 함수 써보기

	A	B	C	D	E
1	날짜	항목	가격		
2	2021-08-16	다과비	₩5,000		
3	2021-08-16	중식	₩10,000		
4	2021-08-16	석식	₩13,000		
5	2021-08-16	유류비	₩100,000		
6	2021-08-16	커피	₩5,000		
7	2021-08-17	다과비	₩5,000		
8	2021-08-17	중식	₩10,000		
9	2021-08-17	석식	₩13,000		
10	2021-08-17	유류비	₩100,000		
11	2021-08-17	커피	₩5,000		
12					
13					

간단한 예시를 보여주기 위해 위 이미지 지출 내역의 총 합계를 구하고 엑셀에 표시해주는 코드를 작성해볼 것입니다. 사실 엑셀에서 SUM 함수를 사용하는게 가장 빠릅니다. 이 책의 목적은 openpyxl을 통해 업무 자동화에 활용하는 방법을 배우는 것이므로 파이썬 코드로 작성해보겠습니다.

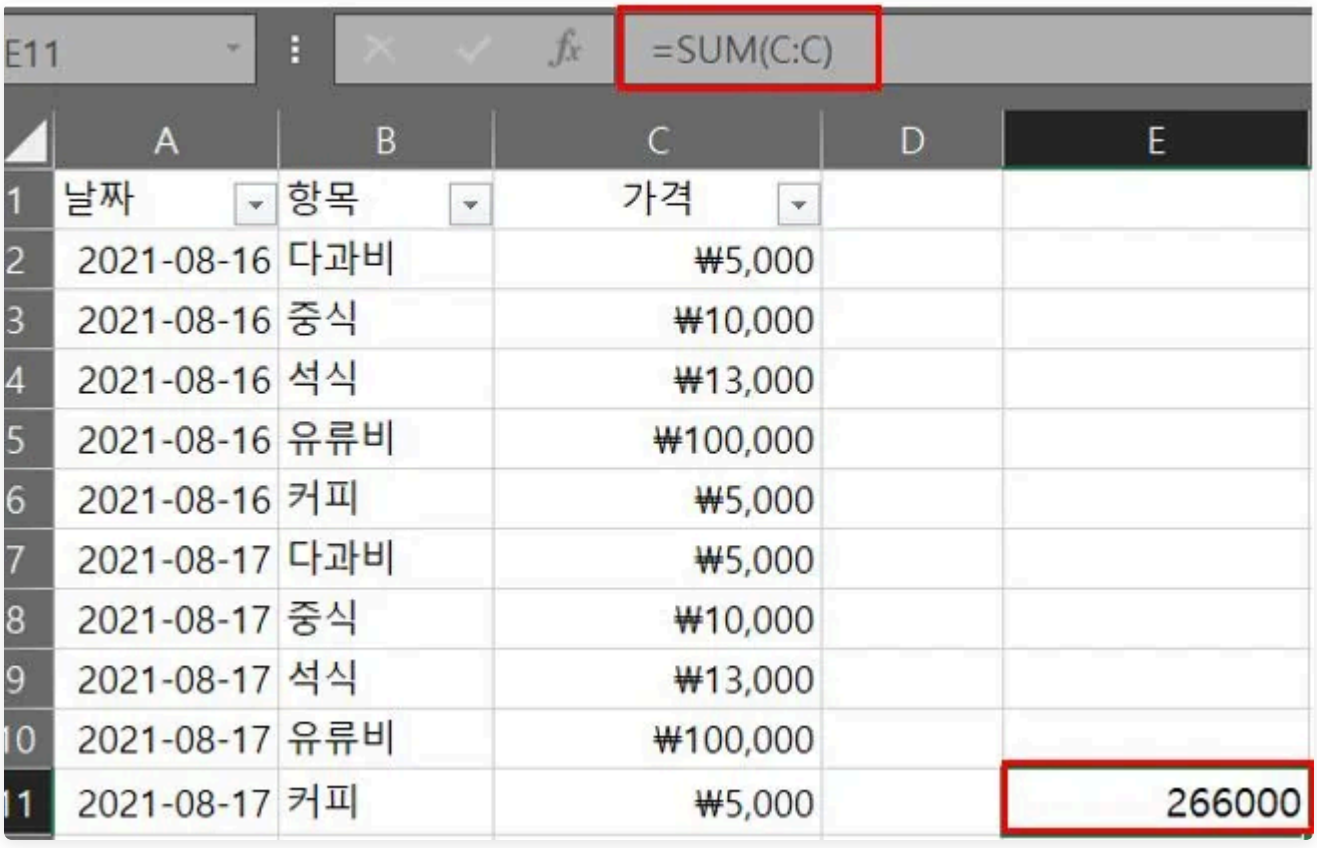
엑셀을 아시는 분이라면 위 지출 합계를 구하기 위해선 "=SUM(C:C)"를 사용하면 된다는 것을 아실겁니다. 실제로 파이썬 코드로 엑셀 SUM 함수를 입력하면 적용되는지 확인해볼까요?

접근하는 엑셀파일명은 "test.xlsx", 위 시트는 활성화되어있는 상태입니다. 입력되는 Cell의 위치는 "E11"로 생각하고 코드를 작성해보겠습니다.

<코드>

```
import  openpyxl  as  op  #openpyxl 모듈 import

wb = op.load_workbook(r"test.xlsx") #워크북 객체 생성
ws = wb.active #시트 객체 생성

#엑셀 함수를 실제 Cell에 써보기
ws["E11"].value = "=SUM(C:C)"

wb.save("result.xlsx")
```

<결과>



결과 이미지에서 보시면 "E11"에 함수가 정상적으로 사용된 것을 확인할 수 있습니다.

2. 여러 줄에 함수 한번에 적용하기

"1. Cell에 엑셀 함수 써보기"에서 다뤘던 내용은 1개 Cell에 함수를 입력하는 방법입니다. 사실 엑셀을 사용하다 보면 여러 줄에 동시에 적용해야 하는 경우가 더 많습니다.

다음 예시에는 코드를 어떻게 작성하면 될지 확인해봅시다. 위 1번에서 다뤘던 예시를 조금 수정하여 지출 내역에서 단가(C열), 인원(D열)을 통해 총 가격을 구해보는 함수를 파이썬을 통해 적용해보겠습니다.

	A	B	C	D	E	F
1	날짜	항목	단가	인원	총 가격(단가*인원)	
2	2021-08-16	다과비	₩5,000	2		
3	2021-08-16	중식	₩10,000	3		
4	2021-08-16	석식	₩13,000	3		
5	2021-08-16	유류비	₩100,000	1		
6	2021-08-16	커피	₩5,000	3		
7	2021-08-17	다과비	₩5,000	3		
8	2021-08-17	중식	₩10,000	3		
9	2021-08-17	석식	₩13,000	3		
10	2021-08-17	유류비	₩100,000	1		
11	2021-08-17	커피	₩5,000	4		
12						
13						

이전과 다른 점이라면 "E"열에 행마다 함수를 다르게 적용해야 한다는 점입니다. "줄 별로 계산식을 다 넣으면 되지 않을까요?" 라고 생각하실 수도 있지만, 이 책은 업무자동화를 다루고 있으므로 파이썬스럽게 for 반복문을 통해 코드를 작성할 것입니다. 지출내역부는 데이터가 계속 추가될 수 있는 점도 고려할겁니다.

코드를 작성하기 전에 먼저 WorkSheet(ws)의 속성 2가지를 추가로 알아보겠습니다. openpyxl에는 이전 파트에서 다뤘던 WorkSheet의 .rows, .columns 말고도 데이터가 입력 된 최대 행 값, 열 값을 정수로 반환하는속성도 존재합니다. 바로 .max_row와 .max_column입니다. 아래 코드를 보며 이해해보겠습니다.

<코드>

```
import  openpyxl  as  op  #openpyxl 모듈 import

wb = op.load_workbook(r"test.xlsx")

ws = wb.active

#해당 시트의 최대 행값, 최대 열값 구하기

col_max = ws.max_column
row_max = ws.max_row

#출력해보기
print("최대행값 : ", row_max)
print("최대열값 : ", col_max)
```

<결과>

```
최대행값 :  11
최대열값 :  5
```

결과에서 보시다시피, 최대 행값과 최대 열값을 정수로 반환합니다. 데이터를 새로 추가하면 최대 행값과 최대 열값은 자동으로 바뀌게 됩니다. 이번 코드는 이 row_max값을 활용해 작성할 것입니다.

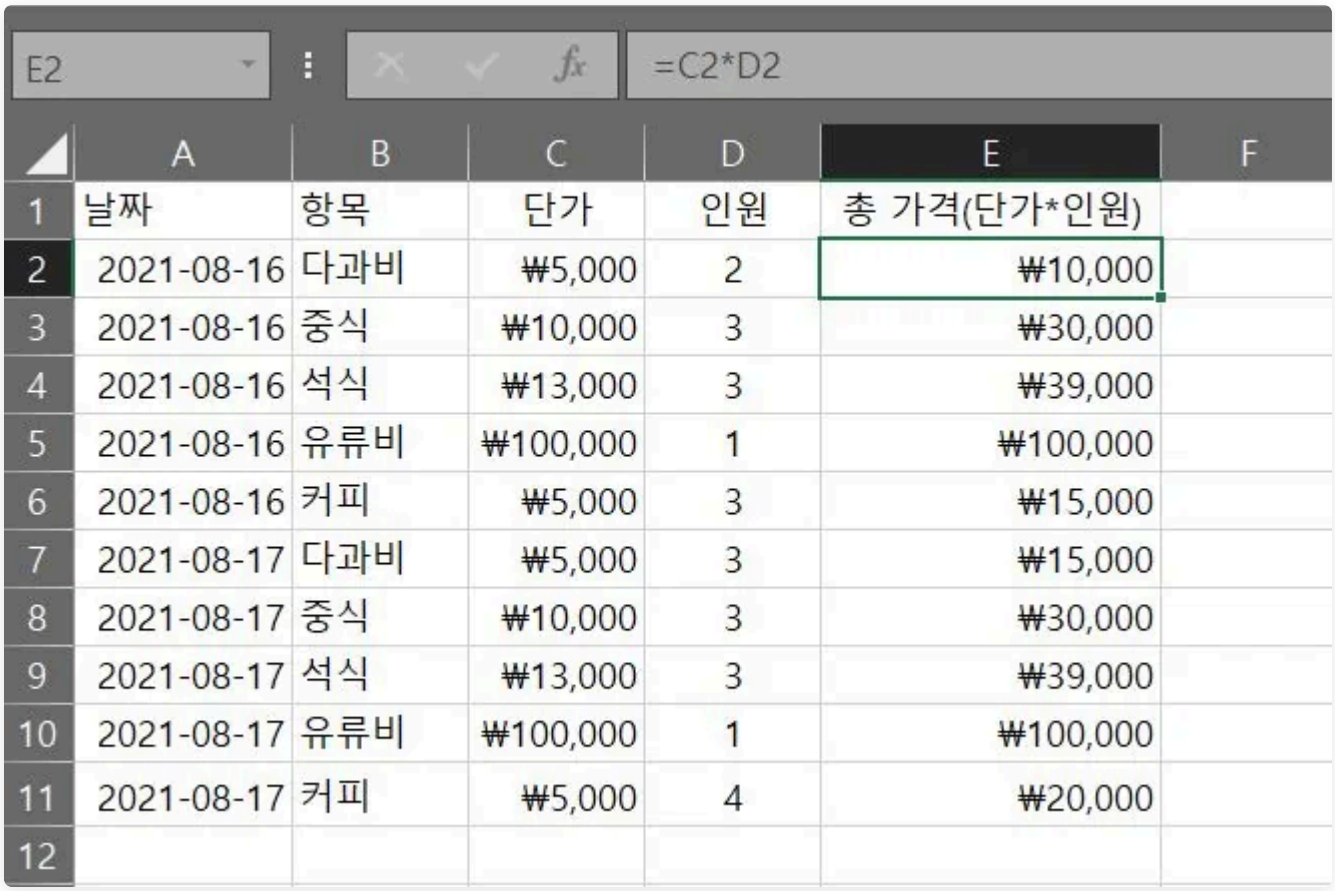
이번에 계산해야 할 총 가격(단가 x 인원)은 엑셀 열로 따지면 C x D입니다. 즉, 두번째 행의 함수는 "=C2 x D2", 3번째 행의 함수는 "=C3 x D3"로 각행마다 열 번호가 바뀌면서 반복됩니다. 따라서 행의 시작값은 2, 행의 마지막 값은 row_max로 반복문이 진행될 것입니다.

아래 코드는 설명 드린 내용을 파이썬 코드로 구현한 것입니다.

<코드>

```
import  openpyxl  as  op  #openpyxl 모듈 import

wb = op.load_workbook(r"test.xlsx") #Workbook 객체 생성
ws = wb.active #활성화 된 시트 객체 생성

row_max = ws.max_row  #최대 행값 저장

#for문 통해 2행~최대행까지 반복문
#range(a,b+1) : a부터 b까지 반복하는 range 구문
for  row  in  range(2, row_max+1):
    #함수 자동 작성
    ws["E"+str(row)].value = "=C"+str(row)+"*"+"D"+str(row)

wb.save(r"result.xlsx") #결과 엑셀파일 저장
```

<결과>



E2 | =C2*D2

	A	B	C	D	E	F
1	날짜	항목	단가	인원	총 가격(단가*인원)	
2	2021-08-16	다과비	₩5,000	2	₩10,000	
3	2021-08-16	중식	₩10,000	3	₩30,000	
4	2021-08-16	석식	₩13,000	3	₩39,000	
5	2021-08-16	유류비	₩100,000	1	₩100,000	
6	2021-08-16	커피	₩5,000	3	₩15,000	
7	2021-08-17	다과비	₩5,000	3	₩15,000	
8	2021-08-17	중식	₩10,000	3	₩30,000	
9	2021-08-17	석식	₩13,000	3	₩39,000	
10	2021-08-17	유류비	₩100,000	1	₩100,000	
11	2021-08-17	커피	₩5,000	4	₩20,000	
12						

E 열에 행마다 함수가 적용되어 계산되어 결과 값을 확인할 수 있습니다. 한가지 주의해야할 점은, 위 반복문의 정수 row를 str(row)로 표기한 것입니다. 엑셀 함수를 적용하는 것은 각 Cell에 문자열을 입력하는 것이므로 row 그대로 입력하면 에러가 발생합니다. 정수 타입의 row를 str( ) 함수를 통해 문자열로 변경함으로써, 에러를 해결할 수 있습니다.

> *각 계산될 대상(C열,D열)을 리스트나 튜플로 데이터를 입력받아 파이썬에서 계산하여 값을 출력하는 방법도 있습니다. 이번 학습 내용은 "엑셀 함수로 적용이 더 쉽다면 위와 같이 openpyxl을 통해 함수를 쓰는 방법도 있다"는 방법을 알려드리기 위해 쓴 파트입니다.*

3. 함수 결과를 다시 읽어보기

위에서 적용한 함수 열 "E"열을 행 별로 읽어와서 리스트로 저장하는 코드를 작성해보겠습니다.

<코드>

```
import  openpyxl  as  op  #openpyxl 모듈 import

#위 결과 파일을 기준으로 Workbook, WorkSheet 객체 생성
wb = op.load_workbook(r"result.xlsx")
ws = wb.active

#빈 리스트 생성
data = []

#ws.rows 속성 활용하여 for문 진행
for  row  in  ws.rows:
    data.append(row[4].value) #E열 데이터를 리스트에 추가

print(data) #최종 리스트 출력
```

<결과>

```
['총 가격(단가*인원)', '=C2*D2', '=C3*D3', '=C4*D4', '=C5*D5', '=C6*D6', '=C7*D7', '=C8*D8', '=
C9*D9', '=C10*D10', '=C11*D11']
```

결과를 확인해보시면 총 가격(E열)에 대한 결과가 리스트로 저장된 것을 확인할 수 있습니다. 그런데 한가지 이상한 점이 보이시죠? 계산된 결과가 아니라 엑셀에 적용했던 수식이 그대로 저장되었습니다. 업무자동화에 활용하려면 수식으로 계산된 값을 읽어오는 방법이 필요할 것입니다.

이 부분은 코드 중 기존 파일을 Workbook 개체로 생성하는 .load_workbook( ) 함수의 내부 옵션값을 수정하면 됩니다. 내부 옵션값에는 data_only와 read_only라는 항목이 있습니다. 2개 옵션에 대해 아래 설명 및 예시를 보며 이해해봅시다.

---

data_only = True일 경우 수식이 계산된 값을 읽어옵니다. (기본 False이므로 설정하지 않으면 함수나 수식 그대로 읽어옵니다.)

read_only = True일 경우 읽기 전용으로 읽어오는 옵션입니다. 엑셀을 수정하지 않고 데이터만 읽어올 경우 read_only=True로 작성하는게 속도 측면에서 빠릅니다. (기본 False이므로 설정하지 않으면 읽기/쓰기 모두 가능합니다.)

그럼 수식으로 계산 된 결과값을 읽어오기 위해 data_only=True를 추가하여 코드를 작성해봤습니다.

<코드>

```
wb = op.load_workbook(r"result.xlsx", data_only=True)
ws = wb.active

data = [] #빈 리스트 생성

for  row  in  ws.rows:
    data.append(row[4].value)

print(data)
```

<결과>

```
['총 가격(단가*인원)', None, None, None, None, None, None, None, None, None, None]
```

이번엔 수식이 아닌 결과값이 없다는 표시의 None이 나오네요. 사용자에 따라 결과값을 바로 확인할 수 있는 분도 있고 위 경우처럼 None이 출력되는 분도 계실겁니다. None 이 나오시는 분은 불러온 엑셀파일을 한번 실행시켜서 저장한 후, 다시 코드를 실행시켜보도록 하겠습니다.

<결과>

```
['총 가격(단가*인원)', 10000, 30000, 39000, 100000, 15000, 15000, 30000, 39000, 100000, 20000]
```

정상적으로 결과를 확인할 수 있습니다. 위와 같은 현상이 나오는 이유는 openpyxl 라이브러리의 특성 때문입니다. 저희는 "2. 여러 줄에 엑셀 함수 적용하기" 파트에서 함수를 적용 후 엑셀 파일을 .save( ) 함수를 통해 저장했었죠?

openpyxl에서는 "=C2D2"와 같은 수식을 문자열로 인식하여 엑셀 파일에 입력은 하지만 실제 위 수식을 계산하여 표시하진 않습니다. 계산의 영역은 파이썬이 아닌 실제 엑셀 프로그램의 영역이기 때문입니다. 그래서 파이썬 코드 상에서 .save( )를 하고 그대로 "result.xlsx' 결과 파일을 읽어오면 저희가 입력한 수식 문자열을 그대로 불러오는겁니다.

따라서, 엑셀 파일을 직접 실행하면 저희가 코드를 통해 입력했던 수식은 자동으로 계산이 되고, 그 때 저장이 되면 수식이 계산된 값을 인식하게 됩니다. 그런데 엑셀파일을 실행시키고 저장하는 부분을 직접하려면 굉장히 번거롭겠죠? 이 부분도 코드로 작성해보겠습니다. 엑셀을 직접 실행시키고 저장시키는 부분은 win32com 라이브러리를 사용하시면 됩니다. 아래 코드를 앞에 추가하시면 됩니다.*

<코드>

```
import  win32com.client #win32com.client 모듈 import

#excel 사용할 수 있게 설정
excel = win32com.client.Dispatch("Excel.Application")

#임시 Workbook 객체 생성 및 엑셀 열기
temp_wb = excel.Workbooks.Open(r"result.xlsx")

#저장
temp_wb.Save()

#excel 종료
excel.quit()
```

이 파트에서는 win32com에 대해서는 자세히 설명하지 않겠지만 win32com을 통해 엑셀을 직접 실행시키고 저장할 수 있습니다. win32com에 대해서는 3-4) 챕터에서 설명드리겠습니다.

아래는 이번 "5)엑셀 함수 쓰기" 파트를 정리한 전체 코드입니다. 본인 엑셀 파일에 맞게 수정하여 테스트해보시기 바랍니다.

<전체 코드>

```
import  openpyxl  as  op  #openpyxl 모듈 import
import  win32com.client



def  writeFunc():
    wb = op.load_workbook(r"test.xlsx") #Workbook 객체 생성
    ws = wb.active  #활성화 된 시트 객체 생성

    row_max = ws.max_row  #최대 행값 저장

    #for문 통해 2행~최대행까지 반복문
    #range(a,b+1) : a부터 b까지 반복하는 range 구문
    for  row  in  range(2, row_max+1):
    #함수 자동 작성
        ws["E"+str(row)].value = "=C"+str(row)+"*"+"D"+str(row)

    wb.save(r"result.xlsx") #결과 엑셀파일 저장



def  loadData():
    #엑셀 직접 실행시키고 저장 및 종료
    excel = win32com.client.Dispatch("Excel.Application")
    temp_wb = excel.Workbooks.Open(r"result.xlsx")
    temp_wb.Save()
    excel.quit()

    #openpyxl 통해 Workbook 객체 및 WorkSheet 객체 생성
    wb = op.load_workbook(r"result.xlsx", data_only=True)
    ws = wb.active

    #빈 리스트 생성
    data = []

    #ws.rows 속성 활용하여 for문 진행
    for  row  in  ws.rows:
        data.append(row[4].value) #E열 데이터를 리스트에 추가

    print(data) #최종 리스트 출력

if  __name__  ==  "__main__":
```

```
    writeFunc() #writeFunc 실행
    loadData() #loadData 실행
```

6) 엑셀 서식 지정하기 - 기본

지금까지 openpyxl 라이브러리를 통해 기본적으로 Cell에 데이터를 읽고 쓰는 방식에 대해 알아봤습니다. 엑셀에는 데이터를 읽고 쓰는 것 말고도 셀 서식(테두리, 셀 색상, 글씨체 등)을 지정하고 조건부 서식을 적용할 수 있습니다. 조건부 서식이란 조건에 따라 다르게 지정하는 셀 서식을 의미합니다.

이번 편은 openpyxl을 통해 셀 서식을 적용하는 방법을 알아봅시다.

파이썬 openpyxl 라이브러리는 엑셀의 셀 서식 제어 기능을 제공합니다. Font, Border, Alignment, patternfill, Protection 등의 모듈을 제공합니다. 이번 셀서식 기본 파트에서는 위에서 언급한 6가지 모듈에 대한 사용법을 알아보도록 하겠습니다.

```
# Copyright (c) 2010-2021 openpyxl


from .alignment import Alignment
from .borders import Border, Side
from .colors import Color
from .fills import PatternFill, GradientFill, Fill
from .fonts import Font, DEFAULT_FONT
from .numbers import NumberFormatDescriptor, is_date_format, is_builtin
from .protection import Protection
from .named_styles import NamedStyle
```

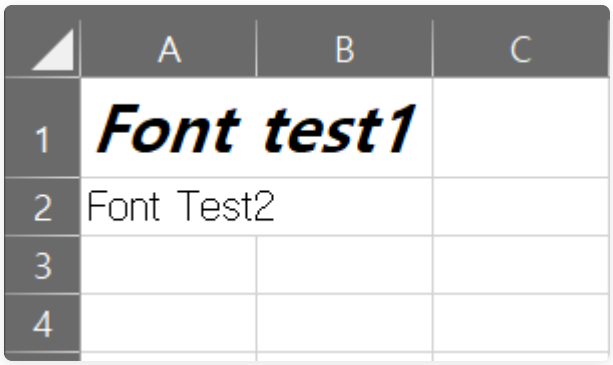
1) Font(글꼴)

openpyxl Font 모듈에서는 글자의 크기, 글꼴, 밑줄 등의 옵션을 설정할 수 있습니다. 먼저 아래 코드를 작성하여 실행시켜보도록 하겠습니다.

<코드>

```
import  openpyxl  as  op
from  openpyxl.styles.fonts  import  Font

#Workbook 및 Worksheet 객체 설정하기
wb = op.Workbook()
ws = wb.active

#font test1 : 직접 font 설정하기
ws["A1"].value = "Font test1"
ws["A1"].font = Font(size=20, italic = True, bold = True)

#font test2 : format을 정해놓고 font 설정하기
ws["A2"].value = "Font Test2"
font_format = Font(size=12, name='굴림', color = 'FF000000')
ws["A2"].font = font_format

#Workbook(엑셀) 저장 및 객체 닫기
wb.save("test_result.xlsx")
wb.close()
```

<결과>



위 코드는 Font 를 설정하는 2가지 방식을 나타내보았습니다. 첫번째 방식(font test1)은 직접 Cell 주소에 직접적으로 font를 설정하는 방식입니다. 두번째 방식(font test2)은 특정 format을 정해 놓고 변수로 지정하여 Cell에 설정하는 방식입니다. 두번째 방식은 밑에서 다룰 조건부 서식에서 사용할 수 있는 방식입니다.

코드 중 "Font(size=20, italic = True, bold = True)"의 괄호 안에는 엑셀에서 font를 설정할 수 있는 여러 설정 값들이 있습니다. size는 폰트 크기, Italic은 글자 기울임, bold는 글자를 굵게 할 것인지 설정할 수 있습니다.

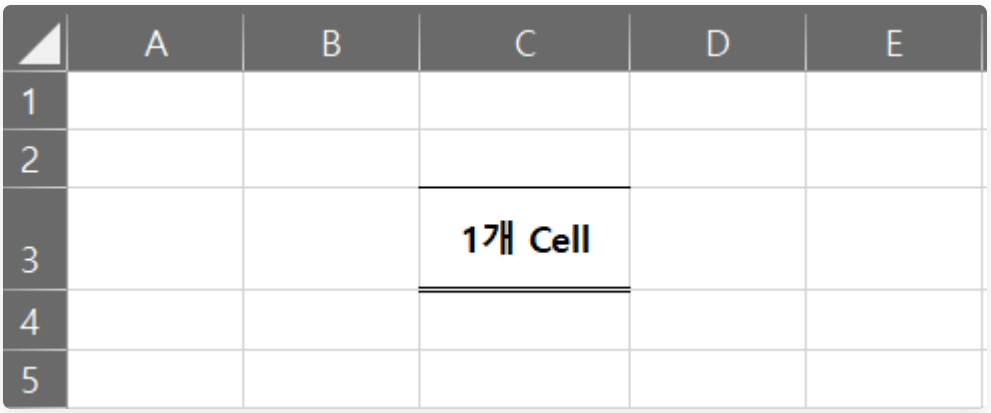
아래는 Font를 설정할 수 있는 여러 설정 값들입니다. 이미지는 openpyxl 공식 홈페이지에서 참고하였습니다. 설정 값들에 대한 자세한 내용은 openpyxl 공식 홈페이지를 참고하시면 됩니다.

```
font = Font(name='Calibri',
                size=11,
                bold=False,
                italic=False,
                vertAlign=None,
                underline='none',
                strike=False,
                color='FF000000')
```

2) Border, Side(테두리)

Border, Side 모듈은 엑셀의 셀 테두리를 설정할 수 있는 모듈입니다. Border의 경우 "선택 Cell에 상하좌우 어떤 부분에 테두리를 설정할 것인가?"에 대한 모듈입니다. Side는 "각 테두리에 어떤 테두리 형식을 적용할 것인가"에 대한 부분으로 이해하시면 됩니다.

예를 들어, 아래와 같이 1개 Cell "C3"에 위에는 실선, 아래에는 이중선을 적용한다고 생각해봅시다. 그럼 위와 아래를 설정하는 코드는 border를 이용해야 합니다. 위에는 실선, 아래는 이중선이라는 형식은 side를 사용합니다. 대신 side는 2가지 변수로 설정 되어있어야 합니다.



아래 코드를 통해 이해해보도록 하겠습니다.

<코드>

```
#모듈 import
import  openpyxl  as  op
from  openpyxl.styles  import  Border, Side
from  openpyxl.styles.colors  import  Color

#Workbook 객체 생성 및 Sheet 설정
wb = op.Workbook()
ws = wb.active

#"C3"에 값 입력
ws["C3"].value = "1개 Cell"

#border test1 : 위에는 실선, 아래에는 이중선 적용 예시 코드
ws["C3"].border = Border(top = Side(border_style="thin"))
ws["C3"].border = Border(bottom = Side(border_style="double"))

#객체 저장 및 닫기
wb.save("style_result.xlsx")
wb.close()
```

위 코드는 이미지의 "C3" 셀에 위는 가는 실선, 아래는 이중선을 적용하는 셀 서식을 Border와 Side 모듈을 통해 구현해본 것입니다. 위 주석을 보며 내용을 이해해보도록 합시다. 주의해야 할 점은 각 셀 위치에 접근할 때 ws["C3"].border와 같이 뒤에 설정할 속성을 써주어야 합니다.

위 코드에서는 각 테두리에 대한 형태만 설정했지만 테두리의 색상도 설정할 수 있습니다. 색상은 아래와 같이 지정하시면 됩니다. 색상은 Side 내부에 있는 color 파라미터를 활용하시면 됩니다.

<테두리 색상 코드 설정 예시>

```
ws["C3"].border = Border(top = Side(border_style="thin", color="000000"))
```

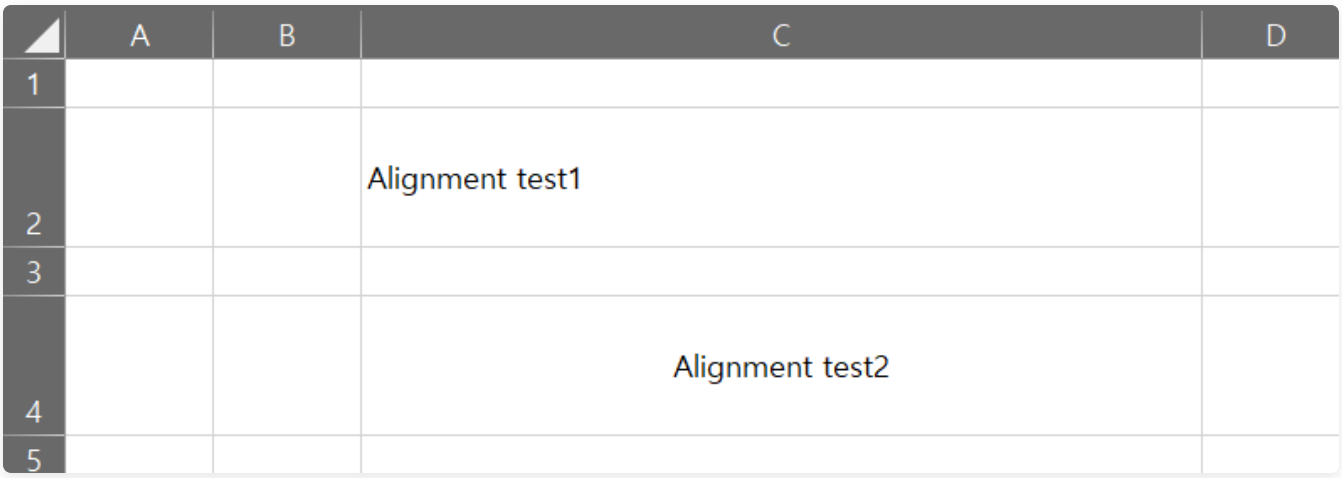
color 의 속성 값인 "000000"은 openpyxl에서 제공하는 IndexedColours 값입니다. "000000"은 검은색을 나타내는 인덱스입니다. "FF0000"는 빨간색, "0000FF"는 파란색 등 색 별로 여러 인덱스가 존재합니다. 여기서는 인덱스 값에 대해 자세히 다루진 않지만 openpyxl tutorial 공식 홈페이지의 'Working with styles'를 참고하시면 자세하게 아실 수 있습니다.

Border에 대한 속성은 아래 표를 참고하시기 바랍니다.

세부 설정 요소	설정 값	기본 값
left / right / top / bottom vertical / horizontal (좌우 모서리) (상하 모서리) outline (네 모서리 모두)	'dashDot' / 'dashDotDot' / 'dashed' / 'dotted' / 'double' / 'hair' / 'medium' / 'mediumDashDot' / 'mediumDashDotDot' / 'mediumDashed' / 'slantDashDot' / **'thick' / 'thin'**	None
diagonal (대각선)	위와 동일	None
diagonal_direction (대각선 방향)	0 / 1	0

3) Alignment(정렬)

Alignment는 셀에서 text를 어떤 방식으로 정렬할 것인지를 설정할 수 있습니다.



위 이미지를 봤을 때, "C2"의 Alignment test1은 수직으로 가운데 정렬, 수평 방향으로 왼쪽 정렬을 하였습니다. "C4"의 Alignment test2는 수평,수직 방향으로 모두 가운데 정렬한 것입니다.

이 내용을 openpyxl 코드로 구현하면 어떻게 될까요?

<코드>

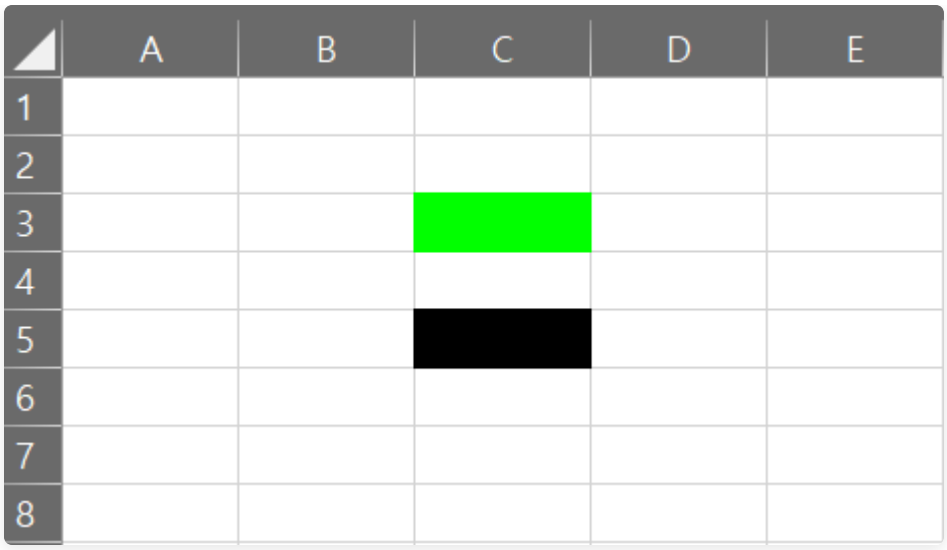
```
import  openpyxl  as  op
from  openpyxl.styles  import  Alignment

wb = op.Workbook()
ws = wb.active

# "C2"와 "C4"에 Text 입력
ws["C2"].value = "Alignment test1"
ws["C4"].value = "Alignment test2"

# 셀 너비, 높이 설정하기
ws.row_dimensions[2].height = 50 #2행의 높이 50으로
ws.row_dimensions[4].height = 50 #4행의 높이 50으로
ws.column_dimensions['C'].width = 50 #C열의 너비 50으로

#Alignment test1
ws["C2"].alignment = Alignment(horizontal = 'left', vertical='center')

#Alignment test2
format1 = Alignment(horizontal = 'center', vertical='center')
ws["C4"].alignment = format1

#Save
wb.save("result.xlsx")
wb.close()
```

위 이미지와 코드를 비교해서 이해해보시길 바랍니다. Alignment 내부 속성인 horizontal, verical은 정렬 방향을 설정하는 속성입니다. 각 속성에 대한 설정값은 아래와 같습니다.

-. horizontal : 'center', 'left' , 'right'
-. vertical : 'top', 'bottom' 'center'

4) PatternFill(채우기)

PatternFill은 셀 채우기를 설정하는 모듈입니다. 예를 들어, 아래와 같이 "C3"와 "C5"에 각각 연두색, 검은색 셀 채우기를 해봤습니다. 이 것을 openpyxl을 통해 구현하려면 어떻게 할까요?



"2) Border, Side" 파트에서도 다뤘지만 openpyxl에서는 컬러에 대한 인덱스를 제공합니다. 아래 연두색은 "00FF00", 검은색은 "000000"입니다. 이 인덱스를 통해 코드를 작성해보겠습니다.

<코드>

```
import  openpyxl  as  op
from  openpyxl.styles  import  PatternFill

#Workbook 생성 및 시트 설정
wb = op.Workbook()
ws = wb.active

#PatternFill test1 : green
ws["C5"].fill = PatternFill(fill_type='solid', fgColor="00FF00")
ws["C3"].fill=green

#PatternFill test2 : Black
ws["C5"].fill = PatternFill(fill_type='solid', fgColor="000000")

wb.save("result.xlsx")
wb.close()
```

PatternFill의 내부 속성에 대해 설명드려보겠습니다. fill_type은 셀을 채우는 형태입니다. "solid"는 설정한 색으로 전부 채우는 속성값입니다. fgColor는 어떤 색으로 채울 것인가에 대한 속성입니다. 이전에도 설명드렸던 바와 마찬가지로 openpyxl의 컬러 인덱스를 사용합니다.

자세한 내용은 아래 표를 참고해주세요. patterType의 GradientFill( )은 셀 채우기 형태의 다른 형태입니다. GradientFill( )을 사용할 경우 셀 배경색을 설정할 때, 속성을 fgColor가 아닌 start_color/end_color를 사용합니다.

세부 설정 요소	설정 값	기본 값
patternType	PatternFill() / GradientFill()	사용 예 참고
fill_type	**'solid'** / 'darkDown' / 'darkGray' / 'darkGrid' / 'darkHorizontal' / 'darkTrellis' / 'darkUp' / 'darkVertical' / 'gray0625' / 'gray125' / 'lightDown' / 'lightGray' / 'lightGrid' / 'lightHorizontal' / 'lightTrellis' / 'lightUp' / 'lightVertical' / 'mediumGray'	None
fgColor (배경색)	'000000~FFFFFF' or '00000000~FFFFFFFF'	000000(검정색)
bgColor (패턴색)	'000000~FFFFFF' or '00000000~FFFFFFFF'	000000(검정색)
start_color / end_color (그라데이션)	'000000~FFFFFF' or '00000000~FFFFFFFF'	None

5) Protection(셀 숨김, 보호)

Protection은 특정 셀 숨김/잠금 속성을 설정해주는 모듈입니다. 아래 코드를 통해 이해해보도록 하겠습니다.

<코드>

```
import  openpyxl  as  op
from  openpyxl.styles  import  Protection

#Workbook 객체 생성 및 시트 설정
wb = op.Workbook()
ws = wb.active

#text 입력하기
ws["C3"].value = "Protection test1 : locked"
ws["C5"].value = "Protection test2 : hidden"

#Protection 속성 설정하기(숨김/잠금)
ws["C3"].protection = Protection(locked = True, hidden = True)
ws["C5"].protection = Protection(locked = False, hidden = False)

#엑셀 파일 저장 및 객체 닫기
wb.save("result.xlsx")
wb.close()
```

7) 엑셀 서식 지정하기 - 조건부 서식

openpyxl의 셀 서식 기능을 업무 자동화에 활용한다면 대표적인 예시가 조건부 서식 적용입니다. 조건부 서식은 어떤 특정 조건에 따라 셀 서식을 달리 적용하도록 할 수 있는 기능입니다. 엑셀에서 기본적으로 조건부 서식이라는 기능을 제공하나, 이 파트에서는 openpyxl을 통해 조건부 서식을 적용하는 방법을 알아보겠습니다.

1. 성적표에 합격/불합격 표시하기

먼저 아래와 같은 예시를 임의로 작성해보았습니다. 저희가 학교에서 볼 수 있는 성적표입니다. 먼저 조건부 서식을 적용하기 전에 평균이 70점 이상일 경우 "합불여부" 열에 합격/불합격을 표시하는 코드를 작성해보겠습니다.

아래 엑셀의 파일명은 "조건부서식 test.xlsx"입니다.

	A	B	C	D	E	F	G
1	이름	국	영	수	평균	합불여부	
2	YB	100	100	100	100		
3	HJ	100	6	80	62		
4	PN	50	40	50	46.66667		
5	SB	100	80	90	90		
6							

<코드>

```
import openpyxl as op

#test 엑셀 파일이 있는 경로 지정
path = r"C:\Users\Desktop\VS CODE\Project"

wb = op.load_workbook(path + "/" + "조건부서식 test.xlsx", data_only=True) #Workbook 객체 생성
ws = wb.active #Worksheet 객체 생성

#합/불 판정해주는 함수
def passfail():
    #최대 행값 구하기
    max_row = ws.max_row
    #최대 행값 활용하여 for문 (2행부터~최대행까지)
    for row_index in range(2, max_row+1):
        #평균값 데이터를 average 변수에 저장
        average = ws.cell(row = row_index, column=5).value
        #평균이 70점 이상이면 '합격' 표시
        if average >= 70:
            ws.cell(row=row_index, column=6).value = "합격"
        #평균이 70점 미만이면 '불합격' 표시
        else:
            ws.cell(row=row_index, column=6).value = "불합격"

#실행부
if __name__ == "__main__":
    passfail()
    wb.save(r"C:\Users\Yongbeom Jeong\Desktop\VS CODE\Project\result.xlsx")
```

위 코드는 이전 openpyxl 내용들을 이해하셨다면 충분히 작성해볼 수 있는 코드입니다. 엑셀파일 "조건부서식 test.xlsx" 파일의 평균 점수를 읽어와서 70점 이상이면 "합격", 미만이면 "불합격"을 표시하는 코드입니다.

결과는 아래와 같습니다. 아래 이미지는 결과파일 "result.xlsx"를 실행시켜서 나온 결과입니다.

	A	B	C	D	E	F	G
1	이름	국	영	수	평균	합불여부	
2	YB	100	100	100	100	합격	
3	HJ	100	6	80	62	불합격	
4	PN	50	40	50	46.66667	불합격	
5	SB	100	80	90	90	합격	
6							

2. 합격/불합격에 대한 조건부 서식 적용하기

조건부서식을 적용하기 전에 1번 합격/불합격을 먼저 표시해보았습니다. 조건부서식 내용이 아닌데 1번을 왜 먼저 다뤘을까요?

조건부서식을 적용하는 방법이 1번의 내용과 크게 다르지 않기 때문입니다. 파이썬 openpyxl을 통해 조건부서식을 적용하는 방법은 아래와 같이 코드를 작성하시면 됩니다.

-. 각 조건에 따른 셀서식 format을 변수로 설정하기

-. if ~ else 등 조건문을 사용하여 조건에 따른 셀서식 format 적용하기

위 내용만으로는 이해하기 어려울 수 있으니 직접 코드를 작성해보겠습니다.

<코드>

```
#조건부 서식 적용 함수
def conditionFormat():
    #합격일 때 format 변수로 설정
    pass_format =  Font(size=12, name='굴림', color = '000000FF') #   000000FF은 파란색
    #불합격일 때 foramt을 변수로 설정
    fail_format =  Font(size=12, name='굴림', color = '00FF0000') #00ff000은 빨간색
    #행 최대값 구하기
    max_row  = ws.max_row

    #행 최대값 사용하여 for문 사용(반복)
    for row_index in range(2, max_row+1):
        #합격/불합격인지 문자열 읽어오기
        result_str = ws.cell(row = row_index, column=6).value
        #합격일 경우 서식
        if result_str == "합격":
            ws.cell(row=row_index, column=6).font = pass_format

        #불합격일 경우 서식
        else:
            ws.cell(row=row_index, column=6).font = fail_format
```

위 코드에 주석을 달아놨으니 천천히 이해해보도록 합시다. 결과 이미지는 아래와 같습니다. 합격일 경우 "굴림", "font size = 12", "파란색"을 적용하고 불합격일 경우 "굴림", "font size = 12", "빨간색"인 조건부 서식을 적용하는 코드입니다.

<결과>

	A	B	C	D	E	F	G
1	이름	국	영	수	평균	합불여부	
2	YB	100	100	100	100	합격	
3	HJ	100	6	80	62	불합격	
4	PN	50	40	50	46.66667	불합격	
5	SB	100	80	90	90	합격	
6							

3. 전체 코드

앞에서 다룬 1,2번에 대한 전체 코드를 각각 함수로 작성해보았습니다. 아래 코드를 통해 서식 format을 수정하여 연습해보시기 바랍니다.

<코드>

```
import openpyxl as op
from openpyxl.styles.fonts import Font

path = r"C:\Users\Desktop\VS CODE\Project"

wb = op.load_workbook(path + "/" + "조건부서식 test.xlsx", data_only=True) #Workbook 객체 생성
ws = wb.active #Worksheet 객체 생성

#합/불 판정해주는 함수
def passfail():
    max_row = ws.max_row
    for row_index in range(2, max_row+1):
        average = ws.cell(row = row_index, column=5).value
        if average ⩾ 70:
            ws.cell(row=row_index, column=6).value = "합격"
        else:
            ws.cell(row=row_index, column=6).value = "불합격"

#조건부 서식 적용 함수
def conditionFormat():
    pass_format =  Font(size=12, name='굴림', color = '000000FF') #   000000FF은 파란색
    fail_format =  Font(size=12, name='굴림', color = '00FF0000') #00ff000은 빨간색
    max_row  = ws.max_row
    for row_index in range(2, max_row+1):
        result_str = ws.cell(row = row_index, column=6).value
        if result_str == "합격":
            ws.cell(row=row_index, column=6).font = pass_format
        else:
            ws.cell(row=row_index, column=6).font = fail_format



#실행부
if __name__ == "__main__":
    passfail()
    conditionFormat()
    wb.save(r"result.xlsx")
```

8) WorkSheet(=Sheet) 조작하기

이번 챕터에서는 openpyxl을 이용해 엑셀의 시트를 조작하는 여러가지 방법을 배워보겠습니다.

먼저, 새로운 Workbook 객체를 생성하고 해당 Workbook의 시트명을 출력하는 코드를 작성해보겠습니다.

<코드>

```
import  openpyxl  as  op

#새로운 Workbook 객체 생성
wb = op.Workbook()

#새로운 Workbook의 시트명 리스트 출력해보기
print(wb.sheetnames)
```

<결과>

```
['Sheet']
```

새로운 Workbook 객체를 생성하고 sheet 리스트를 출력해보면 'Sheet' 라는 리스트 요소 1개가 출력됩니다. 이전 "2) Workbook 설정하기, Sheet 설정하기" 파트에서 새로운 Workbook을 생성할시 'Sheet'라는 시트가 자동 생성된다고 언급한 바가 있습니다.

> *.sheetnames 함수는 해당 Workbook(엑셀파일)의 시트명을 리스트로 출력해주는 함수입니다. openpyxl을 통해 자동화를 진행할 때, 자주 사용하는 부분으로 기억해두도록 합시다.*

위에서 새롭게 생성한 Workbook 객체를 기준으로 아래 4가지 시트 조작 방법을 연습해보겠습니다.

1. 시트 생성/삭제하기

Workbook 객체에 시트를 새로 생성하는 방법은 .create_sheet 함수를 사용하는 것입니다. 시트 생성 함수는 이전 "2) WorkBook, WorkSheet 설정하기" 파트에서도 다룬 적이 있습니다. 이번 편은 해당 내용에 대해 좀더 자세히 다뤄보겠습니다.

create_sheet 함수 사용법은 아래와 같습니다.

```
'Workbook 객체명'.create_sheet('시트명',위치값(정수))
```

아래 예시 코드를 보며 이해해봅시다.

1) 시트 생성

<코드>

```
import  openpyxl  as  op

#새로운 Workbook 객체 생성
wb = op.Workbook()

#시트 새로 생성하기(create sheet)
ws = wb.create_sheet("연습")
print(wb.sheetnames)
```

<결과>

```
['Sheet', '연습']
```

시트를 생성하는 방법은 간단합니다. 위 사용법과 같이 create_sheet("시트명")을 입력해주면 자동으로 해당 Workbook 객체(=wb)에 새로운 Sheet가 자동으로 생성됩니다. 추가로, 변수로 지정한 ws는 새롭게 생성한 '연습' Sheet의 객체가 됩니다.

만약, 기존에 생성 된 시트가 많아 시트의 위치를 지정해주고 싶다면 아래 코드와 같이 사용하시면 됩니다.

<코드>

```
#첫번째 위치에 생성
ws = wb.create_sheet("첫번째", 0)
print("첫번째 : ", wb.sheetnames)

#마지막에서 두번재 위치에 생성
ws = wb.create_sheet("뒤에서 두번째",-1)
print("뒤에서 두번째 : ", wb.sheetnames)

#두번째 위치에 생성
ws = wb.create_sheet("두번째",1)
print("두번째 : ", wb.sheetnames)

#세번째 위치에 생성
ws = wb.create_sheet('세번째',2)
print("세번째 : ", wb.sheetnames)
```

<결과>

```
첫번째 : ['첫번째', 'Sheet', '연습']
뒤에서 두번째 : ['첫번째', 'Sheet', '뒤에서 두번째', '연습']
두번째 : ['첫번째', '두번째', 'Sheet', '뒤에서 두번째', '연습']
세번째 : ['첫번째', '두번째', '세번째', 'Sheet', '뒤에서 두번째', '연습']
```

위 코드와 결과를 대조해서 확인해보시기 바랍니다. create_sheet( ) 내부의 숫자 입력값에 따라 시트의 위치가 다르게 생성 되는 것을 확인하실 수 있습니다.

> *한가지 주의할 점은 위 생성한 시트 내용은 wb.save( ) 함수를 통해 저장을 해야 원본에 반영이 됩니다. 자동화 프로그램을 작성할 때는 save( ) 를 꼭 넣도록 합시다. 이 부분은 시트 생성뿐만 아니라 시트를 제어하는 모든 기능에 해당됩니다.*

2) 시트 삭제

그럼 이번에는 생성한 시트를 다시 삭제하는 코드를 작성해보도록 하겠습니다. 시트를 삭제하는 방법은 간단합니다. Workbook 객체의 remove 함수를 사용하면 됩니다.

사용법은 아래와 같습니다.

```
'Workbook 객체명'.remove('Workbook 객체명'['Sheet명'])
```

이렇게 설명하니 뭔가 복잡하네요. 예시 코드를 보며 쉽게 이해해봅시다. 주의할 점은 괄호 안의 내용을 Sheet명만 쓰는게 아니라 wb['시트명']과 같이 Worksheet 객체로 설정해주어야 한다는 것입니다.

<코드>

```
#원본 시트 리스트 출력
print("원본 : ", wb.sheetnames)

#시트 삭제 : 삭제시 괄호 안의 내용은 Worksheet 객체이다.
ws = wb['First']
wb.remove(ws)

#삭제 후 시트 리스트 출력
print("삭제 후 :", wb.sheetnames)
```

<결과>

```
원본 :  ['First', 'Second', '세번째', 'Sheet', '뒤에서 두번째', '연습', 'Second Copy']
삭제 후 : ['Second', '세번째', 'Sheet', '뒤에서 두번째', '연습', 'Second Copy']
```

2. 시트 이름 변경하기

시트 이름을 변경하는 방법은 WorkSheet 객체의 title 속성을 사용하면 됩니다. 아래 예시 코드를 확인해보겠습니다.

<코드>

```
#'첫번째'라는 이름을 가진 시트를 'First'로 변경
ws1 = wb["첫번째"]
ws1.title = "First"
print(wb.sheetnames)

#'두번째'라는 이름을 가진 시트를 'Second'로 변경
ws2 = wb["두번째"]
ws2.title = "Second"
print(wb.sheetnames)
```

<결과>

```
['First', '두번째', '세번째', 'Sheet', '뒤에서 두번째', '연습']
['First', 'Second', '세번째', 'Sheet', '뒤에서 두번째', '연습']
```

이름을 바꾸려고 하는 WorkSheet 객체의 title 속성 값을 변경 할시트명으로 지정해주시면 결과와 같이 시트명을 변경할 수 있습니다.

3. 시트 이동/복사(같은 파일)

이번엔 같은 파일에서 시트의 위치를 변경하고 복사하는 방법을 알아보겠습니다. 시트 위치 이동과 복사는 .move_sheet 함수와 copy_worksheet 함수를 사용하면 됩니다. "1-2) 시트 삭제"에서 다뤘던 내용과 마찬가지로 함수 괄호 안의 내용은 Worksheet 객체가 들어가야 합니다.

아래 코드를 보며 이해해보겠습니다.

> *다른 파일로 시트를 이동,복사하는 기능은 Openpyx로 구현할 수 없습니다. 파일 간 시트 이동/복사는 win32com.client 모듈을 사용해주시기 바랍니다. 해당 내용은 책의 챕터 3-3에서 배웁니다.*

1) 시트 이동

move_sheet는 현재 위치(기준)에서 상대적인 위치 값(offset)을 입력하여 시트를 이동시킵니다.

```
#원본 출력
print("원본 : ", wb.sheetnames)

#Workshet 객체 설정(시트명 : Second Sheet)
ws = wb['Second sheet']

#'Second sheet'를 기준점에서 앞으로 5칸 이동
wb.move_sheet(ws, -5)

#위치 이동 후 출력
print("이동 후 : " , wb.sheetnames)
```

<결과>

```
원본 :  ['First', 'Second', '세번째', 'Sheet', '뒤에서 두번째', '연습', 'Second Copy']
이동 후 :  ['First', 'Second Copy', 'Second', '세번째', 'Sheet', '뒤에서 두번째', '연습']
```

2) 시트 복사

copy_worksheet를 사용하면 시트를 복사할 수 있습니다. 특정시트 복사시 '해당 시트명' + 'Copy'라는 이름으로 시트가 생성됩니다. 아래 예시를 보겠습니다.

<코드>

```
#'Second' 시트를 Worksheet 객체로 설정
sht = wb["Second"]
#'Second' 시트를 복사하고 복사한 시트를 ws_copy 객체로 설정
ws_copy = wb.copy_worksheet(sht)

#출력해보기
print("복사한 시트명 : ", ws_copy)
print("시트리스트 : ", wb.sheetnames)
```

<결과>

```
복사한 시트명 :  <Worksheet "Second Copy">
시트리스트 :  ['First', 'Second', '세번째', 'Sheet', '뒤에서 두번째', '연습', 'Second Copy']
```

결과를 보시면 'Second Copy'라는 시트가 생성된 것이 확인할 수 있습니다.

4. 시트 탭 속성 설정

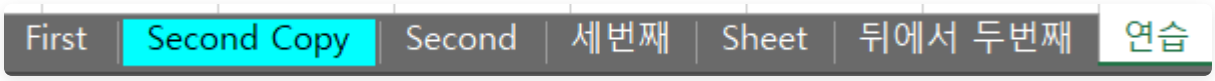
Worksheet 객체의 sheet_properties 속성을 활용하면 탭의 색을 변경할 수 있습니다.

위에서 복사했던 'Second Copy' 시트의 탭 색을 변경해보겠습니다.

<코드>

```
#시트 탭 색깔 변경(하늘색)
ws_copy.sheet_properties.tabColor = '00FFFF'

#다시 저장하여 확인
wb.save("./Project/15. openpyxl 시트 조작 관련/after.xlsx")
```

<결과>



sheet_propertie.tabColor 속성을 활용하면 위 결과와 같이 탭의 색을 변경할 수 있습니다. 참고로 탭색을 확인하시려면 결과 파일을 열어봐야 하므로 wb.save( )를 통해 엑셀파일을 저장했습니다.

> *~~.tabColor ='00FFFF' 에서 '00FFFF'는 하늘색을 가르키는 Color Index입니다. 참고로 이전 OpenPyxl 셀서식 편에서 Color Index에 대해 잠깐 언급한 적이 있으니 내용 참고하시면 되겠습니다.*

3-2. Excel - xlwings 소개

이번 파트에서는 또 다른 엑셀 자동화 모듈인 xlwings에 대해서 공부해보겠습니다. 엑셀을 제어한다는 점에서 openpyxl과 비슷해보이지만 xlwings는 엑셀의 매크로(VBA : Visiaul Basic for Appliacation)와 파이썬을 연동하는 것이 가능합니다.

xlwings 공식 홈페이지 : https://www.xlwings.org/

1) 엑셀 VBA(Visual Basic for Applicaiton) 소개

1. 엑셀 VBA는 무엇인가?

VBA란 Visual Basic for Application의 약자로 Microsoft 사에서 제공하는 프로그래밍 언어입니다. Excel, Powerpoint,Word, Outlook과 같은 Microsoft 응용 프로그램을 위한 언어라고 생각하시면 됩니다. 엑셀 VBA는 Excel에서 사용자가 원하는 기능을 프로그래밍 언어를 통해 개발할 수 있는 도구입니다.

2. 파이썬과 VBA의 연동

개요에서 설명드린 부분이지만, 파이썬의 xlwings 라이브러리를 사용하면 엑셀의 VBA와 연동이 가능해집니다. 엑셀에서 파이썬 코드를 불러와서 사용할 수 있고, 반대로 파이썬에서 엑셀 VBA를 실행시키는 것도 가능합니다.

최근, 파이썬으로 쉽게 엑셀을 제어 할 수 있는 패키지가 많이 개발되어 있어서 VBA를 사용하지 않아도 엑셀 자동화 프로그램을 개발할 수 있습니다. 하지만, 엑셀 VBA는 엑셀을 위한 프로그래밍 언어인만큼 파이썬에 비해 엑셀을 제어하는 기능이나 속도 측면에서 좀더 나은 부분이 있습니다.

따라서, 필요에 따라 엑셀 VBA를 파이썬과 연동한다면 엑셀 업무 자동화에서 좀더 활용도 높은 자동화 프로그램을 개발할 수 있습니다. 해당 책에서는 VBA에 대해 자세히 다루진 않습니다. 다만, VBA는 엑셀과 유래를 같이 한 만큼 인터넷에 자료가 많은 편입니다. 처음부터 직접 모든걸 개발하지 않아도 원하는 기능의 VBA 코드를 인터넷에서 찾아 xlwings와 연동하는 방법만 알고 있어도 유용하게 활용할 수 있습니다.

이번 xlwings 파트에서는 엑셀 VBA를 사용하는 방법 정도만 간단하게 정리하고 넘어가겠습니다.

3. 엑셀 VBA 사용하기

1) 매크로(VBA) 파일 생성하기

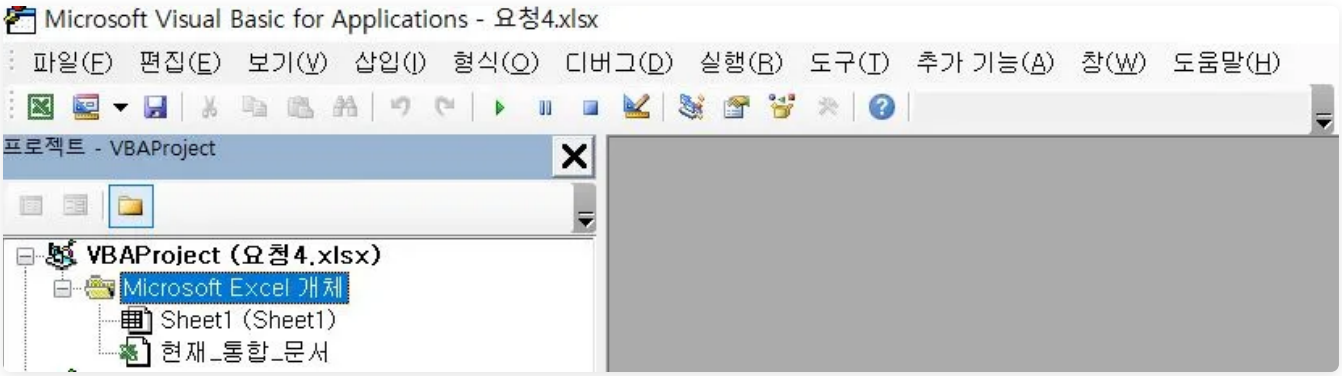
엑셀에는 여러 확장자 파일이 있습니다.(.xlsx, xls, .xlsm, xltm 등) VBA를 동작시키기 위해선 .xlsm 확장자 파일로 생성하시면 됩니다. 앞으로 xlwings로 다루는 코드들은 .xlsm 기준으로 설명하겠습니다.

2) VBA 개발 도구 실행해보기

엑셀 VBA 개발 도구(코드 작성 창)을 실행시키는 방법은 2가지입니다. 메뉴 선택바 중 개발 도구 - Visual Basic을 선택하거나 단축키(Alt+F11)를 누르는 것입니다.

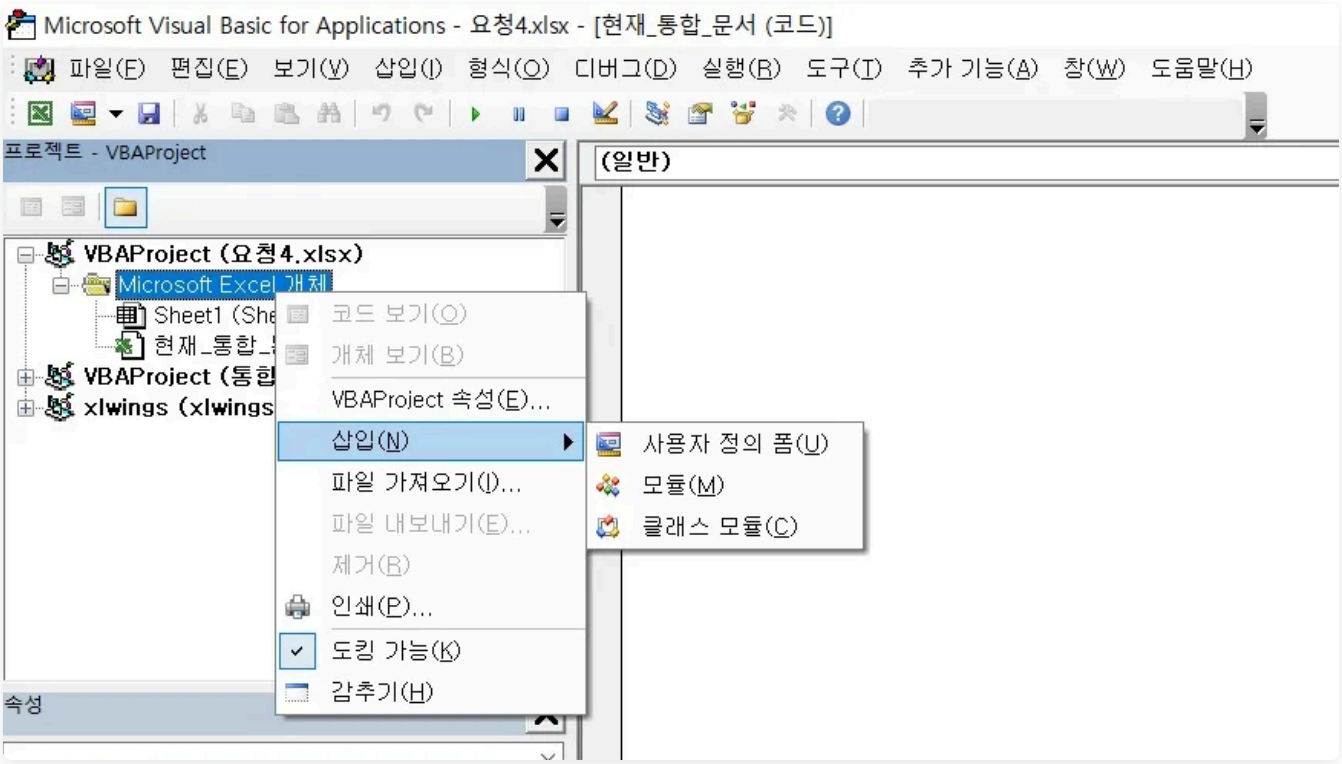
아래 이미지는 엑셀 메뉴 선택바를 통해 개발 도구를 실행시키는 방법입니다.

위 버튼을 클릭(단축키 Alt+F11)하시면 아래와 같은 엑셀 VBA 개발 도구 창이 실행됩니다.



3) 모듈 추가

엑셀 VBA에는 모듈이라는 개념이 있습니다. 모듈은 VBA에서 프로젝트를 구성하는 기본 단위입니다. 프로시저(Prodecure)의 집합으로 표현하기도 합니다. 프로시저는 특정 기능을 실행하기 위한 코드 집합입니다. 개념을 외우실 필요는 없고 '이런게 있구나'정도로 알아두세요.



VBA 코드를 작성하려면 위에서 설명한 모듈을 생성해야 합니다. 위 이미지처럼 'Microsoft Excel 개체' - 마우스 오른쪽 클릭을 하여 삽입-모듈을 클릭하시면 됩니다.

모듈이 정상적으로 생성되었으면 아래 사진처럼 모듈 폴더와 함께 'Module1'이라는 항목이 생깁니다. 'Module1'을 더블 클릭하시면 해당 모듈에 VBA 코드를 작성할 수 있습니다.(아래 이미지)

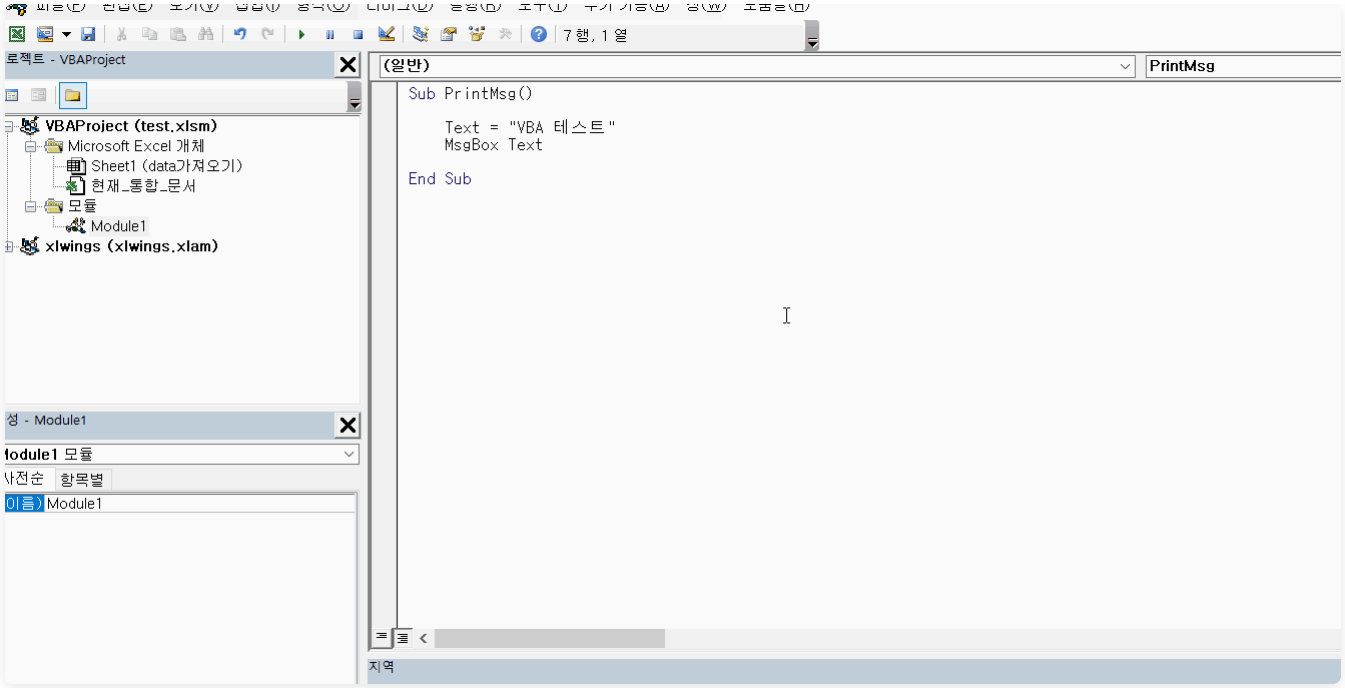
4) VBA 코드 작성 해보기

프로시저 1개를 정의하고 메시지박스를 띄우는 VBA 코드를 작성해보겠습니다. 아래 코드에서 Sub ~ End Sub라는 부분이 하나의 프로시저입니다. 프로시저의 이름은 'PrintMsg'입니다.

<코드>

```
Sub PrintMsg()
    Text = "VBA 테스트"
    MsgBox Text
End Sub
```

<결과>



기본적인 VBA 사용법 설명은 끝났습니다. 앞에서 말씀드린대로, 이 책에서는 VBA에 대해 자세히 다루진 않습니다. 파이썬 xlwings 라이브러리를 사용하기 위한 기본적인 VBA 사용법만 명시하였습니다. 다음 파트에서는 xlwings 설치 및 엑셀 환경 설정에 대한 부분을 알아보겠습니다.

2) xlwings 설치 및 Excel 환경설정하기

이번 파트에서는 파이썬에서 xlwings를 설치하고 엑셀과 연동할 수 있도록 개발환경을 설정하는 방법을 정리해보겠습니다.

1. xlwings 설치하기

xlwings는 파이썬 내장모듈이 아니기 때문에 라이브러리를 사용하기 위해서는 먼저 설치를 해야 합니다.

openpyxl과 다르게 xlwings 라이브러리 말고도 xlwings addin이라는 엑셀 Add-in도 추가로 설치해야 합니다.

설치방법은 어렵지 않습니다. 먼저 xlwings 라이브러리부터 설치해봅시다. 본인의 개발환경 터미널 창에 아래 명령어를 입력하세요.

```
#Python
pip install xlwings

#Anaconda
conda install xlwings
```

그리고 동일한 방법으로 add-in도 설치해줍시다. xlwings를 설치가 완료된 상태에서 아래와 같이 입력해주세요.

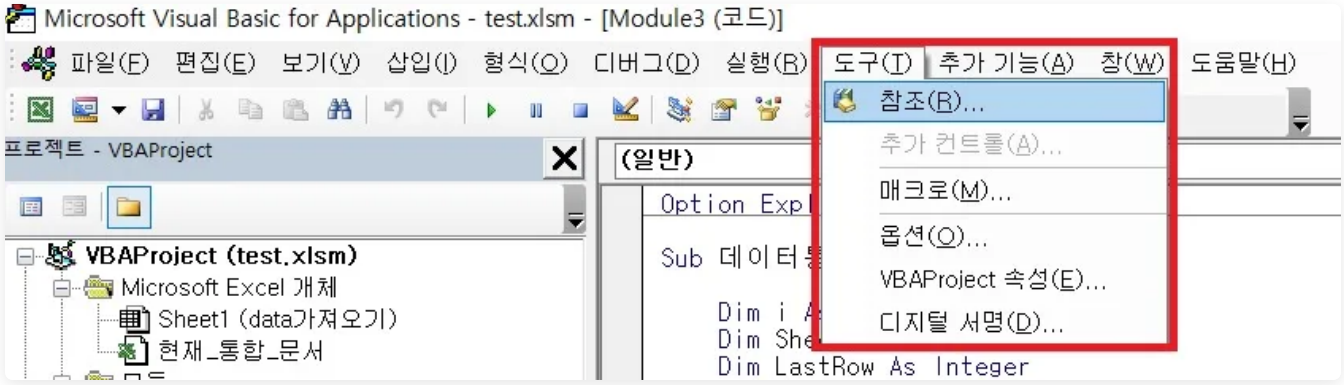
```
xlwings addin install
```

정상적으로 설치가 되었다면 사용자의 PC 엑셀 프로그램을 실행시켜보세요. 아래와 같이 엑셀 메뉴 우측 상단에 'xlwings'라는 메뉴가 생겼다면 정상적으로 설치된 것입니다.

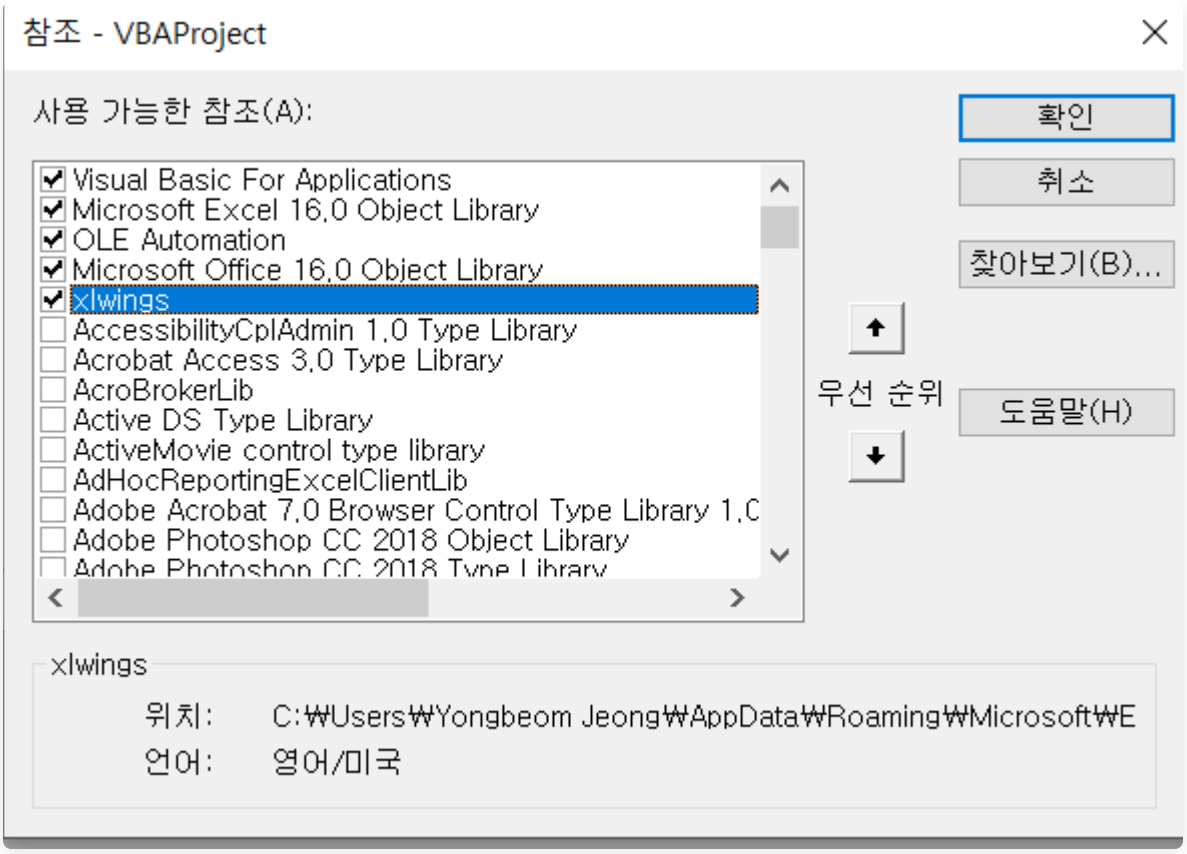
2. 엑셀 VBA 개발 환경 설정하기

엑셀 VBA와 xlwings를 연동하기 위해서는 엑셀 VBA의 개발 환경을 별도로 설정해줘야 합니다.

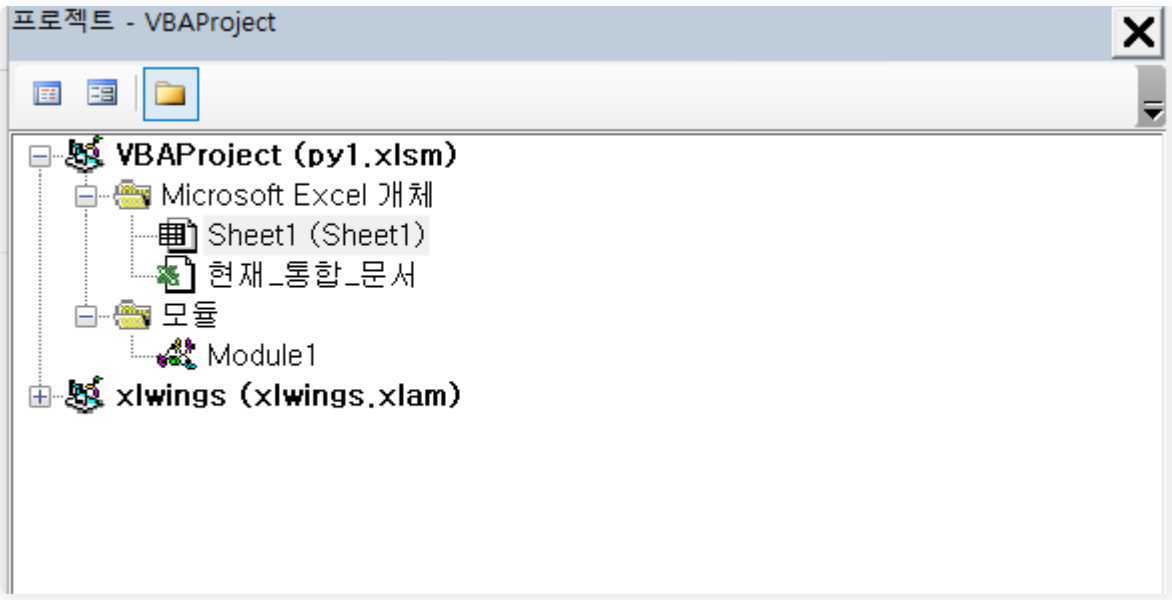
이전 파트에서 엑셀 VBA 개발 도구(코드 작성 창)을 실행시키는 방법을 설명했습니다. 엑셀을 실행시킨 후, Visual Basic 개발 도구를 실행시켜봅시다.(단축키 Alt+F11)



그 다음, 위 이미지를 참고하여 상단의 '도구'-'참조' 메뉴를 선택해봅시다. 그럼 아래와 같은 창이 나오며 'xlwings'라는 항목을 체크하고 확인을 눌러줍시다.



그럼 설정이 끝났습니다. 정상적으로 개발환경이 설정되어 있다면 아래 이미지처럼 VBA Project 창에 xlwings라는 항목이 추가되어있어야 합니다.



다음 파트부터는 xlwings와 파이썬을 연동하는 방법을 알아볼 것입니다.

3) VBA에서 파이썬 코드 실행시키기

이번 파트에서는 VBA에서 파이썬 코드를 실행시키는 방법을 알아보겠습니다.

1. 파이썬 코드 작성하기

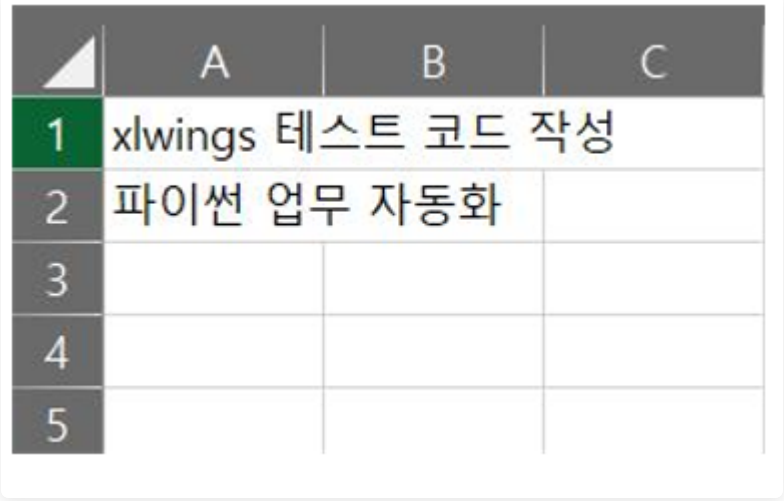
파이썬 코드를 VBA에서 동작시키기 전에 동작 시킬 파이썬 테스트 코드를 작성해보겠습니다. 아래 파이썬 코드 파일명은 "py1.py"입니다.

<코드>

```
import  xlwings

def  main():
    #xlwings 통해 Workbook 호출
    wb = xlwings.Book.caller()

    #Sheet 설정
    sheet = wb.sheets[0]

    #텍스트 입력
    sheet["A1"].value = "xlwings 테스트 코드 작성"
    sheet["A2"].value = "파이썬 업무 자동화"

if  __name__== "__main__":
    path = r"C:\Users\Desktop\VS CODE\Project\practice"
    #매크로 파일 설정
    xlwings.Book(path+"/"+"p1.xlsm").set_mock_caller()
    #main 함수 호출
    main()
```

VBA를 활용하여 실행시키기 전에 파이썬에서 코드를 제대로 작성했는지 py1.py 파일 내에서 main( ) 함수를 호출해봅시다. 결과는 아래와 같습니다. "A1"과 "A2" 셀 위치에 텍스트가 출력됩니다.

<결과>



2. VBA에서 파이썬 코드 호출하기

이제 위 파이썬 코드를 VBA에서 호출해보도록 합시다. 엑셀 VBA를 사용하는 방법에 대해서는 "3-1) 엑셀 VBA(Visual Basic for Application) 소개"에서 다뤘으니 해당 내용은 생략하고 다음 과정부터 진행해보겠습니다. VBA 엑셀 파일의 파일명은 "py1.xlsm"입니다.

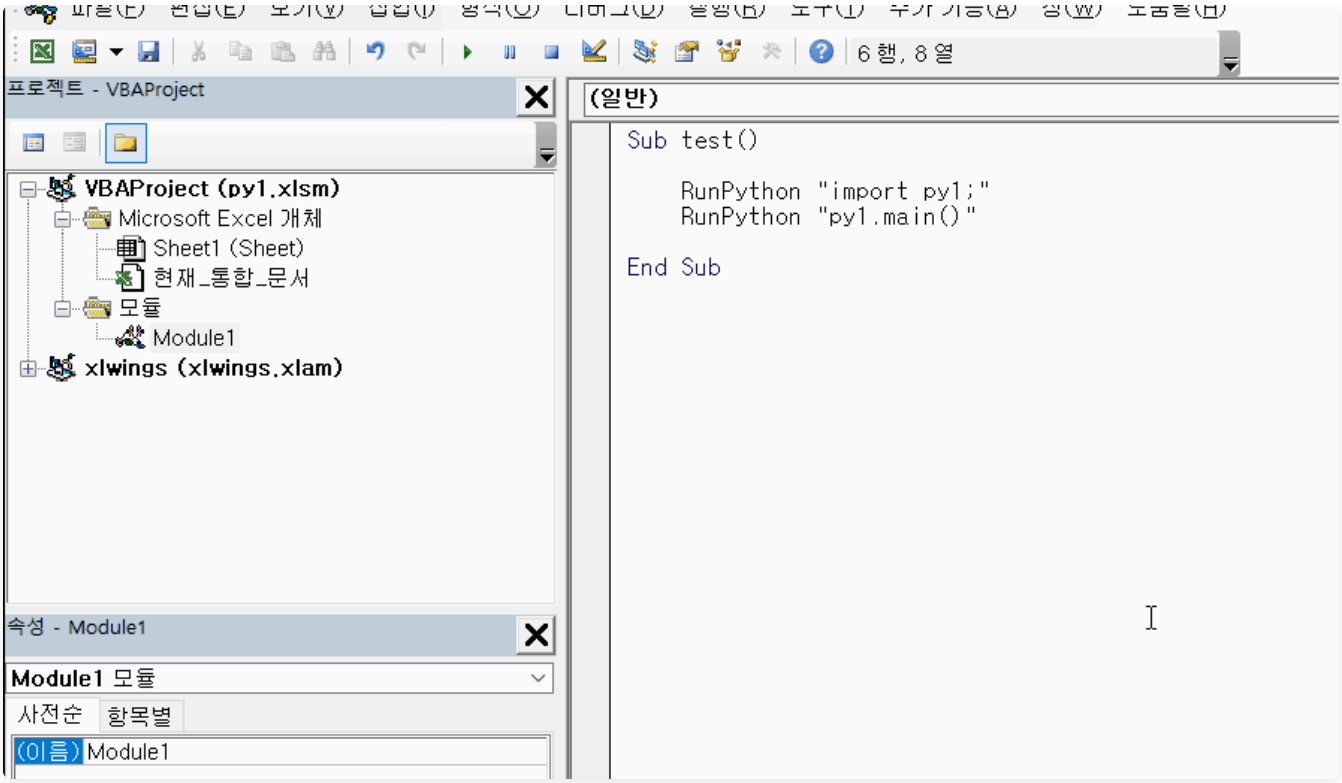
엑셀 VBA에서 파이썬 코드를 호출할 때는, 파이썬 코드 파일이 같은 경로에 있어야 합니다. 즉, 현재 "py1.py" 파일과 "py1.xlsm"파일은 같은 경로에 있는 상태입니다.

그 다음, Alt+F11 을 눌러 VBA 개발 도구 창을 열고 모듈을 생성한 후, 아래 코드를 작성해보도록 합시다. 아래 코드는 엑셀 VBA 코드입니다.

```
Sub test()
    RunPython "import py1;"
    RunPython "py1.main()"
End Sub
```

그리고 실행시켜보겠습니다. xlwings를 통해 파이썬 코드를 실행시킬 땐, 개발 환경 설정시 진행했던 xlwings addin을 활용합니다. 아래 이미지를 참고해봅시다.

<결과>



파이썬 코드를 엑셀 VBA에서 호출할 땐, RunPython이라는 구문을 활용합니다. " " 내부에 들어가는 내용은 파이썬 코드이므로 이 부분만 기억하시면 됩니다.

- 사용법 : Run Python " 파이썬 코드 "

4) 파이썬에서 VBA 코드 실행시키기

이번 파트에서는 파이썬에서 VBA 코드를 실행시키는 방법을 알아보겠습니다.

1. VBA 코드 작성

아래와 같이 "py1.xlsm" 파일에 VBA 코드를 작성해보겠습니다. 1열에 1~10행까지 "xlwings 테스트" 라는 문자열을 출력하는 코드입니다.

<코드>

```
Sub test()
    Dim i As Integer
    Dim sht As Worksheet

    Set sht = Sheets(1)
    With sht
        For i = 1 To 10
            .Cells(i, 1).Value = "xlwings 테스트" & i
        Next i
    End With
End Sub
```

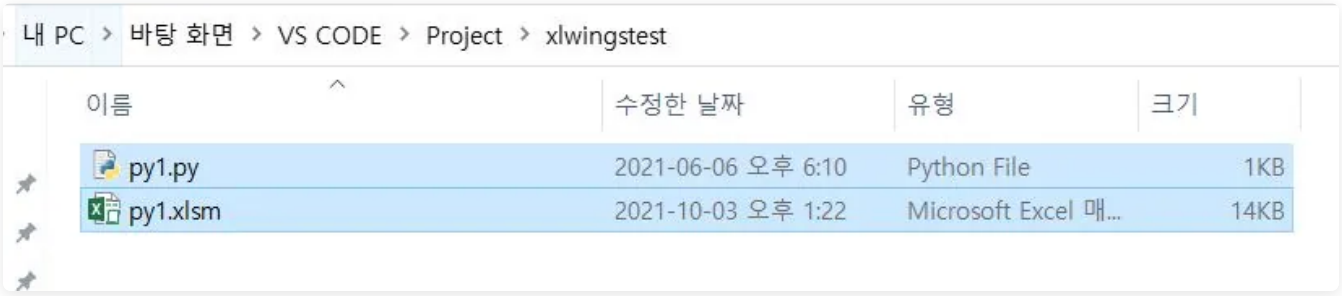
<결과>

	A	B	C
1	xlwings 테스트1		
2	xlwings 테스트2		
3	xlwings 테스트3		
4	xlwings 테스트4		
5	xlwings 테스트5		
6	xlwings 테스트6		
7	xlwings 테스트7		
8	xlwings 테스트8		
9	xlwings 테스트9		
10	xlwings 테스트10		
11			

이제 위 코드를 파이썬 코드에서 호출해보겠습니다. VBA 엑셀 파일의 파일명은 "py1.xlsm"입니다. 위 VBA 프로시저의 명칭이 "test"임을 기억해둡시다.

2. 파이썬에서 VBA 코드 호출하기

파이썬 "py1.py"라는 파일을 생성하였습니다. "py1.py" 파일은 .xlsm과 같은 경로에 있다고 가정하고 코드를 작성해보겠습니다. 아래 이미지는 VBA 매크로 파일과 .py 파일이 같은 경로에 있음을 보여줍니다.



<코드>

```
import  xlwings  as  xw

#엑셀 매크로파일 열기(path는 매크로 파일이 있는 경로)
wb = xw.Book("py1.xlsm")

#엑셀 VBA의 매크로 함수 'test'를 파이썬 함수로 지정
macro_test = wb.macro('test')

#VBA 함수 실행
macro_test()

#함수를 실행한 엑셀파일 따로 저장하기
wb.save("py1_result.xlsm")

#WorkBook 객체 닫기
wb.close
```

파이썬에서 엑셀 VBA를 호출하는 방법은 어렵지 않습니다. Workbook 객체 "wb"를 생성하고 객체의 매크로를 macro_test라는 이름으로 바인딩해줍니다. 그 다음, macro_test( )구문으로 실행시키면 됩니다. 위 코드의 주석을 통해 이해해보시기 바랍니다.

3-3. Win32com (pywin32)

MS office를 제어하기 위한 모듈인 pywin32에 대해 설명하는 파트입니다. pywin32를 공부하면 Excel, Word, Outlook 등을 파이썬으로 제어할 수 있습니다.

1) win32com 소개 및 설치 방법

1. Win32com(pywin32) 소개

Win32com은 Microsoft에서 제공하는 윈도우 프로그램을 파이썬이라는 언어를 매개체로 제어할 수 있도록 만든 API입니다. Win32com을 사용하면 Microsoft Excel, Power point, Word, Outlook 등 프로그램을 제어할 수 있습니다. 이번 파트에서는 Win32com 엑셀을 다루는 기본적인 방법과 Outlook 이메일을 제어하는 방법에 대해 소개하고자 합니다. (Power point, Word 등 프로그램을 다루는 방법은 차후 소개해보도록 하겠습니다.)

앞에서 배웠던 Openpyxl를 공부하신 분이라면 Win32com으로 엑셀을 다루는 것이랑 무슨 차이가 있나 궁금증이 드실 수도 있을 것 같습니다. Openpyxl과 대표적인 차이점이라면, Win32com을 사용했을 때 엑셀 실행 상태에서 제어가 가능하다라는 점을 들 수 있습니다. Openpyxl은 해당 엑셀파일을 실행하면 파이썬 코드에서 접근이 불가능합니다. Win32com 을 사용하여 엑셀을 제어하면 실행상태를 바로 볼 수 있어 편리해집니다.(실시간 확인 가능)

두번째는, 좀더 엑셀 기능적인 측면에서 접근할 수 있습니다. 쉽게 설명하자면 엑셀에서의 자동 줄채우기 기능, 복사 붙여넣기 등 실제 엑셀 프로그램에서 사용하는 기능을 함수 하나로 대체하여 사용할 수 있습니다. 이 부분은 Openpyxl에는 없는 부분으로 차이가 있습니다. 엑셀 VBA를 아시는 분이라면 코드 구조가 VBA랑 매우 비슷함을 느끼실 수 있습니다. 이 부분은 "win32com 통해 엑셀 다루기" 파트에서 자세하게 다뤄보도록 하겠습니다.

추가로, 이번 파트에서 다루는 내용 중 Outlook 이메일 제어는 사내 이메일 파싱 등 이메일 업무를 자동화할 수 있습니다. 이 부분은 "3-1-9) Outlook 이메일 제어'에서 배웁니다.

2. 설치방법

Win32com은 파이썬의 내장모듈이 아니기에 따로 설치가 필요합니다. 본인의 개발환경에 맞게 pywin32이라는 라이브러리를 설치하시면 됩니다.

```
pip install pywin32
```

2021년 12월 16일 기준, pywin32를 설치하면 302 버전이 설치됩니다. 하지만 최신 버전의 안정성 문제 때문인지 아래와 같은 에러가 출력되면서 파이썬 스크립트가 정상적으로 실행되지 않습니다.

ImportError: DLL load failed while importing win32api: 지정된 프로시저를 찾을 수 없습니다.

```
Traceback (most recent call last):
  File "c:/Users/2017363/OneDrive - 주식회사 뷰웍스/바탕 화면/EXCEL_PYTHON/Excel_sum.py", line 3, in <module>
    import win32com.client
  File "C:\Users\2017363\OneDrive - 주식회사 뷰웍스\바탕 화면\EXCEL_PYTHON\newvenv\lib\site-packages\win32com\__init__.py", line 5, in <module>
    import win32api, sys, os
ImportError: DLL load failed while importing win32api: 지정된 프로시저를 찾을 수 없습니다.
```

따라서 해결방법은 버전을 다운그레이드 해주는 것입니다.

```
pip install --upgrade pywin32==225
```

```
Collecting pywin32==225
  Downloading pywin32-225-cp38-cp38-win_amd64.whl (9.1 MB)
     |████████████████████████████████| 9.1 MB 6.8 MB/s
Installing collected packages: pywin32
  Attempting uninstall: pywin32
    Found existing installation: pywin32 302
    Uninstalling pywin32-302:
      Successfully uninstalled pywin32-302
Successfully installed pywin32-225
```

2) win32com 통해 엑셀 다루기 - Cell 데이터 읽기/쓰기

win32com에 속한 client라는 모듈을 활용하여 엑셀을 제어할 수 있습니다. 이번 편에서는 win32com.client를 이용해서 엑셀을 제어하는 기본적인 방법에 대해 알아보겠습니다.

1. import

엑셀을 제어하기 위해선 아래와 같이 win32com.client를 import 시켜야 합니다. win32com은 파이썬 내장 모듈이 아니기에 별도 설치가 완료되어 있어야 합니다.

<코드>

```
import win32com.client
```

2. 엑셀 Application 열기

win32com.client 모듈을 사용하여 엑셀을 제어할 경우 실제 저희가 엑셀을 사용하는 것처럼 엑셀 프로그램을 여는 과정을 코드로 작성해야 합니다.

<코드>

```
excel = win32com.client.Dispatch("Excel.Application") #엑셀 프로그램 실행
```

위와 같이 win32com.client 모듈의 Dispatch 함수를 호출합니다. 괄호 안의 내용은 엑셀 프로그램을 뜻하며, 코드의 전체 의미는 "excel"이라는 변수에 엑셀 프로그램 객체를 설정해주는 것이라고 이해하시면 됩니다.

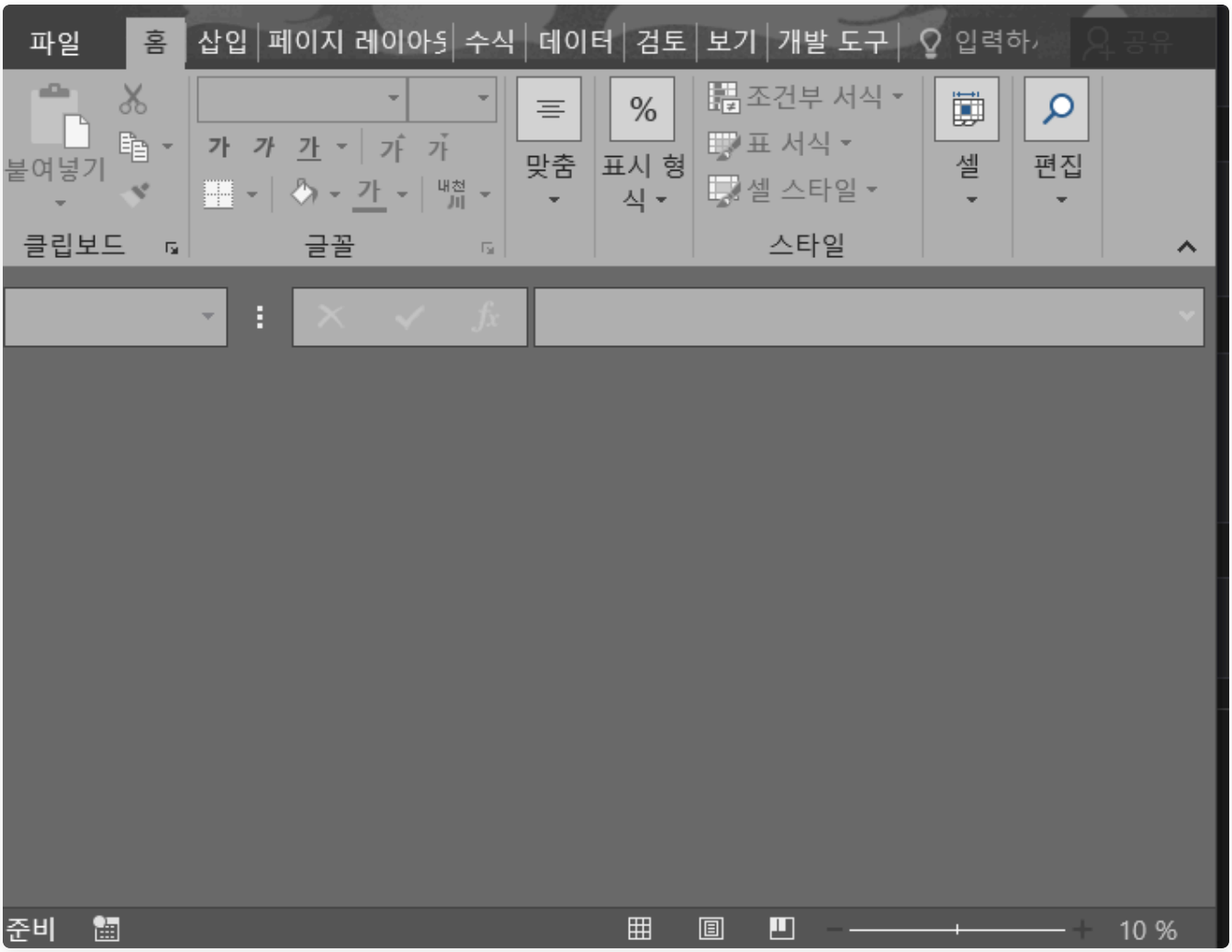
> *위 코드는 엑셀을 다루는 방법이므로 "Excel.Applicaton" 을 괄호 안에 입력하였지만 Microsoft에서 제공하는 Internet Explorer, word, PPT 등 을 바인딩할 수 도있습니다. 예를 들어, 인터넷 익스플로러를 객체로 바인딩하고 싶다면 "InternetExplorer.Application"을 괄호 안에 입력해주시면 됩니다.*

추가로, win32com의 대표적인 특징 중 하나인 실행과정을 실시간으로 확인하려면 아래와 같이 코드를 추가하여 줍니다. 위에서 설정해준 excel이라는 변수의 Visible 속성을 True로 설정해야 합니다.

<코드>

```
excel.Visible = True #앞으로 실행과정을 보이게
```

위 2줄 코드를 실행하면 아래와 같이 빈 엑셀창이 뜬 것을 확인하실 수 있습니다.



3. Workbook 및 Worksheet 객체 생성

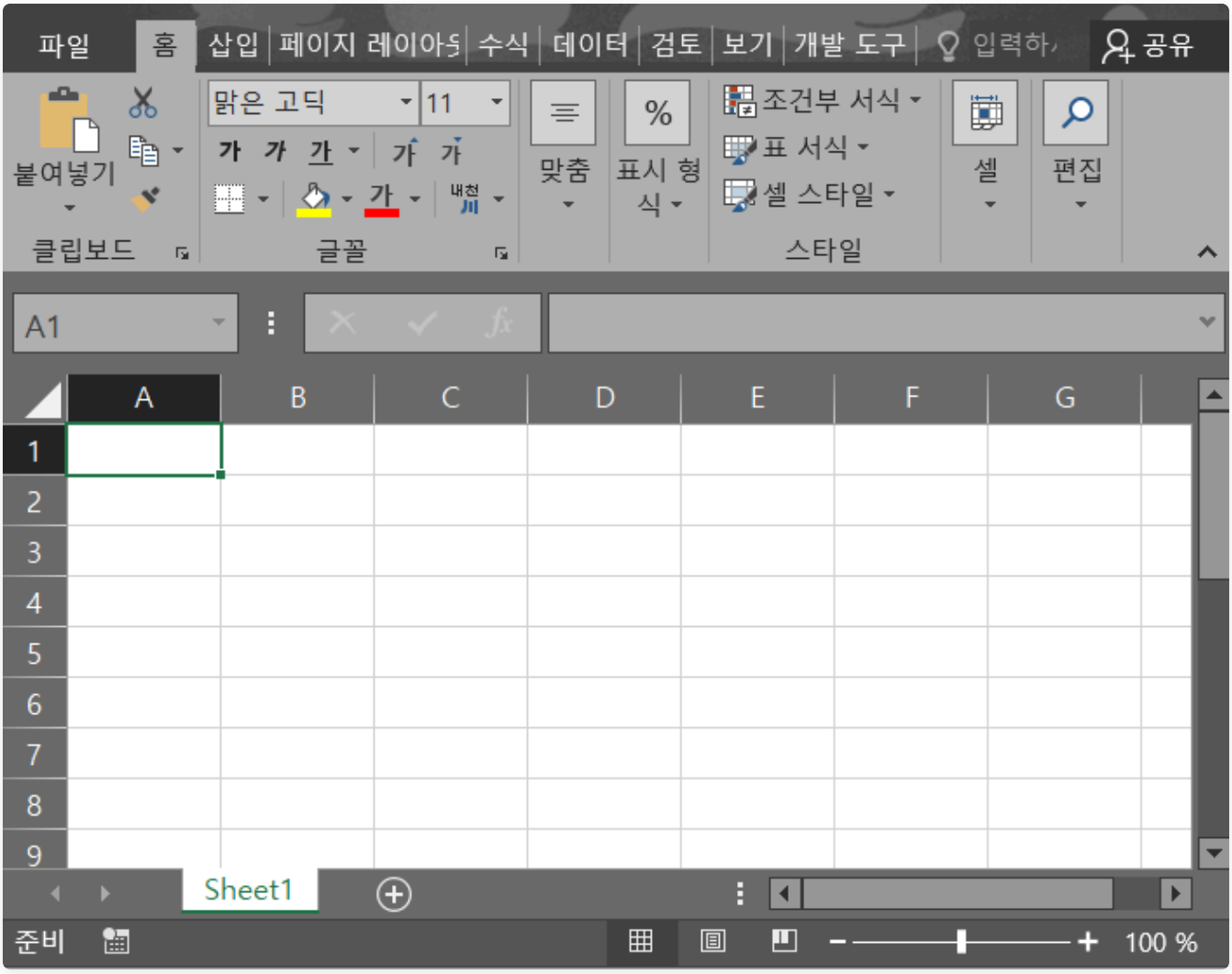
OpenPyxl을 설명하는 챕터에서 엑셀에는 Workbook과 Worksheet의 개념이 있다는 것을 설명했습니다. 마찬가지로, win32com.client 모듈을 사용하여 엑셀을 제어할 경우 Workbook 및 Worksheet 객체를 생성해야 합니다.

Workbook 및 Worksheet의 개념을 모르시는 분들은 "3-1-1) 엑셀의 구성요소"와 "3-1-2) Workbook, Worksheet 객체 생성하기" 파트를 참고하시면 되겠습니다.

<코드>

```
wb = excel.Workbooks.Add() #엑셀 프로그램에 Workbook 추가(객체 설정)
ws = wb.Worksheets("sheet1") #Worksheet 설정
```

OpenPyxl과 문법은 다르나 Workbook과 Worksheet를 객체로 설정한다는 과정은 같습니다. 위 코드를 실행하면 아래와 같이 엑셀 "통합문서"가 만들어지면서 아래와 같이 "Sheet1"이 생성됩니다.



위 방법은 새로운 엑셀 파일을 생성하는 방법입니다. 만약, 원래 있는 엑셀 파일을 Workbook 객체로 생성하고 싶다면 아래와 같이 작성합니다.

<코드>

```
wb = excel.Workbooks.Open(r"C:\Users\Desktop\VS CODE\Project\17. win32com\data.xlsx")
```

Open 함수를 사용하되, 괄호 안의 내용을 엑셀 파일의 전체 경로로 채워주면 됩니다.

4. Cell에 데이터 써보기

특정 Cell에 데이터를 쓰는 방법은 아래와 같습니다. 3가지 방식을 표현해보았습니다. 아래 주석과 코드를 통해 이해해보겠습니다. 참고로 3번째, 4번째 방식은 여러 범위에 한번에 값을 입력하는 방식입니다.

<코드>

```
#셀 row, col 값 지정하여 값넣기(Range("A1")과 동일 함)
ws.cells(1,1).Value = "win32com excel test1"

#ragne로 값 넣기(Cell(1,2)와 동일함)
ws.Range("A2").Value = "win32com excel test2"

#range로 다중범위 지정해서 값 넣기1
ws.Range("A3:C3").Value = "win32com excel test3"

#range로 다중범위 지정해서 값 넣기2(위 코드와 동일 표현)
ws.Range(ws.Cells(3,1), ws.Cells(3,3)).Value = "win32com excel test3"
```

<결과>

	A	B	C	D
1	win32com excel test1			
2	win32com excel test2	win32com excel test2		
3	win32com excel test3	win32com excel test3	win32com excel test3	
4				

만약 엑셀의 자동 채우기 기능을 활용하려면 Autofill( ) 함수를 사용합니다. 사용 방법은 아래와 같습니다.

<코드>

```
ws.Range('A1:A3').AutoFill(ws.Range("A1:A10"))
```

코드에서 첫 번째 Range("A1:A3")는 자동 채우기를 할 기준 데이터 범위입니다. 위 이미지에서 A1:A3 데이터를 지정해서 자동 채우기 할 것입니다.

두 번째 AutoFill( ) 괄호 안의 Range("A1:A10")는 자동 채우기를 어디까지 할 것인가 범위를 지정합니다. 주의해야 할 점은 위에 첫 번째에서 기준이 되었던 A1:A3를 포함 시켜야 합니다. 결과는 아래와 같습니다.

	A	B	C	D	E
1	win32com excel test1				
2	win32com excel test2				
3	win32com excel test3	win32com excel test3	win32com excel test3		
4	win32com excel test4				
5	win32com excel test5				
6	win32com excel test6				
7	win32com excel test7				
8	win32com excel test8				
9	win32com excel test9				
10	win32com excel test10				

5. Cell data 복사/붙여넣기

위 4번의 결과 이미지 중 "A1:A10" 데이터를 복사해서 "B1:B10" 붙여넣기 해보겠습니다. 위에서 생성한 Worksheet 객체를 이용해서 원하는 범위를 복사합니다.(Copy)

그 다음, 복사 시킬 위치를 선택해야 합니다.(Select) 그리고 복사 한 내용을 붙여넣기 합니다.(Paste) 아래 코드는 이 내용을 코드로 구현한 것입니다.

<코드>

```
ws.Range("A1:A10").Copy() # "A1:A10" 데이터 복사하기
ws.Range("B1").Select() # 붙여넣기 할 위치 선택
ws.Paste() #붙여넣기 실행
```

<결과>

	A	B	C	D
1	win32com excel test1	win32com excel test1		
2	win32com excel test2	win32com excel test2		
3	win32com excel test3	win32com excel test3	win32com excel test3	
4	win32com excel test4	win32com excel test4		
5	win32com excel test5	win32com excel test5		
6	win32com excel test6	win32com excel test6		
7	win32com excel test7	win32com excel test7		
8	win32com excel test8	win32com excel test8		
9	win32com excel test9	win32com excel test9		
10	win32com excel test10	win32com excel test10		
11				

6. 파일 저장하기

파일을 저장하는 방법은 Workbook 객체 변수를 통해 Save( ) 함수를 사용하면 됩니다. 이 부분은 OpenPyxl과 비슷합니다.

<코드>

```
wb.Save()
```

wb는 위 3번에서 지정했던 Workbook 객체 변수입니다. 위 코드로 저장할 경우 "내 PC-문서" 위치로 지정되며, 파일명은 "통합문서"라는 이름으로 저장됩니다. 만약 "다른 이름으로 저장하기" 기능을 활용하고 싶다면 SaveAs( ) 함수를 사용합니다

<코드>

```
wb.SaveAs(r"C:\Users\Desktop\VS CODE\Project\17. win32com\data.xlsx")
```

7. 엑셀 Application 닫기

엑셀에서 문서를 다 작성하고 파일까지 저장 완료하였으면, 다음 할 일은 무엇일까요? 당연히 엑셀 프로그램을 닫는 것입니다. 물론 닫지 않아도 되지만, 코드를 여러번 실행할 경우 프로그램 상에서 충돌되거나, 메모리 점유율 측면에서 안좋을 수 있습니다.

반드시 아래 코드를 통해 엑셀 App을 닫도록 합시다. excel은 "2.엑셀 Application 열기"에서 다뤘던 객체 변수입니다. Quit( ) 함수를 사용하면 엑셀 프로그램을 닫을 수 있습니다.

<코드>

```
excel.Quit()
```

3) win32com 통해 엑셀 다루기 - Cell 영역을 선택하는 방법

이번 파트에서는 엑셀에서 셀 영역을 선택하는 방법을 알아보겠습니다. 엑셀에서는 원하는 영역의 데이터를 읽어오거나 쓰는 등 셀 영역을 선택하는 방법이 상당히 중요합니다. win32com.client 모듈을 사용하면 엑셀 영역을 선택하는 여러가지 방법이 있으며 업무자동화에 여러 방면으로 활용할 수 있습니다. 추가로 엑셀 VBA에서도 비슷하게 적용할 수 있는 내용으로 알아두시면 유용합니다.

---

1. Range

2. UsedRange

3. CurrentRegion

4. SpecialCells

내용을 설명하기 전에 import 및 Workbook, Worksheet 객체 설정을 해볼까요? 해당 코드는 이전 편에서도 설명한 동일한 내용이므로 별도의 내용은 생략하겠습니다. 자세한 내용은 아래 주석을 확인해주세요.

<코드>

```
import win32com.client

excel = win32com.client.Dispatch("Excel.Application") #엑셀 프로그램 실행

excel.Visible = True  #앞으로 실행과정을 보이게

wb = excel.Workbooks.Open("test.xlsx") #기존에 생성된 문서를 Workbook 객체로 생성
ws = wb.ActiveSheet #활성화 된 시트 "Sheet1"을 객체로 생성
```

1. Range

win32com.client를 통해 엑셀 셀 영역을 선택할 수 있는 방법 중 가장 대표적인 방법입니다. 이전 OpenPyxl 챕터 중 "3-1-1) 엑셀의 구성요소"에서 공부했던 이미지를 가져와보겠습니다.

아래 이미지의 파일명은 "test.xlsx"이고 위 코드에서 Workbook(wb)및 Worksheet(ws) 객체를 생성했다고 가정하였습니다.

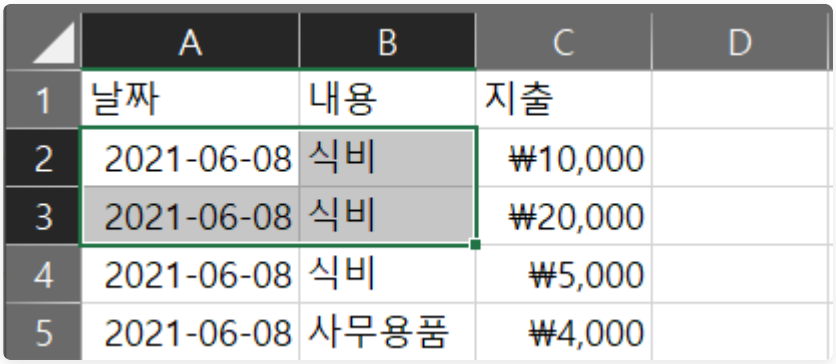
	A	B	C	D
1	날짜	내용	지출	
2	2021-06-08	식비	₩10,000	
3	2021-06-08	식비	₩20,000	
4	2021-06-08	식비	₩5,000	
5	2021-06-08	사무용품	₩4,000	
6	2021-06-08	비품	₩2,000	
7	2021-06-08	유류비	₩1,000	
8	2021-06-08	기타	₩5,000	
9				
10				

위 4가지 예시를 사용하여 셀을 선택하는 방법은 아래와 같습니다. Range(영역) 뒤에 .Select( )를 붙이면 엑셀 파일에서 실제 영역을 선택하게 됩니다.

<코드>

```
ws.Range("A1").Select() # "a1" 영역을 선택

ws.Range("A1,B2").Select() # "A1", "B2" 영역을 각 각 선택

ws.Range("A2:B3").Select() # "A2:B3"의 연속 된 영역 선택
```

위 코드에서 "A2:B3" 영역을 선택하는 코드를 실행하여 엑셀 파일에서 어떤 변화가 일어나는지 확인해봅시다. 코드를 실행하면 아래 이미지와 같이 "A2:B3" 영역이 선택됩니다.



	A	B	C	D
1	날짜	내용	지출	
2	2021-06-08	식비	₩10,000	
3	2021-06-08	식비	₩20,000	
4	2021-06-08	식비	₩5,000	
5	2021-06-08	사무용품	₩4,000	

만약, 특정 영역의 값을 실제로 출력해보고 싶다면 어떻게 해야할까요? 위 코드 에서 Select( )를 지우고 .value를 대신 입력해 보겠습니다.

<코드>

```
ws.Range("A2:B3").value # "A2:B3"의 연속 된 영역 선택
```

<결과>

```
((pywintypes.datetime(2021, 6, 8, 0, 0, tzinfo=TimeZoneInfo('GMT Standard Time', True)), '식비'), (pywintypes.datetime(2021, 6, 8, 0, 0, tzinfo=TimeZoneInfo('GMT Standard Time', True)), '식비'))
```

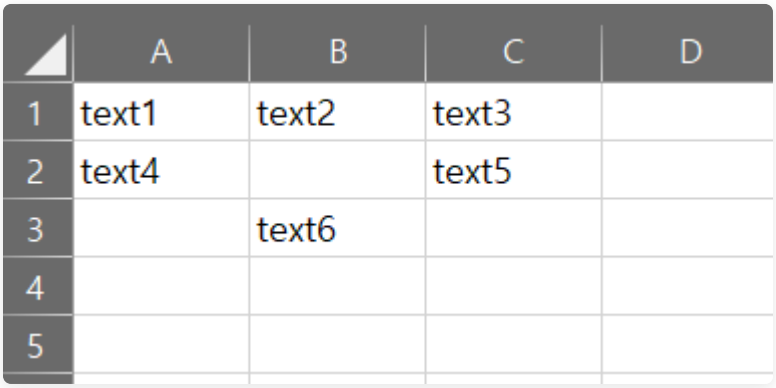
뭔가 복잡해 보이죠? 앞에 괄호가 2개 있는 것을 보시면 2차원 튜플 형태로 "A2:B3" 영역이 출력 된 것을 아실 수 있습니다. 위 결과를 아래 표와 같이 도식화 해보았습니다. 표를 참고하며 이해해봅시다.

값	2021-06-08	식비	2021-06-08	식비
엑셀	A2	A3	B2	B3
Tuple	[0][0]	[0][1]	[1][0]	[1][1]

2. UsedRange

엑셀에서는 1번에서 다뤘던 Range 이외에도 셀 영역을 선택하는 다양한 방법을 제공합니다. 그 중 가장 많이 쓰이는 코드 중 하나가 UsedRange입니다. UsedRange는 WorkSheet 객체의 사용 된 영역을 모두 선택합니다.

이렇게 설명하면 이해가 어려우니 바로 예시로 들어가겠습니다. 엑셀 파일은 아래 이미지를 예시로 사용하겠습니다. 위 1번에서 다뤘던 코드와 동일하게 Workbook 및 WorkSheet 객체를 wb, ws로 설정하였습니다.



	A	B	C	D
1	text1	text2	text3	
2	text4		text5	
3		text6		
4				
5				

먼저 위 영역의 값을 UsedRange를 통해 출력해보겠습니다. 아래 코드와 같이 작성하여 실행해보세요.

<코드>

```
#사용 영역 출력해보기
ws.UsedRange()
```

<결과>

```
(('text1', 'text2', 'text3'), ('text4', None, 'text5'), (None, 'text6', None))
```

결과를 보시면 1번에서 Range로 다뤘던 내용과 마찬가지로 2차원 튜플형태로 출력 되었습니다. 해당 결과를 표로 요약해보면 아래와 같습니다.

값	text1	text2	text3
엑셀 Index	A1	A2	A3
Tuple	[0][1]	[0][1]	[0][2]

값	text4	빈칸	text5
엑셀 Index	B1	B2	B3
Tuple	[1][0]	[1][1]	[1][2]

값	빈칸	text6	빈칸
엑셀 Index	C1	C2	C3
Tuple	[2][0]	[2][1]	[2][2]

각 값들을 2차원 튜플을 통해 출력하거나 접근할 수 있습니다. 예를 들면 아래처럼요.

<코드>

```
#사용 영역 출력해보기
print("A1 값 :", ws.UsedRange()[0][1])
print("B3 값 :", ws.UsedRange()[1][2])
```

<결과>

```
A1 값 : text2
B3 값 : text5
```

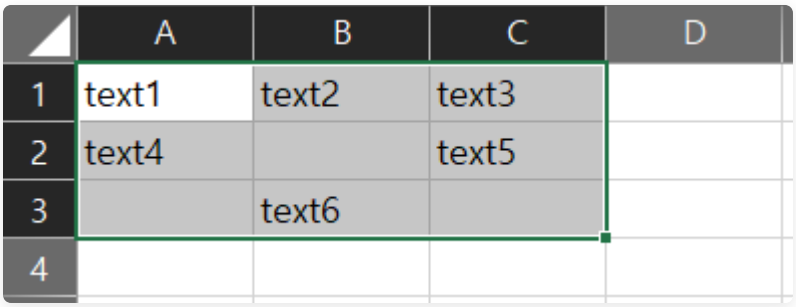
해당 내용은 OpenPyxl의 "3-1-4) rows, columns 속성 이해하기"와 상당히 비슷한 부분이 많습니다. 2차원 튜플이므로 이중 for 문을 통해 모든 요소를 출력할 수 있습니다. 자세한 내용이 필요하신 분은 해당 파트를 참고하세요.

위 내용에 이어서 이번에는 해당 영역을 직접 엑셀에서 선택 처리하는 코드를 작성해보겠습니다. "1. Range"에서도 다뤘지만 UsedRange 뒤에 Select( ) 함수를 사용하면 실제 영역을 선택합니다.

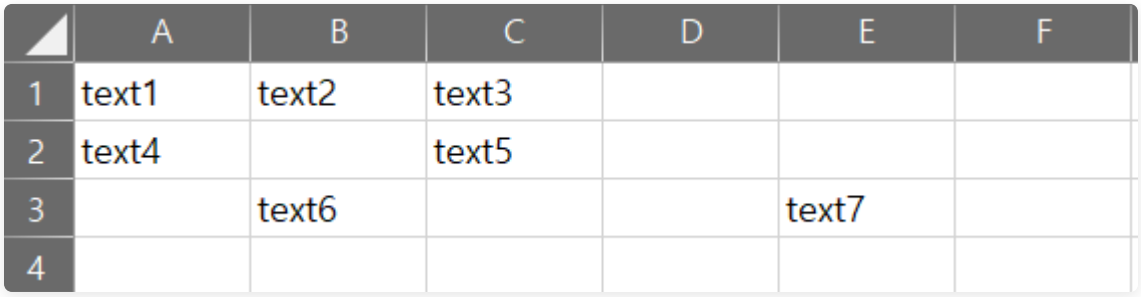
<코드>

```
#사용 영역 선택처리하기
ws.UsedRange.Select()
```

<결과>



이 결과에서 보시다시피 해당 시트의 사용 영역은 모두 선택 처리 됐습니다. UsedRange를 정확히 이해 하기 위해 한가지 예시를 더 들어보겠습니다.

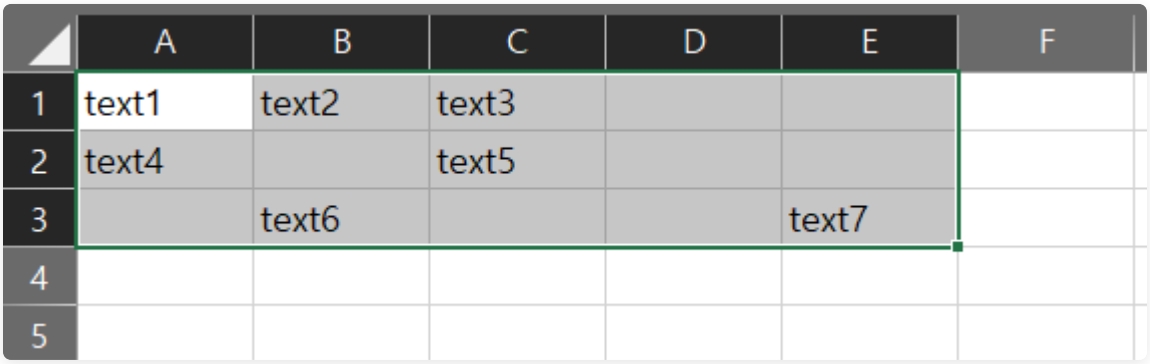


위 이미지에서 동일한 코드를 실행시키면 어떻게 될까요?

<코드>

```
#사용 영역 선택처리하기
ws.UsedRange.Select()
```

<결과>



위 결과에서 보시다시피, text7까지 사용 영역으로 인식하여 연속 된 범위가 모두 선택되었습니다. UsedRange는 위 결과처럼 시트의 연속 된 사용 영역을 모두 선택 처리한다는 점을 이해하시면 됩니다.

참고로, data가 입력되어있는 것 뿐만 아니라 셀 서식이 적용되어있거나 셀의 행/열 높이가 수정 된 경우에도 사용 영역으로 인식합니다.

3. CurrentRegion

세번째 내용은 CurrentRegion에 대한 내용입니다. CurrentRegion은 단어 뜻에 보자면 사실 UsedRange와 무슨 차이가 있나라는 생각이 들 수 있습니다.

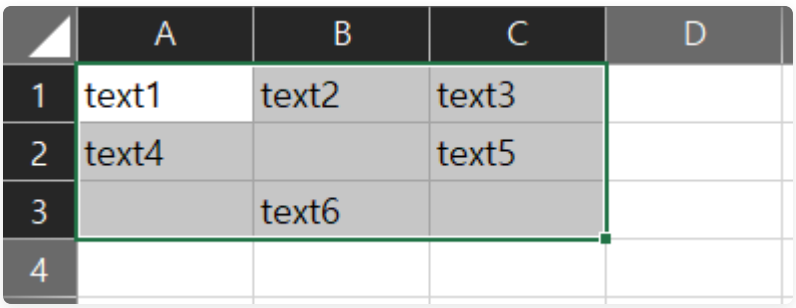
먼저, 사용법에서 차이점이 있습니다. UsedRange는 앞에 대상이 되는 객체가 WorkSheet(=ws)였습니다. CurrentRegion의 대상이 되는 객체는 Range입니다.

예시 코드를 보며 이해해보겠습니다.

<코드>

```
#A:C 열의 사용 영역 선택하기
ws.Range("A:C").CurrentRegion.Select()
```

<결과>



앞에서 설명드렸지만 CurrentRegion은 대상이 되는 객체가 Range입니다. UsedRange에서는 대상이 되는 객체가 ws였던 것 기억하시죠?? 기억이 안나시면 2번 내용을 다시 확인해보세요!

따라서, 아래 와 같은 코드 구성도 가능합니다.

<코드>

```
#시트 사용 영역 선택하기
ws.UsedRange.CurrentRegion.Select()
```

위 코드는 위에서 Range("A:C")라고 표현한 코드와 동일한 결과를 출력합니다. 여기까지 공부해보면 자연스럽게 궁금증이 생기실 수도 있습니다. 결과 만을 놓고 보면 UsedRange와 별 차이가 없기 때문이죠.

차이점을 알기 위해 아래 이미지와 같이 다른 예시를 들어보겠습니다.

	A	B	C	D	E	F
1	text1	text2	text3			
2	text4		text5			
3		text6			text7	
4						

위 UsedRange를 설명했을 때, 사용했던 엑셀 파일 E3에 "text7"이라는 데이터를 추가했습니다. 위 코드를 사용하여 결과가 어떻게 나오는지 확인해보겠습니다.

<코드>

```
#CurrentRegion 사용하기
ws.UsedRange.CurrentRegion.Select()
```

<결과>

	A	B	C	D	E
1	text1	text2	text3		
2	text4		text5		
3		text6			text7
4					
5					

"2. UsedRange" 에서 배웠던 내용을 생각해봅시다. ws.UsedRange.Select()를 사용하면 "text7"의 영역까지 전부 선택처리 되었습니다. 위 코드를 확인하시면 CurrentRegion을 사용하니 "text7"은 선택되지 않았네요.

CurrentRegion은 사용 영역의 첫 번째 영역을 기준으로 연속 된 영역 만을 선택합니다. 즉, "text1"을 기준으로 연속 된 영역(사각형)을 선택하는 것이죠.

이해를 위해 한 가지 더 예시를 들어보겠습니다. 이번에는 위 이미지의 D3에 "text8"을 입력하고 똑같은 코드를 실행시켜보겠습니다.

<결과>

	A	B	C	D	E	F
1	text1	text2	text3			
2	text4		text5			
3		text6		text8	text7	
4						
5						

위 결과와 같이 사용 영역이 전부 선택되었습니다. "text8"이 입력되면서 "text7"까지 연속 된 영역으로 설정됐기 때문입니다. 따라서, 아래처럼 코드를 작성하셔도 같은 결과가 나옵니다. "A1"을 기준으로 연속 된 영역을 선택한다는 코드입니다.

<코드>

```
ws.Range("A1").CurrentRegion.Select()
```

궁금하신 부분은 한번 실행시켜보시기 바랍니다. 따라서, CurrentRegion을 사용해야 할 경우 선택되는 영역이 어딘지 반드시 확인하고 코드를 작성하시기 바랍니다.

4. SpecialCells

마지막으로 배울 셀 선택 방법은 SpecialCells 이라는 함수를 사용하는 것입니다. SpecialCells는 특정 조건을 만족하는 셀을 선택 할 수 있습니다. 예를 들면, 데이터만 있는 셀만 선택하기, 빈 공간 선택하기가 있습니다.

CurrentRegion( )과 마찬가지로 대상이 되는 객체는 Range입니다. 몇 가지 예시를 통해 알아보겠습니다.

	A	B	C	D	E
1	text1	text2	text3		
2	text4		text5		
3		text6		text8	text7
4					

예시는 위 엑셀 시트를 사용하겠습니다. 지금까지 설명해왔던 것과 마찬가지로 WorkSheet의 객체명은 ws임을 가정하고 아래 코드를 작성하겠습니다.

<사용 예시1 : 사용 영역(UsedRange) 중 연속 된 영역 모두 선택>

```
#연속 된 사용영역 선택
ws.UsedRange.SpecialCells(12).Select()
```

<결과>

	A	B	C	D	E	F
1	text1	text2	text3			
2	text4		text5			
3		text6		text8	text7	
4						

사실상 위 결과는 ws.UsedRange.Select( ) 동일합니다. SpecialCells(12)의 숫자 12는 옵션을 뜻합니다. 12는 데이터가 있는 연속 된 영역을 모두 선택하겠다는 옵션 값입니다. 괄호 안의 여러 가지 숫자에 따라 영역 선택 조건을 변경할 수 있습니다.

참고로, 앞 과정과 같이 데이터를 2차원 튜플로 읽어오시려면 Select( ) 대신 Value( ) 를 사용하시면 됩니다.

<사용 예시2 : 사용 영역(UsedRange) 중 데이터가 없는 빈 셀만 선택>

```
#연속 된 사용역역에서 빈 셀만 선택하기
ws.UsedRange.SpecialCells(4).Select()
```

<결과>

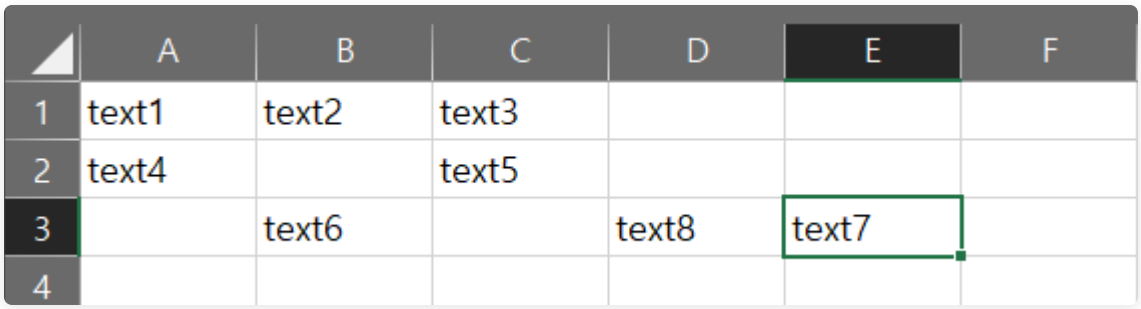
	A	B	C	D	E	F
1	text1	text2	text3			
2	text4		text5			
3		text6		text8	text7	
4						
5						

위 시트에서 ws.UsedRange.Select( ) 코드를 실행하면 "A1:E3"의 연속 된 영역을 모두 선택합니다. 추가로, SpecialCells(4)를 사용하면 위 이미지와 같이 사용 영역(UsedRange) 에서 빈 셀만 선택합니다.

<사용 예시3 : 사용 영역(UsedRange) 중 마지막 셀 선택>

```
#연속 된 사용역역에서 마지막 셀 선택
ws.UsedRange.SpecialCells(11).Select()
```

<결과>



	A	B	C	D	E	F
1	text1	text2	text3			
2	text4		text5			
3		text6		text8	text7	
4						

SpecialCells(11)을 사용하면 이미지와 같이 사용 영역(UsedRange) 중 마지막 셀을 선택하게 됩니다. 즉, "text7"을 선택합니다.

SpecialCells( )의 옵션 값은 Microsoft에서 제공하는 공식 Document를 확하시면 됩니다. 아래 이미지는 그 내용입니다. 엑셀 VBA를 아시는 분이라면 지금까지 배웠던 내용이 엑셀 VBA와 동일한 내용임을 아실 수 있습니다. win32com은 엑셀 VBA와 마찬가지로 Microsoft에서 개발한 라이브러리이기 때문입니다.

궁금하신 분은 아래 표의 Value를 참고하여 테스트해보세요!

Name	Value	Description
xlCellTypeAllFormatConditions	-4172	Cells of any format.
xlCellTypeAllValidation	-4174	Cells having validation criteria.
xlCellTypeBlanks	4	Empty cells.
xlCellTypeComments	-4144	Cells containing notes.
xlCellTypeConstants	2	Cells containing constants.
xlCellTypeFormulas	-4123	Cells containing formulas.
xlCellTypeLastCell	11	The last cell in the used range.
xlCellTypeSameFormatConditions	-4173	Cells having the same format.
xlCellTypeSameValidation	-4175	Cells having the same validation criteria.
xlCellTypeVisible	12	All visible cells.

4) win32com 통해 엑셀 다루기 - Sheet 제어

이번 파트는 win32com.cleint 모듈을 사용하여 엑셀 시트를 다루는법을 알아보겠습니다. 서론에서 win32com.client가 엑셀의 기능적인 측면에서 OpenPyxl보다 더 편리하다라고 언급한 적이 있습니다.

예를 들어, win32com.client로 엑셀 시트를 제어하면 여러 엑셀 파일간 시트 이동/복사가 가능해집니다. (OpenPyxl에서는 불가능한 작업입니다.)

아래 5가지 부분으로 나누어 설명해보도록 하겠습니다.

1 시트 생성 및 설정하기

2 시트 복사/이동하기

3. 시트 삭제하기

4. 시트 이름 변경하기

5. 시트 탭 속성 설정하기

1. 시트 생성 및 설정

"2) win32com 통해 엑셀 다루기 - Cell 데이터 읽기/쓰기" 파트에서도 다뤘지만 win32com.client를 통해 엑셀을 다룰 때는 아래 과정을 필수로 거쳐야 합니다.

win32com.client 모듈을 임포트하고 엑셀 프로그램을 변수(excel)로 바인딩하는 과정입니다. 변수는 꼭 "excel"로 지정하실 필요는 없습니다. 여러분이 편하게 사용할 수 있는 변수로 지정하시면 됩니다.

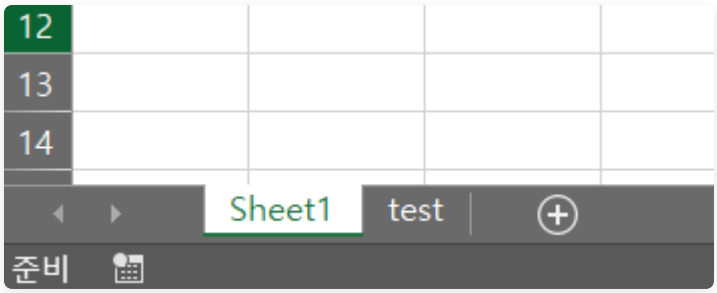
```
#win32com.client 모듈 임포트
import win32com.client

#Excel 프로그램 객체 생성
excel=win32com.client.Dispatch("Excel.Application")

#엑셀 실행과정이 보이게 설정
excel.Visible = True
```

앞으로의 과정은 위 코드가 앞에 작성되어 있다는 가정하에 설명하도록 하겠습니다.

먼저 시트를 생성하는 코드는 아래와 같습니다. 이전 편에서 설명드렸듯이 Workbook 객체를 변수로 설정하고 그 변수를 이용하여 WorkSheet 객체를 생성합니다.

```
#기존에 있는 "data1.xlsx"파일을 Workbook 객체로 설정
wb1 = excel.Workbooks.Open(r"C:\Users\Desktop\VS CODE\Project\17. win32com\data1.xlsx")

#data1.xlsx 파일에 새로운 Worksheet 생성 및 ws1으로 객체 설정
ws1 = wb1.Worksheets.Add()
```

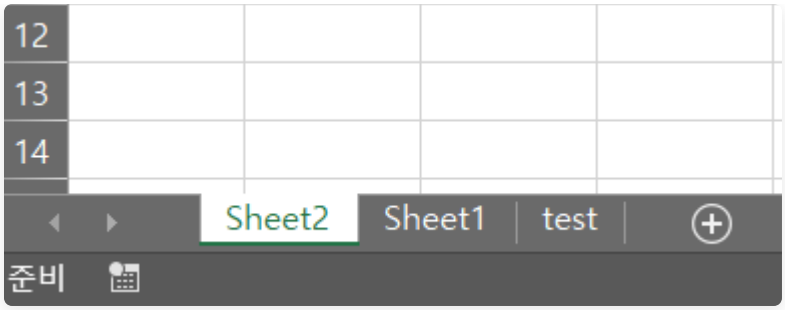
위 과정은 "data1.xlsx"파일에 새로운 "Sheet1"이라는 시트를 생성하여 ws1이라는 변수로 설정합니다.(아래 이미지)



이해를 위해 한번더 똑같은 코드를 실행해보겠습니다. 이번에는 변수 ws2로 설정하였습니다.

```
#data1.xlsx 파일에 새로운 Worksheet 생성 및 ws2으로 객체 설정
ws2 = wb1.Worksheets.Add()
```

이번에는 "Sheet2"가 새로 생성되었죠? WorkSheets.Add( )라는 메서드는 워크시트를 "Sheet"+번호 형식으로 시트를 새로 생성합니다.



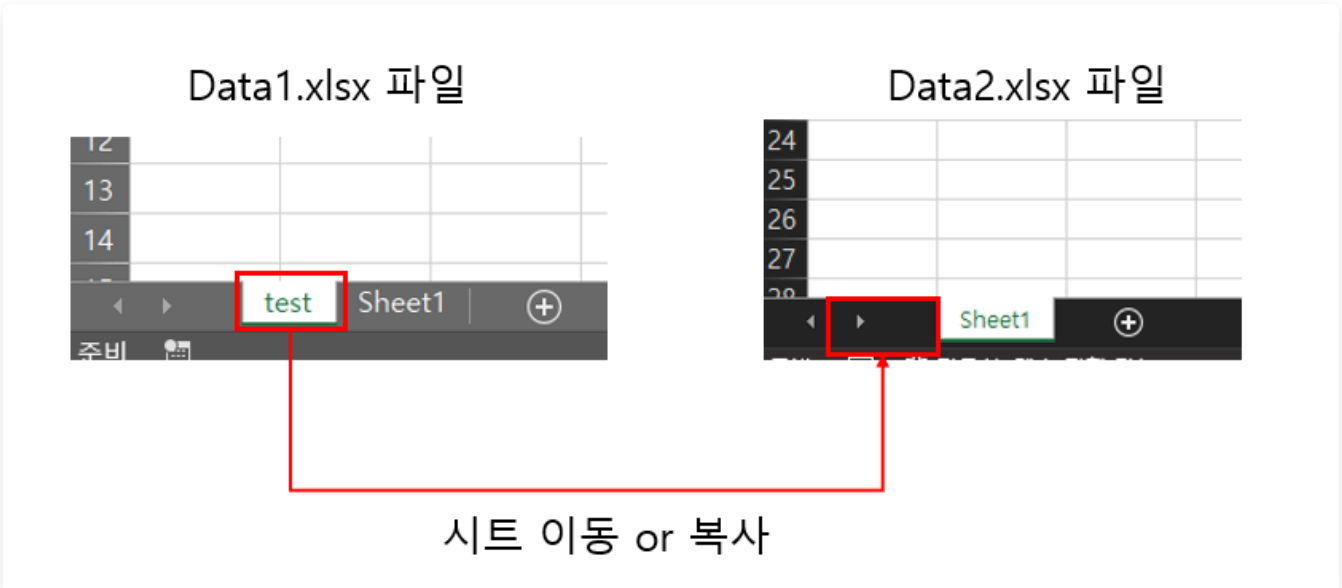
옆에 있는 "test" 시트는 기존 "data1.xlsx"에 만들어져 있던 시트입니다. 만약, 기존에 만들어져 있던 시트에 접근하려면 아래와 같이 코드를 작성합니다.

```
#현재 활성화되어 있는 시트를 객체로 설정(위 이미지 기준 Sheet2)
ws3 = wb1.ActiveSheet

#특정 시트명으로 접근(Sheet1으로 접근)
ws4 = wb1.Worksheets("sheet1")
```

시트를 변수로 설정하는 방법을 알고 있으면 사용자가 원하는 시트에 접근하여 데이터를 읽거나 쓸 수 있습니다.

2. 시트 복사 및 이동

서론에서도 설명하였지만 win32com.client 모듈을 통해 엑셀을 다룰 때 가장 큰 장점은 엑셀의 본래 기능적인 측면에서 프로그램을 제어할 수 있는 것입니다. 따라서, 엑셀에서 제공하는 시트 이동/복사 기능을 제공합니다. 같은 파일 뿐만 아니라 다른 파일 간 시트 이동/복사도 가능합니다.



이번에 저희가 학습해볼 내용은 파일간 시트/이동복사 기능을 구현하는 것입니다. (위 그림)

위 그림과 같이 "data1.xlsx" 파일의 "test" 시트를 "data2.xlsx" 파일에 복사하는 코드를 작성해보겠습니다.

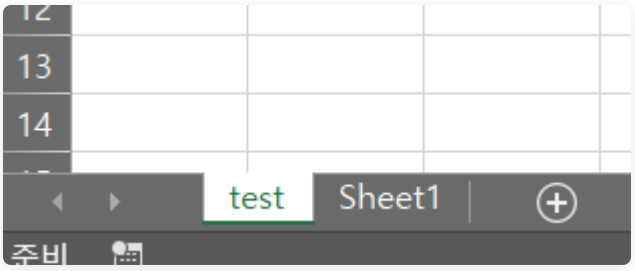
먼저 테스트를 위해 2개 파일을 각각 Workbook 객체로 설정해봅시다.

```
#"data1.xlsx" 파일을 wb1로 설정
wb1 = excel.Workbooks.Open(r"C:\Users\Desktop\VS CODE\Project\17. win32com\data1.xlsx")

#"data2.xlsx" 파일을 wb2로 설정
wb2 = excel.Workbooks.Open(r"C:\Users\Desktop\VS CODE\Project\17. win32com\data2.xlsx")
```

위 이미지 상으로 설명하면 wb1 객체에 속해있는 "test" 시트를 wb2 객체로 복사하면 됩니다.

```
#wb1의 "test" 시트를 wb2의 "Sheet1" 앞으로 복사한다.
wb1.Worksheets("test").Copy(Before=wb2.Worksheets("Sheet1"))
```

위 코드대로 구현하면 아래와 같이 "data2.xlsx" 파일에 "test" 시트가 복사되게 됩니다.



위에서 코드를 보면 궁금하신 부분이 있을겁니다. Copy( ) 함수 내부에 있는 Before라는 옵션인데요. Before 설정값은 시트를 어떤 위치에 복사할지를 설정할 수 있습니다. 위 코드대로 하면 wb2의 "Sheet1" 객체 앞(before)로 복사한다고 이해하시면 됩니다.

만약, 위치를 "Sheet1" 객체 뒤(After)로 복사하고 싶으면 아래와 같이 코드를 작성합니다.

```
#wb1의 "test" 시트를 wb2의 "Sheet1" 뒤로 복사한다.
wb1.Worksheets("test").Copy(After=wb2.Worksheets("Sheet1"))
```

시트 이동은 Copy( )라는 메서드에서 Move( )라는 함수로 변경만 해주시면 됩니다. 프로그램 실행과정에서는 시트가 이동이 됐는데 엑셀파일을 다시 실행시켰을 때 반영이 안되어있다면, 코드 중에 엑셀 파일을 저장하는 부분이 없는 것이니 Save 코드를 작성해주시면 됩니다.(전편 참고)

참고로, win32com.client의 서론에서도 설명한 부분이지만 같은 Microsoft에서 개발 된 언어인 엑셀 VBA와 매우 유사한 형태를 띄고 있습니다. 자료를 참고하고 싶다면 Microsoft의 VBA Documentaion을 확인하시면 됩니다. 아래 내용은 Copy( ) 메서드의 옵션값(Before, After)를 설명하는 내용입니다.

Parameters

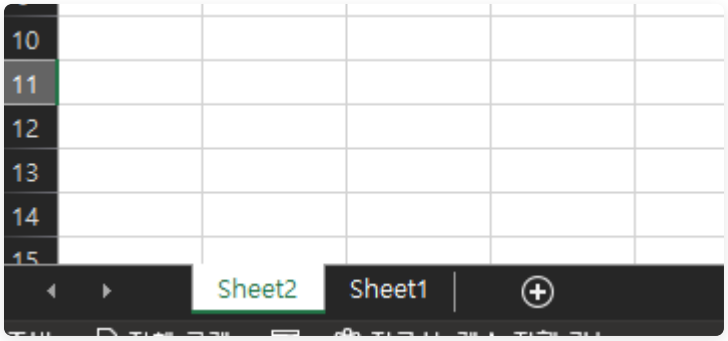
Name	Required/Optional	Data type	Description
Before	Optional	**Variant**	The sheet before which the moved sheet will be placed. You cannot specify *Before* if you specify *After*.
After	Optional	**Variant**	The sheet after which the moved sheet will be placed. You cannot specify *After* if you specify *Before*.

3. 시트삭제

이번에는 시트를 생성해보고 그 시트를 다시 삭제하는 코드를 작성해보겠습니다.

먼저 WorokBook 및 WorkSheet 객체를 새로 생성해보겠습니다.

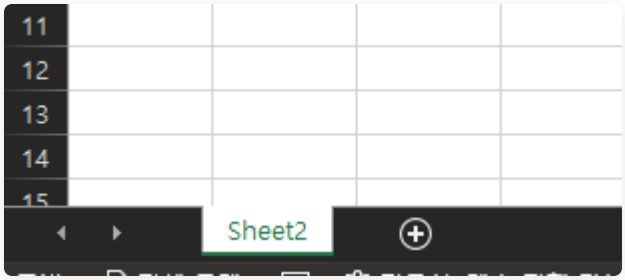
```
wb = excel.Workbooks.Add() #엑셀 Workbook 파일 1개 새로 생성

ws = wb.Worksheets.Add() #엑셀 시트를 새로 생성
```



위 결과 이미지를 보시면 Sheet1, Sheet2가 생성되어 있습니다. 엑셀은 기본적으로 WorkBook 1개를 생성했을 때, "Sheet1"을 가지게 됩니다. 따라서, Worksheets.Add( )를 하면 "Sheet2"가 새로 생성되는 점 참고해주세요.

위 이미지에서 "Sheet1"을 삭제하려고 합니다.

```
#"Sheet1"을 ws로 설정
ws = wb.Worksheets("Sheet1")
#ws 삭제
ws.Delete()
```

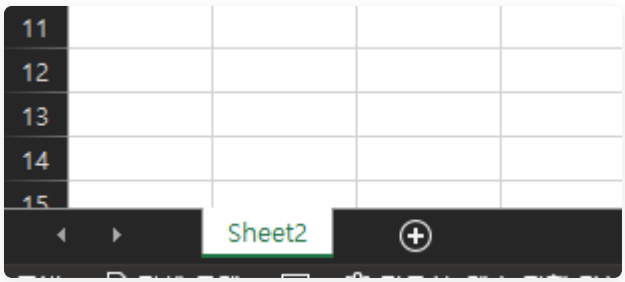


위 코드는 풀어쓰면 아래와 코드와 동일한 코드입니다.

```
#"Sheet1" 삭제하기
wb.Worksheets("Sheet1").Delete()
```

4. 시트 속성 설정(시트명, 시트탭 색)

시트명을 변경하려면 아래와 같이 코드를 작성합니다.




```
#"Sheet2"을 ws로 설정
ws = wb.Worksheets("Sheet2")
#ws 시트명 바꾸기
ws.Name = "시트 이름 바꾸기"
```

위 "시트 이름 바꾸기" 시트탭의 색상을 변경하려면 아래와 같이 코드를작성합니다.

```
ws.Tab.ColorIndex = 30
```

"Worksheet 객체".Tab.ColorIndex = "번호" 와 같은 형태로 작성하면 시트탭 색상을 변경할 수 있습니다.

Microsoft에서 제공하는 엑셀 ColorIndex는 1~56이며, 각 번호가 뜻하는 색상은 구글링을 하면 쉽게 찾아볼 수 있으니 참고하시면 됩니다. (Excel Color Index 검색)

5) win32com 통해 엑셀 다루기 - 엑셀 서식 지정

이번 파트에서는 win32com.client를 통해 엑셀 셀서식을 설정하는 방법을 알아보겠습니다. 셀서식 파트의 내용은 아래와 같은 순서도 다뤄보겠습니다.

---

1. Font Style

2. Font Color(글 색상)

3. Cell 서식 : 색상, 테두리, 정렬

4. Font, Cell 서식 지우기

앞 파트와 마찬가지로 아래 코드가 이미 작성되어 있다는 가정하에 아래 내용을 진행하도록 하겠습니다. 이전에도 많이 설명한 부분이지만, 새로운 Workbook 및 Worksheet 객체를 생성하는 과정입니다.

```
import win32com.client

excel = win32com.client.Dispatch("Excel.Application") #엑셀 프로그램 실행
excel.Visible = True #앞으로 실행과정을 보이게

wb = excel.Workbooks.Add() #엑셀 Workbook 파일 1개 새로 생성
ws = wb.Worksheets.Add() #엑셀 시트를 새로 생성
```

1. Font Style

Font Style은 글꼴, 글씨의 크기 등 말 그대로 글씨의 형태를 지정해주는 것을 말합니다. win32com.client를 사용해서 어떻게 Font Style을 수정할 수 있는지 알아보겠습니다.

먼저 아래와 같이 임의의 텍스트를 엑셀 셀에 입력해보았습니다.

<코드>

```
#다중범위 써보기1
ws.Range(ws.cells(1,1), ws.cells(1,3)).value = "win32com excel 서식(font style)"

#다중범위 써보기2
ws.Range("A2:B2").value = "win32com excel 서식(font 색상)"

#다중범위 써보기3
ws.Range("A3").value = "win32com excel 서식(Cell 색상)"
```

<결과>

	A	B	C
1	win32com excel 서식(font style)	win32com excel 서식(font style)	win32com excel 서식(font style)
2	win32com excel 서식(font 색상)	win32com excel 서식(font 색상)	
3	win32com excel 서식(Cell 색상)		
4			

위 이미지의 각 텍스트에 Font Style 설정 코드를 작성해보겠습니다.

<코드>

```
#셀서식1. font size 및 font 굵게, 글씨체

#"A1" 글씨체 변경
ws.cells(1,1).Font.name = "굴림"

#"B2"글씨 사이즈 변경
ws.cells(1,2).Font.Size = 14

#"C3"글씨 스타일 변경(Bold, Italic, Underline)
ws.cells(1,3).Font.Bold = True #굵게
ws.cells(1,3).Font.italic = True #글씨 기울기
ws.cells(1,3).Font.Underline = True #밑줄
```

<결과>

	A	B	C
1	win32com excel 서식(font style)	win32com excel 서식(font style)	***win32com excel 서식(font style)***
2	win32com excel 서식(font 색상)	win32com excel 서식(font 색상)	
3	win32com excel 서식(Cell 색상)		
4			

위 코드에서는 cells(row, column) 을 사용했지만 Range( )로 범위를 설정해도 무관합니다. 셀 범위 선택 방법에 대해서는 "3-3-3) Cell 영역 선택하는 방법" 편을 참고해주세요.

참고로 "C3"의 글씨 스타일을 변경하는 코드(Bold, Italic, Underline)은 True로 설정하면 적용되며, False로 할 경우 적용이 취소됩니다. 코드를 작성하지 않은 경우 기본이 False 상태라고 생각하시면 됩니다

2. Font Color

이번에는 글씨의 색상을 지정하는 코드를 작성해보겠습니다.

<코드>

```
#셀서식2. font 색상 지정
ws.Range("A2").font.ColorIndex = 40
ws.Range("B2").font.ColorIndex = 50
```

<결과>

	A	B	C
1	win32com excel 서식(font style)	win32com excel 서식(font style)	***win32com excel 서식(font style)***
2	win32com excel 서식(font 색상)	win32com excel 서식(font 색상)	
3	win32com excel 서식(Cell 색상)		
4			

글씨 색상을 지정하는 방법은 Font.ColorIndex 라는 속성을 사용하는 것입니다. 위 코드에서 보시면 숫자 값(40, 50)을 설정한 것을 확인할 수 있습니다.

Microsoft에서는 엑셀의 Color를 설정하는 고유의 숫자를 설정해두었습니다. 이를 컬러 인덱스(ColorIndex)라고 하며, 1~56가지의 번호로 56가지 색상을 설정할 수 있습니다. 인덱스별 색상은 아래 이미지를 참고해봅시다.

Color 1		Color 11		Color 21		Color 31		Color 41		Color 51	
Color 2		Color 12		Color 22		Color 32		Color 42		Color 52	
Color 3		Color 13		Color 23		Color 33		Color 43		Color 53	
Color 4		Color 14		Color 24		Color 34		Color 44		Color 54	
Color 5		Color 15		Color 25		Color 35		Color 45		Color 55	
Color 6		Color 16		Color 26		Color 36		Color 46		Color 56	
Color 7		Color 17		Color 27		Color 37		Color 47			
Color 8		Color 18		Color 28		Color 38		Color 48			
Color 9		Color 19		Color 29		Color 39		Color 49			
Color 10		Color 20		Color 30		Color 40		Color 50			

사장님 몰래 하는 파이썬 업무 자동화

3. Cell 서식 : 색상, 테두리, 정렬

1) Cell 색상

셀 서식-셀 색상은 셀의 배경색을 지정하는 방법입니다. 위에서 다뤘던 ColorIndex를 그대로 활용하며 .Interior.ColorInddex 라는 속성을 활용합니다. 사용법은 아래 코드를 통해 이해해보겠습니다.

<코드>

```
#"A3" 셀 배경색 지정
ws.Range("A3").Interior.ColorIndex = 30
```

<결과>

	A	B	C
1	win32com excel 서식(font style)	win32com excel 서식(font style)	***win32com excel 서식(font style)***
2	win32com excel 서식(font 색상)	win32com excel 서식(font 색상)	
3	win32com excel 서식(Cell 색상)		
4			

2) Cell 테두리 지정

이번에는 셀의 테두리를 지정하는 방법에 대해 알아보겠습니다. 먼저 정해진 영역의 외곽 테두리만 지정하는 코드를 작성해보죠.

<코드>

```
#Weight : 선 굵기, LineStyle의 경우 1:실선, 2:짧은점선, 3:긴점선
ws.Range("A1:C3").BorderAround(ColorIndex = 1,Weight = 2,LineStyle = 1)
```

<결과>

	A	B	C
1	win32com excel 서식(font style)	win32com excel 서식(font style)	***win32com excel 서식(font style)***
2	win32com excel 서식(font 색상)	win32com excel 서식(font 색상)	
3	win32com excel 서식(Cell 색상)		
4			

외곽 테두리만 지정하는 함수는 BorderAround( )라는 함수입니다. 괄호 안의 옵션 값은 테두리 색을 지정하는 ColorIndex, 두께를 지정하는 Weight, 선의 스타일을 설정하는 LineStyle입니다. 자세한 내용은 위 주석을 확인해주세요.

참고로, 이번 코드에는 Range("A1:C3")라고 표기하였지만 사용 영역에 다라 UsedRange, CurrentRegion 등 셀 영역을 설정하는 개체들을 사용할 수 있습니다. 관련 내용은 "3-3-3) 셀 영역 선택하기"에서 배웠던 것 기억나시죠? 혹시 내용이 궁금하신 분들은 해당 파트를 참고하시면 됩니다.

만약, 외곽 테두리가 아닌 선택 영역 안쪽까지 모두 테두리 지정하려면 아래 코드를 사용하시면 됩니다.

<코드>

```
#셀 안쪽까지 모두 테두리 지정
rng = ws.Range("A1:C3") #범위 설정
rng.Borders.LineStyle = 1 #선 스타일
rng.Borders.ColorIndex = 14 #선 색상
rng.Borders.Weight = 2 #선 굵기
```

<결과>

	A	B	C	D
1	win32com excel 서식(font style)	win32com excel 서식(font style)	***win32com excel 서식(font style)***	
2	win32com excel 서식(font 색상)	win32com excel 서식(font 색상)		
3	win32com excel 서식(Cell 색상)			
4				

rng라는 변수에 선택 영역을 설정하여 Borders 라는 객체의 속성값(LineStyle, ColorIndex, Weight)을 ㅅ함으로써 테두리를 지정할 수 있습니다.

3) Cell 정렬

아래와 같이 임의의 양식을 작성해보았습니다. 이번 내용의 목표는 각 텍스트에 맞게 정렬 코드를 구현해보는 겁니다.

	A	B	C	D
1		위로 정렬		
2	왼쪽정렬	가운데정렬	우측으로 정렬	
3		아래로 정렬		
4				

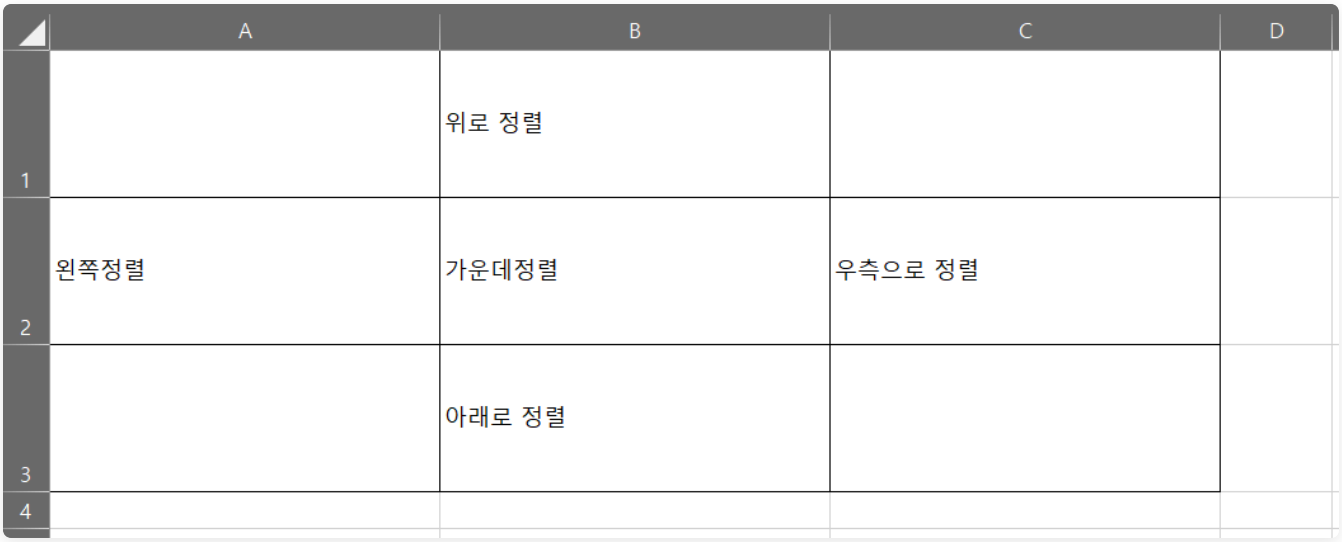
셀 정렬 코드를 배우기 전에 행,열크기를 먼저 조정해보도록 하겠습니다. 행,열 크기를 조정하는 이유는 정렬 된 내용이 더 잘보이게 하기 위함입니다. 선택 범위의 rowHeight와 ColumnWidth 속성을 이용하여 셀의 행,열 크기를 조절할 수 있습니다.

추가로 위에서 배운 테두리 코드도 적용해볼까요?

<코드>

```
#행, 열크기 조정
rng = ws.UsedRange # 사용 영역 선택
rng.rowHeight = 70  # 선택 영역 행 크기 설정
rng.ColumnWidth = 30  # 선택 영역 열 크기 설정

#테두리 설정
rng.Borders.LineStyle = 1 #선 스타일
rng.Borders.ColorIndex = 1 #선 색상 : Black
rng.Borders.Weight = 2 #선 굵기
```

<결과>



	A	B	C	D
1		위로 정렬		
2	왼쪽정렬	가운데정렬	우측으로 정렬	
3		아래로 정렬		
4				

양식 설정은 끝났습니다. 이번에는 셀 정렬 코드를 배워보겠습니다. 셀 정렬 코드는 2가지 속성을 통해 설정할 수 있습니다. HorizotalAlignment, VerticalAlignment입니다. 영어 뜻대로 HorozontalAlignment는 수평 방향으로의 정렬, VerticalAlignment는 수직 방향으로의 정렬을 뜻합니다.

코드를 통해 배워볼까요?

<코드>

```
#B1
ws.Range("B1").VerticalAlignment = -4160  #위로 정렬
ws.Range("B1").HorizontalAlignment = -4108  #가운데 정렬(수평)

#B3
ws.Range("B3").VerticalAlignment = -4107  #아래로 정렬
ws.Range("B3").HorizontalAlignment = -4108  #가운데 정렬(수평)

#A2
ws.Range("A2").VerticalAlignment = -4108  #가운데 정렬(수직)
ws.Range("A2").HorizontalAlignment = -4131  #왼쪽으로 정렬

#C2
ws.Range("C2").VerticalAlignment = -4108  #가운데 정렬(수직)
ws.Range("C2").HorizontalAlignment = -4152  #오른쪽으로 정렬

#B2
ws.Range("B2").VerticalAlignment = -4108  #가운데 정렬(수직)
ws.Range("B2").HorizontalAlignment = -4108  #가운데 정렬(수평)
```

<결과>

	A	B	C	D
1		위로 정렬		
2	왼쪽정렬	가운데정렬	우측으로 정렬	
3		아래로 정렬		
4				

각 설정 값들은 Microsoft에서 제공하는 공식 문서에서 각 값들이 의미하는 바를 찾아볼 수 있습니다. 아래 이미지를 참고해보도록 합시다.

-. VertialAlignment 옵션 값에 따른 정렬

Name	Value	Description
xlVAlignBottom	-4107	Bottom
xlVAlignCenter	-4108	Center
xlVAlignDistributed	-4117	Distributed
xlVAlignJustify	-4130	Justify
xlVAlignTop	-4160	Top

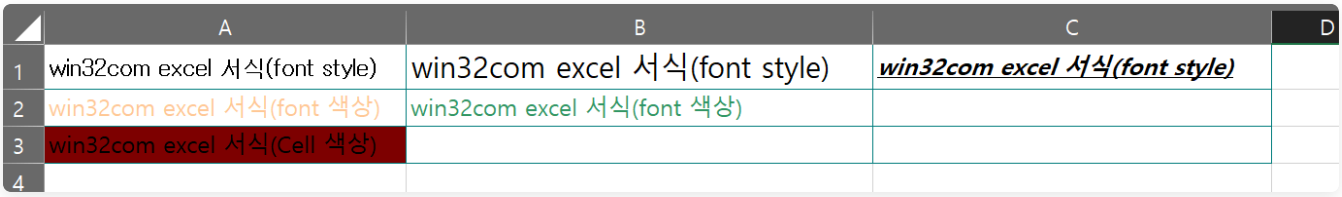
-. HorizontalAlignment 옵션 값에 따른 정렬

Name	Value	Description
xlHAlignCenter	-4108	Center.
xlHAlignCenterAcrossSelection	7	Center across selection.
xlHAlignDistributed	-4117	Distribute.
xlHAlignFill	5	Fill.
xlHAlignGeneral	1	Align according to data type.
xlHAlignJustify	-4130	Justify.
xlHAlignLeft	-4131	Left.
xlHAlignRight	-4152	Right.

4. Font, Cell 서식 지우기

3번까지는 폰트, 셀 서식을 설정하는 방법에 대해 공부해보았습니다. 이번에는 지금까지 설정한 폰트 및 셀 서식을 초기화하는 방법에 대해 배워보겠습니다. 바로 ClearFormats( )이라는 함수입니다. 추가로 비슷한 종류인 Clear( ), ClearContents( ) 함수까지 알아보죠.

먼저, 위에서 작성했던 이미지를 그대로 들고오겠습니다.



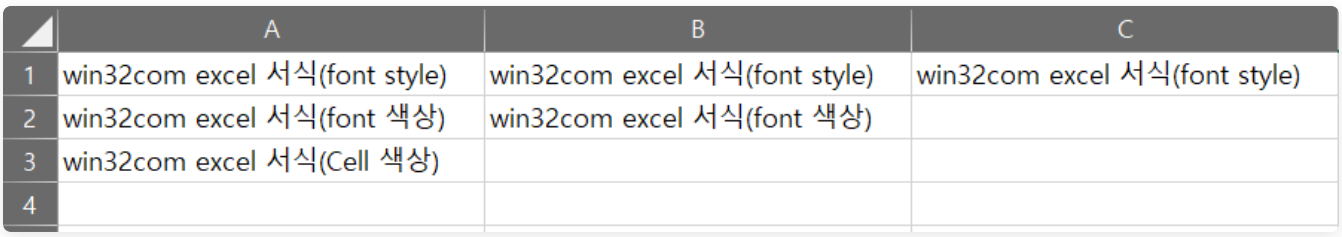
	A	B	C	D
1	win32com excel 서식(font style)	win32com excel 서식(font style)	***win32com excel 서식(font style)***	
2	win32com excel 서식(font 색상)	win32com excel 서식(font 색상)		
3	win32com excel 서식(Cell 색상)			
4				

위에서 설정했던 폰트와 셀 서식을 아래 한줄로 초기화 할 수 있습니다.

<코드>

```
#폰트, 셀서식 초기화(ClearContents일 겨우 내용만 삭제, Clear는 내용,서식 모두 초기화)
ws.Range("A1:C3").ClearFormats()
```

<결과>



	A	B	C
1	win32com excel 서식(font style)	win32com excel 서식(font style)	win32com excel 서식(font style)
2	win32com excel 서식(font 색상)	win32com excel 서식(font 색상)	
3	win32com excel 서식(Cell 색상)		
4			

ClearFormats( )를 사용하면 선택한 영역의 서식들이 위 결과 이미지와 같이 모두 삭제됩니다. 행과 열의 크기가 바뀐 부분에 대해서는 적용되지 않습니다.

주석에서도 설명하였지만 Clear( )를 사용하면 서식 및 내용이 모두 지워집니다. ClearContents( )는 내용만 지우는 코드입니다. 내용만 지운다는 것은 서식은 그대로 남아있다는 점 참고해주세요.

6) win32com 통해 엑셀 다루기 - 차트

이번 파트는 win32com.client 모듈을 통해 엑셀 차트를 다루는 방법을 소개하겠습니다. win32com.client를 사용하여 차트를 표시하는 코드를 순서대로 설명하겠습니다.

1. 모듈 import

지금까지 계속 반복했던 내용으로 모듈 import는 별도 설명은 하지 않고 코드만 작성하고 넘어가겠습니다.

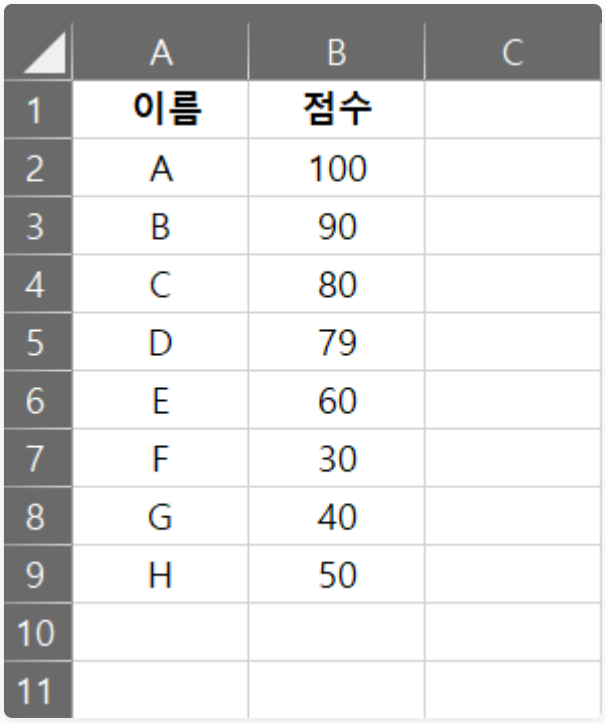
```
import win32com.client
```

2. 엑셀 App 열기(객체 생성)

이 부분도 많이 설명했던 부분이죠? win32com 을 통해 Microsft Office를 제외할 때 관련 Application 객체를 생성하는 과정은 필수이니 이 점 꼭 기억해주세요.

```
excel = win32com.client.Dispatch("Excel.Application") #엑셀 프로그램을 객체로 생성
excel.Visible = True #실행과정 보이게
```

3. Workbook 및 Worksheet 객체 생성



	A	B	C
1	이름	점수	
2	A	100	
3	B	90	
4	C	80	
5	D	79	
6	E	60	
7	F	30	
8	G	40	
9	H	50	
10			
11			

이번에는 "chart_data.xlsx"라는 파일명을 가진 엑셀 파일을 가지고 객체를 생성해보겠습니다. 위 이미지가 설정할 파일입니다. 현재 이름별 점수가 기록되어 있는 시트가 선택되어있습니다.

```
#기존 존재하는 data파일을 Workbook 객체로 생성
wb = excel.Workbooks.Open(r"C:\Users\Desktop\VS CODE\Project\17. win32com\chart_data.xlsx")

#활성화되어잇는 시트를 객체로 생성
ws = wb.ActiveSheet
```

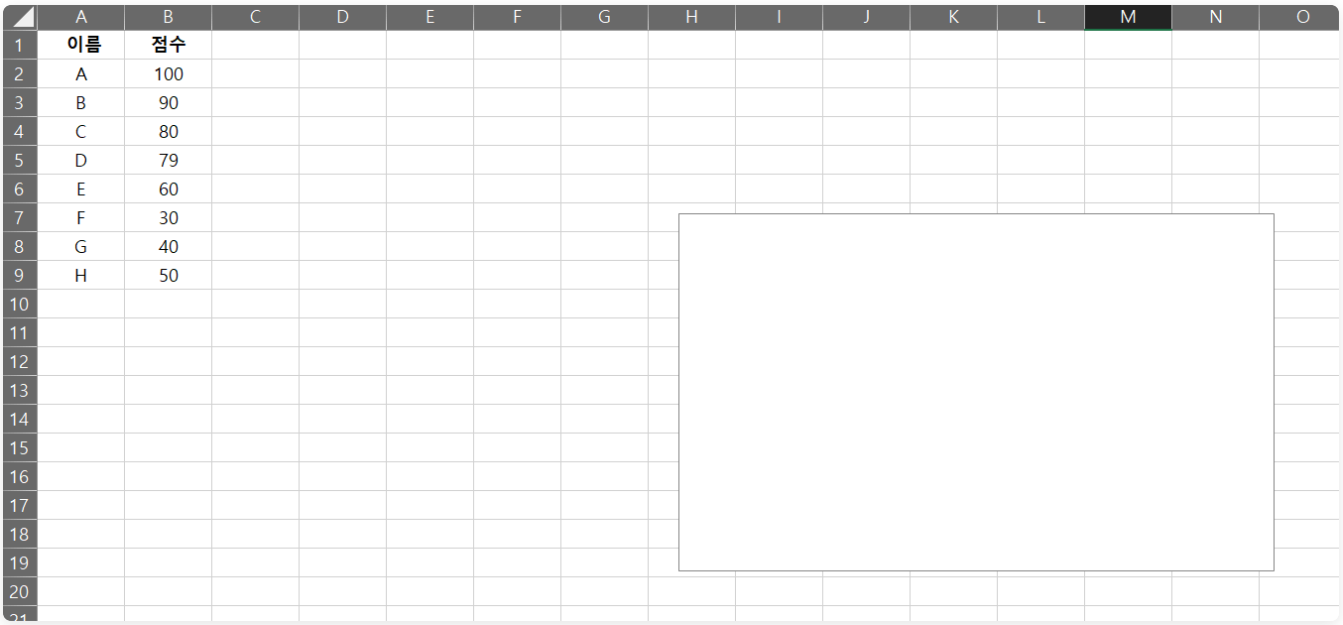
저희는 위 성적표 양식을 막대 차트로 표현해볼 것입니다.

4. 빈 차트 양식 생성하기

차트를 만드는 과정 중 첫번째는 엑셀에 빈 차트 양식을 생성하는 것입니다. 아래와 같이 코드를 작성해보겠습니다.

```
#차트를 새로 추가하고 선택하기
ws.Shapes.AddChart().Select()
```

차트를 생성하는 코드는 Worksheet(=ws)에 Shapes.AddChart( )라는 함수를 사용하면 됩니다. 위 코드를 실행하면 설정한 WorkSheet에 빈 차트 양식이 만들어집니다.(아래 빈 사각형 공간)



	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	이름	점수													
2	A	100													
3	B	90													
4	C	80													
5	D	79													
6	E	60													
7	F	30													
8	G	40													
9	H	50													
10															
11															
12															
13															
14															
15															
16															
17															
18															
19															
20															

5. 차트 Data 영역 설정하기

4번에서 빈 차트 양식을 생성했습니다. 이번에는 빈 차트 양식에 실제 막대 차트를 생성해보겠습니다.

```
# 선택 된 차트의 데이터영역을 설정하기
excel.ActiveChart.SetSourceData(Source=ws.Range("A1:B9"))
```

코드가 길지만 순서대로 설명해볼게요. 처음에 나온 excel은 위 2번 "엑셀 App 열기"에서 다뤘던 excel 변수입니다.

그 다음, ActiveChart는 현재 선택 된 차트를 의미합니다. 위 4번 "빈 차트 양식 생성하기"에서 저희는 아래와 같은 코드를 작성했었습니다.

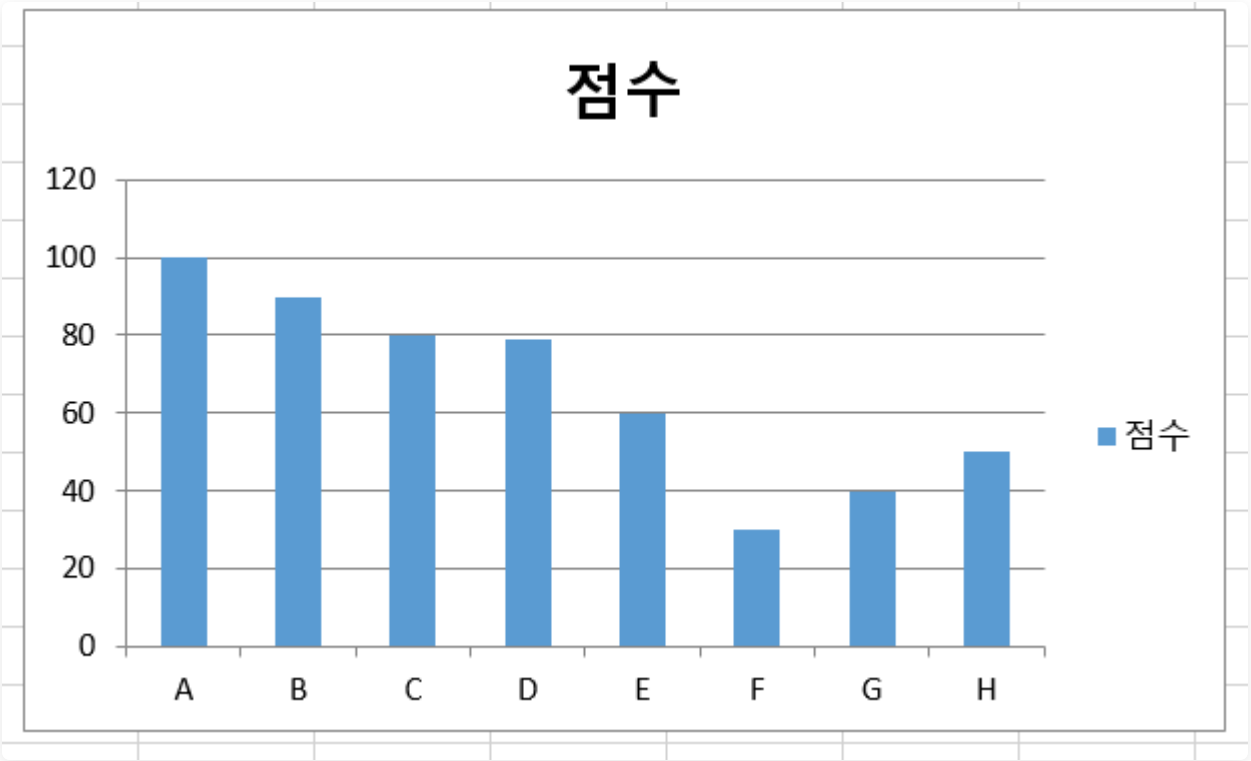
```
#차트를 새로 추가하고 선택하기
ws.Shapes.AddChart().Select()
```

차트를 생성하고 선택을 했으므로 ActiveChart는 위에서 생성했던 차트 빈 양식을 의미합니다.

SetSourceData는 저희가 선택한 빈 차트 양식에 차트를 생성할 데이터 범위를 선택하는 코드입니다. ws.Range("A1:B9")는 많이 보았던 코드죠? "A1:B9" 영역을 차트 데이터 영역으로 선택합니다. win32com 엑셀 파트를 쭉 공부해오신 분이라면 ws.Range("A1:B9") 대신 ws.UsedRange( )를 사용해도 동일한 결과가 나옴을 이해하실겁니다.

해당 내용은 "3-3-2) win32com 엑셀 다루기 - Cell 영역 선택 방법"을 참고해주세요.

위 코드를 순서대로 실행하면 아래와 같은 세로 막대 차트가 생성됩니다.



6. 차트 생성시 옵션

1) 차트 형식 설정하기

위에서는 기본적인 세로 막대 차트를 생성했지만 실제로 엑셀에서는 다양한 형식의 차트를 제공합니다. 사용자가 원하는 차트를 생성하게 하려면 어떻게 해야할까요?

Microsoft에서는 엑셀 차트 번호라는 옵션값을 제공합니다. (구글에 XlChartType Enumeration을 검색해보시기 바랍니다.)

XlChartType Enumeration

Article • 09/14/2015 • 2 minutes to read

Specifies the chart type.

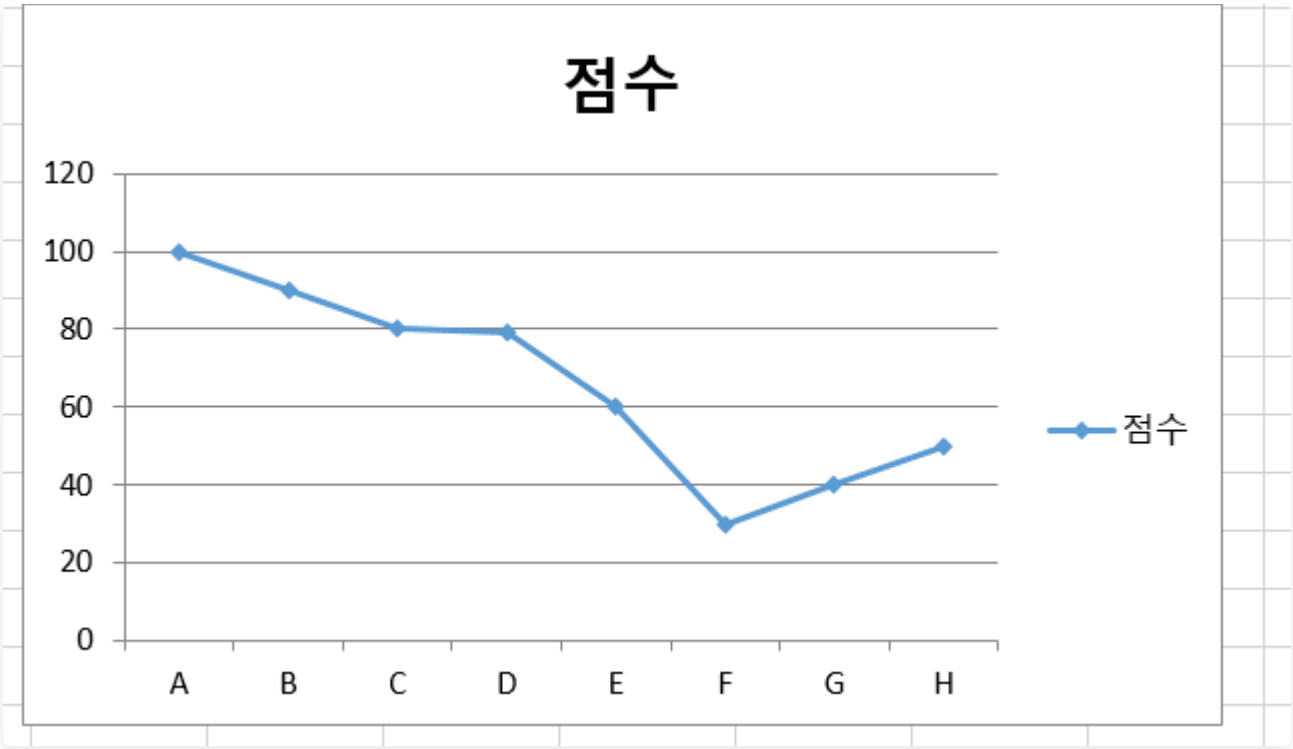
Name	Value	Description
xl3DArea	-4098	3D Area.
xl3DAreaStacked	78	3D Stacked Area.
xl3DAreaStacked100	79	100% Stacked Area.
xl3DBarClustered	60	3D Clustered Bar.
xl3DBarStacked	61	3D Stacked Bar.
xl3DBarStacked100	62	3D 100% Stacked Bar.
xl3DColumn	-4100	3D Column.
xl3DColumnClustered	54	3D Clustered Column.
xl3DColumnStacked	55	3D Stacked Column.
xl3DColumnStacked100	56	3D 100% Stacked Column.

위 이미지가 바로 그 내용인데요. 차트 옵션값을 설정함에 따라 사용자가 원하는 형식의 차트를 선택할 수 있습니다. (모든 내용이 있는 것은 아니므로 자세한 내용은 Microsoft에서 제공하는 문서를 찾아보시기 바랍니다.)

먼저 앞 과정에서 차트를 생성했던 코드를 다시 가져와 보겠습니다.

```
#꺾은선 차트 생성
chart = ws.Shapes.AddChart(65).Select()
#데이터 범위 설정
excel.ActiveChart.SetSourceData(Source=ws.Range("A1:B9"))
```

이 전 코드랑 달라진 점은 Shapes.AddChart(65) 부분입니다. 이전에 빈 차트를 생성했을 때는 괄호 안에 아무 값도 입력하지 않았습니다. 이번에는 65라는 숫자 값을 입력하고 코드를 실행시켜보겠습니다.



원래 막대 그래프였던 부분이 숫자 65를 입력함으로써 꺾은선 그래프가 됐네요. 다시 위 XlChartType Enumeration에서 65라는 값에 대한 옵션을 검색해보겠습니다.

xlLineMarkers	65	Line with Markers.

숫자 65는 "Line with Markers"를 의미합니다. 즉, 위와 같은 그래프 형태를 사용하려면 숫자 65를 입력하면 되는 것이죠. AddChart( ) 괄호 안의 숫자 값 변경을 통해서 테스트해보시기 바랍니다.

2) Chart 위치 및 크기 설정하기

이번에는 설정한 Chart의 위치 및 크기를 설정하는 코드를 작성해보겠습니다.

```
chart = ws.Shapes.AddChart(4,100,100,200,200).Select() #막대 차트 200*200 크기 생성
excel.ActiveChart.SetSourceData(Source=ws.Range("A1:B9")) #데이터 범위 설정
```

이전 코드와 달라진 점이라면 AddChart에 차트 타입을 설정하는 숫자뿐만 아니라 위치, 크기 값이 추가되었다는 것입니다.

괄호 안의 옵션 값 입력 형태는 아래와 같습니다.

```
"Worksheet 객체명".Shapes.AddChart(차트타입, x축 위치, y축 위치, 가로크기, 세로크기)
```

-. 차트타입 : 어떤 타입의 차트를 사용할 것인가?(숫자 값)

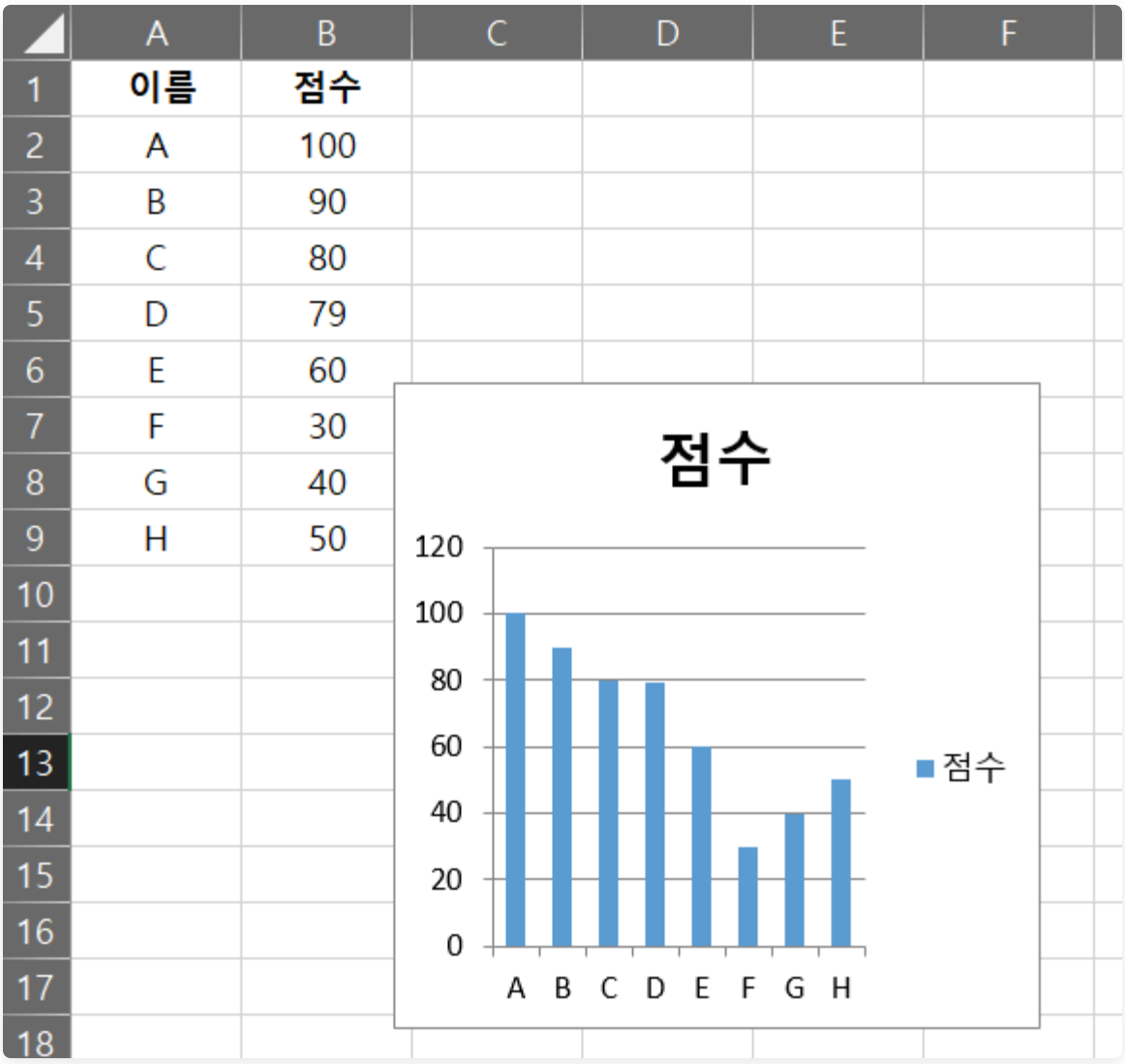
-. x축위치 : 왼쪽으로부터 시작해서 위치 값

-. y축위치 : 위부터 시작해서 위치 값

-. 가로크기 : 차트의 가로크기

-. 세로크기 : 차트의 세로크기

설명한 내용을 참고하셔서 차트 옵션 값들을 이해해보세요. 코드를 실행시키면 아래와 같은 결과가 나옵니다.



7. 차트 제목 및 범례 설정하기

이번에는 차트의 제목과 범례(label)을 설정하는 방법을 알아보겠습니다. 해당 내용은 어려운 내용은 없어서 주석으로 설명을 대신하겠습니다. 필요하신 코드를 그냥 그대로 써서 사용해주시면 됩니다.

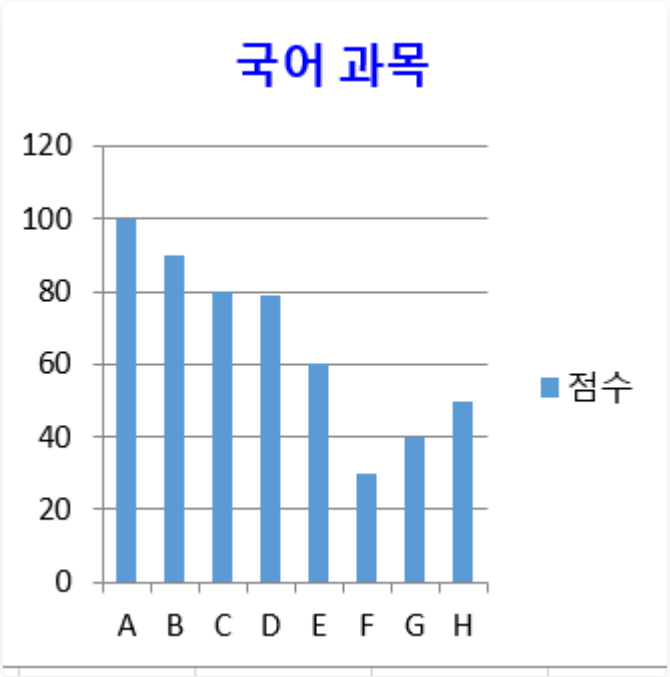
참고로, AcitveChart라는 코드는 위에서도 설명했었지만 현재 선택되어 있는 차트임을 참고해주세요.

1) 차트 제목 설정하기

<코드>

```
#차트 제목 나오게(False면 안보임)
excel.ActiveChart.HasTitle = True

#차트 제목 설정
excel.ActiveChart.Charttitle.Text = "국어 과목"

#차트 제목 글씨 크기
excel.ActiveChart.Charttitle.Characters.Font.Size=14

#차트 제목 색상(파란색)
excel.ActiveChart.Charttitle.Characters.Font.ColorIndex = 5

#제목 굵게
excel.ActiveChart.Charttitle.Characters.Font.Bold=True
```

<결과>

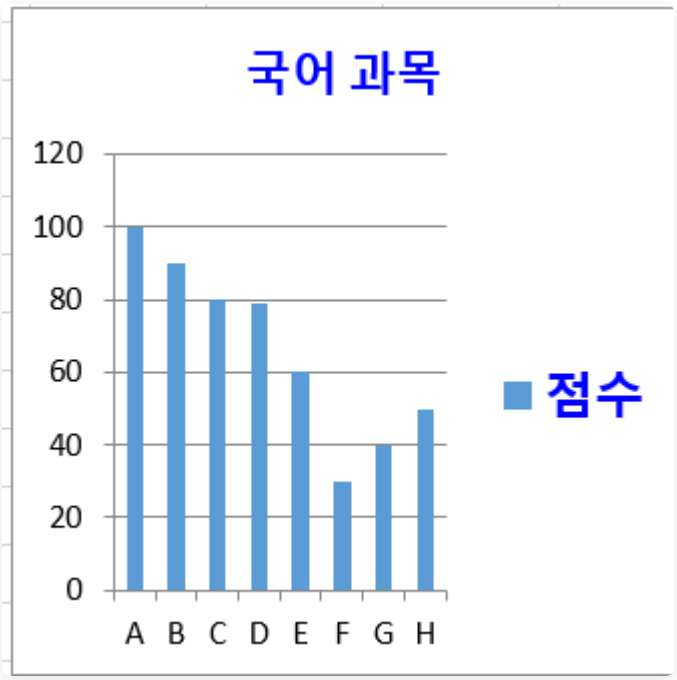


2) 차트 범례 설정하기

<코드>

```
#차트 범례 보이게(Fasle면 안보임)
excel.ActiveChart.HasLegend = True

#차트 범례 글씨 크기
excel.ActiveChart.Legend.Font.Size = 15

#차트 범례 굵게
excel.ActiveChart.Legend.Font.Bold = True

#차트 범례색상 설정(파란색)
excel.ActiveChart.Legend.Font.ColorIndex = 5
```

<결과>



7) win32com 통해 엑셀 다루기 - 파일 저장 옵션

이번 파트에서는 win32com.client를 통해 엑셀 파일을 저장하는 여러가지 방법에 대해 알아보겠습니다. 엑셀 업무자동화에서 중요한 부분 중 하나는 원하는 파일을 사용자가 원하는 형태로 저장하는 것입니다. 사용자에 따라 엑셀 파일을 필요한 시트만 저장하거나, pdf 파일 등 다른 형식의 파일로 저장해야하는 경우도 생깁니다. 아래 내용을 공부하시면 저장 방법에 대한 기초적인 내용을 아실 수 있습니다.

아래와 같은 3가지 함수를 배워보겠습니다.

1. Save( )

2. SaveAs( )

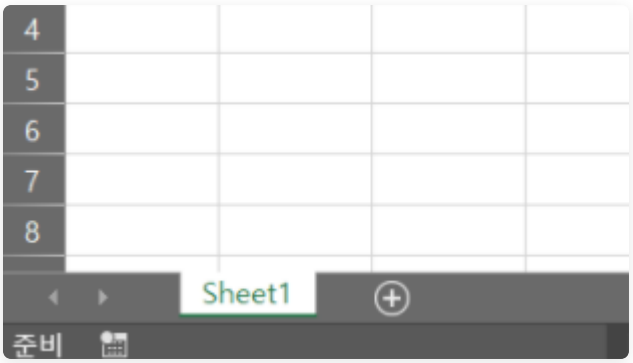
3. ExportAsFixedFormat( )

1. Save( )

Save 함수는 별다른 설정 옵션 없이 "통합문서"라는 이름으로 파일을 저장합니다. Save 함수를 사용해보기 전에 먼저 임의의 Workbook 객체를 생성하는 코드를 작성해보겠습니다.

```
import win32com.client

#excel App 열기(객체 생성)
excel = win32com.client.Dispatch("Excel.Application")

#엑셀 실행과정이 보이게
excel.Visible = True

#Workbook 객체 생성(새로 추가)
wb = excel.Workbooks.Add()
```

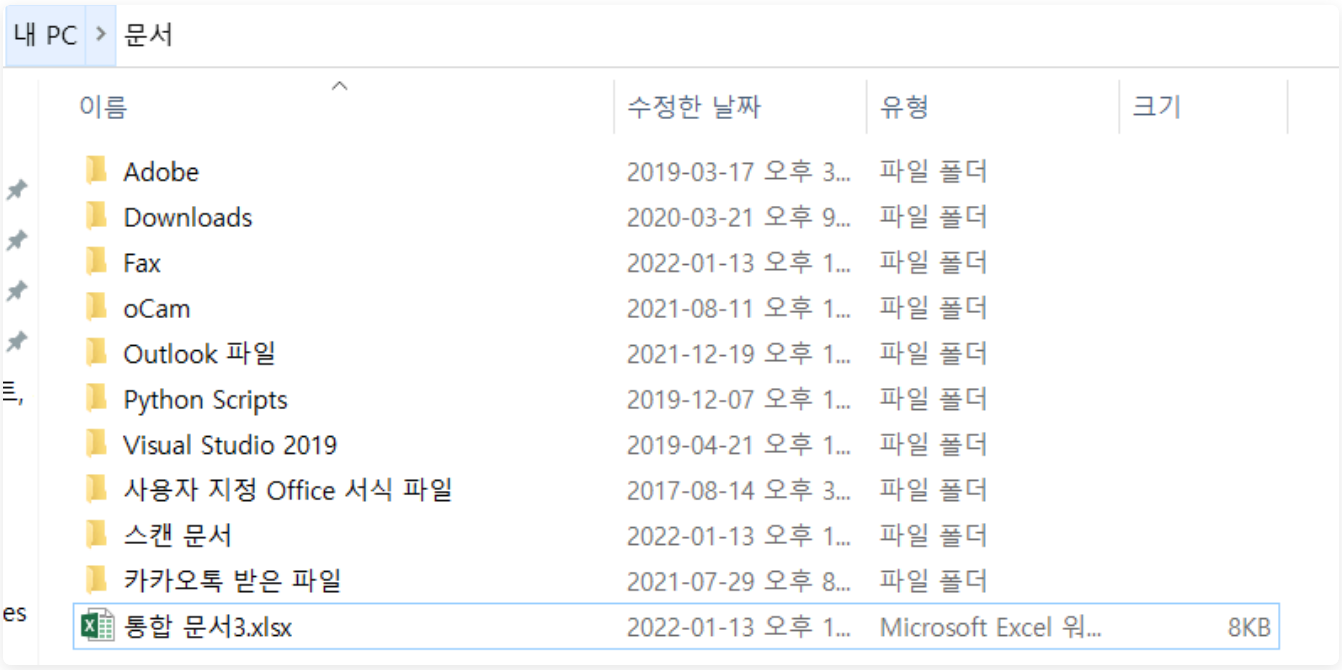
지금까지 win32com.client 모듈을 공부해오셨던 분이라면 많이 봐왔던 코드들이죠?? 위 코드를 실행하면 아래와 같이 엑셀 통합 문서와 함께 기본 시트 "Sheet1"을 생성하게 됩니다.



그럼 위 상태에서 문서를 저장하는 Save( ) 함수를 사용하는 코드를 한줄 추가해보겠습니다. Save( ) 함수를 사용할 때 대상이 되는 객체는 Workbook입니다. 따라서, 아래와 같이 작성합니다.

```
#Workbook(통합문서) 저장
wb.Save()
```

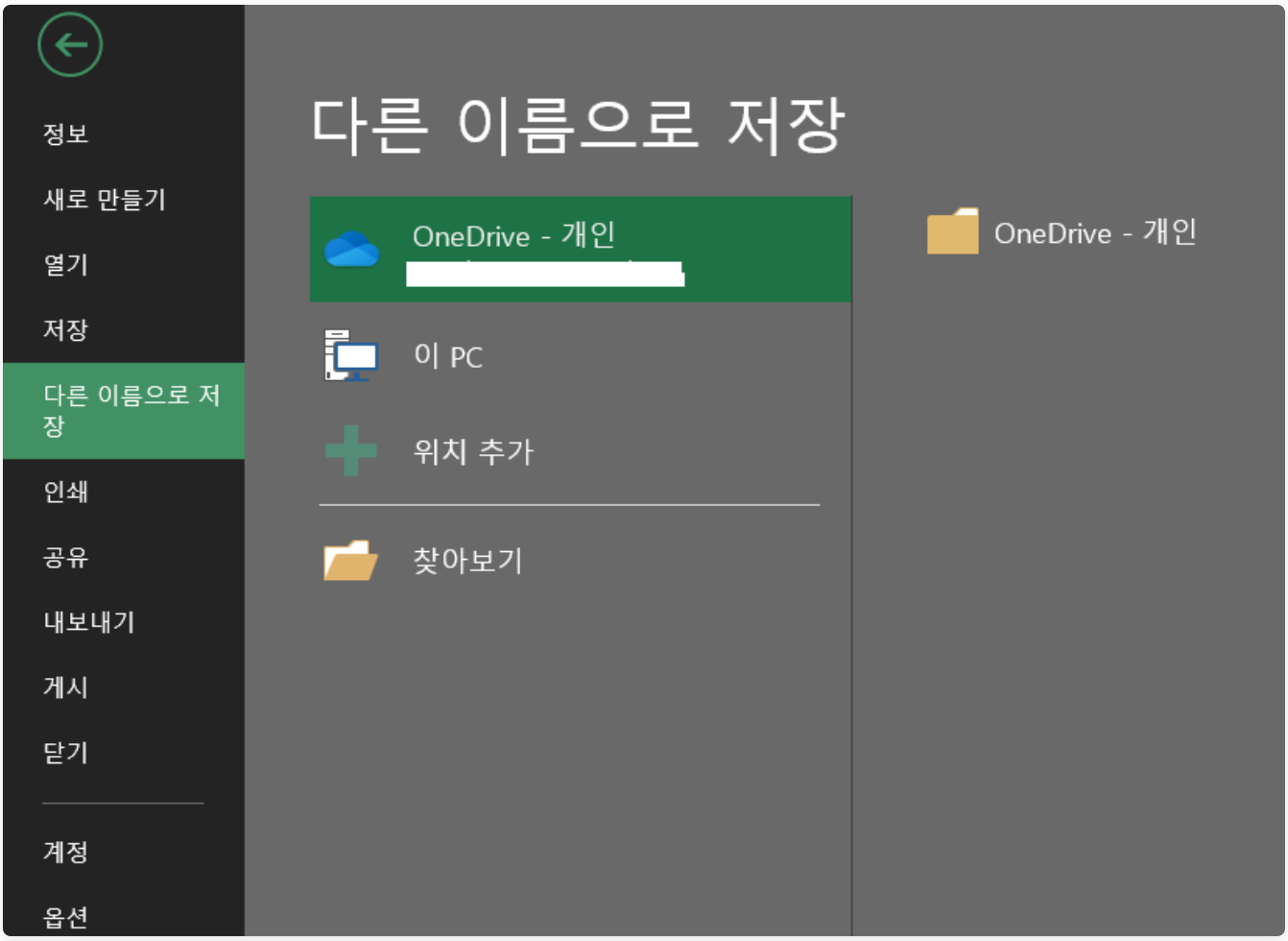
위 코드와 같이 작성하여 실행하면 내 PC-문서라는 폴더에 "통합문서"라는 이름으로 엑셀 파일이 저장되게 됩니다. 넘버링의 경우 그전 문서에 이어 순서대로 저장되게 됩니다.(통합문서1,2,3....)



그럼 만약 엑셀의 "다른 이름으로 저장하기"와 같은 기능을 사용하려면 어떻게 해야할까요? 파일명과 저장 경로를 직접 설정할 수 있게 하려면 아래 SaveAs( ) 함수를 사용해야 합니다.

2. SaveAs( )

위에서 언급했다시피 SaveAs( )는 엑셀의 "다른 이름으로 저장하기"와 같은 기능을 하는 함수입니다.



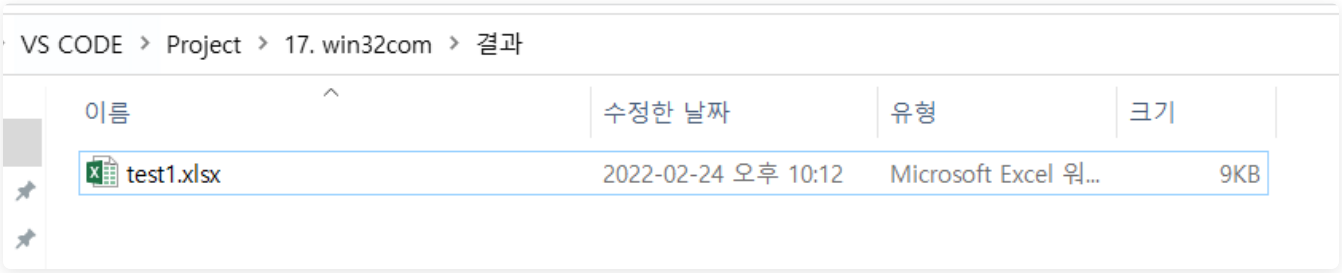
Save( )와 마찬가지로 대상이 되는 객체는 Workbook 입니다. 기본적인 사용법은 아래와 같습니다. SaveAs의 옵션은 여러가지가 있으나 대표적으로 쓰는 3가지만 표시해보았습니다.

```
"Workbook 객체".SaveAs(파일명, 파일포맷, 비밀번호...)
```

직접 사용해보며 익혀볼까요? 아래와 같이 코드를 작성하여 실행시켜보겠습니다. 파일포맷, 비밀번호 등의 옵션은 생각하지 않고 경로와 파일명만 지정하였습니다.

```
#path 지정
path = r"C:\Users\Desktop\VS CODE\Project\17. win32com\결과"

#저장
wb.SaveAs(path+"/test1.xlsx)
```

<결과>



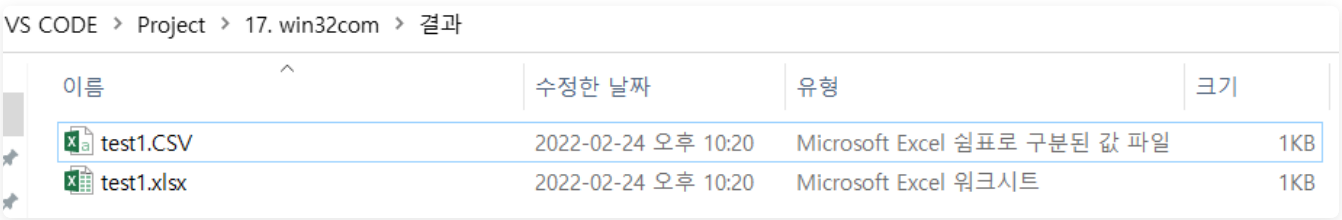
위에서 SaveAs( ) 함수를 사용했을 때, 파일 확장자(.xlsx)를 직접 지정했지만 함수의 옵션 값을 통해 파일 형식을 지정하는 것도 가능합니다.

대표적인 예로 .csv 확장자로 저장하는 코드를 작성해보겠습니다. 아래 코드처럼 파일포맷 자리에 6을 입력하면 .csv 파일로 저장하게 됩니다. 대신 파일명을 지정할 때, 확장자는 제외시켜주어야 합니다.

(위에서 실행했던 코드는 test1.xlsx이었던 기억해보세요)

```
#path 지정
path = r"C:\Users\Desktop\VS CODE\Project\17. win32com\결과"

#.csv 확장자로 저장(file format=6)
wb.SaveAs(path+"/test1, 6)
```

<결과>



파일 포맷에 대한 정보는 Microsoft 공식가이드 문서를 찾아보시면 됩니다. 아래 자료에서 보듯이 숫자 6은 .csv 파일을 의미하는 숫자입니다.

Name	Value	Description	Extension
xlAddIn	18	Microsoft Excel 97-2003 Add-In	*.xla
xlAddIn8	18	Microsoft Excel 97-2003 Add-In	*.xla
xlCSV	6	CSV	*.csv

3. ExportAsFixedFormat( )

위에서 배워왔던 저장 방법은 단순히, 엑셀 파일을 저장하는 방법이었습니다. 엑셀 업무자동화에서 가장 많이 쓰이는 부분 중 하나는 특정 엑셀 시트를 .pdf 파일과 같이 원하는 확장자로 추출하는 방법입니다.

이런 경우에는 win32com 에서 제공하는 ExportAsFixedFormat( )이라는 함수를 사용합니다.

먼저, 해당 함수의 사용법은 아래와 같습니다.

```
"WorkSheet 객체".ExportAsFixedFormat(Type, FileName, From, To)
```

함수의 대상이 되는 객체는 Worksheet 객체입니다. 함수 내부의 옵션값은 매우 많습니다. 하지만, 대표적으로 사용하는 옵션은 Type, FileName, From, To 정도입니다.

1) Type

Type은 추출할 형식을 선택합니다. 0을 입력할시 pdf 형식, 1을 입력할시 xps 형식으로 파일을 추출합니다. 해당 내용은 Microsoft 공식 문서에도 나와있습니다.

xlTypePDF	0	"PDF" — Portable Document Format file (.pdf).
xlTypeXPS	1	"XPS" — XPS Document (.xps).

2) FileName

이 부분은 SaveAs 함수에서 배웠던 내용과 동일합니다. 경로 및 파일명을 지정할 수 있습니다.

3) From, To

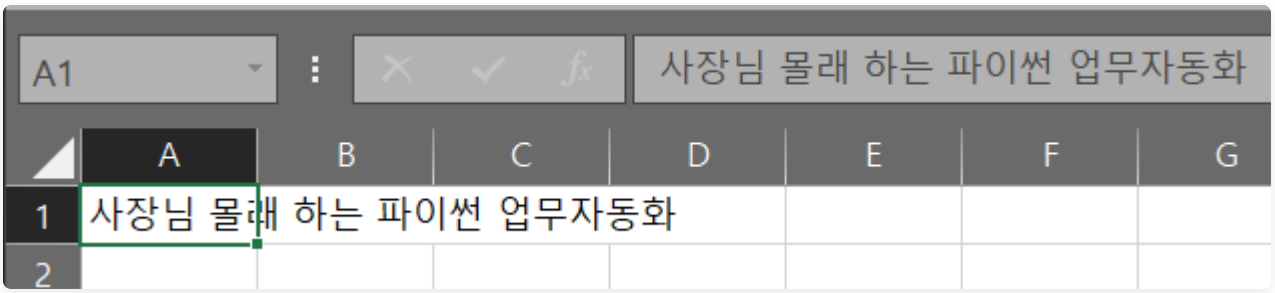
From은 해당 엑셀 시트에서 저장할 시작 페이지를 의미합니다. To는 해당 엑셀 시트에서 저장할 마지막 페이지입니다.

이번엔 사용 예시를 들어보겠습니다.

<코드>

```
#모듈 임포트
import  win32com.client

#excel 객체 생성 및 실행과정 보이게
excel = win32com.client.Dispatch("Excel.Application")
excel.Visible = True

#신규 엑셀 문서 생성
wb = excel.Workbooks.Add()
#활성화 된 시트를 ws로 설정
ws = wb.ActiveSheet

# 1행 1열에 텍스트 입력
ws.cells(1,1).Value = "사장님 몰래 하는 파이썬 업무자동화"
```

<결과>



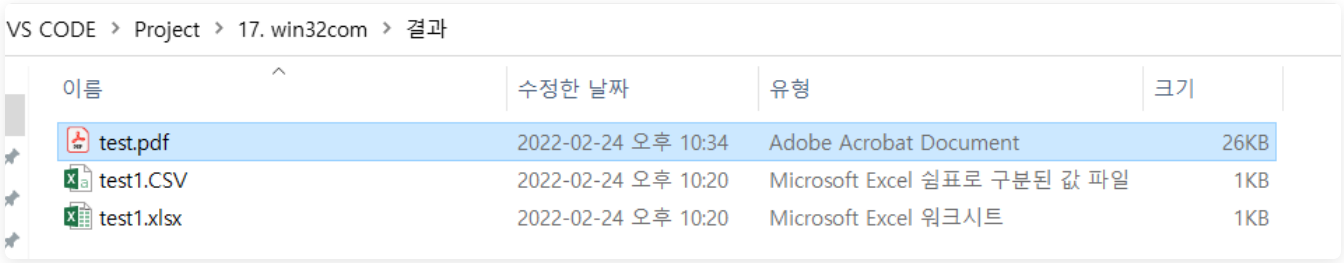
저희는 위 입력 된 엑셀 시트를 pdf 확장자로 추출해볼 것입니다. 아래와 같이 코드를 추가 입력해보겠습니다. 저장 경로는 앞에서 실행했던 코드들과 동일 경로입니다.

From, To의 경우 생략하면 설정 된 페이지 영역을 모두 저장합니다. 아래 예시는 이해를 위해 From과 To를 입력하였습니다. 생략하여도 결과는 동일합니다.

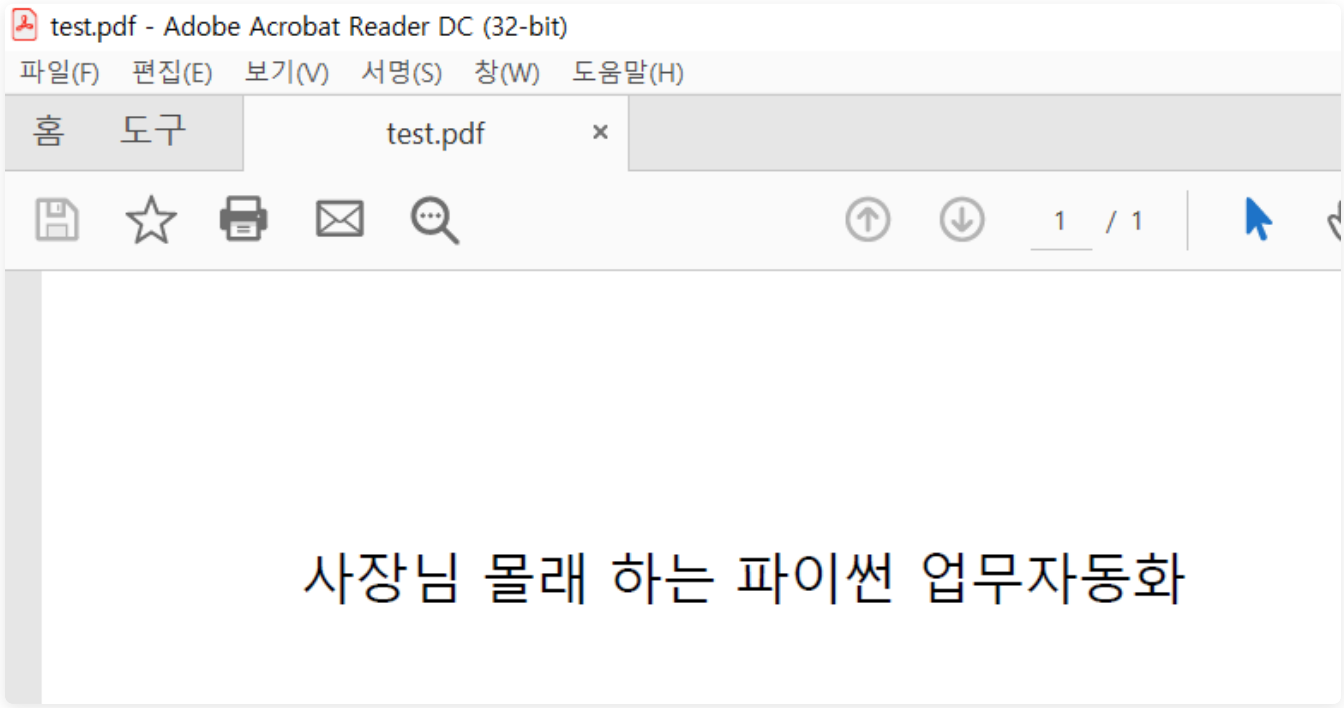
<코드>

```
#저장할 경로 설정
path = r"C:\Users\Desktop\VS CODE\Project\17. win32com\결과"

#pdf 파일로 저장하기 (1 page)
ws.ExportAsFixedFormat(Type=0, Filename=path+"/test.pdf",From=1,To=1)
```

<결과>



pdf 파일을 실행시켜보면 엑셀 시트 내용이 그대로 저장 된 것을 확인하실 수 있습니다.



이번 파트 내용을 응용하시면 여러 엑셀파일을 모두 pdf 파일로 한번에 변환하거나 하는 등의 업무 자동화 프로그램을 작성할 수 있습니다. 관련 프로그램은 "Ch4. 업무자동화 프로젝트 사례"에서 다룹니다.

9) win32com 통해 전자메일(Outlook) 다루기

win32com.client 모듈을 사용하면 사내에서 사용하는 Outlook 전자메일을 자동화할 수 있습니다. 대표적인 자동화 예로, Outlook 메일을 다수의 사람들에게 자동 발송할 수 있습니다. 또한, 첨부파일들을 한번에 다운받거나 메일의 내용들을 엑셀 파일에 한번에 취합하는 것도 가능합니다.

위와 같은 자동화 기능을 구현하려면 win32com.client를 사용해서 outlook 전자메일을 제어할 수 있는 기초적인 방법에 대해 알아야 합니다. 이번 파트에서는 Outlook 이메일을 파이썬을 통해 제어하는 방법에 대해 알아보겠습니다.

1. import

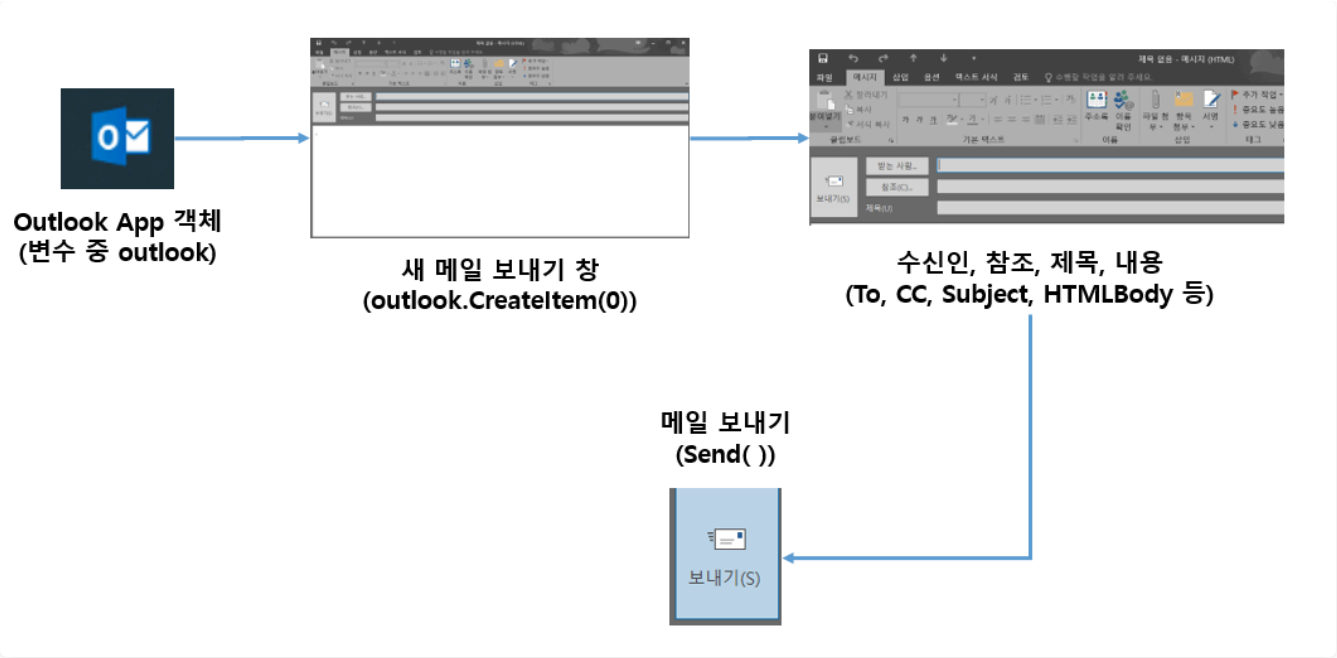
이 부분은 엑셀을 제어하는 방법과 동일합니다. win32com.client를 아래 코드로 import 시켜줍니다.

```
import win32com.client
```

2. 메일 보내기(송신)

코드를 공부하기 전에 먼저 메일을 보내는 과정을 생각해봅시다. Outlook 프로그램을 열고, 메일 보내기 창을 선택하여 메일 정보(내용, 수신자 등)을 입력합니다. 그 다음, 메일 보내기 버튼을 누르면 지정한 수신자에게 메일을 보낼 수 있습니다.

win32com.client를 통해 메일을 제어하는 과정도 동일합니다. 그 과정을 아래와 같이 도식화해보았습니다. 앞으로의 작성 코드는 그림과 같이 순서대로 과정을 따라가도록 하겠습니다.



1) Outlook Application에 대한 객체 생성하기

Outlook을 제어할 때 가장 첫번째로 해야할 일은 Outlook 프로그램을 여는 것입니다. 코드에서는 Outlook 프로그램에 대한 객체를 설정해준다고 표현합니다. (위 이미지에서 첫번째 과정)

```
outlook=win32com.client.Dispatch("Outlook.Application")
```

2) 메일을 보내기 위한 객체 생성

1)에서 생성한 outlook 객체를 이용해 "메일 보내기" 창을 열어보겠습니다. CreateItem( )이라는 함수를 사용합니다. CreateItem(0)은 메일 보내기 창을 새로 생성하는 코드입니다. send_mail이라는 변수로 설정하여 이 변수가 메일보내기 창의 객체 성질을 가질 수 있게 합니다.

```
send_mail = outlook.CreateItem(0)
```

3) 메일 보내기에 필요한 정보 입력

2)에서 send_mail은 메일 보내기 창의 객체 성질을 가진다고 언급했습니다. 저희가 메일을 보낼 때, 기본적으로 입력해야 하는 정보들이 있습니다. send_mail은 이 메일 보내기 창의 성질(속성)들을 가지게 됩니다. 아래 코드처럼요.

수신인, 참조, 제목, 메일내용 등을 문자열을 입력해보겠습니다.

```
send_mail.To = "수신인@gmail.com" #메일 수신인
send.mail.CC = "참조@gmail.com" #메일 참조
send_mail.Subject = "사장님 몰래 하는 파이썬 업무 자동화" #메일 제목
send_mail.HTMLBody = "들키면 일이 많아지니 몰래할 것!" #내용
```

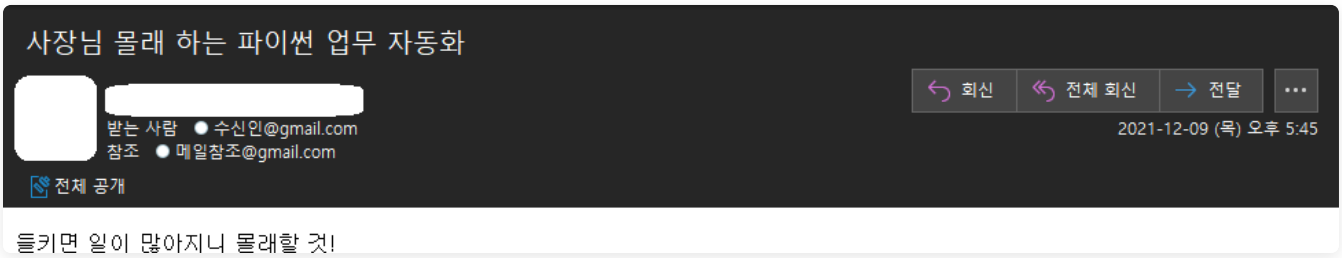
한가지 알아두어야 할 점은 메일의 본문 작성은 HTML 형태라는 것입니다. HTML의 각 태그를 통해 메일을 꾸밀 수도 있다는 점 참고하시면 됩니다.

4) 메일 보내기

메일은 Send( ) 라는 함수를 통해 보낼 수 있습니다. 위에서 설정했던 send_mail 변수를 통해 Send( ) 를 사용해봅시다.

```
send_mail.Send() #메일 보내기
```

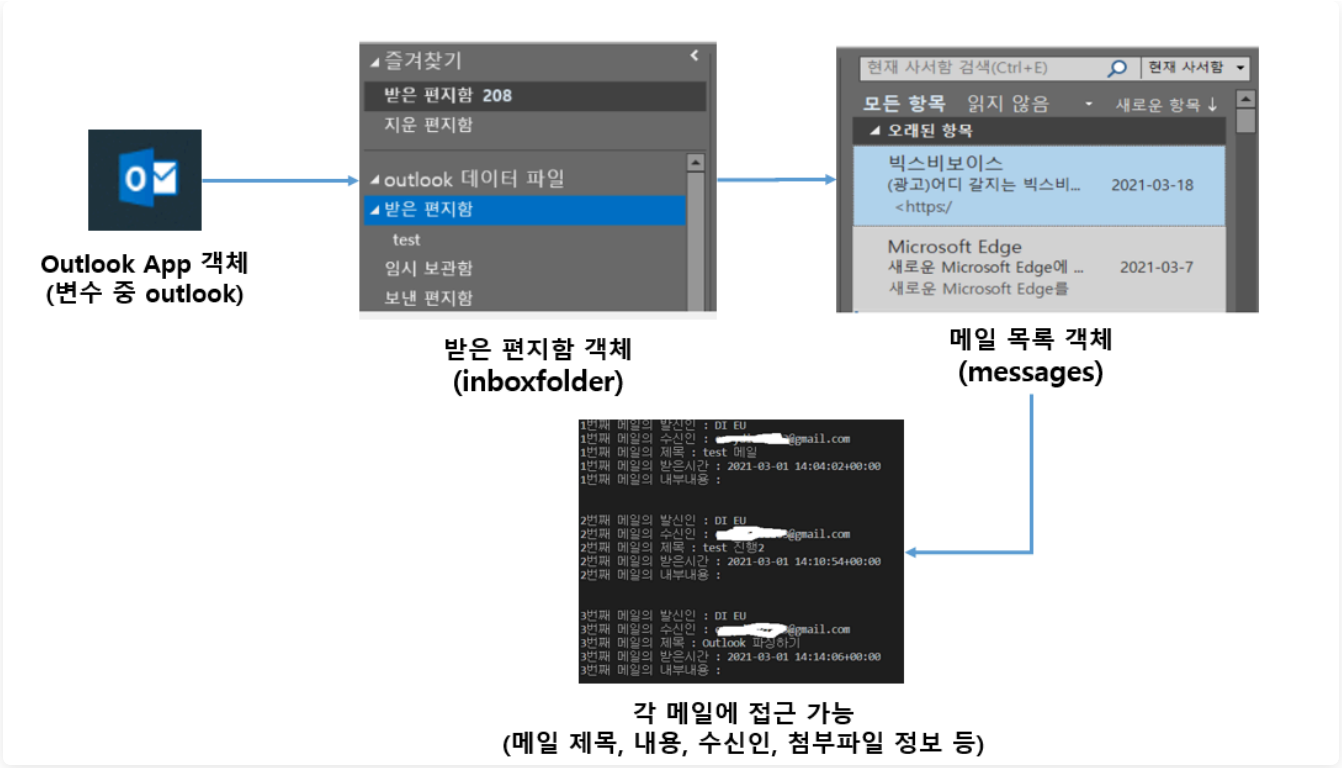
코드를 작성해서 실행해보면 Outlook에서 아래 메일이 작성되어 수신자, 참조자에게 송신된 것을 확인하실 수 있습니다.



3. 메일 받기(수신)

지금까지 2번 항목에서는 win32com.client 모듈을 사용하여 Outlook 메일을 보내는 법을 순서대로 설명했습니다. 이번에는 반대로 받아온 메일을 어떻게 파이썬을 통해 읽어올 수 있는지 설명해보도록 하겠습니다.

먼저, 메일을 받는 수신 과정을 코드로 구현하는 것은 아래 그림으로 요약할 수 있습니다. 앞으로의 과정은 이 그림을 기준으로 설명하겠습니다.



1) Outlook Application에 대한 객체 생성하기

메일 송신과 마찬가지로 수신시에도 Outlook에 대한 객체를 생성해주어야 합니다. 위 그림에서 Outlook App 객체를 생성하는 부분입니다. 아래와 같이 코드를 작성해보겠습니다.

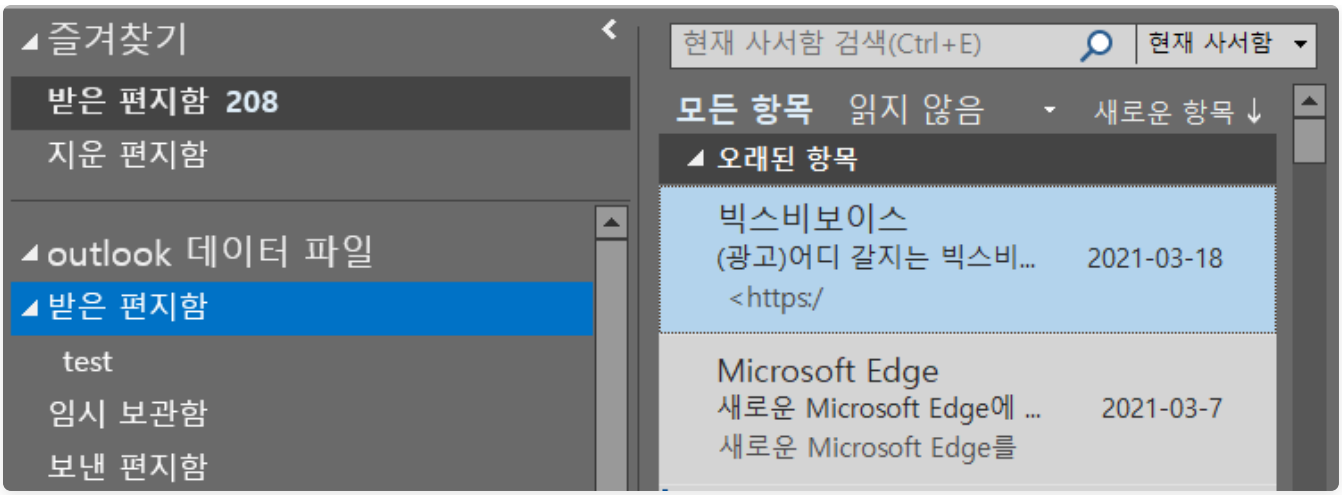
```
outlook=win32com.client.Dispatch("Outlook.Application").GetNamespace("MAPI")
```

메일 송신시와 차이가 있는 부분은 뒤에 .GetNamespace("MAPI")라는 부분입니다.

MAPI는 메일을 수신할 때 필요한 프로토콜입니다. 프로토콜은 표준, 규약이라는 의미입니다. Outlook 메일을 사용할 때 송신자와 수신자 간의 약속이라고 생각하시면 됩니다. 자세한 내용은 모르셔도 사용하시는데 무관하며, win32com.client를 사용하여 메일 수신시 사용해야 하는 코드라고 이해하시면 됩니다.

2) 메일 수신함 설정하기

Outlook 전자메일을 사용할 때, 받은 메일을 확인하는 과정을 생각해봅시다. Outlook 프로그램을 실행하고 받은 편지함을 선택합니다. 그 다음, 오른쪽 메일 리스트에서 보고 싶은 메일을 선택합니다. 위 그림에서 inboxfolder와 messges에 해당하는 부분입니다.



파이썬 코드에서도 이 과정은 동일합니다. 아래 코드 및 주석을 통해 확인해보겠습니다.

```
# 받은 편지함 선택(객체 생성), 6번은 받은 편지함(inbox folder) 의미
inboxfolder = outlook.GetDefaultFolder(6)

#받은 편지함의 messgae 받아오기(모든 메시지)
messages = inboxfolder.Items
```

위 코드에서 messages는 받은 편지함의 메시지 목록을 의미합니다. 따라서 반복문을 통해 접근할 수 있습니다. for문을 통해 메시지 목록을 출력해보겠습니다.

```
#메시지 목록 출력해보기
for  mail  in  messages:
    print(mail)
```

<결과>

```
#메일 목록 출력
<COMObject <unknown>>
<COMObject <unknown>>
```

메일 목록을 출력해보니 COMObject 라는 텍스트가 2건 출력 됐습니다. COMObject는 Win32com.client 모듈을 사용할 때, COM 객체를 의미합니다. 업무 자동화에 이용하려는 목적에서는 COM 객체가 무엇인지 자세히 모르셔도 무관합니다. 단순히, 각각 1개의 COM객체는 메일 1건을 의미하는 것이라고 이해하시면 됩니다.

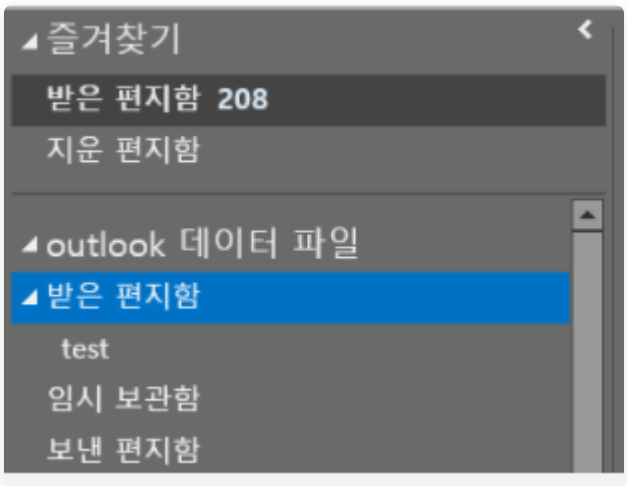
COMObject가 2건 출력됐다는 것은 현재 "받은 편지함"에 메일이 2건이 있는 것을 의미합니다. 받은 편지함의 메일 건수를 실제 숫자로 확인해보고 싶다면 아래와 같이 코드를 작성합니다.

```
#메일 건수 출력하기
msg_count = messages.count
print("받은편지함의 메일 건수는? : ", msg_count,"건")
```

<결과>

```
받은편지함의 메일 건수는? : 2 건
```

추가로, 받은 편지함의 특정 메일함에 접근하려면 어떻게 할까요? 예를 들면, 아래 이미지처럼 받은 편지함 내부에서도 "test" 폴더와 같이 여러 폴더로 구분하여 관리하는 경우입니다.



받은편지함 - test 폴더에 접근하려면 위 코드 중 inboxfolder를 아래와 같이 작성하시면 됩니다.

```
#받은편지함 중 "test" 메일함을 객체로 설정
inboxfolder = outlook.GetDefaultFolder(6).Folders["test"]
```

3) 받은 메일 내용 출력해보기

이번에는 실제로 받은 메일의 보낸이, 제목, 메일 내용 등 메일 정보를 출력하는 코드를 작성해보겠습니다.

```
#받은 편지함 설정
inboxfolder = outlook.GetDefaultFolder(6)

#받은 편지함의 messgae 받아오기(모든 메시지)
messages = inboxfolder.Items
```

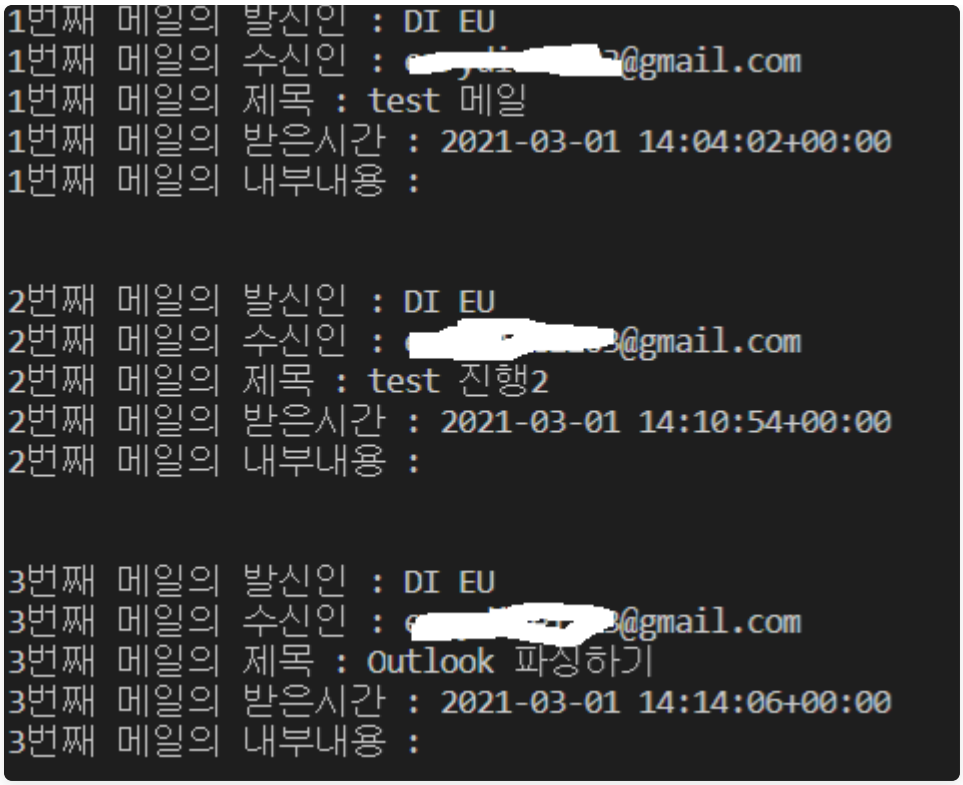
위에서 생성했던 2)에서 설정했던 객체인 messges를 for 반복문을 사용해서 각 메일의 정보를 출력해보겠습니다.

```
i=1 #여러 메일 출력을 위한 인덱싱

#각 메일에 대한 정보를 for 반복문을 통해 출력
for  mail  in  messages:
    print(str(i) + "번째 메일의 발신인 : " + mail.SenderName)
    print(str(i) + "번째 메일의 수신인 : " + mail.To)
    print(str(i) + "번째 메일의 제목 : " + mail.Subject)
    print(str(i) + "번째 메일의 받은시간 : " + str(mail.ReceivedTime))
    print(str(i) + "번째 메일의 내부내용 : " + mail.Body)
    print('\n')
    i = i+1
```

위 코드에서 SenderName(보낸이), To(받는이), Subject(메일제목), Body(메일내용), ReceivedTime(받은시간) 등은 mail 1개 객체가 가진 속성입니다. 이 속성들을 활용해서 메일의 내용들을 읽어올 수 있습니다. 위 코드의 결과는 아래와 같습니다.

ReceivedTime의 경우 문자열이 아니기에 문자열로 변경시켜주기 위해 str( )을 사용했습니다.

<결과>



4) 첨부파일 다운 받기

메일에는 내용뿐만 아니라 별도 파일이 첨부되어 있는 경우도 있습니다. 만약, 업무자동화 내용 중 메일의 첨부파일을 모두 자동으로 다운 받아야하는 경우에는 해당 코드를 응용하시면 됩니다.

```
#받은 편지함 설정
inboxfolder = outlook.GetDefaultFolder(6)

#받은 편지함의 messgae 받아오기(모든 메시지)
messages = inboxfolder.Items
```

마찬가지로 받은 편지함의 메일 목록을 가진 위 코드의 messages 객체를 통해 기능을 구현해보겠습니다. 먼저, 전체 코드부터 볼까요?

<전체코드>

```
no = 1 #중복 파일명이 있을 것을 대비해 추가한 파일명 인덱스
m_count = 1 #메일 카운트
#각 메일마다 첨부파일을 다운로드 하는 루프
for ms in messages:

    attachments = ms.Attachments #해당 메일의 첨부파일 객체 설정
    r = attachments.count #해당 메일의 첨부파일 개수(integer)
    print(str(m_count)+"번째 메일 첨부파일 개수 : ",str(r)+"개")
    m_count=m_count+1
    #해당 메일의 첨부파일을 모두 저장
    for i in range(1, r + 1):
        attachment = attachments.Item(i)
        attachment.SaveASFile("경로" + str(no) + "_" + str(attachment))  # 파일명 설정
        no = no + 1
```

각 코드마다 주석을 달아 놨으나 중요한 부분만 다시 짚고 넘어가도록 하겠습니다. 먼저 첫번째 메일의 첨부파일 객체를 생성하는 부분입니다.

```
#각 메일마다 첨부파일 객체 생성
for  ms  in  messages:

    attachments = ms.Attachments #해당 메일의 첨부파일 객체 설정
    print("첨부파일 목록 객체 생성 :", attachments)
    #첨부파일 개수 확인하기
    print("첨부파일 개수 : ", attachments.count)
```

<결과>

```
첨부파일 목록 객체 생성 : <COMObject <unknown>> 첨부파일 개수 : 1
첨부파일 목록 객체 생성 : <COMObject <unknown>> 첨부파일 개수 : 2
첨부파일 목록 객체 생성 : <COMObject <unknown>> 첨부파일 개수 : 1
```

"2) 메일 수신함 설정하기"에서도 다뤘지만 inboxFolder.Items 라는 구문을 통해 받은 편지함의 메일 리스트를 객체로 생성할 수 있다고 언급했습니다.(위 변수 중 messages)

객체 변수 messages가 for문을 통해 반복하면 각 받은편지함의 1개 메일의 특성을 갖게 됩니다. 따라서 위 코드처럼 ms.Attachments의 의미는 각 메일 1개에 포함되어있는 첨부파일 목록을 의미합니다.

ms.attachments는 첨부파일 목록이므로 다시 아래 코드와 같이 개별 파일에 접근할 수 있게됩니다.

```
#각 메일마다 첨부파일 객체 생성
for  ms  in  messages:
    attachments = ms.Attachments #해당 메일의 첨부파일 목록 객체 설정
        for  onefile  in  attachments: #첨부파일 목록을 다시 개별파일로 접근
            print(onefile)
```

<결과>

```
요청1.xlsx
요청2.xlsx
요청3.xlsx
요청4.xlsx
```

전 과정에서 첨부파일 개수가 총 4개였던 것 기억하시죠?? 그에 맞게 첨부파일 4개가 파일명으로 출력 되었습니다. 위 코드를 응용해서 첨부파일을 저장하는 코드를 작성할 수 있습니다.

```
#각 메일에 접근하는 루프
for  ms  in  messages:

    attachments = ms.Attachments #해당 메일의 첨부파일 객체 설정
    file_num = attachments.count #해당 메일의 첨부파일 개수 확인
    print("현 메일의 파일 건수 : ", file_num)

#파일을 저장하는 루프
    for  i  in  range(1, file_num+1):
        one_file = attachments.Item(i) #메일의 개별 첨부 파일을 객체로 생성
        print(one_file) #파일명 출력

        one_file.SaveASFile(str(one_file)) #파일 명 그대로 저장
```

3-4. Word문서 자동화 (python-docx)

이번 장에서 배워볼 내용은 우리가 정말 자주 사용하는 MS Word를 자동화 할 수 있는 python-docx라는 라이브러리입니다.

앞에서 배우셨던 pyWin32 패키지로도 워드 문서를 다룰 수 있지만 친절한 자료가 부족할 뿐만 아니라 Windows OS 전용이라서 호환성에 아쉬움이 있습니다.

물론 python-docx의 공식문서도 그리 친절하지 않고, 관련 자료도 다른 패키지들에 비해서 적었지만 제가 잘 정리해보았으니 도움이 되시길 바랍니다.

만약 워드를 자동화 할 수 있다면 앞서 배웠던 엑셀 자동화와 함께 응용하여 동일한 내용이지만 받는 사람, 연락처, 회사 등이 다른 경우에 효과적인 업무자동화가 가능할 것으로 예상됩니다.

1) 문서 생성, 저장, 불러오기, 글자 입력

오늘은 python-docx 라이브러리를 설치하는 것부터 시작해서 기본적인 사용법을 익혀보겠습니다.

1. 패키지 설치

아래의 코드를 터미널 창 혹은 주피터 노트북의 셀(cell)에 입력하고 실행시켜주세요. 참고로 해당 패키지는 Windows OS 뿐만 아니라 Mac OS에서도 잘 작동합니다.

```
pip install python-docx
```

2. 패키지 불러오기

python-docx 라이브러리에는 다양한 클래스들이 존재하며, 원하는 기능(스타일, 정렬, 표 등등)에 따라서 import 해주는 것이 조금씩 다릅니다.

아래는 몇가지 예로 든 것 외에도 많은 것들이 있으며, 오늘은 Document 클래스를 이용한 기능들만 알아보겠습니다.

```
# 가장 기본적인 기능(문서 열기, 저장, 글자 쓰기 등등)
from docx import Document

# 문단 정렬
from docx.enum.text import WD_ALIGN_PARAGRAPH

# 문자 스타일 변경
from docx.enum.style import WD_STYLE_TYPE
```

3. 새 워드문서 만들기

위에서 import를 했다면 문서 객체(Object)를 만들어 줄 차례입니다. 참고로 객체란 클래스(Class)를 변수로 선언해줌으로써 만들 수 있습니다.

```
doc = Document()
```

4. 워드문서 저장하기

현재는 그냥 새 워드문서가 파이썬 상에서만 존재하는 상태입니다. 이것을 우리가 사용하는 실제 워드 파일로 만들어주고 싶으면 아래의 코드로 객체를 저장해주어야합니다.

```
# 현재 작업경로에 저장
doc.save('저장하고 싶은 파일명.docx')
```

저장하실 때 특정 위치에 저장하고 싶으시면, 절대경로를 입력해주시면 됩니다.

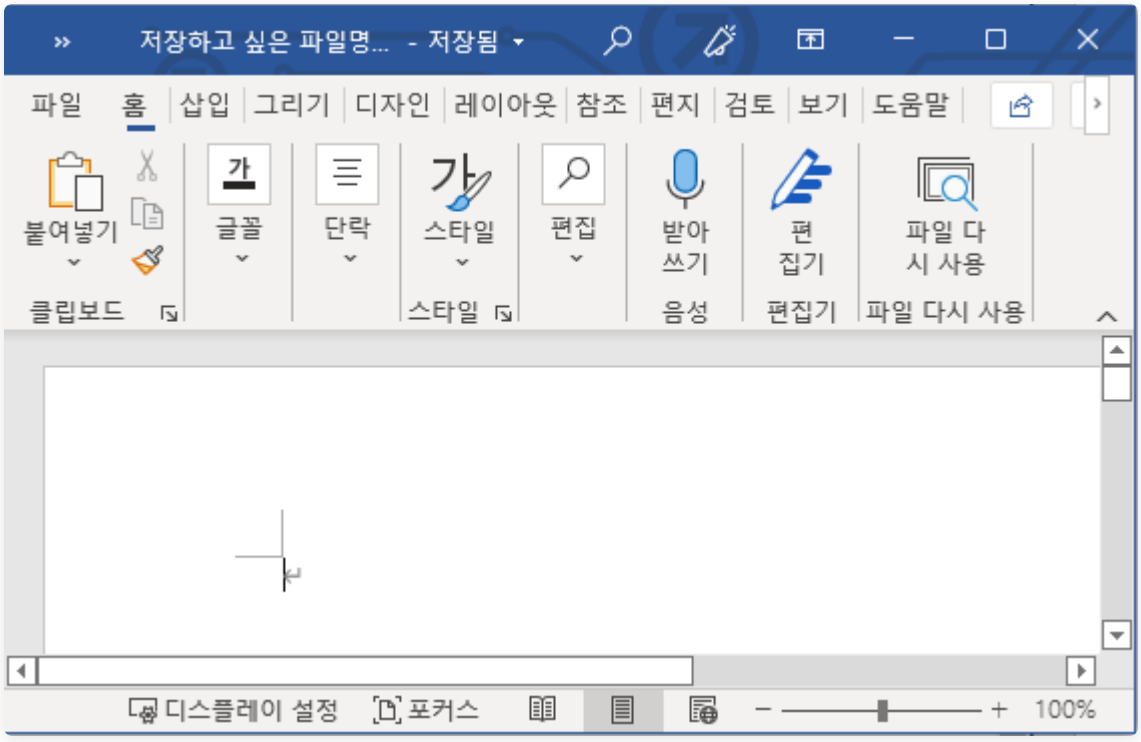
저장된 docx 파일

5. 저장된 워드문서 불러오기

만약 기존의 워드파일을 불러오고 싶으시다면 아래와 같이 코딩해주시면 됩니다.

여기서 '저장하고 싶은 파일명'은 위에서 저장했던 파일명을 그대로 사용한 것입니다.

```
# 절대경로를 이용하는 방법
doc = Document(r'C:\Users\SANGWOO\Desktop\VSCODE\저장하고 싶은 파일명.docx')

# 현재작업 경로에 '파일명.docx'가 위치할 때
doc = Document('저장하고 싶은 파일명.docx')
```



불러온 파일 (실제 출력되지는 않음)

6. 워드문서에 글자 입력하기

워드의 가장 기본적인 기능인 글자를 입력하는 방법을 알아보겠습니다.

1) 제목 넣기

위와 같이 제목을 넣는 방법은 아래의 코드를 사용합니다. 코드와 이미지를 보면서 비교해보세요.

```
# 제목
doc.add_heading('가장 큰 제목 (아래에 밑줄)', level=0)
doc.add_heading('제목 크기, H1', level=1)
doc.add_heading('제목 크기, H2', level=2)
doc.add_heading('제목 크기, H3', level=3)
doc.add_heading('제목 크기, H4', level=4)
doc.add_heading('제목 크기, H5', level=5)
doc.add_heading('제목 크기, H6', level=6)
```

level이 어디까지 가능한지 확인은 해보지 않았으며, 레벨 3 밑으로는 글자 크기가 더 작아지지 않는 것으로 보입니다.

2) 문단(paragraph) 넣기

여기서 말하는 문단이란 그냥 한줄로 쭉 이어지는 글을 말합니다. 즉, 엔터를 치지 않은 상태의 글을 의미합니다.

참고로 제목도 하나의 문단이라고 할 수 있습니다.

```
doc.add_paragraph('여기에 원하는 텍스트를 마음껏 입력하면 됩니다.')
```

3) 문단에 문자 추가하기

문단에 글자를 쭉 이어서 쓸 수도 있지만 특정한 문자를 강조하고 싶거나 다른 스타일을 주고 싶을 때는 add_run( ) 함수를 사용합니다. 참고로 add_paragraph( ) 함수로 문단을 생성하고 내용을 적으면, 자동으로 하나의 run 객체가 문단에 포함되어 만들어집니다.

우선 위에서와 같이 add_paragraph( ) 함수를 사용하되 변수에 담아줍니다.

```
p = doc.add_paragraph('두번째 문단: 여기에 원하는 텍스트를 마음껏 입력하면 됩니다.')
```

변수 p에 .add_run( ) 함수로 글자를 추가해줍니다. 스타일 적용 방법은 아래의 코드를 참고하세요.

```
# 굵은 글씨(Bold) 적용
p.add_run('문단에 굵은 글자 추가').bold = True

# 기울임꼴(Italic) 적용
p.add_run('문단에 이텔릭 글자 추가').italic = True

# 밑줄(Underline) 적용
p.add_run('문단에 밑줄 글자 추가').underline = True
```

add_run( ) 함수는 반드시 paragraph 객체를 기반으로 사용할 수 있다는 것 주의해주세요.

2) 이미지 삽입, 표 삽입

오늘은 지난 시간에 이어서 python-docx 라이브러리로 워드 파일에 이미지와 표를 삽입하는 방법을 공부해보겠습니다.

이번 시간 역시 VScode에서 쥬피터 노트북 파일(.ipynb)을 생성하여 코드를 실행해보겠습니다.

우선 가장 기본이 되는 Document 클래스를 import 해주세요.

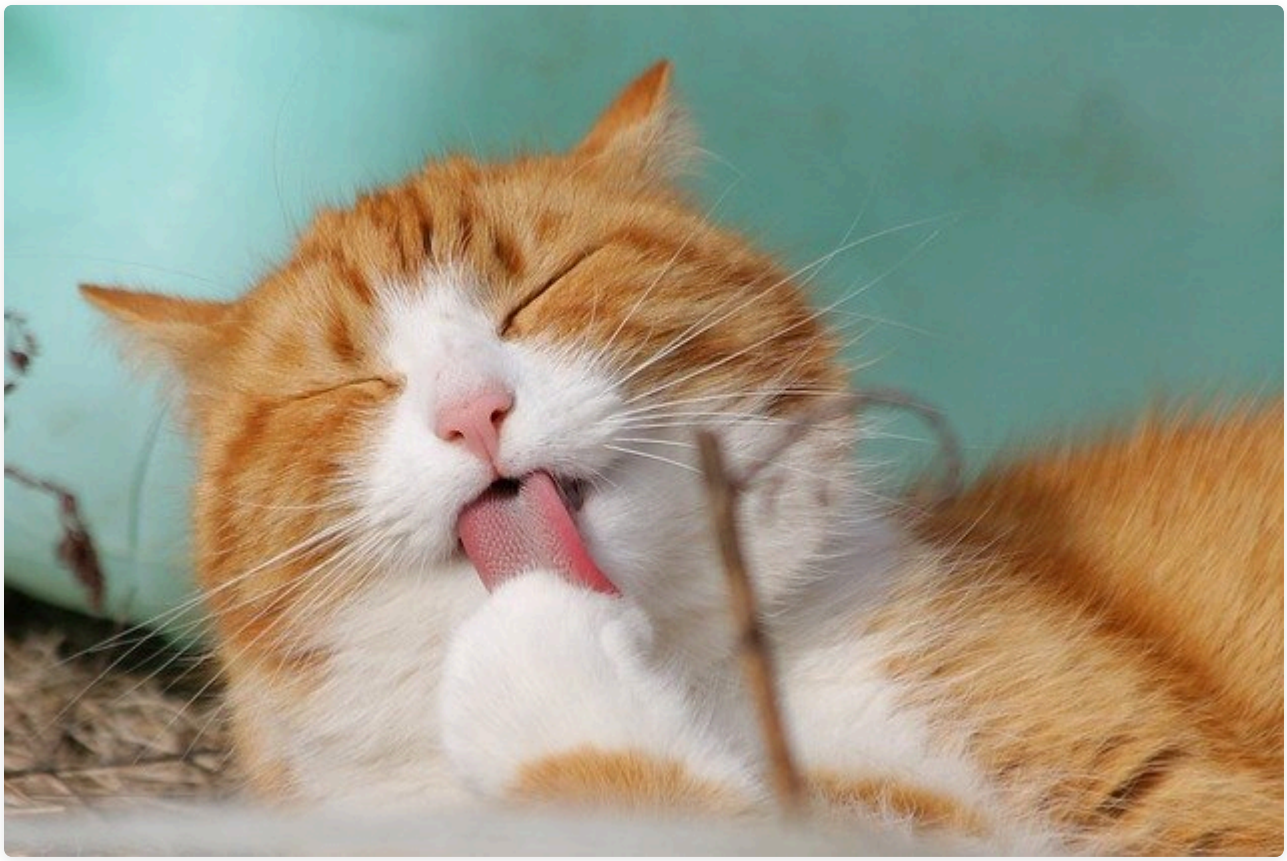
```
from docx import Document
```

1. 이미지 삽입

우선 이미지를 삽입하려면 삽입할 이미지가 있어야겠죠?

이미지 파일을 하나 준비해주세요.

뭘 준비해야 할지 모르겠다면 그냥 화면을 캡쳐해서 jpg 혹은 png 파일로 저장해주세요.



cat.jpg

저는 귀여운 고양이 사진을 워드 문서에 삽입해보겠습니다.

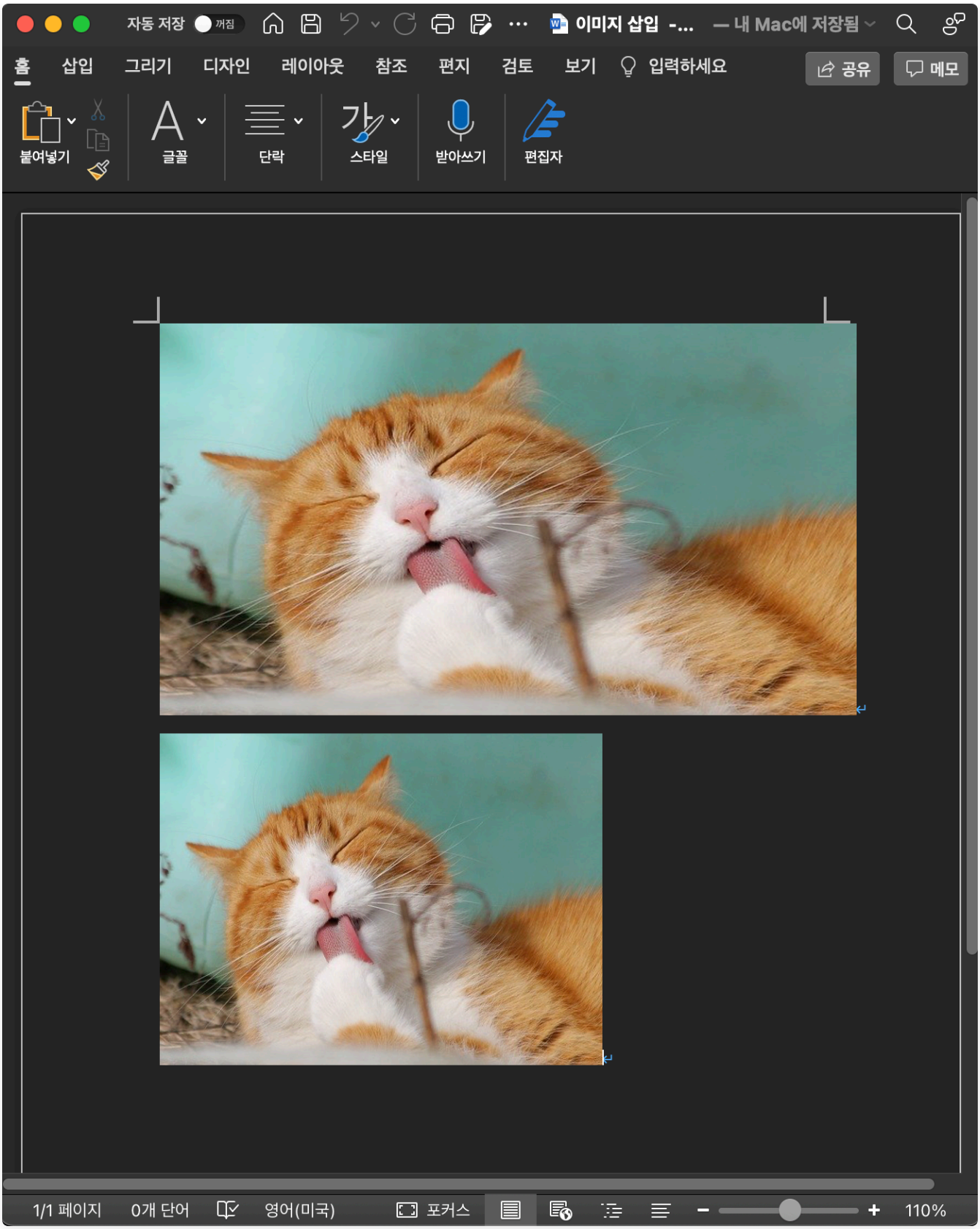
코드 설명은 아래에 주석으로 달아두었으며, 주석을 달지 않은 부분이 잘 이해가 안가신다면 지난 글을 참고해주세요.

<이미지 삽입 코드>

```
# Cm와 Inch 단위를 사용하기 위한 모듈
from docx.shared import Cm, Inches

doc = Document()

# 사진의 크기를 Cm 단위로 설정하여 삽입
doc.add_picture('cat.jpg',width= Cm(16), height= Cm(9))

# 사진의 크기를 Inch 단위로 설정하여 삽입
doc.add_picture('cat.jpg',width= Inches(4), height= Inches(3))

doc.save('이미지 삽입.docx')
```

<실행 결과>

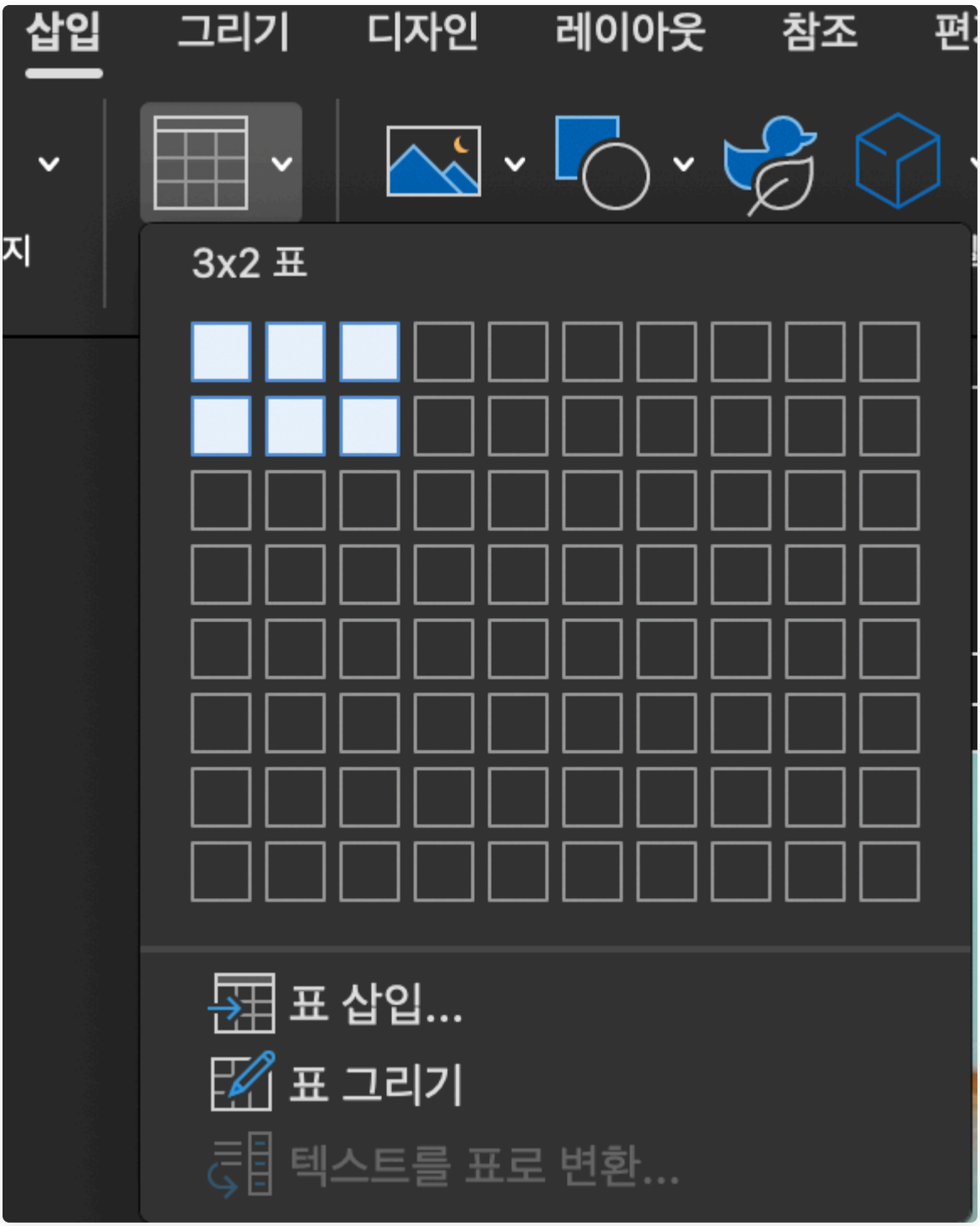
워드 파일을 직접 열어주어야 함.


자동 저장
이미지 삽입 -...
내 Mac에 저장됨
홈
삽입
그리기
디자인
레이아웃
참조
편지
검토
보기
입력하세요
공유
메모
붙여넣기
글꼴
단락
스타일
받아쓰기
편집자
1/1 페이지
0개 단어
영어(미국)
포커스
110%

2. 표 삽입

1) 행과 열 설정 및 표 만들기

우리가 워드를 사용할 때 표를 삽입하려면 가장 먼저 행과 열의 갯수를 설정해주어야 하죠?



Word 사용 시, 일반적인 표 삽입 과정

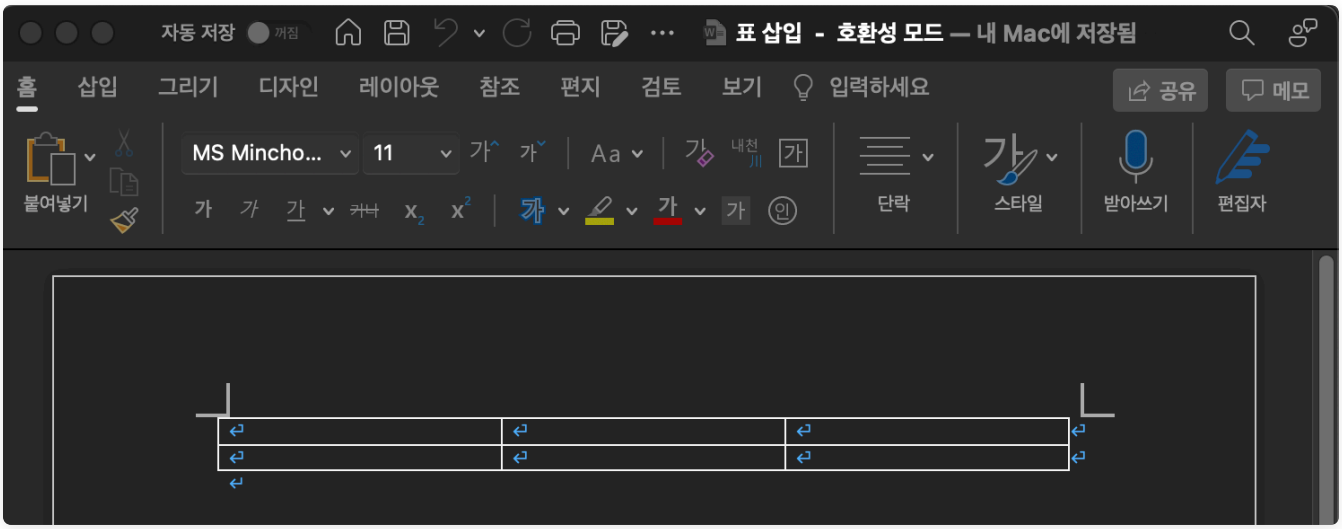
python-docx를 사용할 때에도 마찬가지로 가장 먼저 행(rows)와 열(cols)의 갯수를 설정해줍니다.

이전 시간에 배우셨던 add_paragraph의 개념과 동일하게 변수에 대입해줌으로써 객체화가 가능합니다.

<표 삽입 코드>

```
# 새로운 문서 만들기
doc = Document()

# 2행 3열의 표 만들기
table = doc.add_table(rows = 2, cols = 3)

# 만든 표의 스타일을 가장 기본 스타일인 'Table Grid'로 설정
table.style = doc.styles['Table Grid']

# 저장
doc.save('표 삽입.docx')
```

<실행 결과>



참고로 위에서 표 스타일을 Table Grid로 해주었는데, 이 스타일은 가장 기본적인 스타일입니다.

추후에 문단과 표를 정렬하고 스타일을 변경하는 방법에 대해서 다루어 보겠습니다.

2) 각 셀에 내용 입력

각 셀에는 아래와 같은 인덱싱 방법으로 표에 접근하여, 텍스트를 입력할 수 있습니다.

<표에 텍스트 삽입 코드>

```
# 표의 첫 행을 리스트로 가져오기
first_row = table.rows[0].cells

# 첫 행의 각 열들에 접근해서 값 입력
first_row[0].text = 'a'
first_row[1].text = 'b'
first_row[2].text = 'c'

# 표의 두번째 행을 리스트로 가져온 후, 각 셀에 값 입력
second_row = table.rows[1].cells
second_row[0].text = 'd'
second_row[1].text = 'e'
second_row[2].text = 'f'

# 저장
doc.save('표 삽입.docx')
```

<실행 결과>

3) 행과 열 추가하기

행 추가

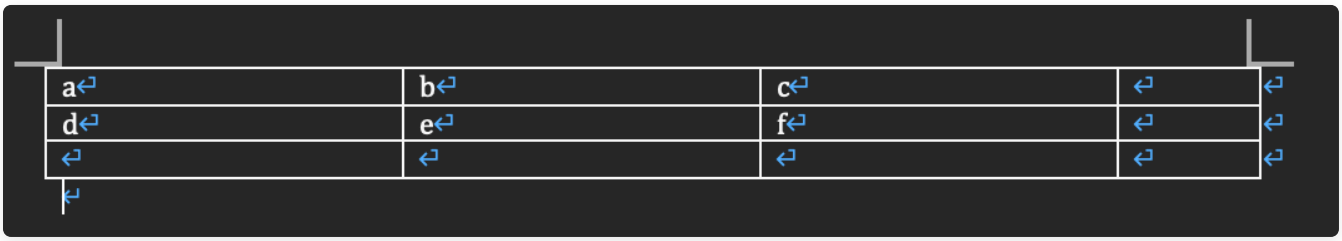
```
row_cells = table.add_row().cells
```

위에서 굳이 변수에 대입해준 이유는 나중에 새로 만든 셀들에 값을 입력할 때, row_cells[0]과 같이 인덱싱하여 텍스트 입력이 가능하기 때문입니다.

열 추가

열을 추가할 때에는 행을 추가할 때와 다르게 반드시 width 값을 꼭 입력해주어야 합니다.

```
col_cells = table.add_column(width=Cm(2)).cells
```



3) 기존문서 내용 읽기

지난 두 시간에 걸쳐서 python-docx 라이브러리로 워드 파일에 글자, 그림, 표를 삽입하는 방법을 배워봤습니다.

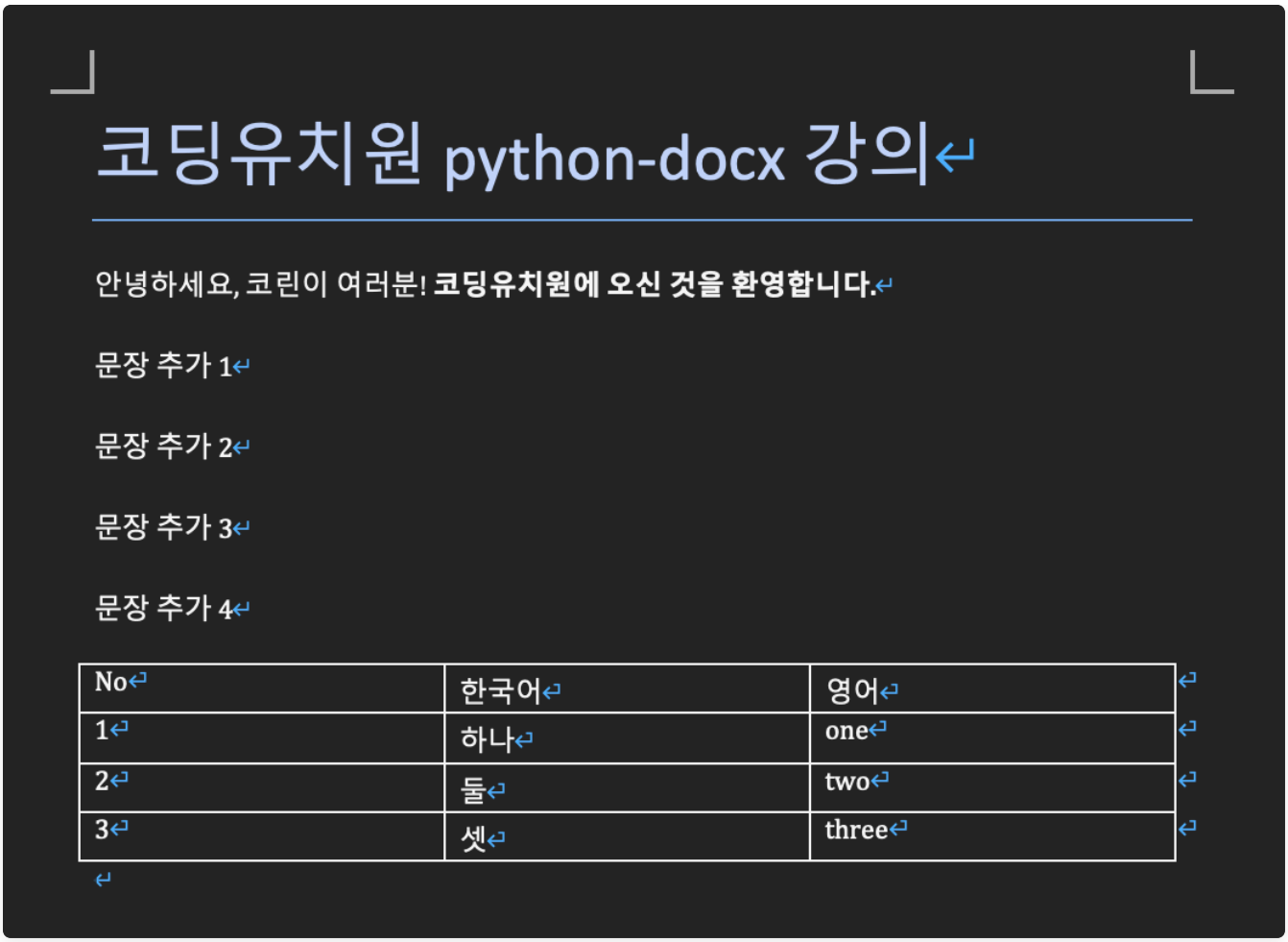
새로운 문서를 만들 때에도 필요하겠지만, 기존 문서의 내용에 무언가 추가하고 변경하기 위해서는 인덱싱(indexing)이 필요합니다.

엑셀에 행(row)과 열(column)의 개념이 있어, 원하는 셀(cell)의 내용을 인덱싱해서 가져올 수 있듯이 워드 파일도 이러한 개념을 잘 파악하고 있어야만이 마음대로 문서를 주무를 수 있답니다.

우선 오늘의 수업을 위해서 아래의 코드를 실행하셔서 '예제 문서.docx'를 만들어주세요.

```
from docx import Document

document = Document()

document.add_heading('코딩유치원 python-docx 강의', level = 0)

p = document.add_paragraph('안녕하세요, 코린이 여러분!')
p.add_run(' 코딩유치원에 오신 것을 환영합니다.').bold = True

document.add_paragraph('문장 추가 1')
document.add_paragraph('문장 추가 2')
document.add_paragraph('문장 추가 3')
document.add_paragraph('문장 추가 4')

records = (
    (1, '하나', 'one'),
    (2, '둘', 'two'),
    (3, '셋', 'three')
)

table = document.add_table(rows=1, cols=3)

# 만든 표의 스타일을 가장 기본 스타일인 'Table Grid'로 설정
table.style = document.styles['Table Grid']

hdr_cells = table.rows[0].cells
hdr_cells[0].text = 'No'
hdr_cells[1].text = '한국어'
hdr_cells[2].text = '영어'
for qty, id, desc in records:
    row_cells = table.add_row().cells
    row_cells[0].text = str(qty)
    row_cells[1].text = id
    row_cells[2].text = desc

document.save('예제 문서.docx')
```

<실행 결과>



1. paragraph 인덱싱

먼저 문단(paragraph)을 인덱싱 하는 방법부터 시작해보겠습니다. 아래의 코드를 실행해보세요.

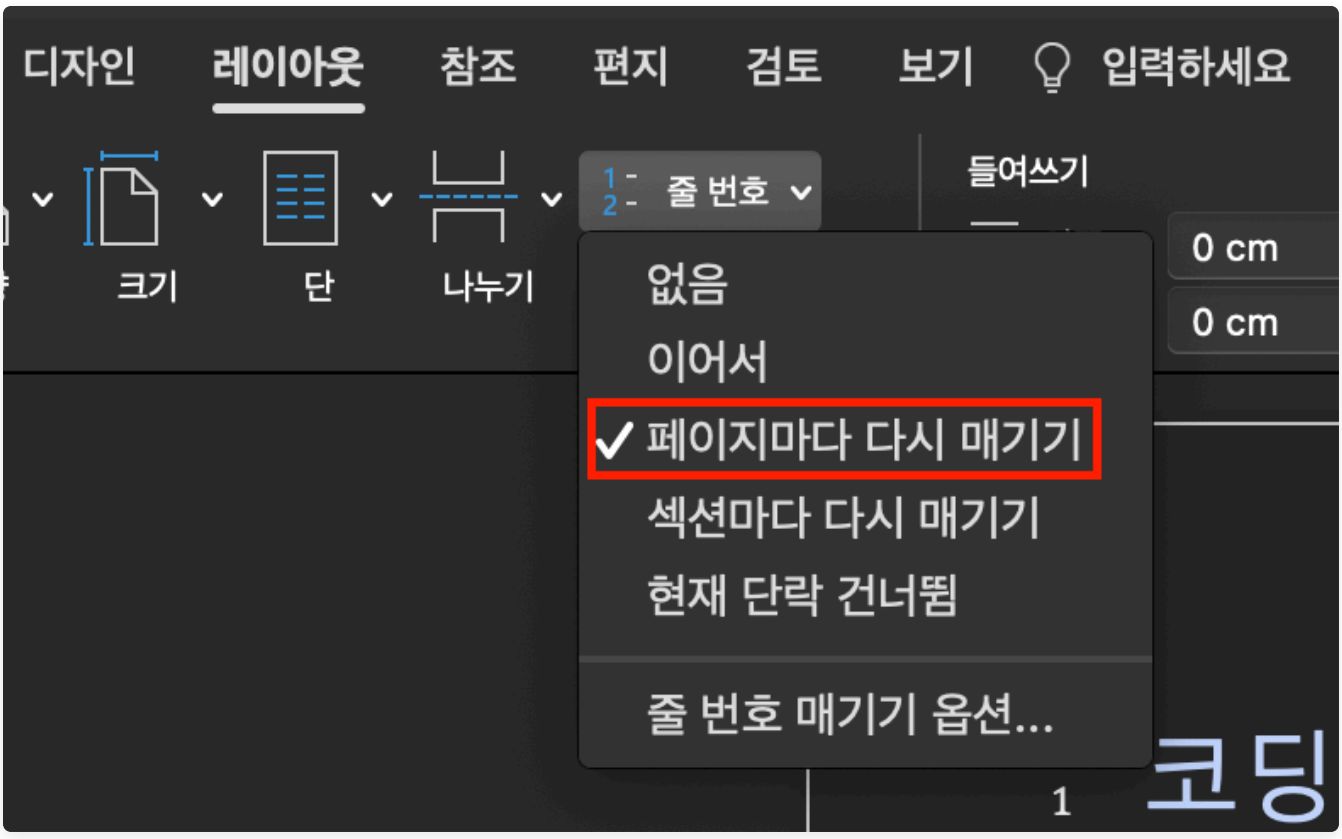
```
doc = Document('예제 문서.docx')

for i, paragraph in enumerate(doc.paragraphs):
    print(str(i+1) + ": " + paragraph.text)
```

그럼 아래와 같이 문단의 번호와 내용들이 출력됩니다.

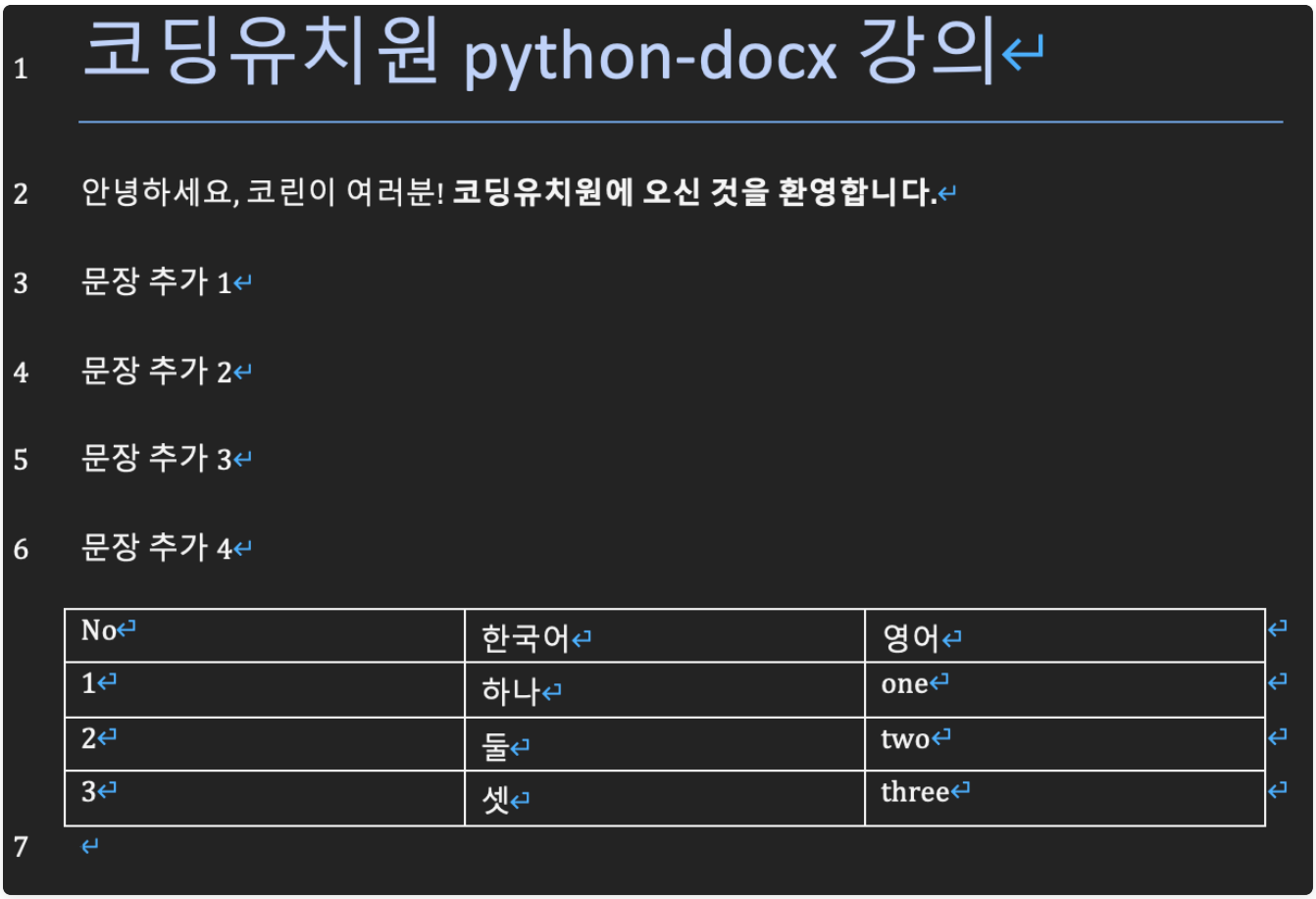
```
1: 코딩유치원 python-docx 강의
2: 안녕하세요, 코린이 여러분! 코딩유치원에 오신 것을 환영합니다.
3: 문장 추가 1
4: 문장 추가 2
5: 문장 추가 3
6: 문장 추가 4
```

> *여기서 잠깐*

워드 문서에서 '레이아웃-줄 번호-페이지마다 다시 매기기'를 클릭하시면 문단 옆에 문단 번호가 표시된답니다.



<문단 좌측에 인덱스가 추가된 모습>



만약 특정 문단에 글을 추가하고 싶으시면 지난 시간에 배웠던 add_run( ) 함수를 이용해 주시면 됩니다.

양식에 내용을 추가할 때 사용하면 되겠죠?

```
# 추가하고 싶은 문단 (*실제 인덱스는 0부터 시작하므로 원하는 문단번호보다 +1 해주어야함에 주의)
p = doc.paragraphs[4]

p.add_run('문단에 글자 추가')
```

그리고 이런 경우가 자주 있을지는 모르겠지만 문단 중간에 새로운 문단을 추가하고 싶으면 아래의 코드를 이용하시면 됩니다.

```
# 3번째 문장 다음에 삽입 (*paragraphs[3]은 4번째 문장을 의미)
dco.paragraphs[3].insert_paragraph_before("문장을 삽입한다.")
```

2. table 인덱싱

위에서 문단을 인덱싱하고 내용을 가져왔을 때, 표(table)은 포함되지 않은 것을 보셨나요?

python-docx에서 표는 따로 분류되어 관리되며, 인덱싱 하는 방법도 조금 다릅니다.

No	한국어	영어
1	하나	one
2	둘	two
3	셋	three

1) 표의 특정 셀의 값 읽는 방법

엑셀과 매우 비슷하면서도 조금 다릅니다. 엑셀로 치면 sheet와 같은 것이 table입니다. 첫번째 시트, 두번째 시트가 있듯이 첫번째 표와 두번째 표를 table[인덱스]로 접근합니다.

rows[0]과 cells[0]은 행과 열로 셀 하나를 인덱싱하는 것이며, paragraphs[0]은 해당 셀의 내용(정확히는 문단)을 인덱싱 하는 것입니다. 만약 한 셀에 엔터를 치고 글자를 입력되어 있다면 paragraphs[1]로 인덱싱 해주어야겠죠?

```
# 문서 안의 모든 표를 가져옴
tables = doc.tables

# 가장 처음 표의 첫행, 첫열의 첫문단 내용 가져오기
tables[0].rows[0].cells[0].paragraphs[0].text
```

당연히 'No' 출력되겠죠? (No.라고 했어야했는데 잘 못한걸 이제 발견했네요)

2) 표의 모든 값에 접근하는 방법

표의 모든 값을 출력할 일이 있을지 모르겠지만 아래와 같이 가능합니다.

```
table = doc.tables[0]

for row in table.rows:
    for cell in row.cells:
        for para in cell.paragraphs:
            print(para.text)
```

이걸 응용해서 표의 모든 내용 중, '하나'를 찾아볼까요? 만약 그게 가능하면 글자를 추가해주거나 수정할 수도 있답니다.

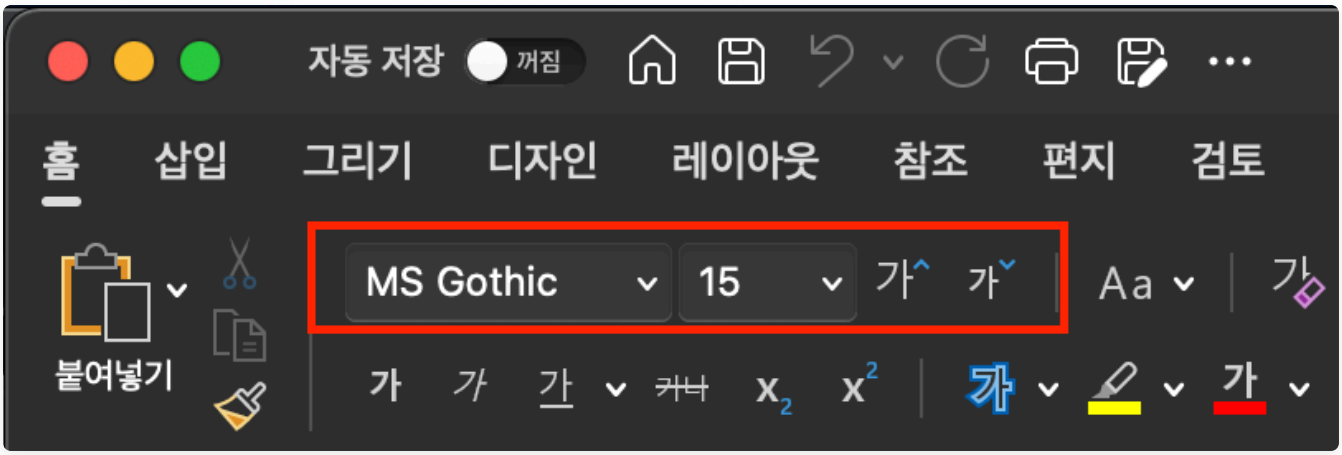
```
# 첫번째 표 인덱싱
table = doc.tables[0]

for row in table.rows:
    for cell in row.cells:
        for para in cell.paragraphs:
            if(para.text == "하나"):
                para.add_run(' ⟵ 찾았다 하나')

# 확인을 위해서 저장하기
doc.save('예제 문서.docx')
```

실행해보니 하나를 잘 찾아서 글자도 잘 추가했네요.

No	한국어	영어
1	하나 <-- 찾았다 하나	one
2	둘	two
3	셋	three

4) 글자 폰트, 크기, 색깔 변경

이번 시간에는 글자의 스타일(폰트,크기,색상)을 적용하는 방법을 공부해보겠습니다.



1. 폰트

우선 폰트를 적용할 때에는 추가적인 import는 필요하지 않습니다.

<먼저 스타일 적용 후에 문단 추가하기>

```
from docx import Document

# 새로운 문서 만들기
doc = Document()

#스타일 적용하기
style = doc.styles['Normal']
font = style.font
font.name = 'Arial'

para = doc.add_paragraph('Some text\n')
```

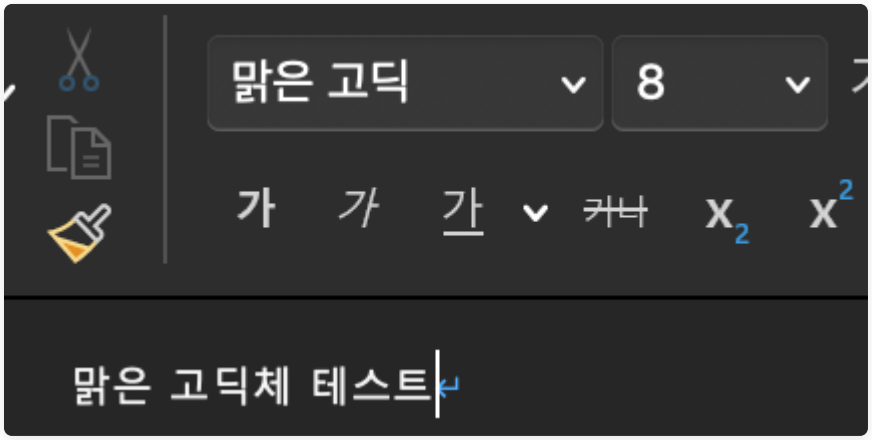
위의 방법은 add_paragraph에도 적용할 수 있고, 위와 아래의 방법모두 add_run으로 쓰인 문장에 적용 가능합니다.

<먼저 문단 입력 후에 스타일 적용하기>

```
para.add_run('코딩유치원에 오신 것을 환영합니다.').bold = True

run = doc.paragraphs[0].runs[0]

run.font.name = 'Arial'
```

여기서 조금 불편한 점은 한글은 위의 방식으로 폰트 적용이 안되어서 다른 방법을 사용해야한다는 것입니다.

```
doc = Document('test.docx')

from docx.oxml.ns import qn

style = doc.styles['Normal']
style._element.rPr.rFonts.set(qn('w:eastAsia'), '맑은 고딕')
style.font.name = '맑은 고딕'
style.font.size = Pt(8)

para = doc.add_paragraph('맑은 고딕체 테스트')

# 저장
doc.save('test.docx')
```



2. 크기

문자 크기를 변경하기 위해서는 우선 아래와 같이 폰트 크기와 관련된 클래스를 import 해주어야 합니다.

```
from docx.shared import Pt
```

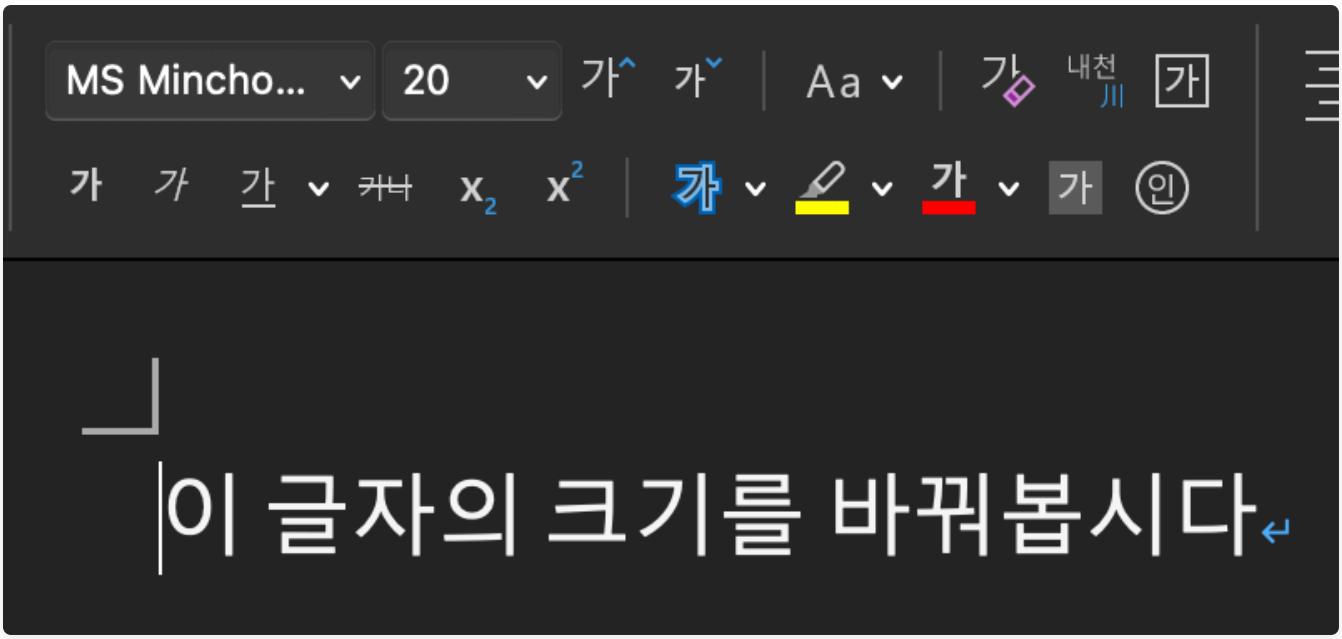
다음으로는 run 객체를 지정해주어야 하는데요. 저는 문단(paragraphs) 안의 모든 run 객체를 바꿔주는 코드를 짜보았습니다.

```
# 첫번째 문단의 문장(run)들을 리스트로 받기
para1 = doc.paragraphs[0].runs

# for 문을 이용해서
for run in para1:
    run.font.size = Pt(20)
```

<동작 확인용 샘플코드>

```
from docx import Document
from docx.shared import Pt

doc = Document()

para = doc.add_paragraph('이 글자의 크기를 바꿔봅시다')

# 첫번째 문단의 문장(run)들을 리스트로 받기
para = doc.paragraphs[0].runs

# for 문을 이용해서
for run in para:
    run.font.size = Pt(20)

# 저장
doc.save('test.docx')
```

<실행 결과>



3. 색깔

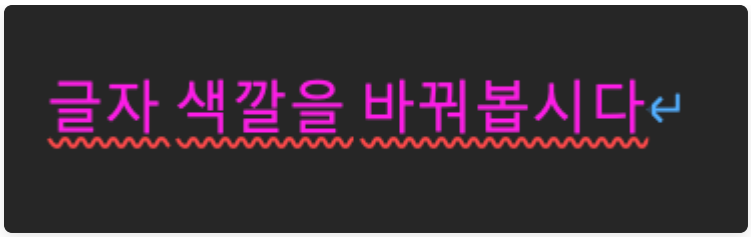
문자 색깔을 변경하기 위해서는 RGBColor 클래스를 import 해주어야 합니다.

```
from docx.shared import RGBColor
```

그 다음엔 문단을 하나 추가하고, 추가한 문단의 첫 문장을 run 객체로 선언해줍니다.

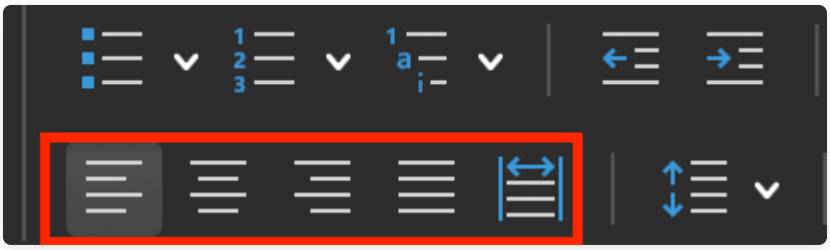
사실 paragraph를 생성하면 자동으로 run 객체를 생성하고 그것으로 문장을 써주는 개념이랍니다. 그래서 아래와 같이 add_run( )으로 문장을 써준 것이 아님에도 para2가 runs[0]를 가질 수 있는 것이죠.

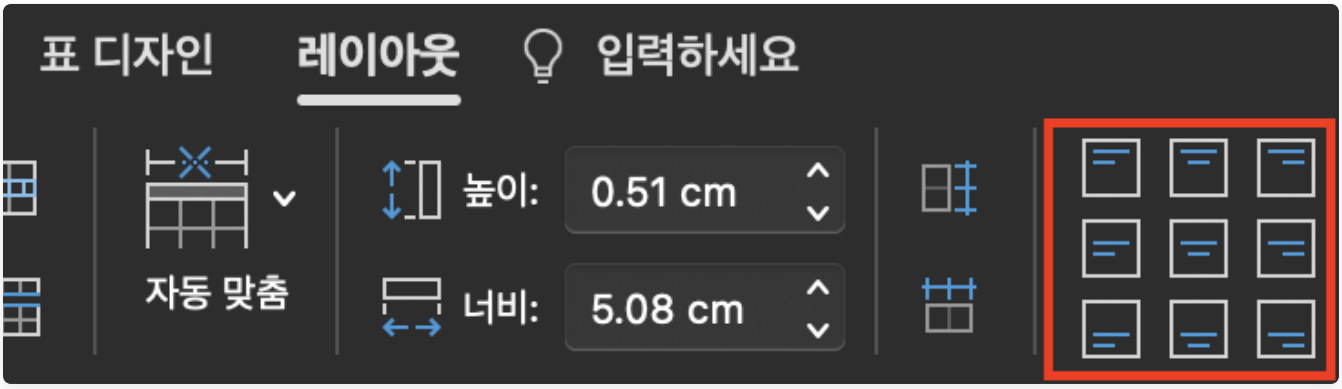
```
para2 = doc.add_paragraph('글자 색깔을 바꿔봅시다')
run = para2.runs[0]
font = run.font

# RGB 컬러를 각각 16진수로 표현 (R, G, B)
font.color.rgb = RGBColor(0×FF, 0×24, 0×E9)

# 저장
doc.save('test.docx')
```


글자 색깔을 바꿔봅시다

5) 문단 정렬, 표 셀 정렬

이번 시간에는 문단을 정렬하는 방법과 표의 셀 안의 내용을 정렬하는 방법에 대해서 공부해보겠습니다.





실습을 위해서 '3) 기존문서 내용 읽기'에서 사용했던 예제 문서를 사용하도록 하겠습니다.

3)을 보지 않고 오신분은 아래의 코드를 실행하시면 예제 문서가 자동으로 생성될 거예요.

```
from docx import Document

document = Document()

document.add_heading('코딩유치원 python-docx 강의', level = 0)

p = document.add_paragraph('안녕하세요, 코린이 여러분!')
p.add_run(' 코딩유치원에 오신 것을 환영합니다.').bold = True

document.add_paragraph('문장 추가 1')
document.add_paragraph('문장 추가 2')
document.add_paragraph('문장 추가 3')
document.add_paragraph('문장 추가 4')

records = (
    (1, '하나', 'one'),
    (2, '둘', 'two'),
    (3, '셋', 'three')
)

table = document.add_table(rows=1, cols=3)

# 만든 표의 스타일을 가장 기본 스타일인 'Table Grid'로 설정
table.style = document.styles['Table Grid']

hdr_cells = table.rows[0].cells
hdr_cells[0].text = 'No'
hdr_cells[1].text = '한국어'
hdr_cells[2].text = '영어'
for qty, id, desc in records:
    row_cells = table.add_row().cells
    row_cells[0].text = str(qty)
    row_cells[1].text = id
    row_cells[2].text = desc

document.save('예제 문서.docx')
```

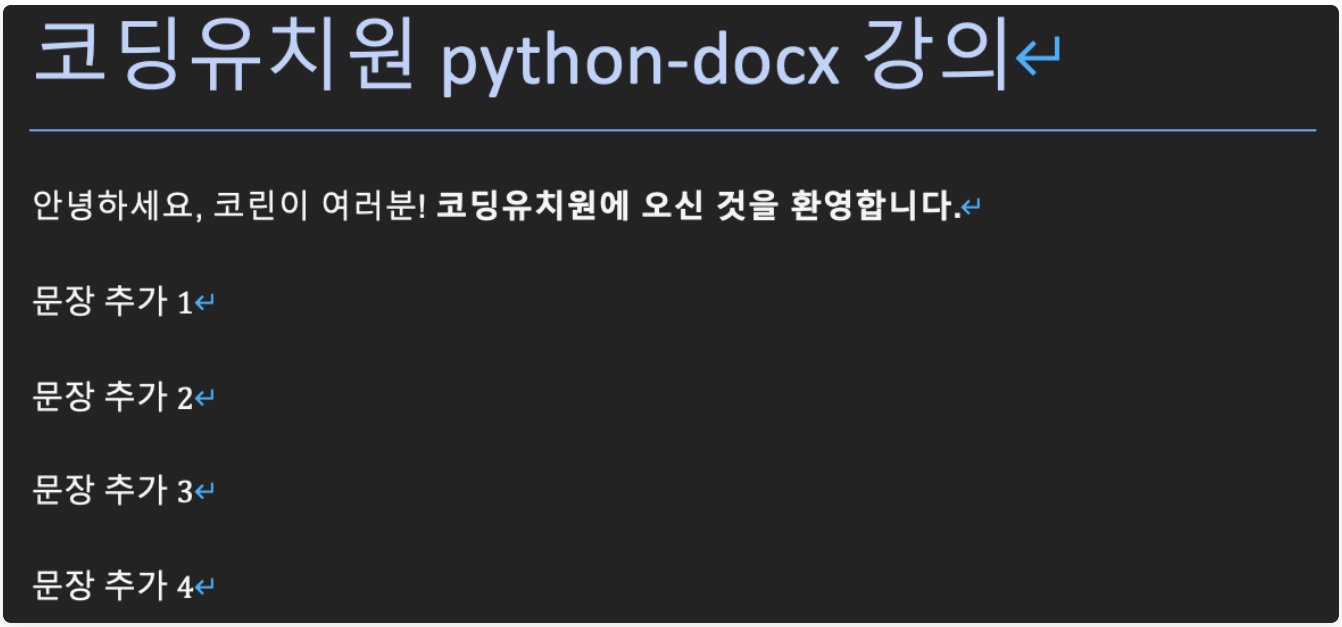
1. 문단 정렬

가장 처음 배울 내용은 우리가 자주 사용하는 왼쪽, 오른쪽, 가운데, 양쪽 정렬을 적용하는 방법입니다.

적용 전과 적용 후의 문단 상태를 비교해보세요. 아시겠지만 paragraphs[0]이 첫번째 문단이므로 두번째 문단부터 적용한 것이랍니다.

```
from docx import Document
from docx.enum.text import WD_ALIGN_PARAGRAPH

document = Document('예제 문서.docx')

# 왼쪽 정렬
paragraph1 = document.paragraphs[1]
paragraph1.alignment = WD_ALIGN_PARAGRAPH.LEFT

# 가운데 정렬
paragraph2 = document.paragraphs[2]
paragraph2.alignment = WD_ALIGN_PARAGRAPH.CENTER

# 오른쪽 정렬
paragraph3 = document.paragraphs[3]
paragraph3.alignment = WD_ALIGN_PARAGRAPH.RIGHT

# 양쪽 정렬
paragraph4 = document.paragraphs[4]
paragraph4.alignment = WD_ALIGN_PARAGRAPH.JUSTIFY

# 텍스트 배분 (글자를 흩어서 배치)
paragraph_last = document.paragraphs[-1]  # 마지막 문단
paragraph_last.alignment = WD_ALIGN_PARAGRAPH.DISTRIBUTE

# 현재 작업경로에 저장
document.save('예제 문서.docx')
```

<기존 예제 문서>



<정렬 후 예제 문서>

2. 셀 정렬

워드에서 표, 정확히는 셀 안의 내용들을 정렬하는 방법을 공부해 보겠습니다.

셀 정렬은 크게 수평 정렬과 수직 정렬로 나뉩니다.

1) 수평 정렬

먼저 예제 문서의 첫번째 열에 수평 정렬(좌측, 가운데, 우측)을 적용해보겠습니다. 공식 문서에서는 table 객체에다가 .alignment를 하라고 되어 있지만 해본 결과 되지 않는 것을 확인했습니다.

꼭 paragraphs 객체에 정렬을 적용해주세요.

```
from docx.enum.table import WD_TABLE_ALIGNMENT

document = Document('예제 문서.docx')

# LEFT : 왼쪽 정렬, CENTER: 가운데 정렬, RIGHT: 오른쪽 정렬
document.tables[0].rows[0].cells[0].paragraphs[0].alignment = WD_TABLE_ALIGNMENT.LEFT
document.tables[0].rows[0].cells[1].paragraphs[0].alignment = WD_TABLE_ALIGNMENT.CENTER
document.tables[0].rows[0].cells[2].paragraphs[0].alignment = WD_TABLE_ALIGNMENT.RIGHT

# 현재 작업경로에 저장
document.save('예제 문서.docx')
```

<실행 결과>

No	한국어	영어
1	하나 <-- 찾았다 하나	one
2	둘	two
3	셋	three

2) 수직 정렬

수직 정렬은 위에서 배운 수평 정렬이 paragraphs 객체에 적용한 것과 다르게 cells 객체에 적용해야 합니다.

(이 부분도 공식문서에 틀리게 작성되어 있어서 알아내느라 고생했네요ㅜㅜ)

```
from docx.enum.table import WD_CELL_VERTICAL_ALIGNMENT

document = Document('예제 문서.docx')

# LEFT : 위쪽 정렬, CENTER: 가운데 정렬, RIGHT: 아래쪽 정렬
document.tables[0].rows[0].cells[0].vertical_alignment = WD_CELL_VERTICAL_ALIGNMENT.TOP
document.tables[0].rows[0].cells[1].vertical_alignment = WD_CELL_VERTICAL_ALIGNMENT.CENTER
document.tables[0].rows[0].cells[2].vertical_alignment = WD_CELL_VERTICAL_ALIGNMENT.BOTTOM

# 현재 작업경로에 저장
document.save('예제 문서.docx')
```

<확인 결과>_수직 정렬 확인을 위해서 표 사이즈 크게 변경

No	한국어	영어
1	하나 <-- 찾았다 하나	one
2	둘	two
3	셋	three

3-5. 파일과 폴더 다루기

경로 관련 라이브러리

1. os

os 모듈은 폴더(디렉토리)를 생성하거나 경로(path)를 다룰 때 사용합니다.

2. shutil

shutil 모듈은 파일을 복사하거나 이동시킬 때 사용합니다.

3. glob

glob 모듈은 패턴을 사용하여 현재 폴더(디렉토리)는 물론 하위 경로의 파일들을 검색할 때 사용합니다.

1) 경로(path)와 디렉토리(directory)

오늘 배울 내용은 바로 경로(path)와 디렉토리(directory)입니다.

자세한 내용으로 들어가기 전에 경로와 디렉토리가 각각 무엇이고, 왜 배워야하는지에 대해 간단히 설명드려보겠습니다.

1. 경로와 디렉토리의 개념

일상 생활에서 경로란 어떤 목적지로 향하는 길이라는 의미로 쓰입니다.

컴퓨터에서 목적지란 디렉토리이며, 찾아가는 길을 경로라고 부릅니다. 우리에게 아주 익숙한 폴더라는 것이 디렉토리라 불리는 것입니다.

참고로 경로는 폴더 뿐만 아니라 파일로 향하는 길을 이르기도 합니다.

이때의 경로는 디렉토리 안의 디렉토리 안의 디렉토리 안의 파일 정도가 되겠죠?

실제 모습은 흔히 보셨겠지만 C:\Users\username\Desktop\filename.txt 이런 식입니다.

참고로, Windows os는 디렉토리를 역슬레쉬(\ )로 나타내고, Mac os에서는 슬레쉬(/)로 나타냅니다.

프로그래밍 공부를 하다보면, 한번쯤 겪게 되는 경로 설정의 오류는 '경로가 없거나 경로가 올바르지 않게 입력되었을 때' 생깁니다.

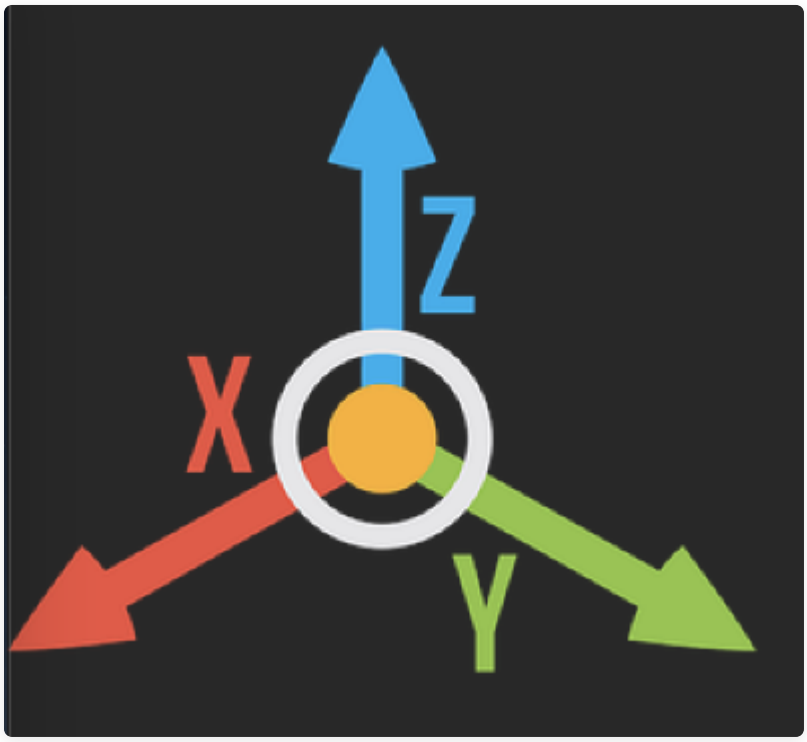
즉, 여기 어떤 파일이 있다고 코드에 잘 적어놓지 않았거나, 그 경로를 따라 갔더니 찾는 파일이 존재하지 않거나 하는 경우입니다.

추후에 프로그램을 자신의 컴퓨터에서 만들어서, 다른 사람에게 배포할 때에도 경로에 대한 고려가 꼭 필요하답니다.

내 컴퓨터에서 실컷 잘돌아가다가도 경로 설정을 신경쓰지 못 하면 다른 컴퓨터에서는 쓸모가 없어지니까요!

2) 절대 경로와 상대 경로

좌표계에도 절대 좌표와 상대 좌표가 있듯이 경로에도 절대 경로와 상대 경로가 있습니다.



1. 절대 경로

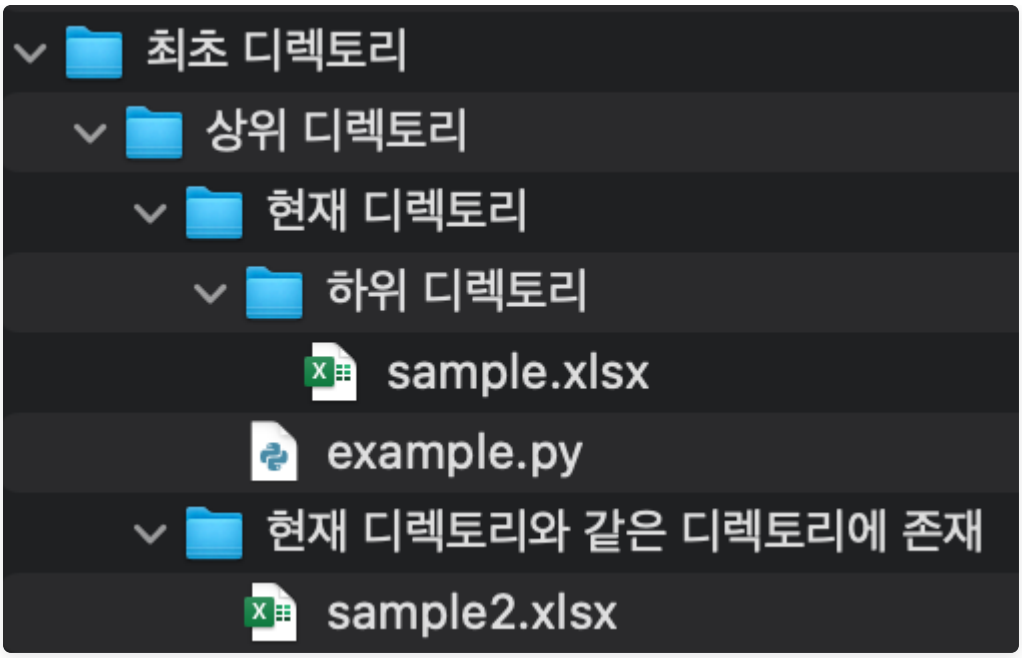
절대적인 기준(최초 디렉토리)를 기준으로 경유한 경로를 모두 기입하는 방식입니다.

Ex) C:\Users\username\Desktop\filename.txt

2. 상대 경로

상대 경로는 최초 디렉토리가 아닌 특정 경로를 기준으로 다른 경로를 표시하는 방식입니다.

주로 현재 작업하고 있는 폴더를 기준으로 합니다.



1) 상대 경로 사용법

예를 들어 **현재 디렉토리에서 제가 example.py라는 파이썬 파일에 코드를 짜고 있다고 생각해봅시다.**코드를 짜다보니 하위 디렉토리에 위치하는 sample.xlsx를 참조해야하는 상황이 발생했습니다.

이때 상대 경로의 개념을 이용하면 **./하위 디렉토리/sample.xlsx** 라고 써주면 됩니다. 즉, 여기서 사용된 **dot(.)**은 현재 디렉토리가 어디에 위치하던 **현재 파이썬 파일이 위치한 디렉토리의 경로**를 뜻합니다. 여기서는 dot(.)의 의미는 '최초 디렉토리/상위 디렉토리/현재 디렉토리'가 되겠죠!

추가적으로 설명드리면, **dot 두 개(..)**는 **현재 디렉토리의 상위 디렉토리**를 뜻합니다.

만약 example.py에서 sample2.xlsx를 상대 경로를 이용해서 참조하고 싶다면 이렇게 하시면 됩니다.

'./../현재 디렉토리와 같은 디렉토리에 존재/sample2.xlsx'

그러니깐, 현재 경로(.)를 기준으로 상위 디렉토리(/..)로 올라간 후, '현재 디렉토리와 같은 디렉토리에 존재'라 이름 붙인 디렉토리로 내려가서 sample2.xlsx를 참조하는 것입니다.

2) 상대 경로 사용하는 이유

그렇다면 절대 경로도 있는데 왜 굳이 상대 경로는 왜 사용하는걸까요?

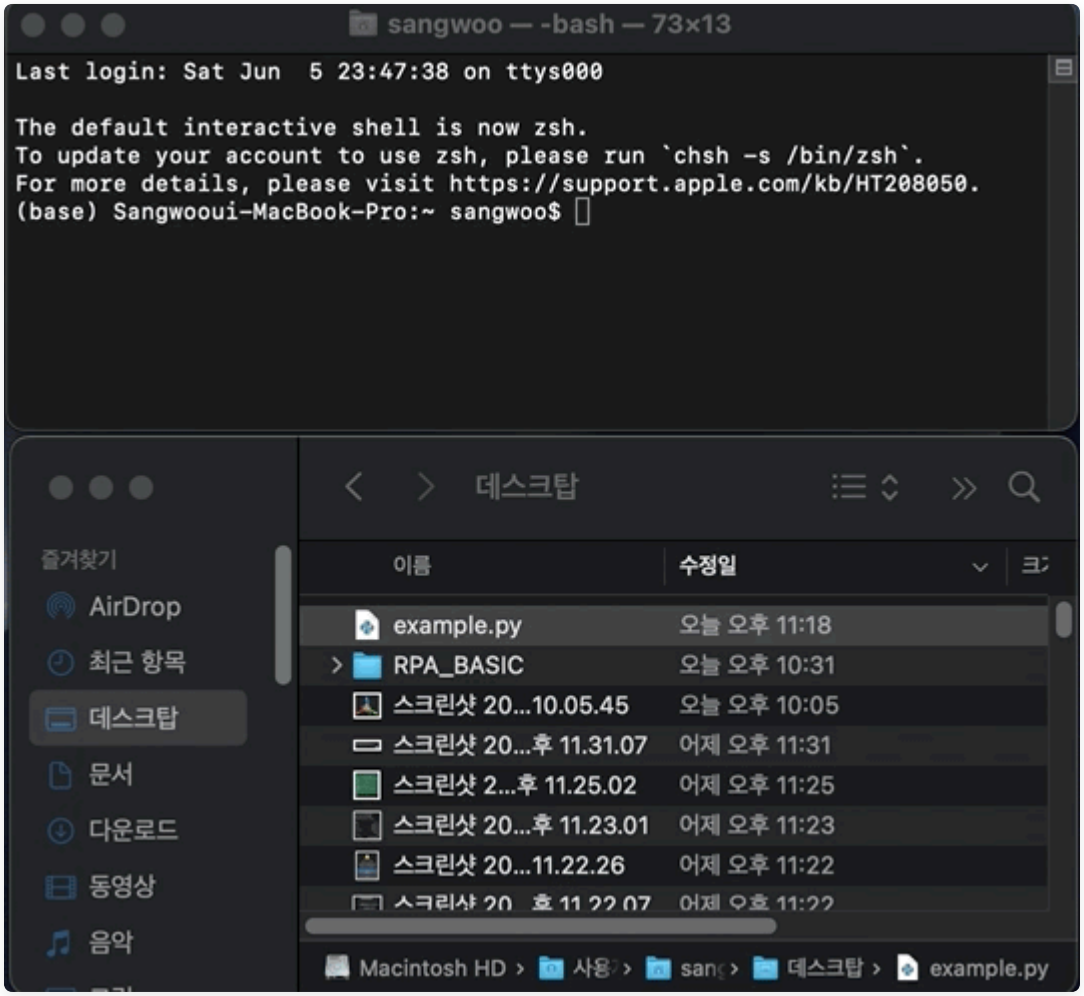
예를들어 filename.txt의 경로가 자주 바뀌거나 최초 디렉토리가 서로 다른 Windows와 Mac OS 모두에서 작동해야 하는 프로그램을 만드는 경우엔 그때 그때 경로를 다시 작성하거나 OS 별로 따로 관리해야 하는 것은 너무 비효율적인 일입니다.

그래서 이럴 때는 절대 경로가 아닌 상대 경로를 이용해 주어야 합니다.

3) 절대 경로 가져오는 꿀팁

window는 cmd, mac은 terminal에 경로가 필요한 파일이나 폴더를 드래그해서 가져다 놓으시면 됩니다.

만약 vscode와 같은 ide를 사용하시면 그 곳의 터미널 창에다가 끌어다 놓으셔도 되니깐 유용하게 사용하시면 좋겠습니다.



---

3) os 모듈

1. os 모듈 불러오기

os 모듈은 기본적으로 설치되어 있는 파이썬 내장 라이브러리에 포함된 모듈입니다. 참고로 파이썬 내장 라이브러리에는 우리가 자주 사용하는 sys, time, random 모듈도 있답니다.

이미 설치되어 있기에 간단하게 import 해주면 되겠습니다.

```
import os
```

2. os 모듈의 함수들

아래의 os 모듈 공식 문서를 보시면 os 모듈에는 정말 많은 함수들이 내장되어 있습니다.

https://docs.python.org/ko/3/library/os.html

수 십 가지의 함수를 모두 공부하는 것은 너무나도 비효율적인 일이니, 제가 생각하기에 가장 많이 쓸만한 함수만 골라서 알려드리겠습니다.

1) os.getcwd( )

현재 작업중인 경로를 문자열로 돌려주는 getcwd( ) 함수입니다. 저의 경우는 실행한 파이썬 파일이 위치한 VSCODE BASIC 이라는 디렉토리(폴더)의 경로를 출력해주는 것을 확인할 수 있습니다.

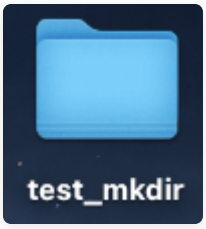
```
print(os.getcwd())

<결과>
>>> /Users/sangwoo/Desktop/VSCODE BASIC
```

2) os.mkdir( )

괄호 안에 '원하는 경로' + '/새로운 디렉토리'를 써주면 새로운 디렉토리가 생겨납니다. 만약 이미 있는 디렉토리를 생성해주려고 하면 에러가 발생하니 주의하세요.

```
os.mkdir("/Users/sangwoo/Desktop/test_mkdir")
```

<결과>

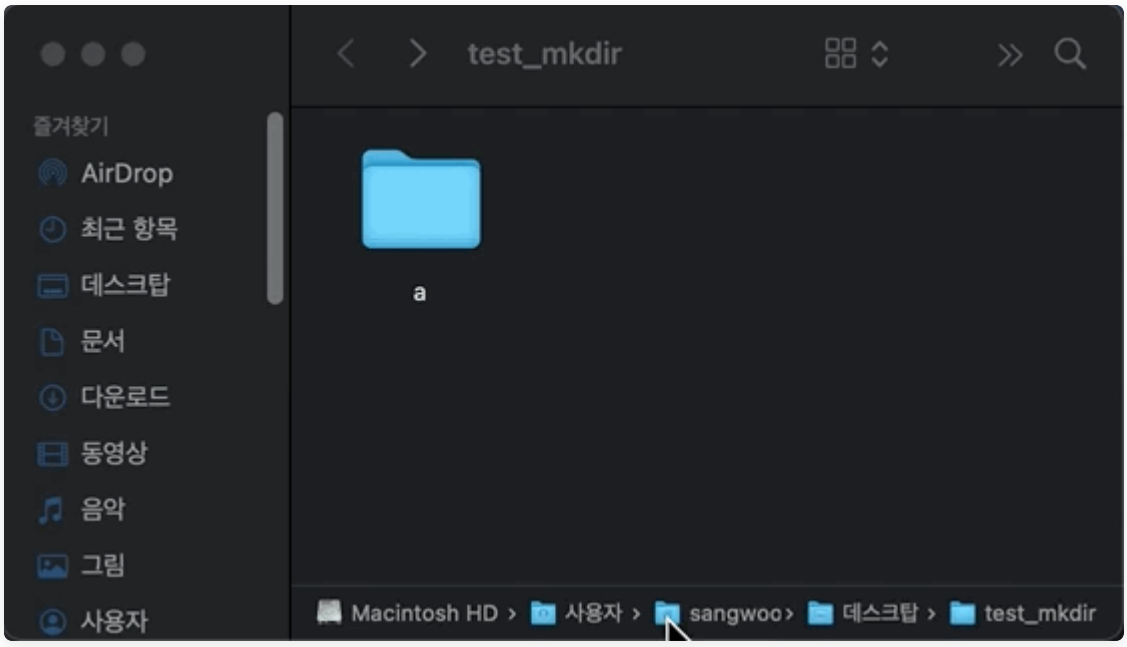


3) os.makedirs( )

os.mkdir( )과 비슷하지만 조금 다릅니다. 뭐랄까.. 디렉토리를 연달아서 적어주면 줄줄이 소세지 처럼 디렉토리가 생성됩니다.

```
os.mkdir("/Users/sangwoo/Desktop/test_mkdir/a/b/c")
```

<결과>

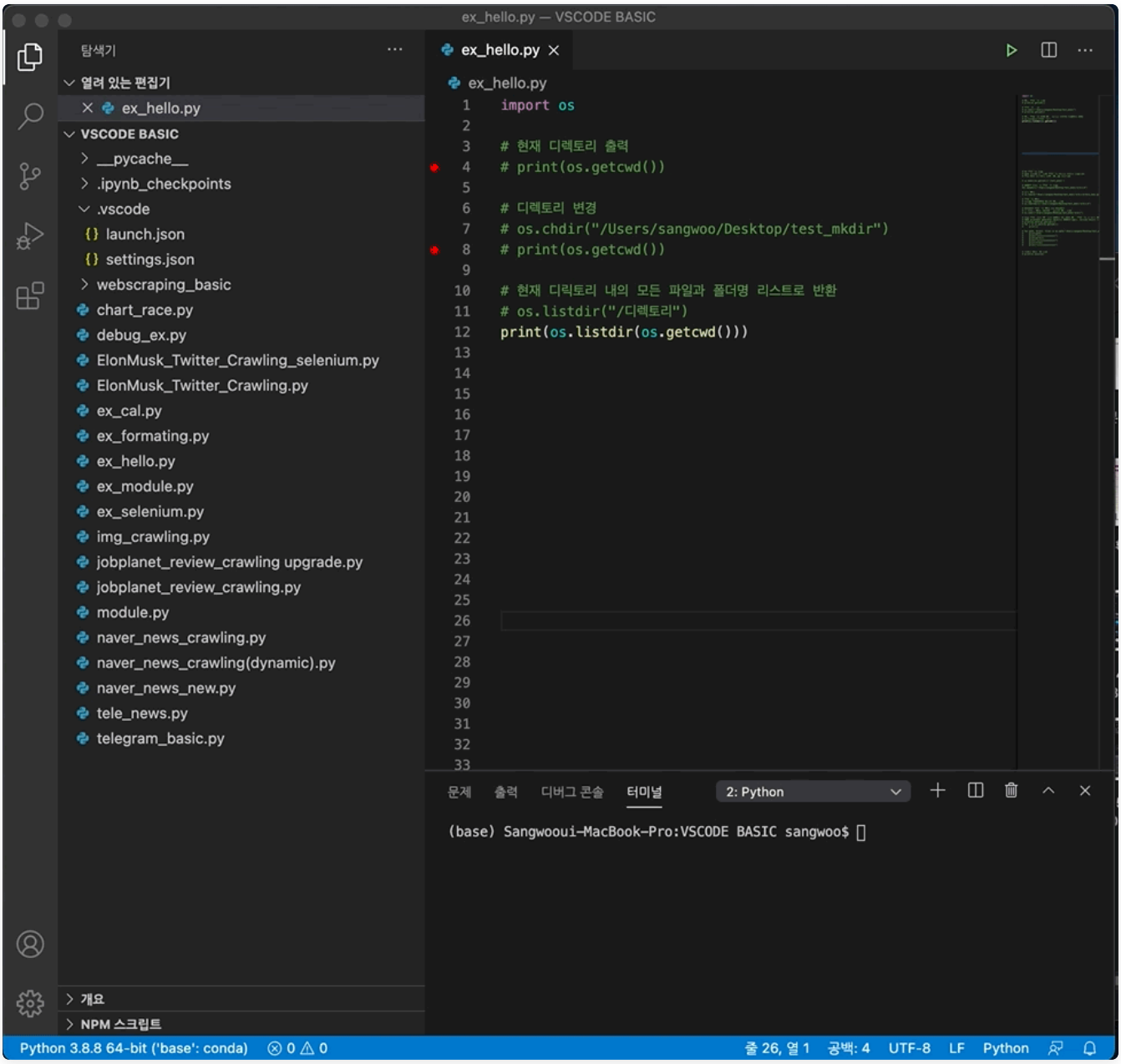


4) os.listdir( )

정말 많이 사용하는 함수인 해당 경로에 위치하는 파일, 디렉토리 이름(확장자 포함)을 리스트로 돌려주는 함수입니다.

```
# os.listdir("/디렉토리")

# 현재 작업 경로에 있는 파일, 디렉토리 리스트 얻기
print(os.listdir(os.getcwd()))
```

좌측의 실제 디렉토리 목록들이 터미널에 출력되는 것을 보실 수 있습니다. 가만보니 .DS_Store라는 이상한 친구가 끼어 있는데 뭔지 나중에 알아봐야겠네요.



5) os.chdir( )

원하는 경로를 현재 작업 경로로 변환해주는 함수입니다. 함수를 사용하기 전과 후의 경로 차이가 나는 것을 확인할 수 있습니다.

```
print(os.getcwd())
os.chdir("/Users/sangwoo/Desktop/")
print(os.getcwd())

<결과>
>>> /Users/sangwoo/Desktop/RPA BASIC
>>> /Users/sangwoo/Desktop
```

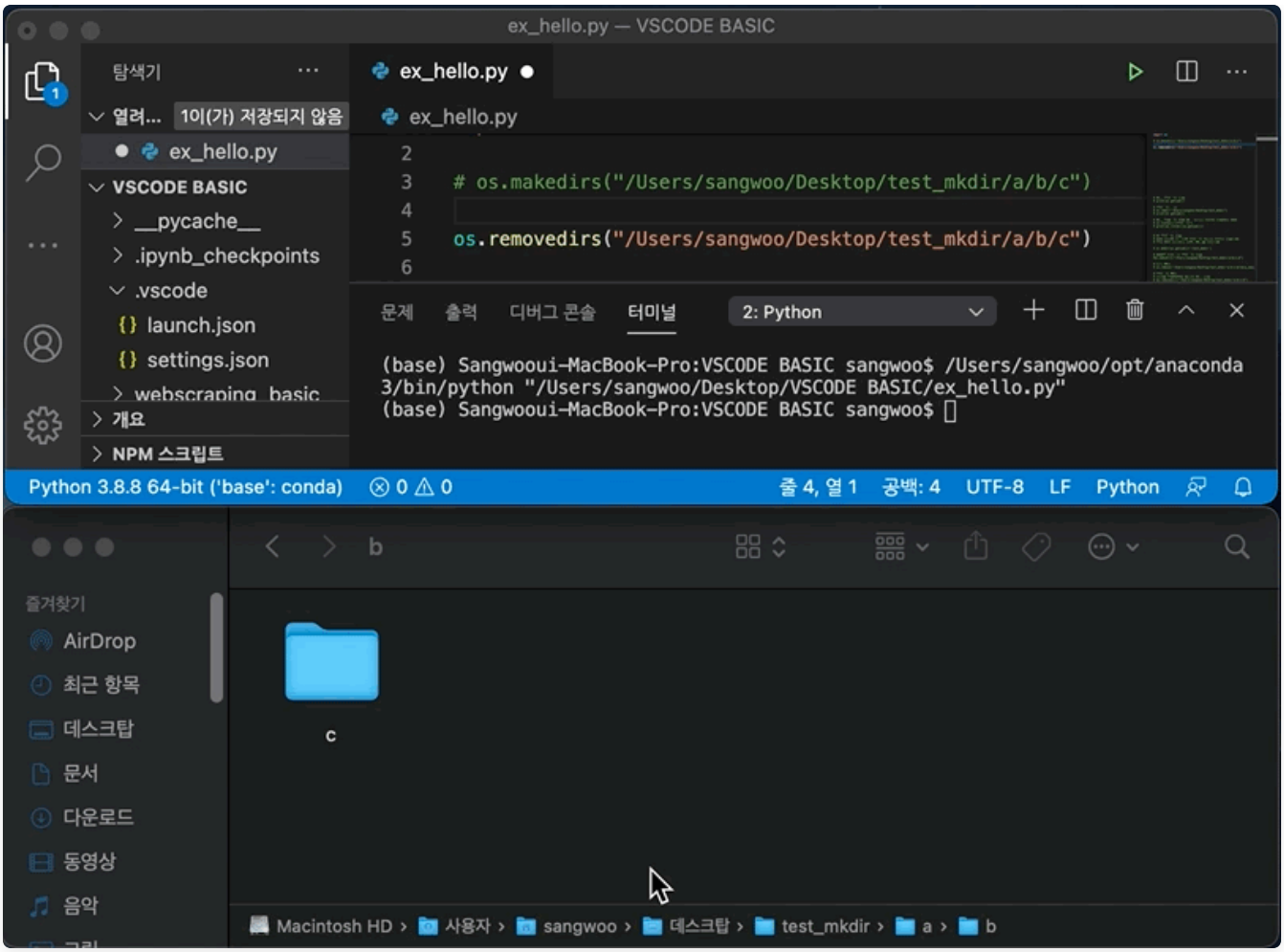
6) os.removedirs( )

함수 이름에서 알 수 있듯이 디렉토리를 제거하는 함수입니다. 만들 때와 같이 제거하고자 하는 디렉토리의 경로를 써주시면 됩니다.

만약 경로가 열려있거나 존재하지 않으면 에러가 발생하니 주의하세요.

```
os.removedirs("/Users/sangwoo/Desktop/test_mkdir/a/b/c")
```

<결과>



7) os.remove( ) / os.unlink( )

위의 removedirs( ) 함수가 디렉토리를 삭제하는 함수였다면 이 함수는 파일을 삭제하는 함수입니다. 두 함수 모두 같은 기능을 합니다.

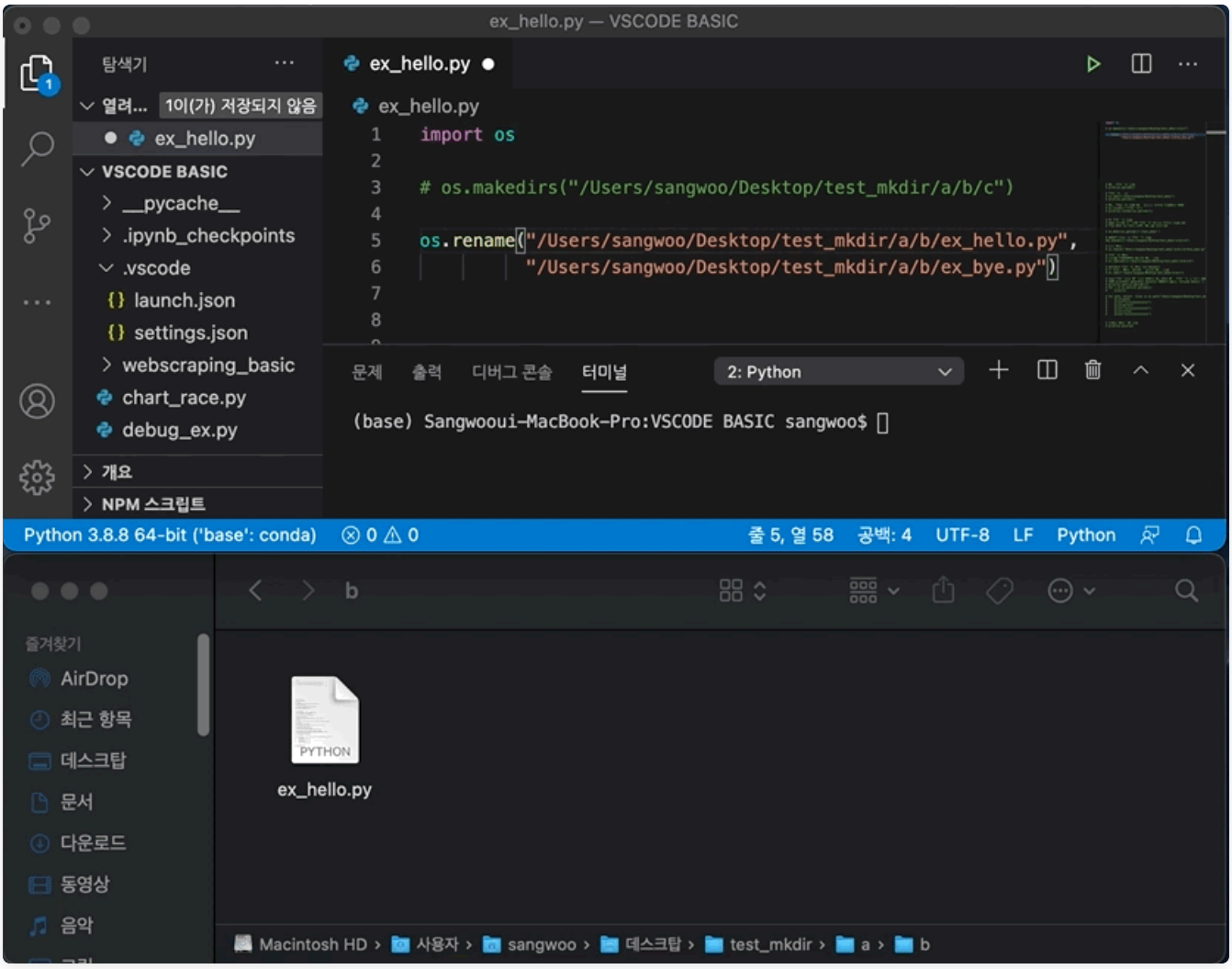
```
os.remove("/Users/sangwoo/Desktop/test_mkdir/a/b/ex_hello.py")
```

<결과>

ex_hello.py 사라짐. (귀찮아서 결과 사진 안올린거 아님.. 암튼 아님..)

8) os.rename(path 1, path 2)

파일의 이름을 바꿔주는 함수입니다. 기존 파일의 경로와 바꾸고자 하는 파일명이 들어간 경로를 차례대로 써주시면 되겠습니다.


ex_hello.py — VSCODE BASIC
탐색기
열려... 1이(가) 저장되지 않음
ex_hello.py
VSCODE BASIC
__pycache__
.ipynb_checkpoints
.vscode
launch.json
settings.json
webscraping_basic
chart_race.py
debug_ex.py
개요
NPM 스크립트
ex_hello.py
1 import os
2
3 # os.makedirs("/Users/sangwoo/Desktop/test_mkdir/a/b/c")
4
5 os.rename("/Users/sangwoo/Desktop/test_mkdir/a/b/ex_hello.py",
6 "/Users/sangwoo/Desktop/test_mkdir/a/b/ex_bye.py")
7
8
문제 출력 디버그 콘솔 터미널
2: Python
(base) Sangwooui-MacBook-Pro:VSCODE BASIC sangwoo$
Python 3.8.8 64-bit ('base': conda)
⊗ 0 ⚠ 0
줄 5, 열 58 공백: 4 UTF-8 LF Python
b
즐겨찾기
AirDrop
최근 항목
데스크탑
문서
다운로드
동영상
음악
PYTHON
ex_hello.py
Macintosh HD › 사용자 › sangwoo › 데스크탑 › test_mkdir › a › b

4) glob 모듈

glob 모듈은 패턴을 사용하여 현재 폴더(디렉토리)는 물론 하위 경로의 파일들을 검색할 때 사용합니다.

1. 모듈 불러오기

glob 모듈을 불러오는 방법은 다음과 같습니다.

```
import glob
```

2. 모듈 사용법

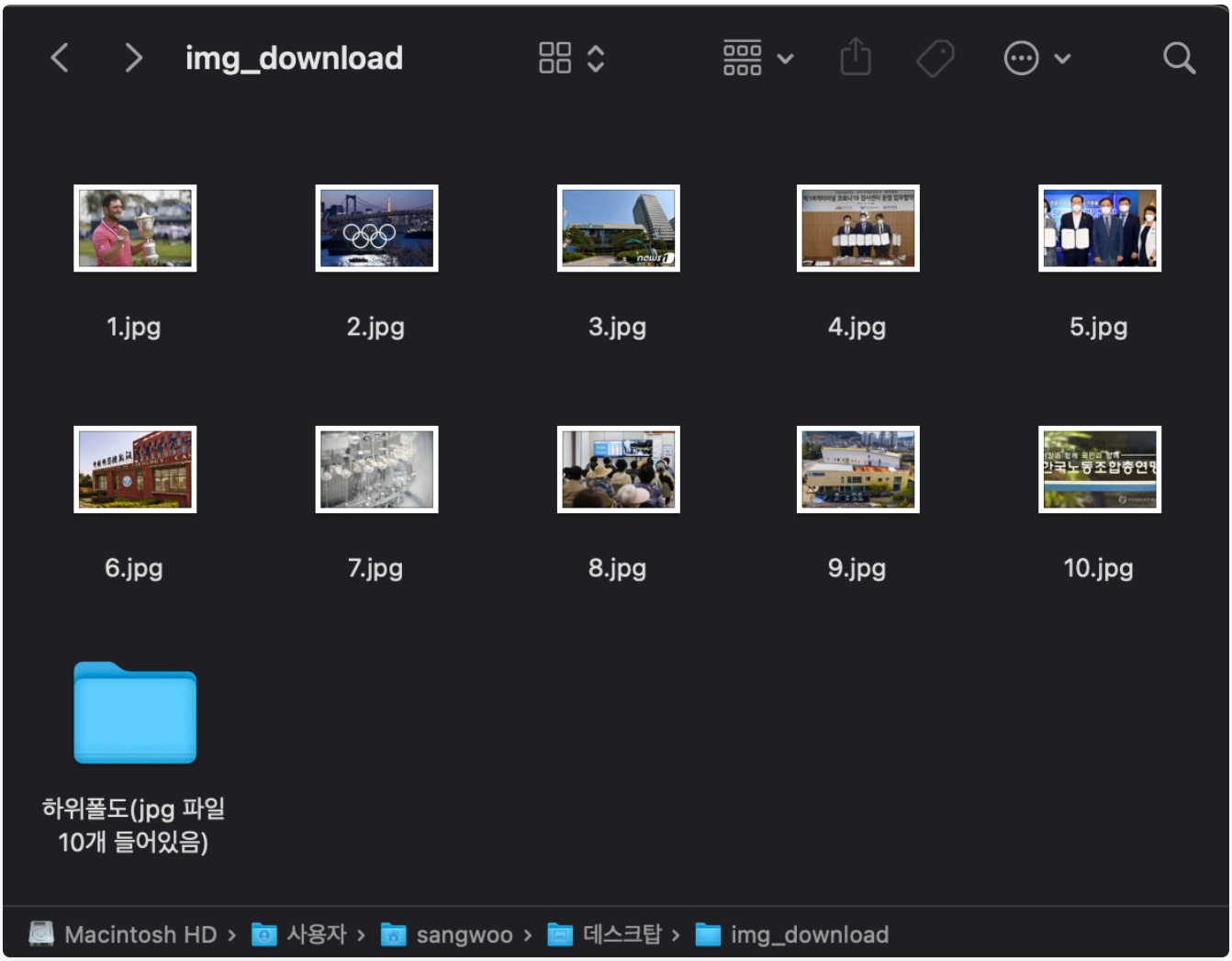
glob 모듈을 불러왔으니 이제 사용법을 알아보겠습니다.

복잡했던 os 모듈과 다르게 glob 모듈은 함수 하나만 알아두시면 됩니다. 바로 glob 함수입니다. 말장난 같지만 glob.glob( )로 사용한답니다.

```
# 확장자를 정하지 않으려면 '경로/*.*'로 입력해주면 됨

glob.glob('참조하길 원하는 폴더의 경로/*.확장자')
```

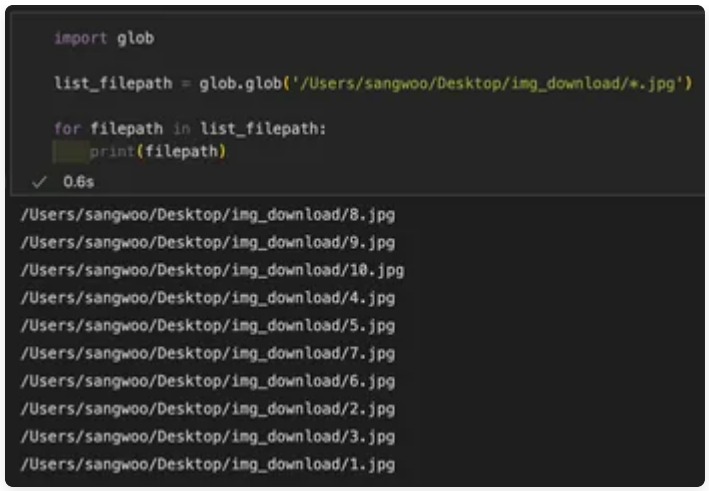
간단한 예시로 원하는 폴더의 모든 jpg 파일을 찾아서 리스트로 받아보겠습니다.

참조하려는 폴더는 아래와 같이 jpg 파일이 10개 들어있는 폴더입니다.




```
import glob

list_filepath = glob.glob('/Users/sangwoo/Desktop/img_download/*.jpg')

for filepath in list_filepath:
    print(filepath)
```

<실행 결과>



하위 폴더까지 참조하고 싶을 때

다시 우리가 참조한 폴더를 잘 살펴보면 하위 폴더에 jpg 파일이 10개 더 들어있음에도 딱 해당 폴더까지만 참조하고 더이상 가져오지 않은 것을 확인 하실 수 있습니다.

만약 하위 폴더까지 모두 참조하고 싶을 때는 아래와 같이 코딩 해주셔야합니다.

```
import glob

list_filepath = glob.glob('/Users/sangwoo/Desktop/img_download/**/*.jpg', recursive=True)

for filepath in list_filepath:
    print(filepath)
```

여기서 주의할 점은 꼭 recursive=True를 넣어주셔야 한다는 것입니다!

이걸 안넣어주고 단순히 `파일경로/**/*/jpg` 로 하시면 하위 폴더만 참조하게 됩니다.

5) shutil 모듈

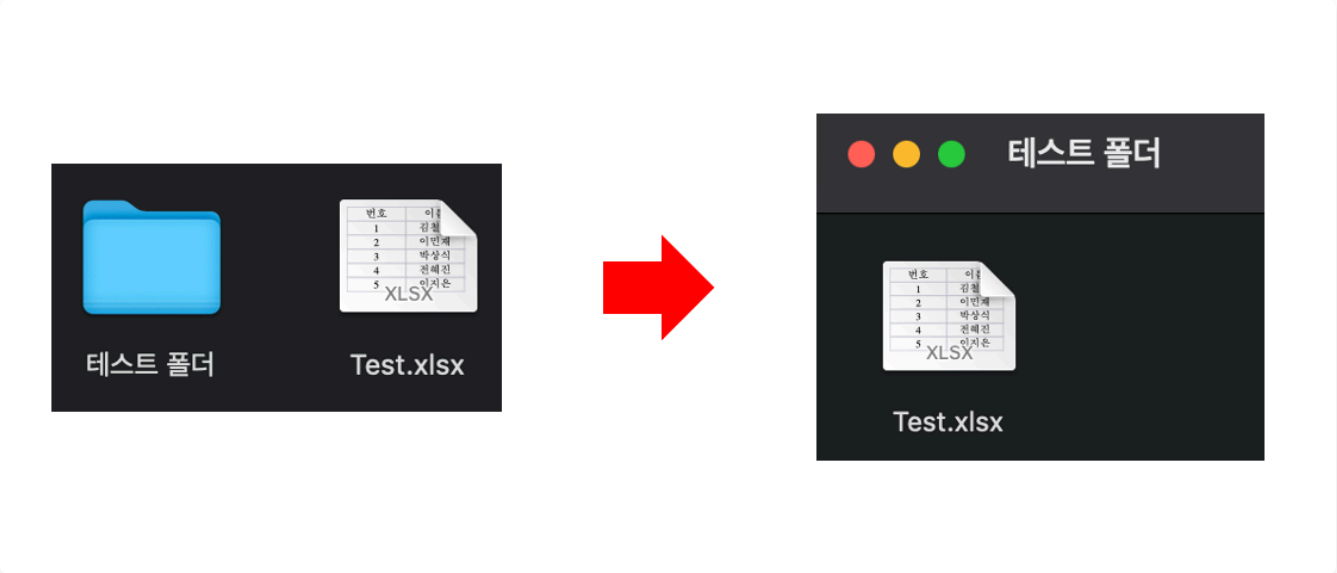
오늘은 파일과 폴더를 이동하거나 복사하고 싶을 때 사용하는 shutil 모듈에 대해서 공부해 보겠습니다.

1. 모듈 불러오기

우선 shutil 모듈은 따로 설치할 필요가 없이 바로 import 해주시면 됩니다. 지난 시간에 배웠던 os 모듈과 같이 파이썬 내장 라이브러리에 포함되어 있는 모듈이기 때문입니다.

```
import shutil
```

2. 파일을 특정 폴더로 이동하기_move( )



1) 상대경로 방식

```
# 방법 1 - 파일과 폴더가 현재 작업경로에 있는 경우만 가능
shutil.move('이동시킬 파일명.확장자', '파일을 이동시킬 폴더명')

# 예시
shutil.move('Test.xlsx', '테스트 폴더')
```

2) 절대경로 방식

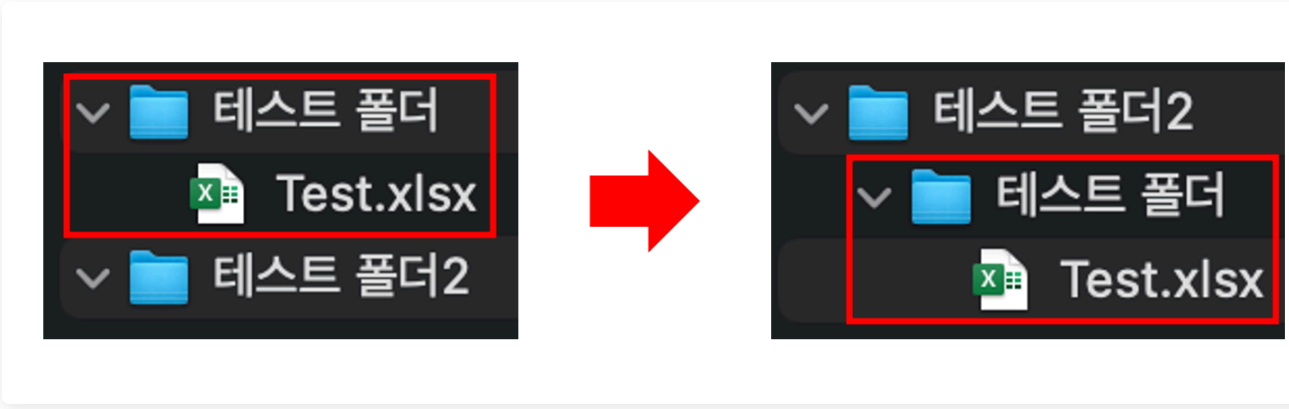
```
# 방법 2 - 현재 작업경로와 상관없이 절대경로를 입력
shutil.move('이동시킬 파일 경로', '파일을 이동시킬 폴더의 경로')

# 예시
shutil.move('/Users/sangwoo/Desktop/PYTHON BASIC/Shutil Basic/Test.xlsx',
            '/Users/sangwoo/Desktop/PYTHON BASIC/Shutil Basic/테스트 폴더')
```

아래부터는 간결한 글을 위해서 상대 경로 방식으로만 코드를 작성하였으나, 위와 동일하게 절대경로 방식또한 가능한 점 참고 해주세요.

3. 폴더를 특정 폴더로 이동하기_move( )

파일과 마찬가지로 폴더도 이동시킬 수 있습니다. 이때 폴더 안의 모든 내용물이 함께 이동됩니다.

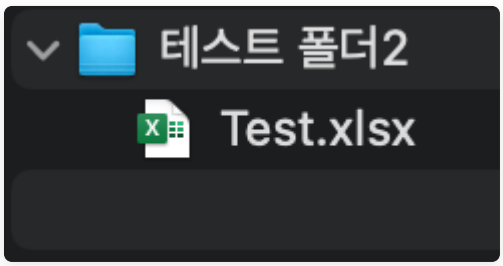
1) 폴더가 존재하는 경우



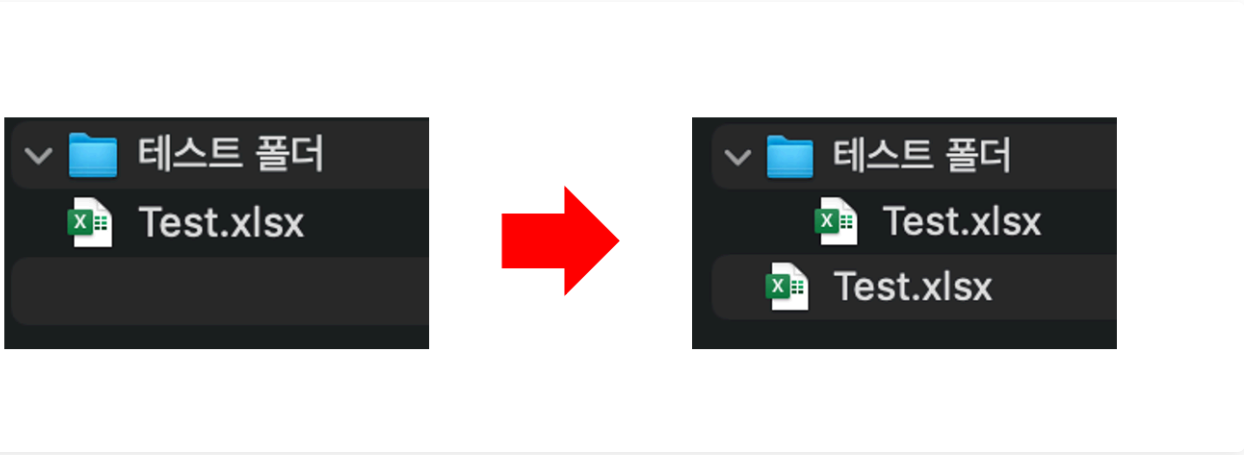

```
# 방법 1 - 파일과 폴더가 현재 작업경로에 있는 경우만 가능
shutil.move('이동시킬 폴더명', '이동할 폴더를 담을 폴더명')

# 예시
shutil.move('테스트 폴더', '테스트 폴더2')
```

2) 폴더가 존재하지 않는 경우

코드는 동일하지만 그 결과가 다릅니다. 만약 '테스트 폴더2'가 없는 경우, 기존의 '테스트 폴더'의 이름에 2가 추가되면서 안에는 'Test.xlsx' 파일이 위치하게 됩니다.

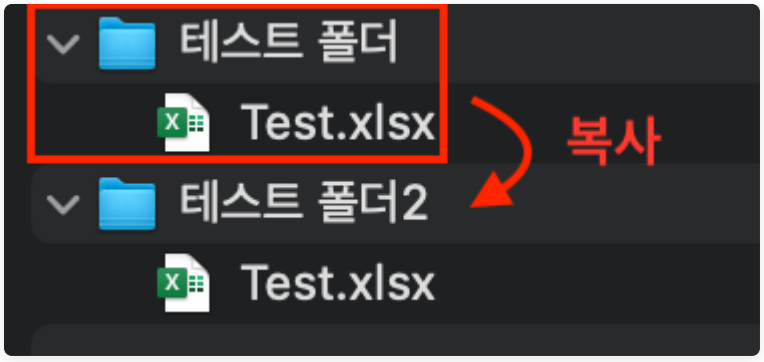


4. 파일을 특정 폴더로 복사하기_copy( )




```
# 방법 1 - 파일과 폴더가 현재 작업경로에 있는 경우만 가능
shutil.copy('복사할 파일명.확장자', '복사한 파일을 담을 폴더명')

# 예시
shutil.copy('Test.xlsx', '테스트 폴더')
```

5. 폴더를 특정 폴더로 복사하기_copytree( )




```
# 방법 1 - 파일과 폴더가 현재 작업경로에 있는 경우만 가능
shutil.copytree('복사할 폴더명', '복사될 새로운 폴더명')

# 예시
shutil.copytree('테스트 폴더', '테스트 폴더2')
```

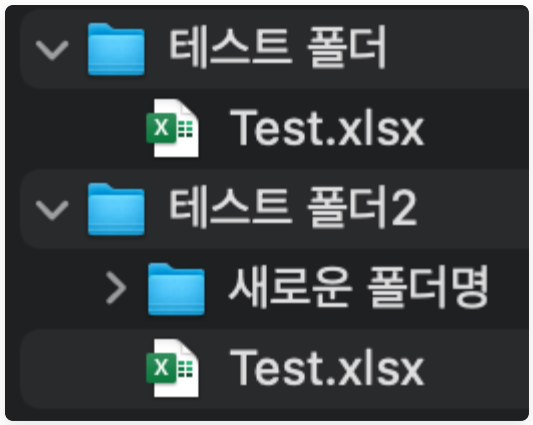
폴더를 복사할 때 주의할 점은 '복사될 새로운 폴더명'과 동일한 폴더명이 존재하면 아래와 같은 에러 메시지를 띄운다는 것입니다.

FileExistsError: [Errno 17] File exists: '테스트 폴더2'

간단히 말해서, '테스트 폴더2'는 코드가 실행될 때 존재해서는 안됩니다.

만약 원하시는 기능이 기존 존재하는 폴더 안에 특정 폴더를 복사해야 하는 것이라면 아래와 같이 하셔야 합니다.

```
shutil.copytree('테스트 폴더', '테스트 폴더2/새로운 폴더명')
```



3-6. 이미지 처리

오늘은 이미지를 많이 다루고 처리하는 분들을 위해서 파이썬으로 이미지를 다룰 수 있게 해주는 Pillow라는 라이브러리에 대해서 공부해보겠습니다.

1. Pillow란?

Python Imaging Library(PIL)라는 파이썬 이미지 처리 라이브러리의 지원이 2011년 중단되고, Pillow가 PIL의 후속 프로젝트로 나왔습니다. 즉, Pillow가 공식 PIL이라고 보면 되겠네요. 실제로 import 할 때에도 PIL이라는 이름을 사용한답니다.

다룰 수 있는 이미지 파일 형식으로는 PPM, PNG, JPEG, GIF, TIFF, BMP 등이 있으며, 지원하지 않는 파일 형식은 라이브러리를 확장해서 새로운 파일 디코더를 만드는 것이 가능하다고 합니다.

PIL 이미지 작업으로는 아래의 기능들이 가능합니다.

- 픽셀 단위의 조작
- 마스킹 및 투명도 제어
- 흐림, 윤곽 보정 다듬어 윤곽 검출 등의 이미지 필터
- 선명하게, 밝기 보정, 명암 보정, 색 보정 등의 화상 조정
- 이미지에 텍스트 추가
- 기타 등등

2. Pillow 설치하고 불러오기

pillow를 설치하는 방법은 매우 간단합니다. 아래의 코드를 커맨드 창에서 입력해주시면 됩니다. 만약 아나콘다 패키지를 설치하신 분들은 conda 명령어를 이용하여 설치해주셔도 됩니다. (만약 이미 설치되어 있다면 이 과정을 생략하셔도 됩니다)

```
pip install pillow
```

설치가 완료되었거나 이미 설치되어 있으신 분들은 아래의 코드로 Pillow를 불러올 수 있습니다.

```
#import PIL로 불러올 시, Image 모듈은 안불러와져서 에러가 뜸

from PIL import Image
```

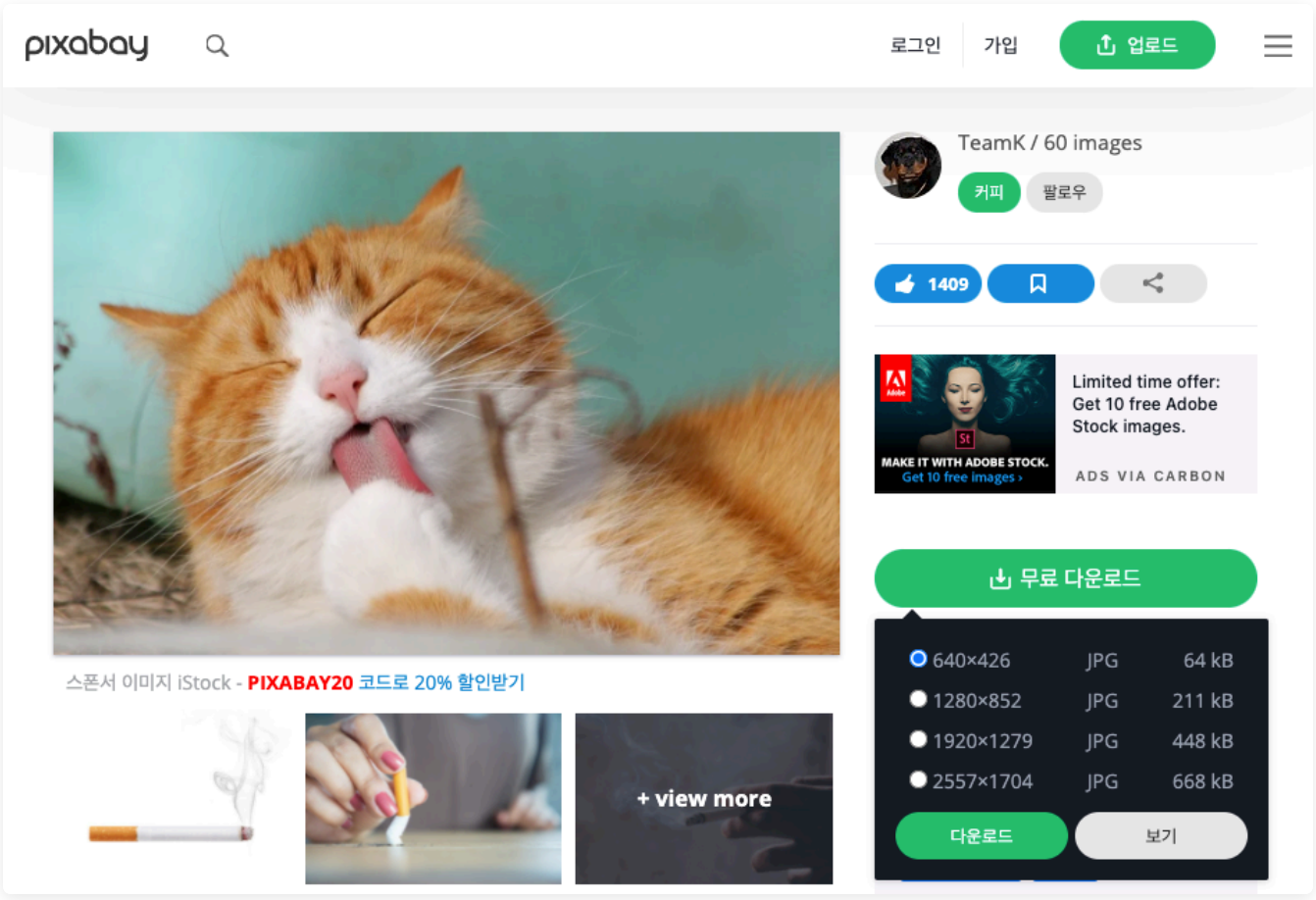
그냥 import PIL로 하면 될 것 같지만 Image 모듈이 불러와지지 않는답니다. 이 부분은 다른 패키지나 라이브러리를 다룰 때에도 종종 볼 수 있는 현상인데, 아직 공부가 부족해서 왜 그런지는 잘 모르겠습니다.

3. 실습 이미지 다운로드 및 Pillow로 Open하기

함께 실습할 이미지를 다운로드 하기 위해서 저작권이 무료이미지를 마음껏 받을 수 있는 Pixabay로 접속해 보겠습니다.

https://pixabay.com/ko/

해당 링크에 접속하셨으면 검색창에 고양이를 검색하고 아래의 이미지를 다운로드해주세요. 사진 사이즈는 가장 작은 640x426 사이즈로 받아볼게요.

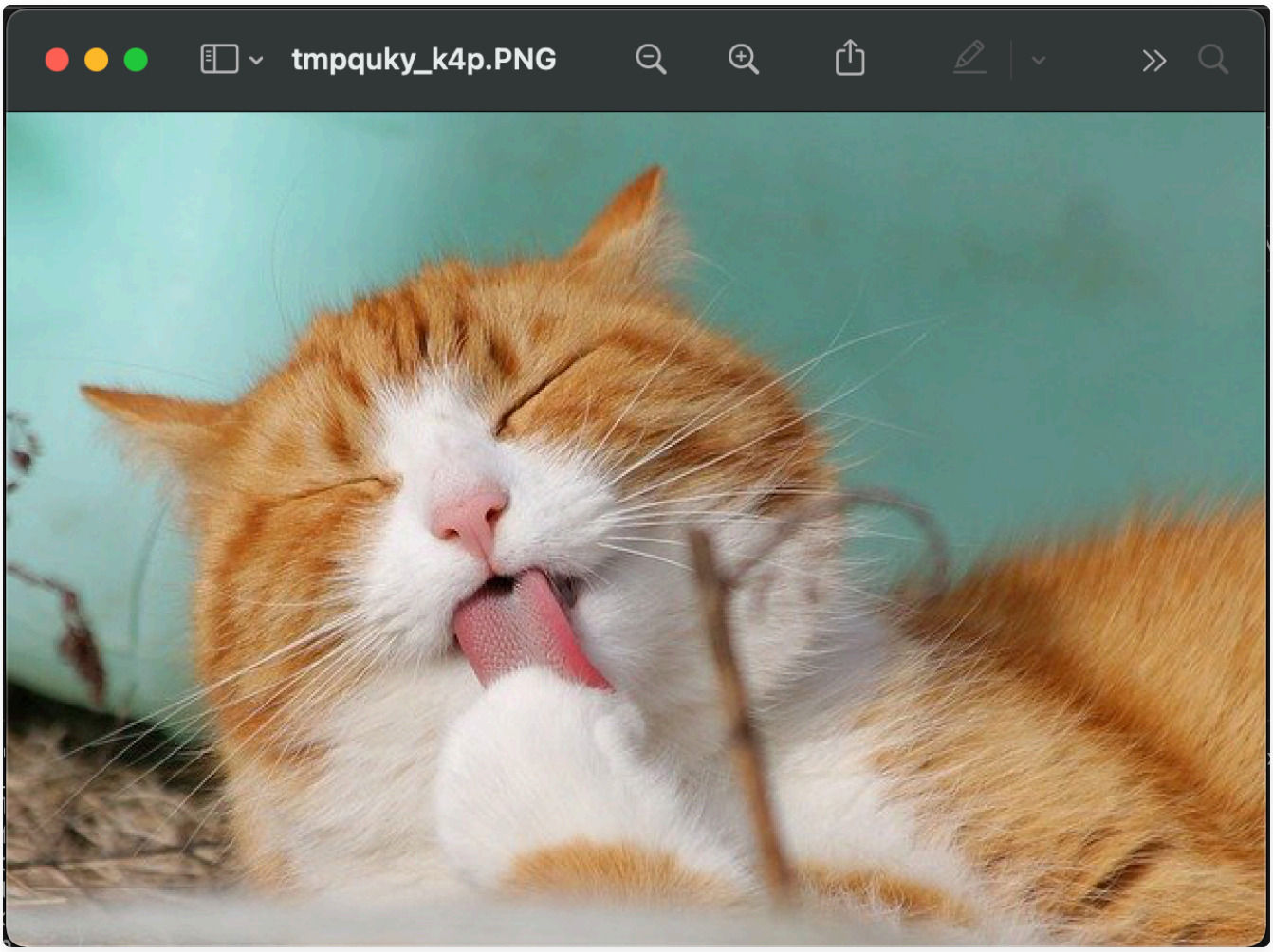


아마 파일명이 'cat-gf5d169137_640.jpg'로 다운로드 받아졌을텐데 사용하기쉽도록 'cat.jpg'로 바꿔주세요.

파일의 위치는 현재 파이썬 파일이 존재하는 폴더에 함께 놔둬 주세요. 이렇게 하면 복잡하게 경로를 입력해줄 필요없이 불러올 때 'cat.jpg'만 입력해주면 되거든요. 아래와 같이 말이죠.

```
# 고양이 이미지 불러와서 img라는 변수에 입력
img = Image.open('cat.jpg')

# 이미지 보여주기
img.show()
```

결과를 볼까요? 요상한 이름과 PNG로 불러와졌지만 일단 사진은 맞네요.



4. 이미지 속성 확인

불러온 이미지 이름, 확장자, 사이즈, 색상 모드 등의 속성을 확인 가능합니다. 출력결과는 >>> 로 표시하였습니다.

```
# 이미지 파일명
print(img.filename)
>>> cat.jpg

# 이미지 형식
print(img.format)
>>> JPEG

# 이미지 크기
print(img.size)
>>> (640, 426)

# 이미지 너비
print(img.width)
>>> 640

# 이미지 높이
print(img.height)
>>> 426

# 이미지의 색상 모드
print(img.mode)
>>> RGB
```

1) Pillow 설치 및 이미지 불러오기

오늘은 이미지를 많이 다루고 처리하는 분들을 위해서 파이썬으로 이미지를 다룰 수 있게 해주는 Pillow라는 라이브러리에 대해서 공부해보겠습니다.

1. Pillow란?

Python Imaging Library(PIL)라는 파이썬 이미지 처리 라이브러리의 지원이 2011년 중단되고, Pillow가 PIL의 후속 프로젝트로 나왔습니다. 즉, Pillow가 공식 PIL이라고 보면 되겠네요. 실제로 import 할 때에도 PIL이라는 이름을 사용한답니다.

다룰 수 있는 이미지 파일 형식으로는 PPM, PNG, JPEG, GIF, TIFF, BMP 등이 있으며, 지원하지 않는 파일 형식은 라이브러리를 확장해서 새로운 파일 디코더를 만드는 것이 가능하다고 합니다.

PIL 이미지 작업으로는 아래의 기능들이 가능합니다.

- 픽셀 단위의 조작
- 마스킹 및 투명도 제어
- 흐림, 윤곽 보정 다듬어 윤곽 검출 등의 이미지 필터
- 선명하게, 밝기 보정, 명암 보정, 색 보정 등의 화상 조정
- 이미지에 텍스트 추가
- 기타 등등

2. Pillow 설치하고 불러오기

pillow를 설치하는 방법은 매우 간단합니다. 아래의 코드를 커맨드 창에서 입력해주시면 됩니다. 만약 아나콘다 패키지를 설치하신 분들은 conda 명령어를 이용하여 설치해주셔도 됩니다. (만약 이미 설치되어 있다면 이 과정을 생략하셔도 됩니다)

```
pip install pillow
```

설치가 완료되었거나 이미 설치되어 있으신 분들은 아래의 코드로 Pillow를 불러올 수 있습니다.

```
#import PIL로 불러올 시, Image 모듈은 안불러와져서 에러가 뜸

from PIL import Image
```

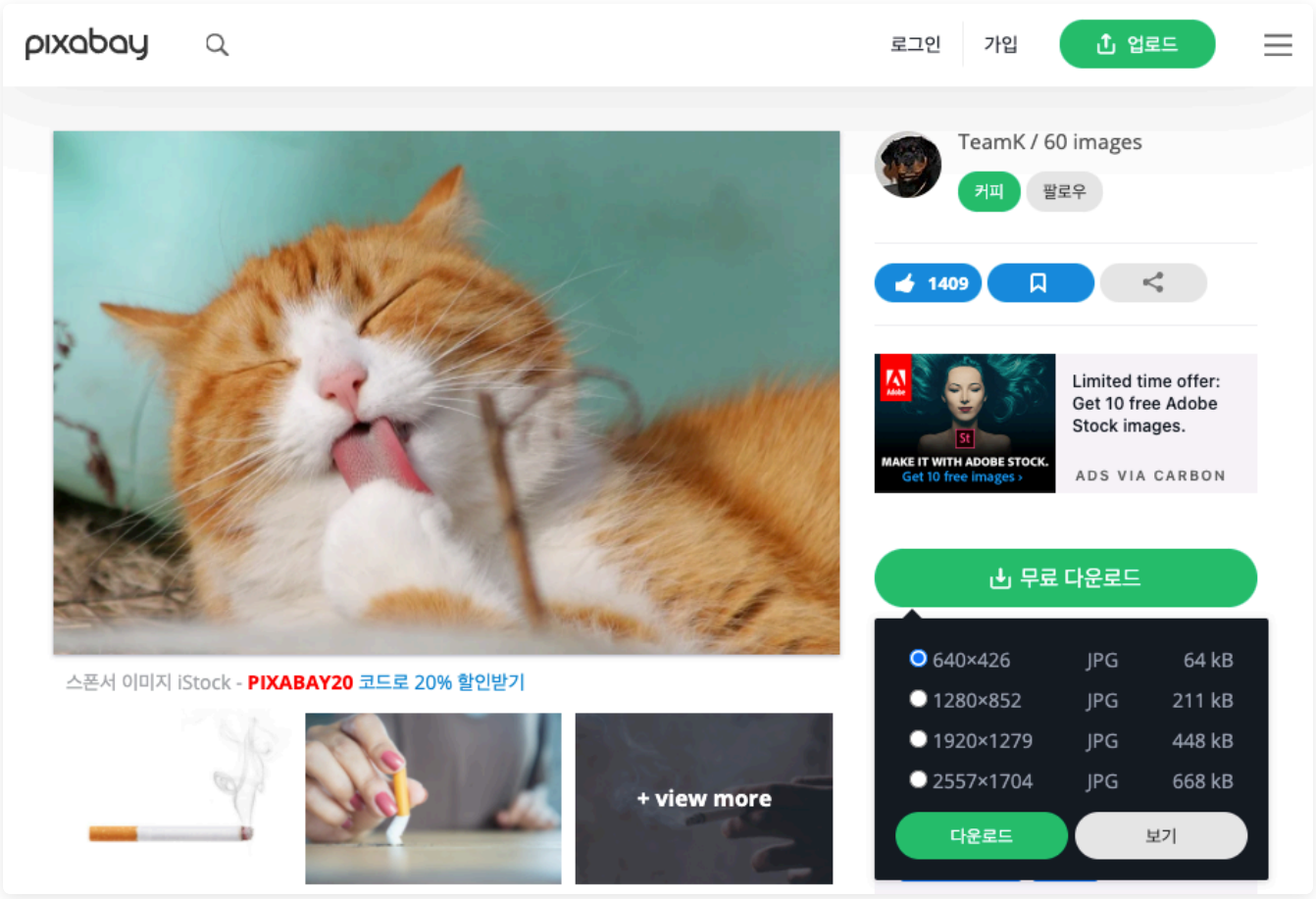
그냥 import PIL로 하면 될 것 같지만 Image 모듈이 불러와지지 않는답니다. 이 부분은 다른 패키지나 라이브러리를 다룰 때에도 종종 볼 수 있는 현상인데, 아직 공부가 부족해서 왜 그런지는 잘 모르겠습니다.

3. 실습 이미지 다운로드 및 Pillow로 Open하기

함께 실습할 이미지를 다운로드 하기 위해서 저작권이 무료이미지를 마음껏 받을 수 있는 Pixabay로 접속해 보겠습니다.

https://pixabay.com/ko/

해당 링크에 접속하셨으면 검색창에 고양이를 검색하고 아래의 이미지를 다운로드해주세요. 사진 사이즈는 가장 작은 640x426 사이즈로 받아볼게요.

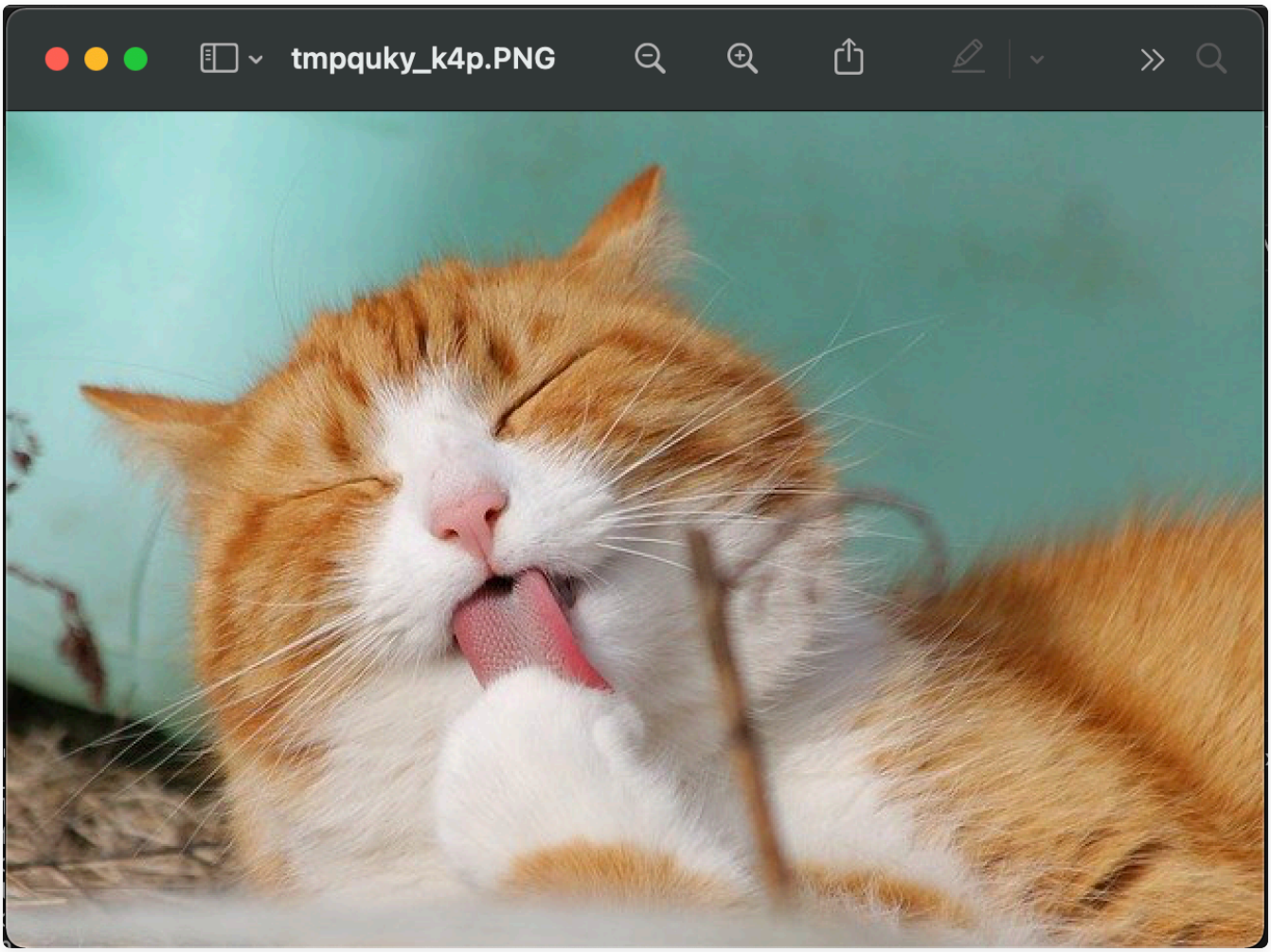


아마 파일명이 'cat-gf5d169137_640.jpg'로 다운로드 받아졌을텐데 사용하기쉽도록 'cat.jpg'로 바꿔주세요.

파일의 위치는 현재 파이썬 파일이 존재하는 폴더에 함께 놔둬 주세요. 이렇게 하면 복잡하게 경로를 입력해줄 필요없이 불러올 때 'cat.jpg'만 입력해주면 되거든요. 아래와 같이 말이죠.

```
# 고양이 이미지 불러와서 img라는 변수에 입력
img = Image.open('cat.jpg')

# 이미지 보여주기
img.show()
```

결과를 볼까요? 요상한 이름과 PNG로 불러와졌지만 일단 사진은 맞네요.



4. 이미지 속성 확인

불러온 이미지 이름, 확장자, 사이즈, 색상 모드 등의 속성을 확인 가능합니다. 출력결과는 >>> 로 표시하였습니다.

```
# 이미지 파일명
print(img.filename)
>>> cat.jpg

# 이미지 형식
print(img.format)
>>> JPEG

# 이미지 크기
print(img.size)
>>> (640, 426)

# 이미지 너비
print(img.width)
>>> 640

# 이미지 높이
print(img.height)
>>> 426

# 이미지의 색상 모드
print(img.mode)
>>> RGB
```

2) 이미지 편집(resize,crop,rotate,flip)

시작하기에 앞서 관련 패키지를 import하고 지난 시간에 다운로드 받았던 고양이 사진을 불러오겠습니다.

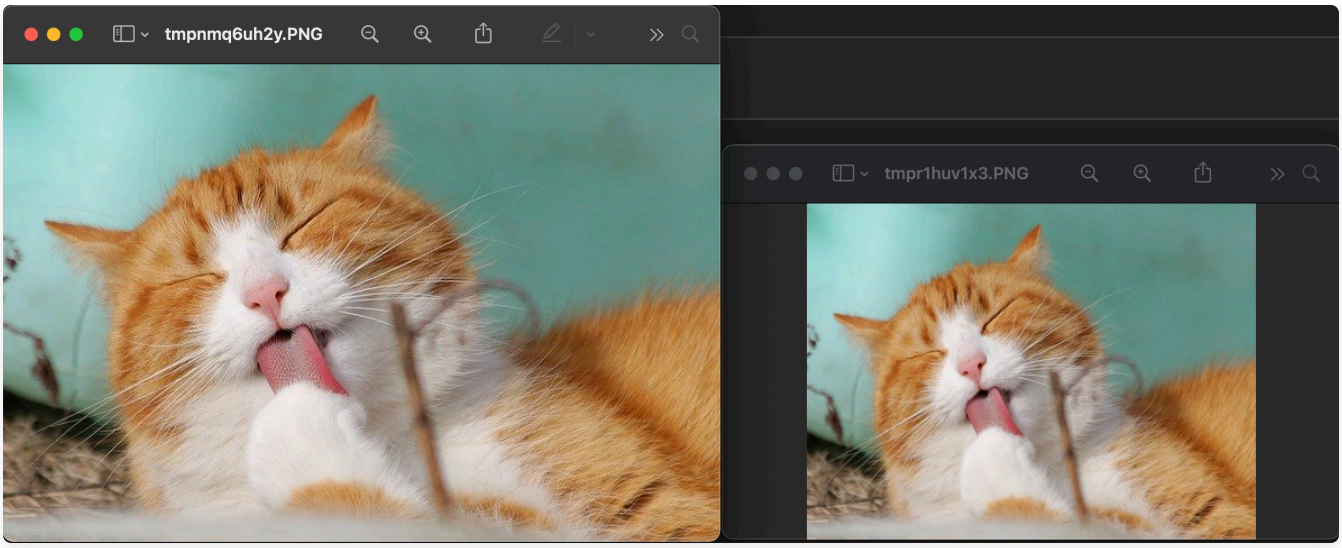
```
# Pillow 라이브러리 불러오기
from PIL import Image

# 고양이 이미지 불러와서 img라는 변수에 입력
img = Image.open('cat.jpg')
```

1. 이미지 크기 변경

이미지 크기 변경은 resize( ) 함수를 사용합니다. 사이즈는 튜플 형식 (width, height)으로 입력해줍니다.

```
# 이미지 사이즈 변경
img_resized = img.resize((400,300))

# 변경한 이미지 출력
img_resized.show()

# 비교를 위해 원본 이미지 출력
img.show()
```

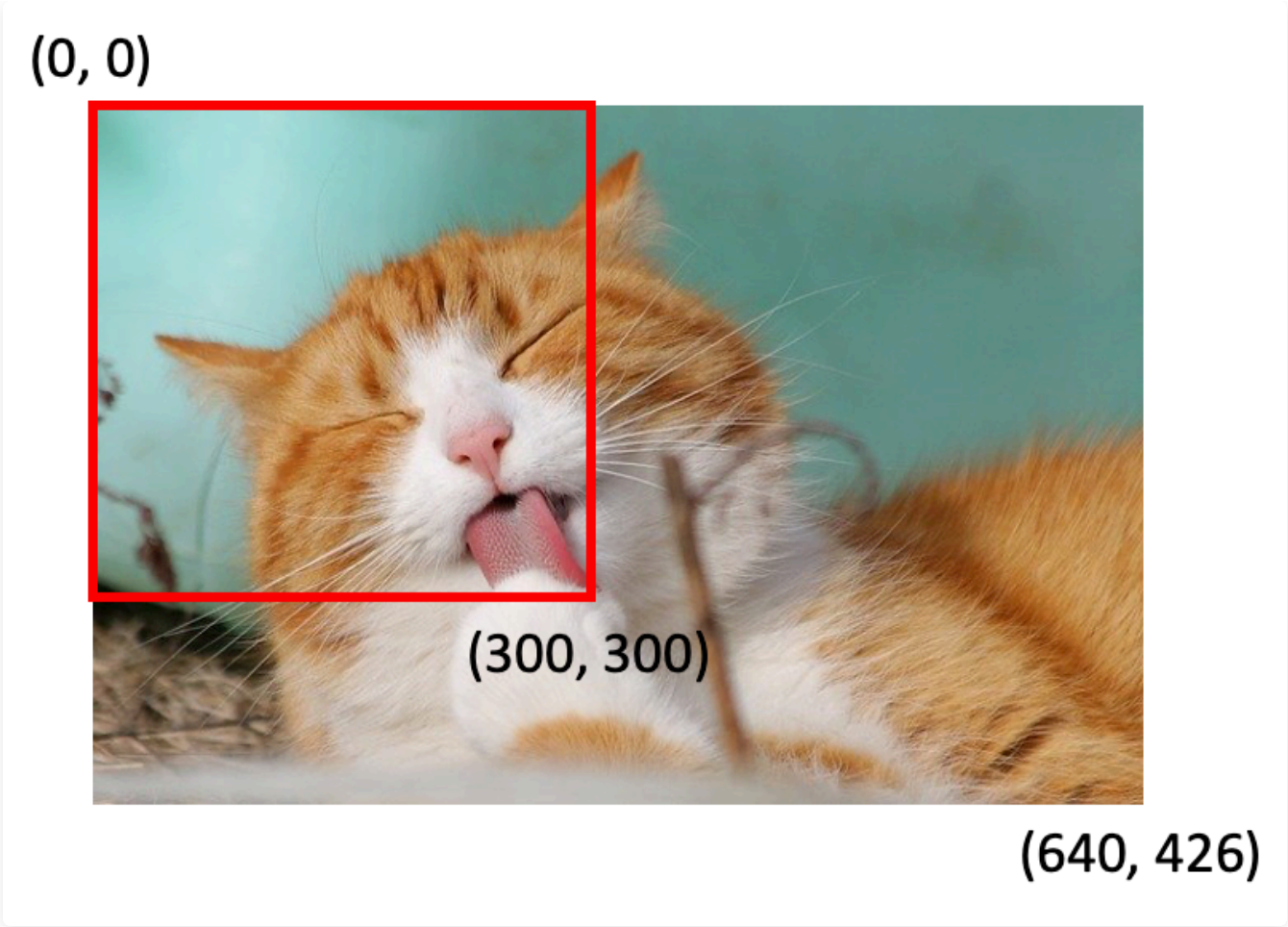
<출력 결과>



2. 이미지 자르기

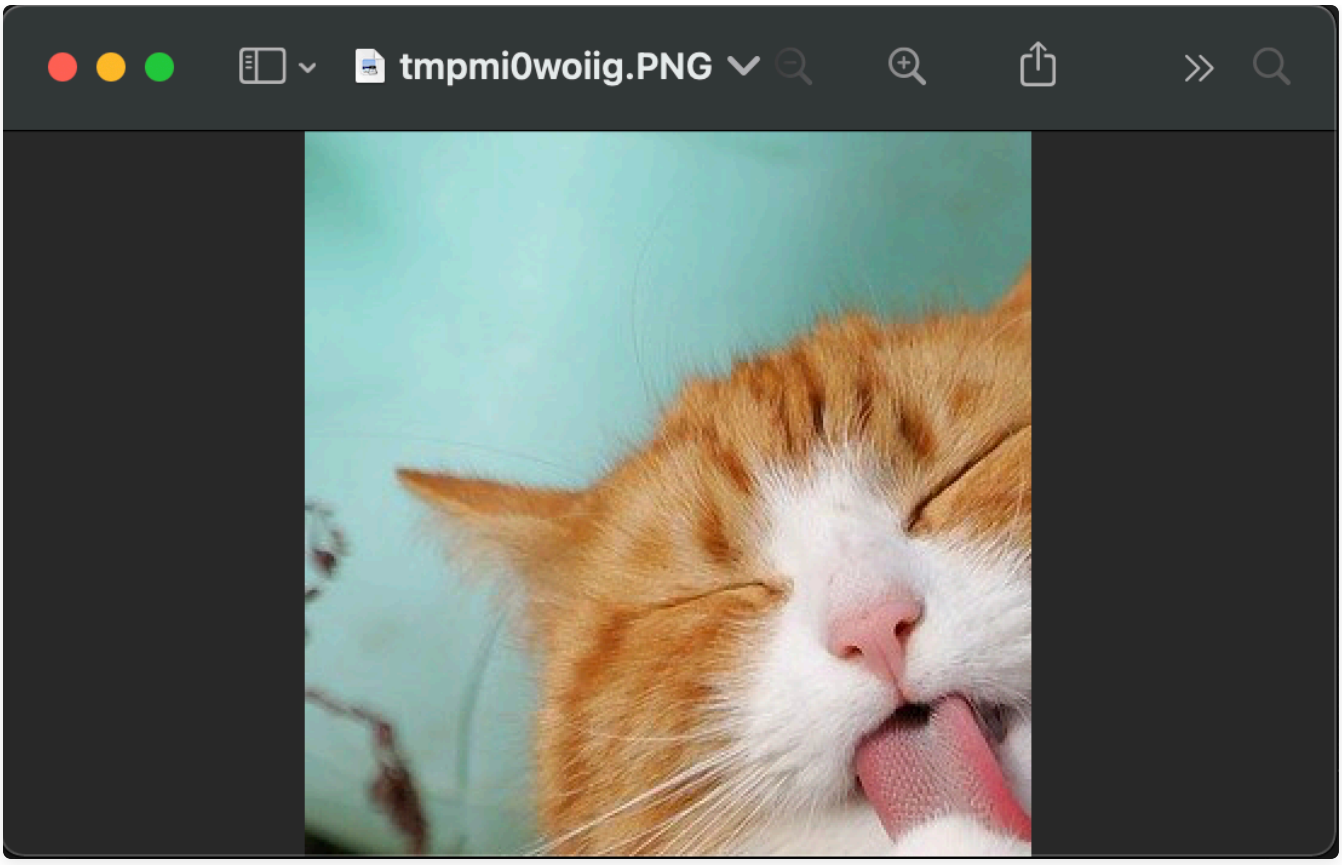
이미지의 일부분을 얻고 싶을 때는 crop( ) 함수를 사용합니다. 마찬가지로 인자를 튜플 형태 (left, upper, right, lower)로 입력 받습니다.

쉽게 생각해서 좌측상단의 xy좌표, 우측하단의 xy좌표를 차례대로 받는다고 생각하시면 돼요.




```
# 이미지 자르기
img_cropped = img.crop((0,0,300,300))

# 잘라낸 이미지 출력
img_cropped.show()
```

<출력 결과>

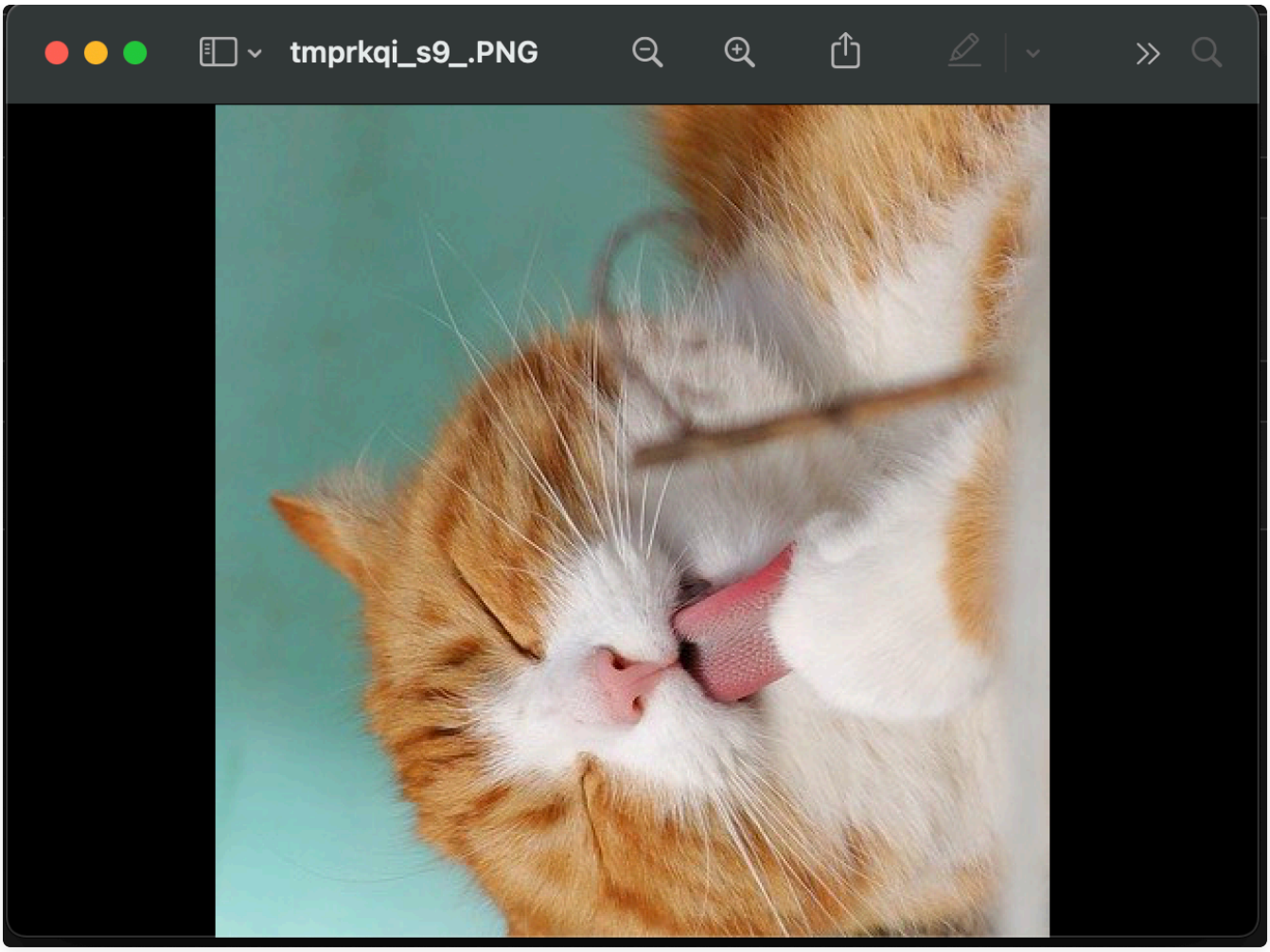


3. 이미지 회전

이미지를 회전 하고 싶으면 rotate( ) 함수를 사용합니다. 입력 인자는 각도입니다.

```
# 이미지 90도 회전
img_rotated = img.rotate(90)

# 회전한 이미지 출력
img_rotated.show()
```

<출력 결과>



4. 이미지 대칭

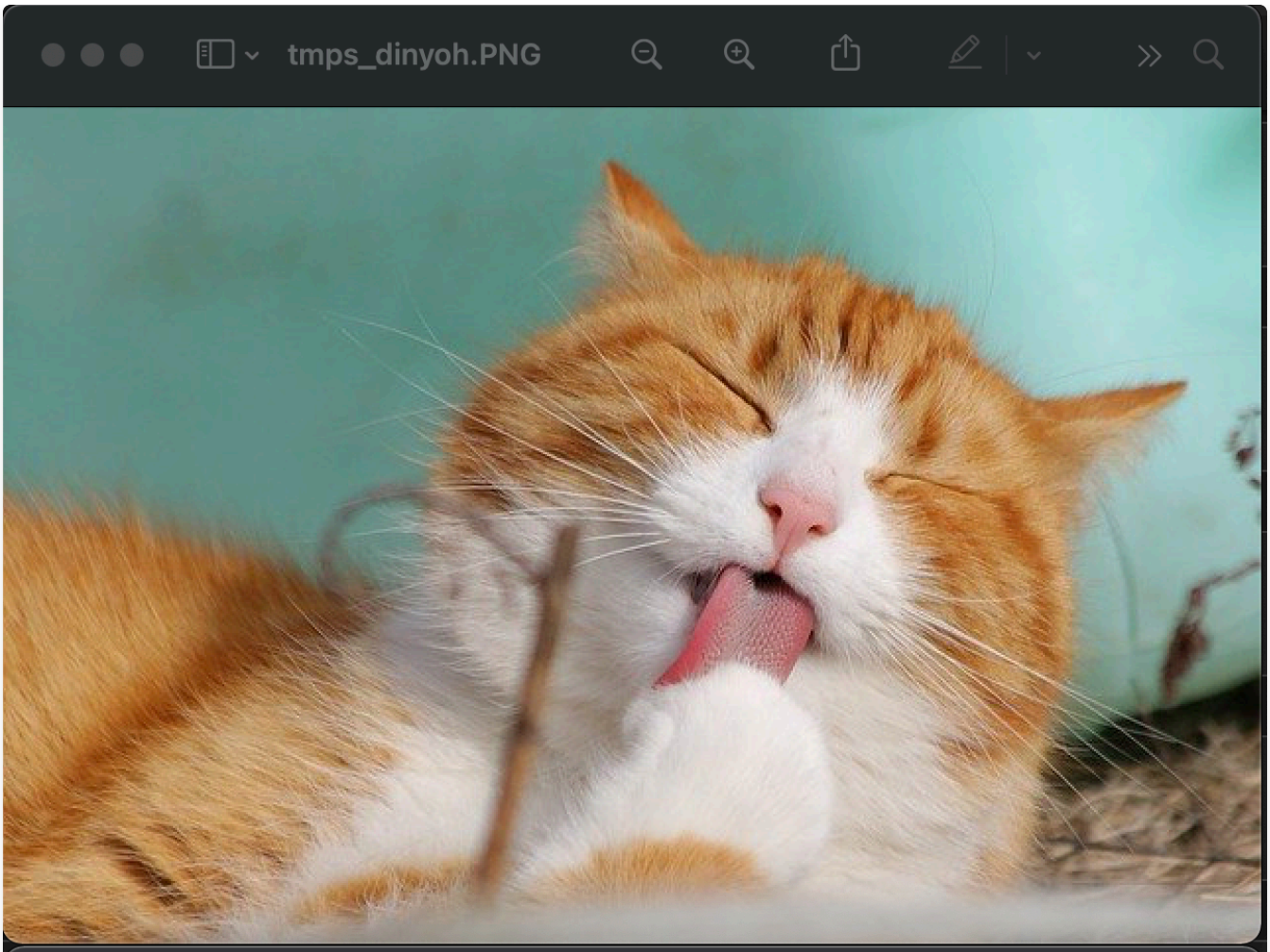
이미지를 상하 혹은 좌우 대칭시키고 싶으면 transpose( ) 함수를 사용합니다. 입력 인자는 6가지가 있는데 대칭에 사용하는 인자는 아래의 2가지입니다. 나머지 인자들은 90도, 180도, 270도 회전과 관련되어 있습니다.

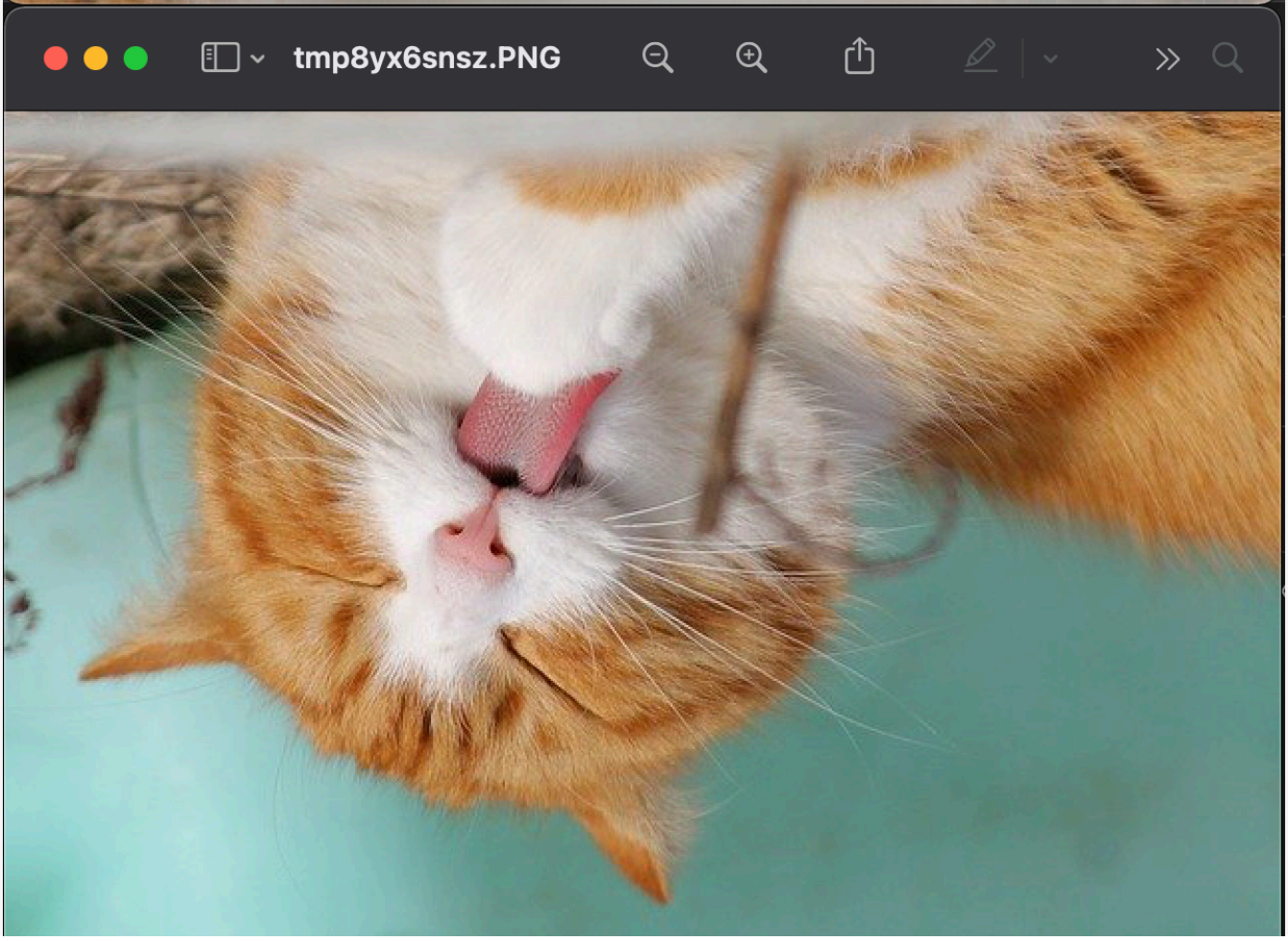
좌우대칭: Image.FLIP_LEFT_RIGHT

상하대칭: Image.FLIP_TOP_BOTTOM

```
# 좌우대칭
img_flipped_LR = img.transpose(Image.FLIP_LEFT_RIGHT)
img_flipped_LR.show()

# 상하대칭
img_flipped_TB = img.transpose(Image.FLIP_TOP_BOTTOM)
img_flipped_TB.show()
```

〈출력 결과〉


tmps_dinyoh.PNG

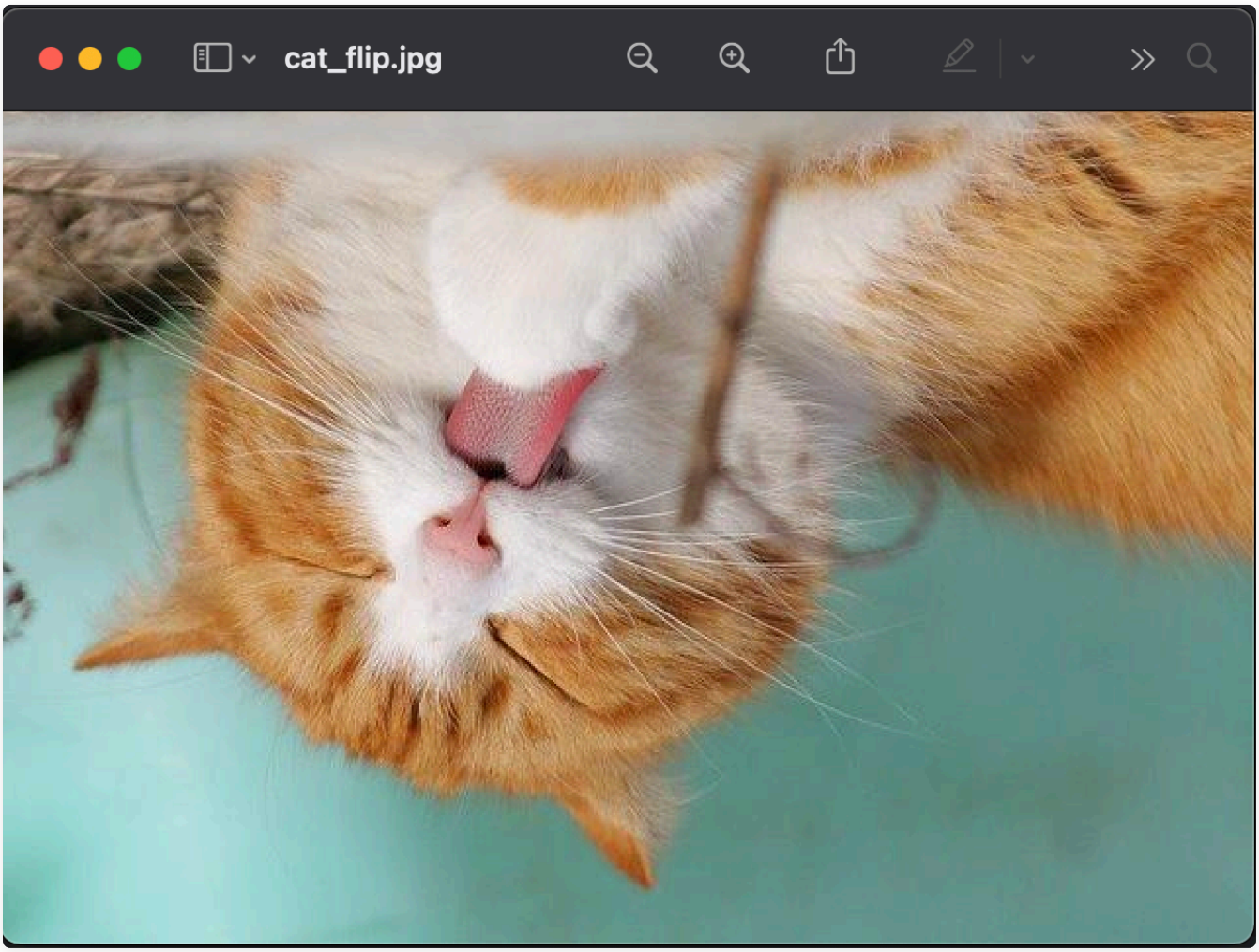

tmp8yx6snsz.PNG

5. 이미지 저장

지금까지 편집한 이미지를 저장하려면 save( ) 함수를 사용하면 됩니다. 입력 받는 인자로는 '파일명.확장자' 혹은 '파일의 경로'입니다.

```
# 상하반전한 이미지 저장
img_flipped_TB.save('cat_flip.jpg')
```

<저장된 이미지>



3) 이미지 필터 효과(흑백, 블러, 엣지)

오늘 배워볼 내용은 컬러 이미지를 흑백으로 만드는 것과 이미지에 필터 효과를 주는 방법입니다.

시작하기에 앞서 관련 패키지를 import하고 첫 시간에 다운로드 받았던 고양이 사진을 불러오겠습니다.

```
# Pillow 라이브러리 불러오기
from PIL import Image

# 고양이 이미지 불러와서 img라는 변수에 입력
img = Image.open('cat.jpg')
```

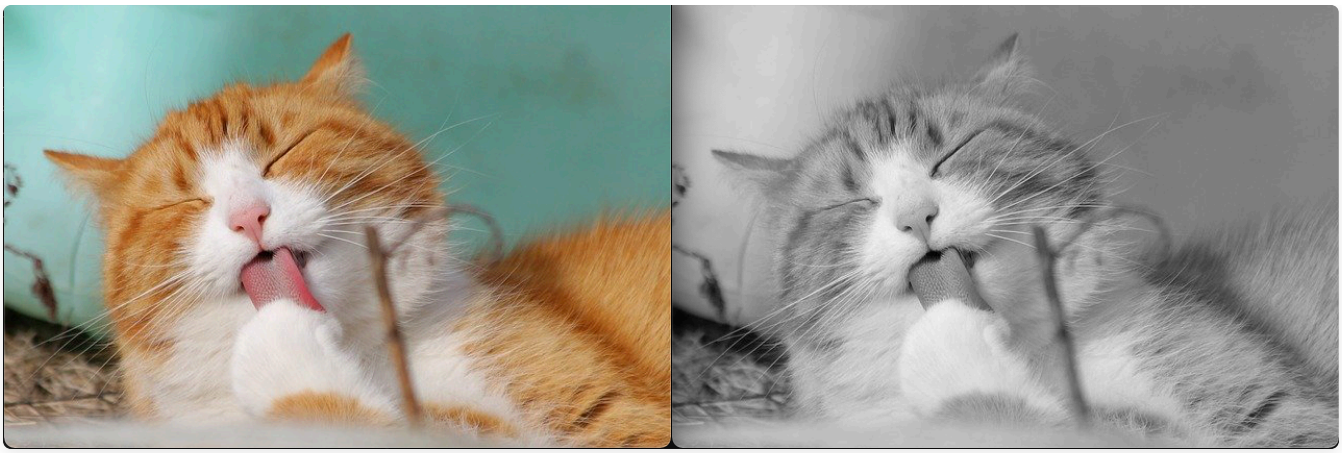
1. 컬러 이미지 흑백으로 만들기

이미지의 모드(mode)를 변환시킬 때는 convert( ) 함수를 사용합니다. 이미지의 모드에는 정말 다양한 형식들이 있지만 우리가 일상에서 사용할만한 것은 RGB 트루 컬러(8bit)와 흑백(8bit)가 거의 대부분이 아닐까 합니다.

```
# RGB로 표현된 컬러 이미지를 흑백으로 변경
img_gray = img.convert("L")

# 흑백으로 변환된 이미지 출력
img_gray.show()
```

<출력 결과>

참고로 위의 코드를 실행하시면 우측의 흑색 사진만 출력되지만 비교를 위해서 원본 이미지도 넣었습니다.



'L' 대신에 다양한 mode들을 넣으시면 다른 이미지가 출력되니 궁금하신 분들은 테스트 해보세요.

- 1 (1비트 픽셀, 흑백, 바이트당 1픽셀로 저장)
- L (8비트 픽셀, 흑백)
- P (8비트 픽셀, 색상 팔레트를 사용하여 다른 모드에 매핑됨)
- RGB (3x8비트 픽셀, 트루 컬러)
- RGBA (4x8비트 픽셀, 투명 마스크가 있는 트루 컬러)
- CMYK (4x8비트 픽셀, 색상 분리)
- YCbCr (3x8비트 픽셀, 컬러 비디오 형식)
- LAB (3x8비트 픽셀, L*a*b 색 공간)
- HSV (3x8비트 픽셀, 색조, 채도, 값 색 공간)
- I (32비트 부호 있는 정수 픽셀)
- F (32비트 부동 소수점 픽셀)

참고링크: https://pillow.readthedocs.io/en/stable/handbook/concepts.html

2. 이미지에 필터 부여하기

이미지에 필터를 부여하고 싶을 때에는 filter( ) 함수를 사용합니다. 단, 필터 함수를 사용하려면 아래의 모듈을 import 해주어야 합니다.

```
from PIL import ImageFilter
```

제공 되는 필터는 아래의 필터들 외에도 다양하며, 아래의 링크를 참고하셔서 사용하시면 됩니다.

https://pillow.readthedocs.io/en/stable/_modules/PIL/ImageFilter.html#MaxFilter

- BLUR : BLUR, BoxBlur( ), GaussianBlur( )

- MedianFilter( ), MinFilter( ), MaxFilter( ) 등
- CONTOUR
- DETAIL
- EDGE_ENHANCE, EDGE_ENHANCE_MORE
- EMBOSS
- FIND_EDGES
- SHARPEN
- SMOOTH, SMOOTH_MORE

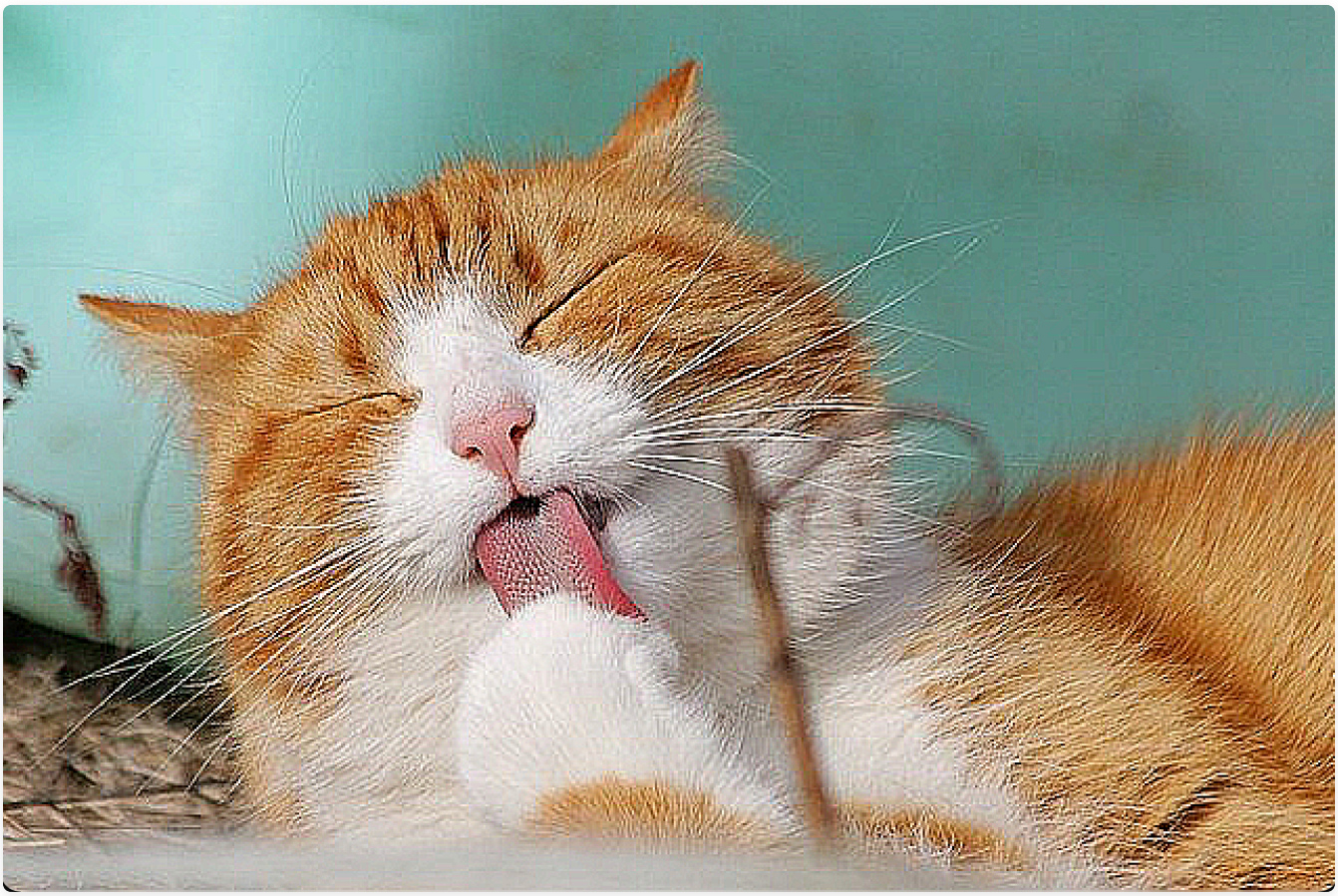
사용 방법은 아래와 같습니다.

1) 블러 효과를 주고 싶을 때

```
# Blur 효과를 줄 때 가장 많이 사용하는 가우시안 블러 --> 숫자를 크게 할 수록 흐려짐
img_blur = img.filter(ImageFilter.GaussianBlur(10))

# 이미지 출력
img_blur.show()
```

2) 이미지의 엣지를 더 강조하고 싶을 때

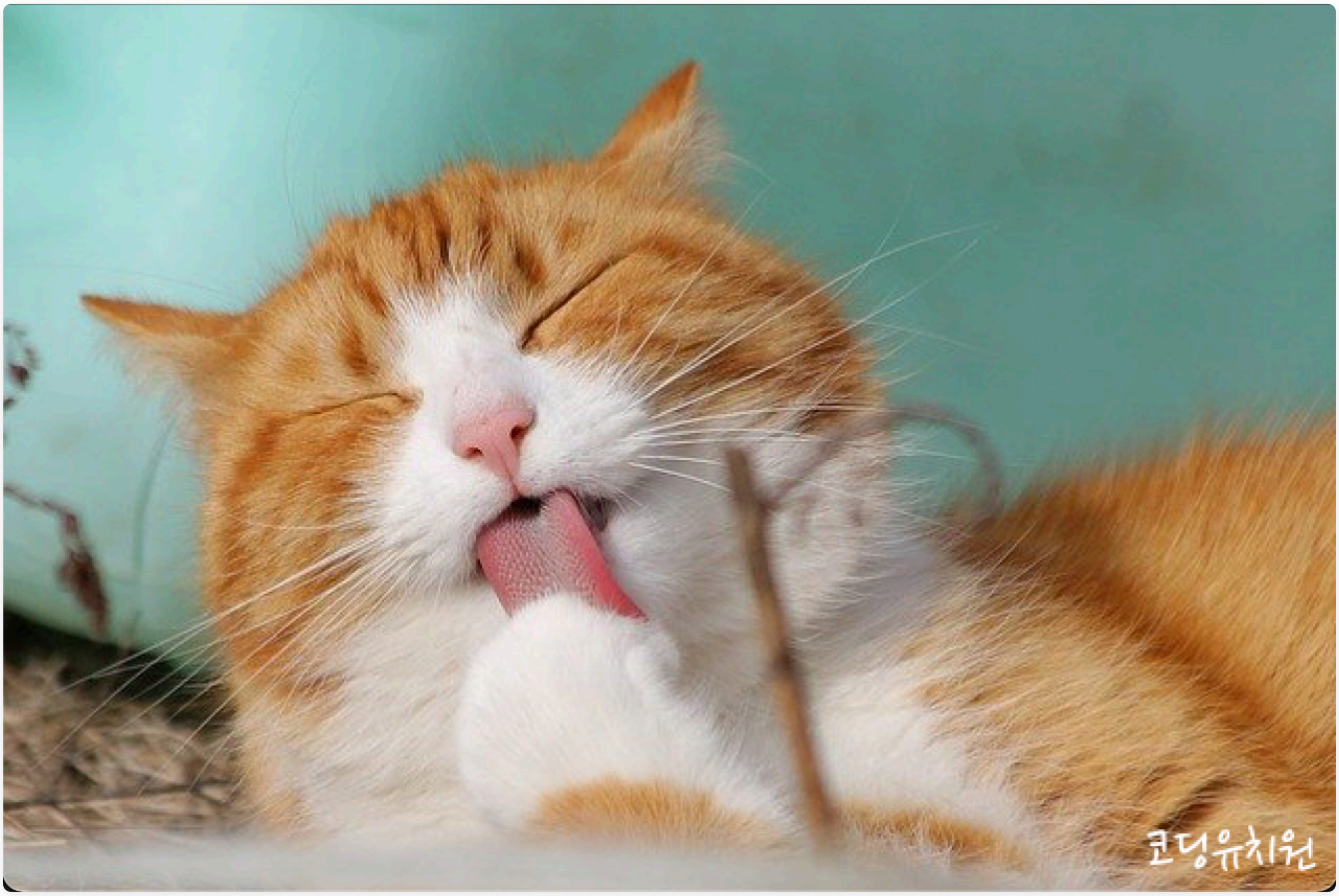
```
img_edge = img.filter(ImageFilter.EDGE_ENHANCE)

img_edge.show()
```



4) 이미지 워터마크(텍스트)

인터넷 상에 사진이 포함된 글을 보다보면 아래와 같이 워터마크를 포함한 사진을 보신적이 있으실 거예요.

이런 워터마크 작업을 많이 하시는 분들은 사진들을 한 폴더에 두고, 자동으로 워터마크를 찍어주는 코드를 사용하면 참 편하겠죠?



먼저 전체 코드를 보겠습니다.

```
# step1.필요한 PIL의 모듈 3가지 import
from PIL import  Image, ImageDraw, ImageFont

# step2.워터마크 삽입할 이미지 불러오기
img = Image.open('cat.jpg')
width, height = img.size

# step3.그림판에 이미지를 그대로 붙여넣는 느낌의 Draw() 함수
draw = ImageDraw.Draw(img)

# step4.삽입할 워터마크 문자
text = "코딩유치원"

# step5.삽입할 문자의 폰트 설정
font = ImageFont.truetype('/Users/sangwoo/Downloads/나눔 글꼴/나눔손글씨_펜/NanumPen.ttf', 30)

# step6.삽입할 문자의 높이, 너비 정보 가져오기
width_txt, height_txt = draw.textsize(text, font)

# step7.워터마크 위치 설정
margin = 10
x = width - width_txt - margin
y = height - height_txt - margin

# step8.텍스트 적용하기
draw.text((x, y), text, fill='white', font=font)

# step9.이미지 출력
img.show()

# step10.현재작업 경로에 완성 이미지 저장
img.save("cat_watermakr.jpg")
```

너무 길어서 조금 부담스러우신가요? 걱정마시고 저랑 차근차근 하나씩 살펴 보시죠.

step1 & step2

```
# 1.필요한 PIL의 모듈 3가지 import
from PIL import  Image, ImageDraw, ImageFont

# 2.워터마크 삽입할 이미지 불러오기
img = Image.open('cat.jpg')
width, height = img.size
```

우선 Pillow 라이브러리에서 필요한 친구들만 import 해줍니다.

Image: 이미지를 불러오고 보여주는 역할(가장 기본적이고 필수적인 모듈)

ImageDraw: 우리가 사진을 편집해 줄 그림판과 같은 역할

ImageFont: 폰트 종류와 크기 설정

그 다음엔 워터마크를 넣어 줄 이미지를 한장 불러온 뒤, 나중에 사용하기 위해서 이미지의 사이즈 정보도 변수에 담아줍니다.

step3 & step4

```
# step3.그림판에 이미지를 그대로 붙여넣는 느낌의 Draw() 함수
draw = ImageDraw.Draw(img)

# step4.삽입할 워터마크 문자
text = "코딩유치원"
```

step3은 불러온 이미지를 그림판에 복붙해주는 느낌으로 생각하시면 됩니다. step4에서는 넣어줄 워터마크의 내용을 입력해주세요.

step5

```
# step5.삽입할 문자의 폰트 설정
font = ImageFont.truetype('/Users/sangwoo/Downloads/나눔 글꼴/나눔손글씨_펜/NanumPen.ttf', 30)
```

가장 핵심인 5단계는 따로 떼어내서 설명드리려 합니다. 왜 핵심이냐?! 에러가 가장 많이 발생할 가능성이 높기 때문입니다.

아래는 구글링을 하다보면 볼 수 있는 코드입니다.

```
font = ImageFont.truetype('arial.ttf', 30)
```

이 코드를 그대로 입력하셨다면 거의 99% 확률로 아래와 같은 문구의 에러가 출력됩니다.

(1%는 해당 ttf 파일이 현재 작업 경로에 들어있거나 폰트가 들어있는 디렉토리를 환경변수로 설정해둔 경우입니다.)

OSError: cannot open resource

한마디로 너가 사용하겠다는 폰트를 못찾겠다는 말입니다.

그렇다면 우리는 어떻게 해야할까요? 사용하고 싶은 폰트를 다운로드 받고 폰트의 위치를 컴퓨터에게 알려주면 됩니다.

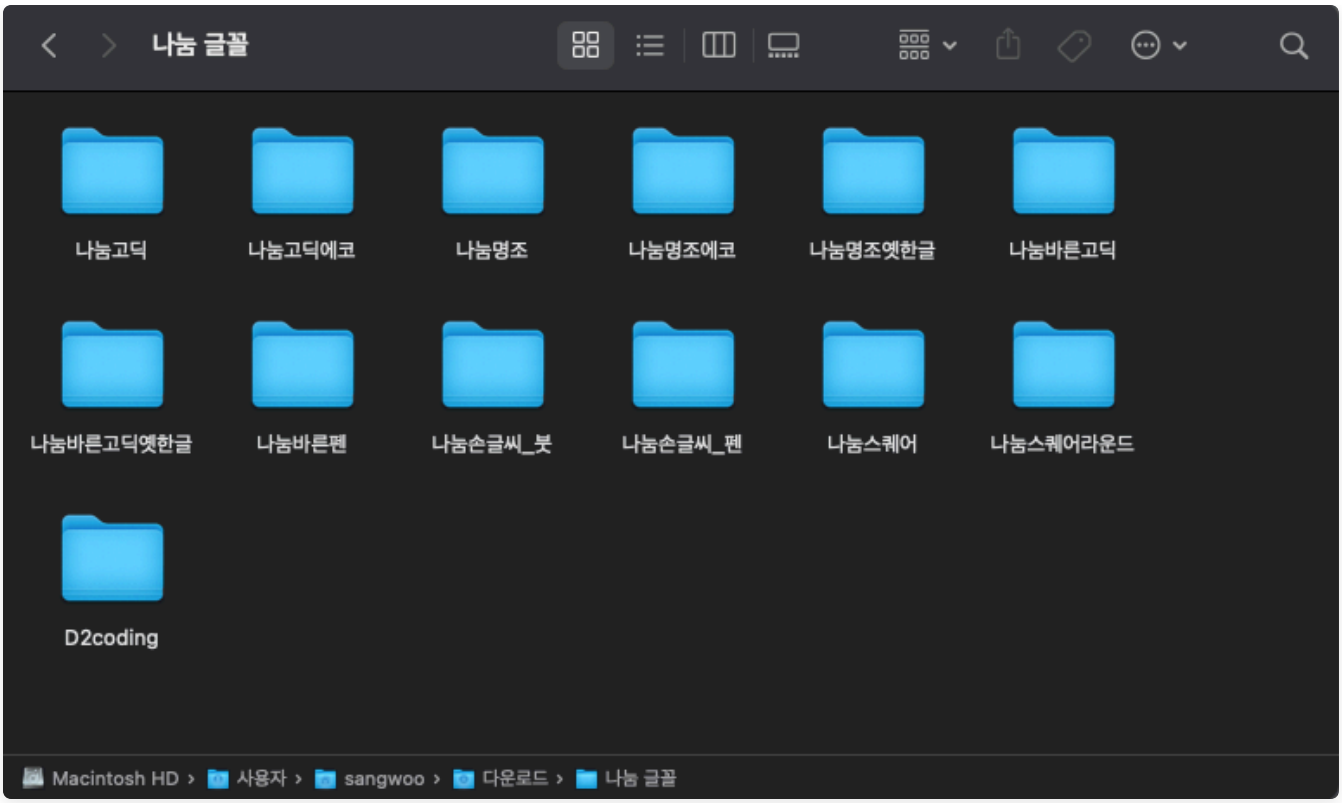
폰트는 저작권이 무료이고, 나름 퀄리티 좋은 네이버 나눔 글꼴을 추천드립니다.

(마음에 안드신다면 무료 글꼴을 제공하는 기업들이 있으니 검색해보시길 추천드립니다)

https://hangeul.naver.com/2017/nanum

위의 링크에서 나눔글꼴을 다운로드 받으셨다면 압축을 푸시고 폴더를 열어주세요.

저는 아래의 여러 글꼴들 중에서 나눔손글씨_펜 폴더 안의 NanumPen.ttf 파일을 선택했습니다.



만약 경로의 개념을 잘 몰라서 경로를 어떻게 입력해야할지 모르시겠다면, 해당 파일을 커맨드(cmd) 창에 드래그&드롭 해보세요.

경로가 출력 된답니다. 그걸 그대로 복사해서 아래의 코드에 붙여넣으시면 됩니다. 참고로 오른쪽에 숫자는 글자 크기입니다.

```
font = ImageFont.truetype('여기 경로를 붙여넣으세요', 30)
```

<주의>

만약 나눔폰트가 아닌 한글을 지원하지 않는 폰트를 선정하면 아래와 같은 오류가 출력됩니다.

UnicodeEncodeError: 'latin-1' codec can't encode characters in position 0-4: ordinal not in range(256)

---

step6 & step7

```
# step6.삽입할 문자의 높이, 너비 정보 가져오기
width_txt, height_txt = draw.textsize(text, font)

# step7.워터마크 위치 설정
margin = 10
x = width - width_txt - margin
y = height - height_txt - margin
```

길었던 폰트 설정을 마치고 이제 해줄 부분은 워터마크의 위치에 대한 설정입니다.

아까 저장해뒀던 draw 객체(안에는 사진이 복붙된 그림판이 들어있음)는 textsize( ) 함수를 이용해서 텍스트의 너비와 높이 정보를 가져 올 수 있습니다.

이렇게 얻은 정보를 이미지 전체의 너비, 높이, 그리고 사진 구석에서 얼마나 띄울 것인지(margin)를 계산해서 워터마크의 시작 위치(왼쪽위 꼭지점)의 위치를 정해줍니다.

step8 & step9 & step10

```
# step8.텍스트 적용하기
draw.text((x, y), text, fill=white, font=font)

# step9.이미지 출력
img.show()

# step10.현재작업 경로에 완성 이미지 저장
img.save("cat_watermakr.jpg")
```

이제 남은 것은 그림판에 문자를 입력하고, 결과를 출력하고 저장하는 것입니다.

만약 글자 색상을 설정해주는 fill을 설정 해주지 않으면 'white'가 기본값으로 들어가 있으며, 이 색깔은 CSS 컬러, HEX 컬러 모두 적용 가능합니다.

색상은 아래의 링크를 참고해서 마음에 드시는 것으로 골라서 넣으시면 됩니다.

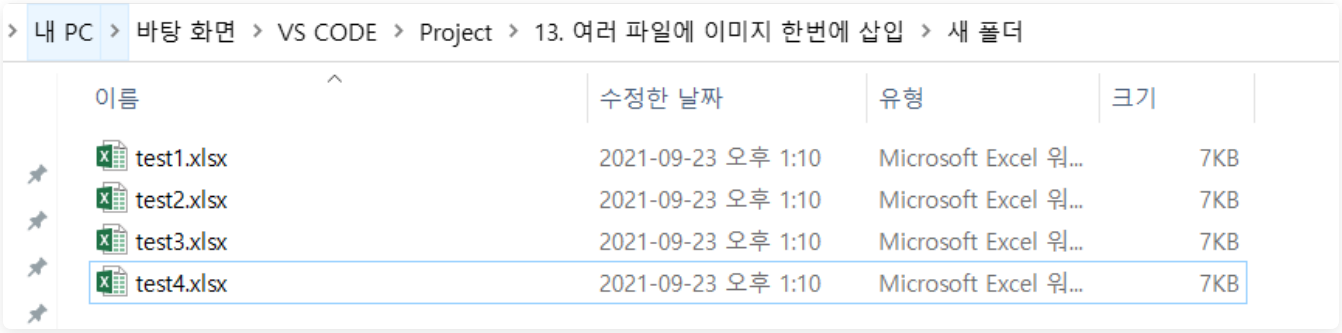
https://namu.wiki/w/%ED%97%A5%EC%8A%A4%20%EC%BD%94%EB%93%9C

마지막 step10의 경우는 저장이 필요없으시면 하지 않으셔도 됩니다.

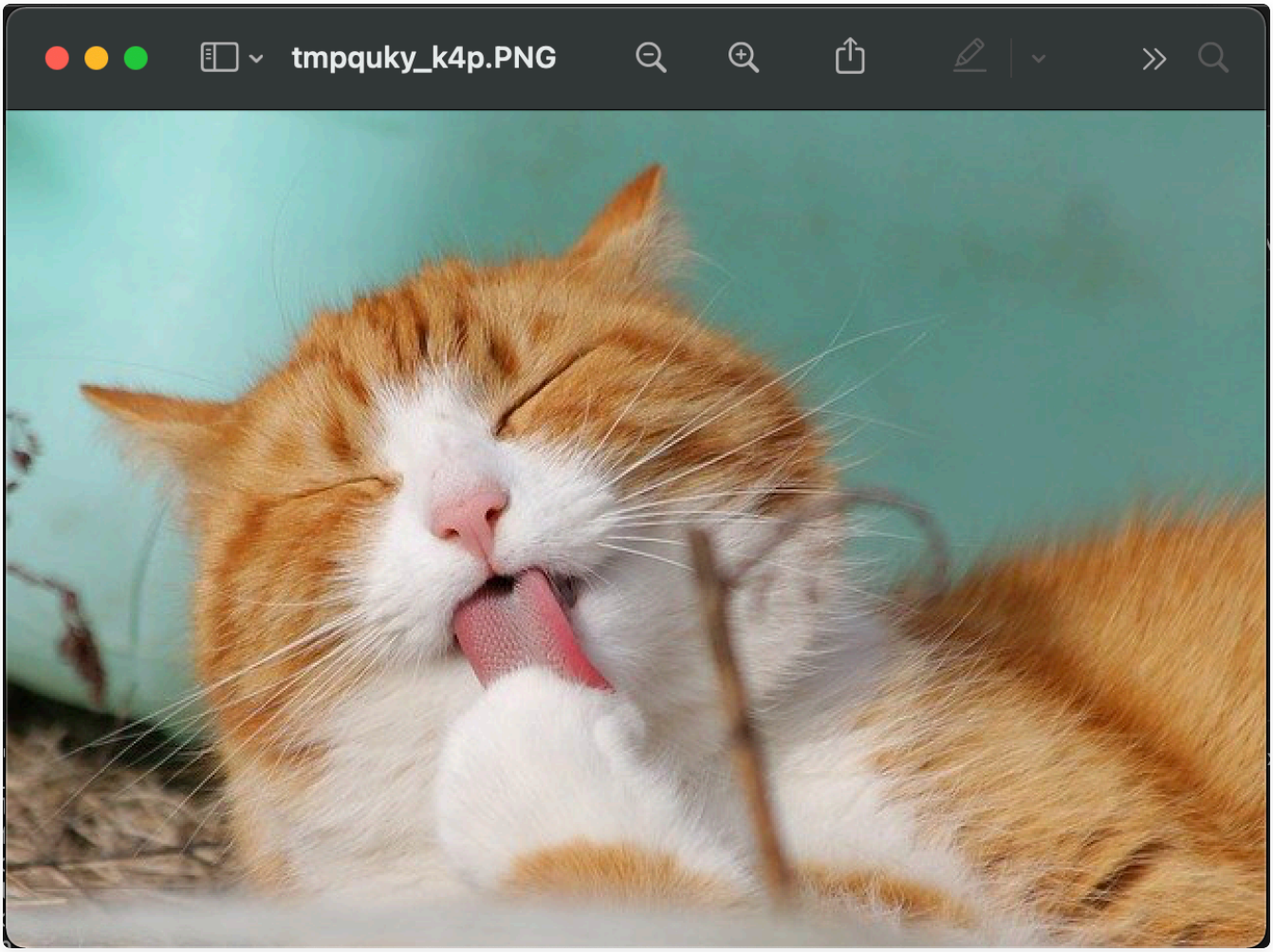
5) 이미지 엑셀에 삽입

이번 시간에는 한 폴더에 여러 엑셀파일(동일 양식)이 있을 경우 한번에 이미지를 삽입하는 방법을 배워보겠습니다.

먼저 테스트를 위해 아래 빈 엑셀 파일 4개를 생성하였습니다.



이미지는 지난 시간에 다뤘던 "cat.jpg"를 이용하겠습니다.



1. Import

<코드>

```
import os
import openpyxl as op
#엑셀 이미지 삽입을 위한 openpyxl import
from openpyxl.drawing.image import Image
#openpyxl Image 클래스와 중복되므로 PIL은 pi로 설정
from PIL import Image as pi
```

필요한 라이브러리 및 모듈을 import 하였습니다.

참고로 openpyxl.drawing.image 모듈에서 제공하는 Image 클래스와 PIL에서 제공하는 Image 클래스의 명칭이 똑같아서 PIL을 pi로 바꿔서 사용하도록 변경한 점 주의 부탁드립니다.

2. 함수 설명

함수명	매개변수	설명
resizeImage	size, image_path, img_name	-. 이미지의 사이즈를 조절하는 함수 정의 -. size : 이미지를 조절할 사이즈 -. image_path : image가 있는 원본 위치(절대 경로 또는 상대 경로) -. img_name : image 파일명
insertImage	file_path, img_path, img_name, save_path	-. 이미지를 엑셀파일 설정 시트 "A1"에 불러오는 함수 정의 -. file_path : 엑셀파일이 있는 경로 -. img_path, img_name : 위와 동일 -. save_path : 결과파일을 저장할 경로

3. 함수 구현

<코드>

```
#사이즈를 조절하는 함수
def resizeImg(size, img_path, img_name):
    #이미지 객체 생성
    img = pi.open(img_path+"/"+img_name)
    #이미지 resize
    resize_img = img.resize(size)
    #수정한 이미지 저장
    resize_img.save(img_path+"/"+img_name, "JPEG", quality=95)

#이미지를 엑셀에 삽입하는 함수
def insertImg(file_path, img_path, img_name, save_path):
    #file_path 폴더에 파일 목록을 리스트화
    file_list = os.listdir(file_path)
    #리스트 for문 실행
    for file in file_list:
        #각 파일 openpyxl Workbook 객체 생성
        wb = op.load_workbook(file_path+"/"+file)
        #시트 설정
        ws = wb.active
        #이미지 객체 생성
        img = Image(img_path+"/"+img_name)
        #이미지를 "A1"에 추가
        ws.add_image(img, "A1")
        #엑셀 파일 저장
        wb.save(save_path+"/"+file)
```

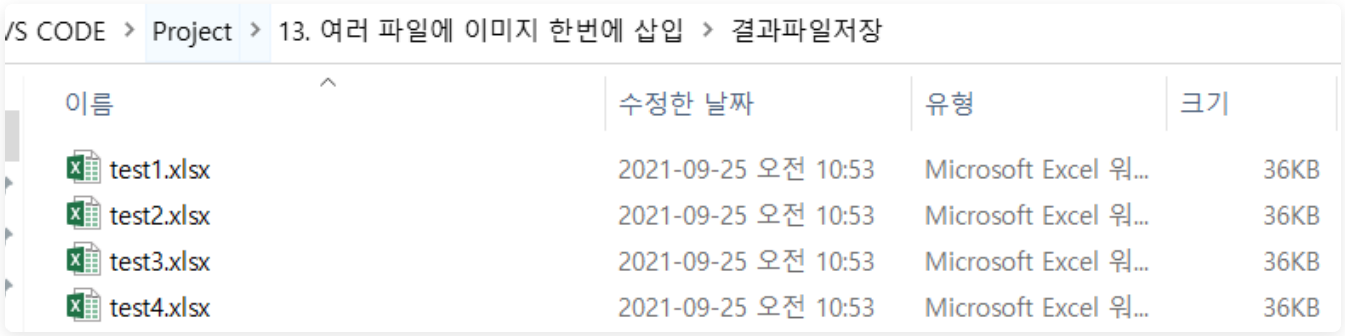
4. 실행부

<코드>

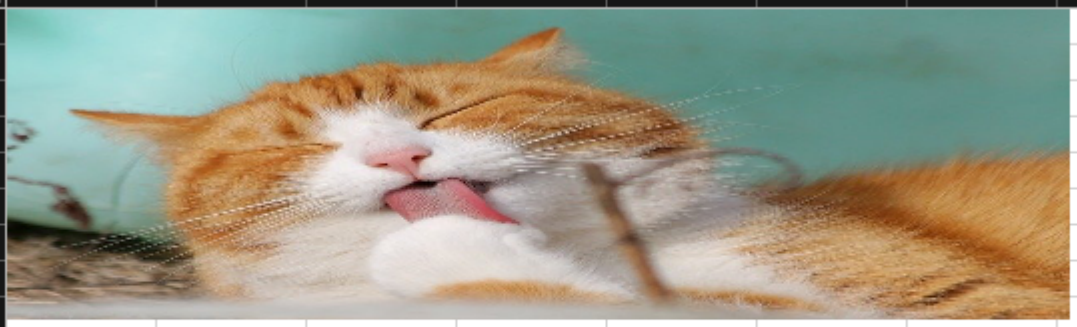
```
if __name__ == "__main__":
    #엑셀파일 PATH
    file_path = r"C:\Users\Desktop\VS CODE\Project\13. 여러 파일에 이미지 한번에 삽입\새 폴더"
    #불러올 이미지 파일 PATH
    img_path = r"C:\Users\Desktop\VS CODE\Project\13. 여러 파일에 이미지 한번에 삽입"
    #결과파일을 저장할 PATH
    save_path = r"C:\Users\Desktop\VS CODE\Project\13. 여러 파일에 이미지 한번에 삽입\결과파일저
장"
    #이미지파일명
    img_name = "cat_and_piano.JPG"
    #조절할 사이즈(500*200)
    size = (500,200)

    #함수 호출
    resizeImg(size, img_path, img_name)
    insertImg(file_path, img_path, img_name, save_path)
```

5) 결과

각 엑셀 파일에 사이즈가 조절 된 이미지가 삽입된 것을 확인할 수 있습니다.



<결과 파일> test1.xlsx 실행

	A	B	C	D	E	F	G	H
1								
2								
3								
4								
5								
6								
7								
8								
9								
10								
11								
12								
13								

3-7. PyAutoGUI 모듈

엑셀이나 웹브라우저처럼 파이썬 전용 패키지가 없어서 조작을 할 수 없는 경우도 있습니다.

예를 들어, 규모가 있는 기업에서 많이 사용하는 SAP사의 ERP(Enterprise Resource Planning)를 자동화 하고 싶은데, 이 프로그램에 자동화를 적용하려면 아마도 많은 돈이 들어갈 것입니다.

이럴 때 키보드와 마우스를 이용해서 GUI 기반의 자동화를 한다면 비용을 들이지 않고 간단한 반복업무 정도는 쉽게 처리할 수 있을거예요.

이를 위한 패키지가 PyAutoGUI라는 패키지입니다.

하나도 어렵지 않지만, 잘 모르는 사람들이 보면 감탄하는 프로그램을 만들어 볼 수 있답니다.

1) 마우스 조작하기

오늘은 엑셀이나 웹브라우저처럼 파이썬 전용 패키지가 없어서 조작을 할 수 없는 경우에 아주 유용한 PyAutoGUI라는 패키지에 대해서 배워보겠습니다.

PyAutoGUI란?

PyAutoGUI는 마우스/키보드 자동제어를 위한 패키지입니다. 흔히 알고계시는 수강신청, 콘서트 티켓 예매 매크로 같은 것을 가능케 하는 것이라 이해하시면 되겠습니다.

흔히 운영체제(OS)에 관계없이 모두 사용가능한 패키지를 크로스 플랫폼이라고 부르는데, PyAutoGUI가 바로 크로스 플랫폼 패키지입니다. 즉, Window, macOS, Linux와 같은 대중적인 운영체제에 모두 적용가능하다는 말입니다.

https://pyautogui.readthedocs.io/en/latest/index.html

PyAutoGUI 설치 방법

터미널 창에 pip 또는 conda를 통해서 설치를 하시면 됩니다. Anaconda 패키지로 파이썬을 설치하신 분이라면 두가지 중 아무거나 선택하시면 되고, 순정 파이썬을 설치하셨다면 pip로만 설치 가능합니다.

```
pip install pyautogui
```

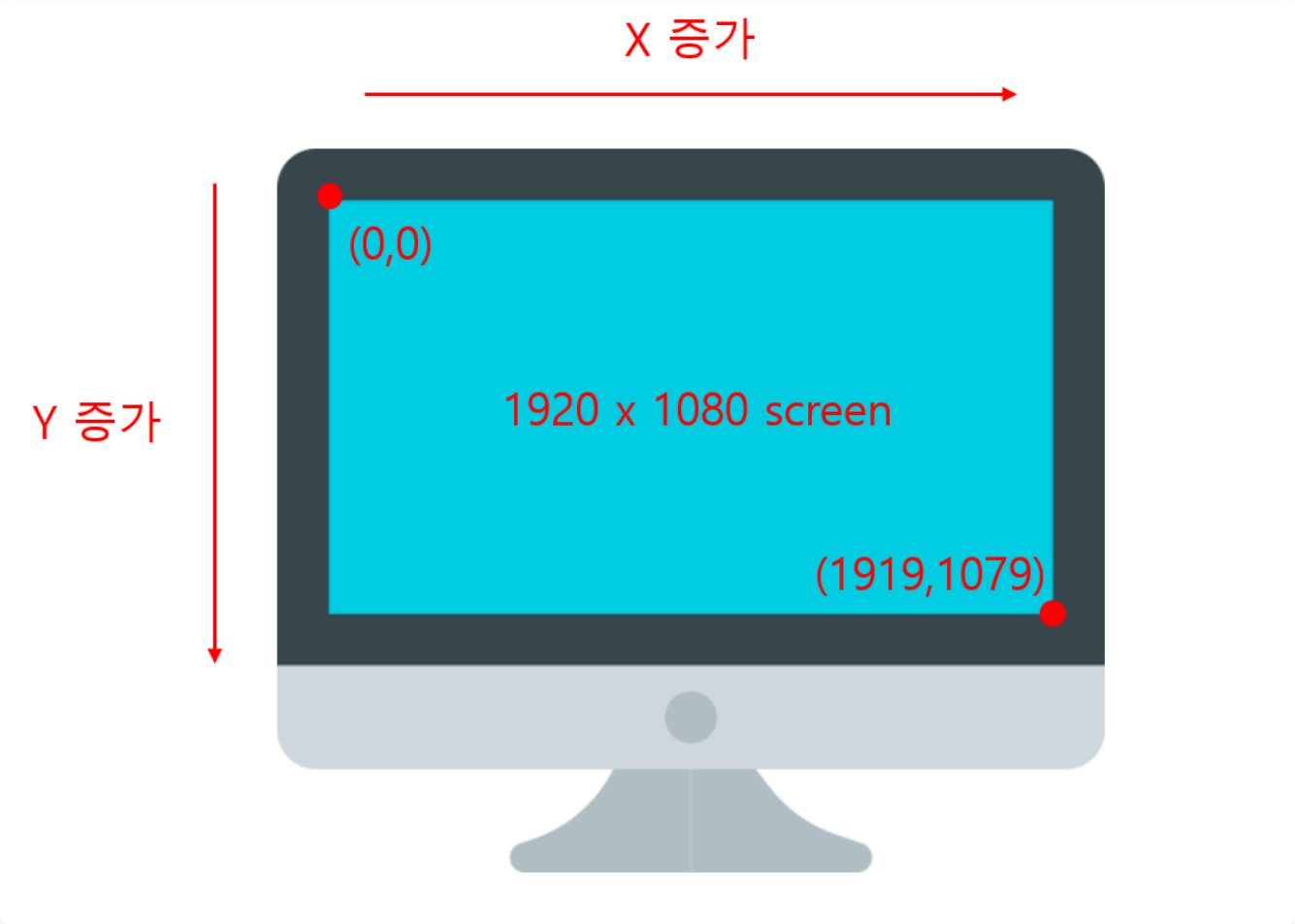
```
conda install -c conda-forge pyautogui
```

PyAutoGUI 사용 방법

1. 모니터 크기(해상도)와 마우스 현재 위치 얻기

PyAutoGUI는 크게 마우스와 키보드 조작 기능으로 나눌 수 있습니다. 그리고 마우스 조작을 위해서는 좌표에 대한 이해가 필요합니다.

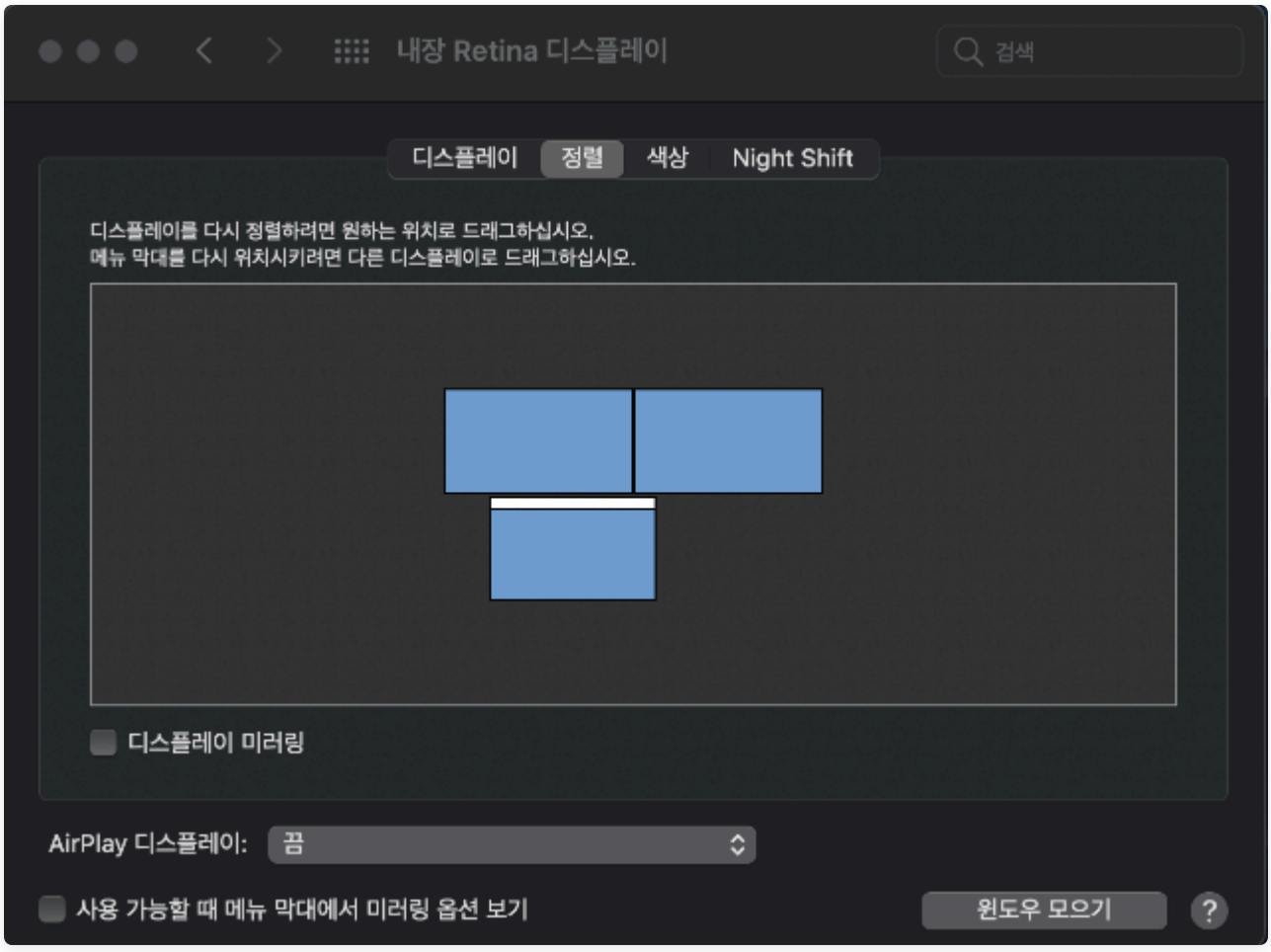
PyAutoGUI는 모니터 화면의 가장 왼쪽 위 꼭지점을 영점(0,0)으로 하며, 픽셀 단위로 x,y좌표를 가집니다.



여러분들의 모니터 해상도와 현재 마우스 위치를 확인해보기 위해서 아래의 코드를 실행해봅시다.

```
import pyautogui

# 현재 사용하는 모니터의 해상도 출력
print(pyautogui.size())

# 현재 마우스 커서의 위치 출력
print(pyautogui.position())


<결과>

>>> Size(width=1680, height=1050)
>>> Point(x=1675, y=18)
```

참고로, 저는 트리플 모니터를 사용하는데, 사이즈는 항상 1680, 1050으로 나오며, 마우스 커서의 위치는 가장 기본 모니터인 맥북 화면의 좌측상단을 기준으로 마이너스 값이 나오기도 하고, x가 3000이 넘는 값이 나오기도 하네요.

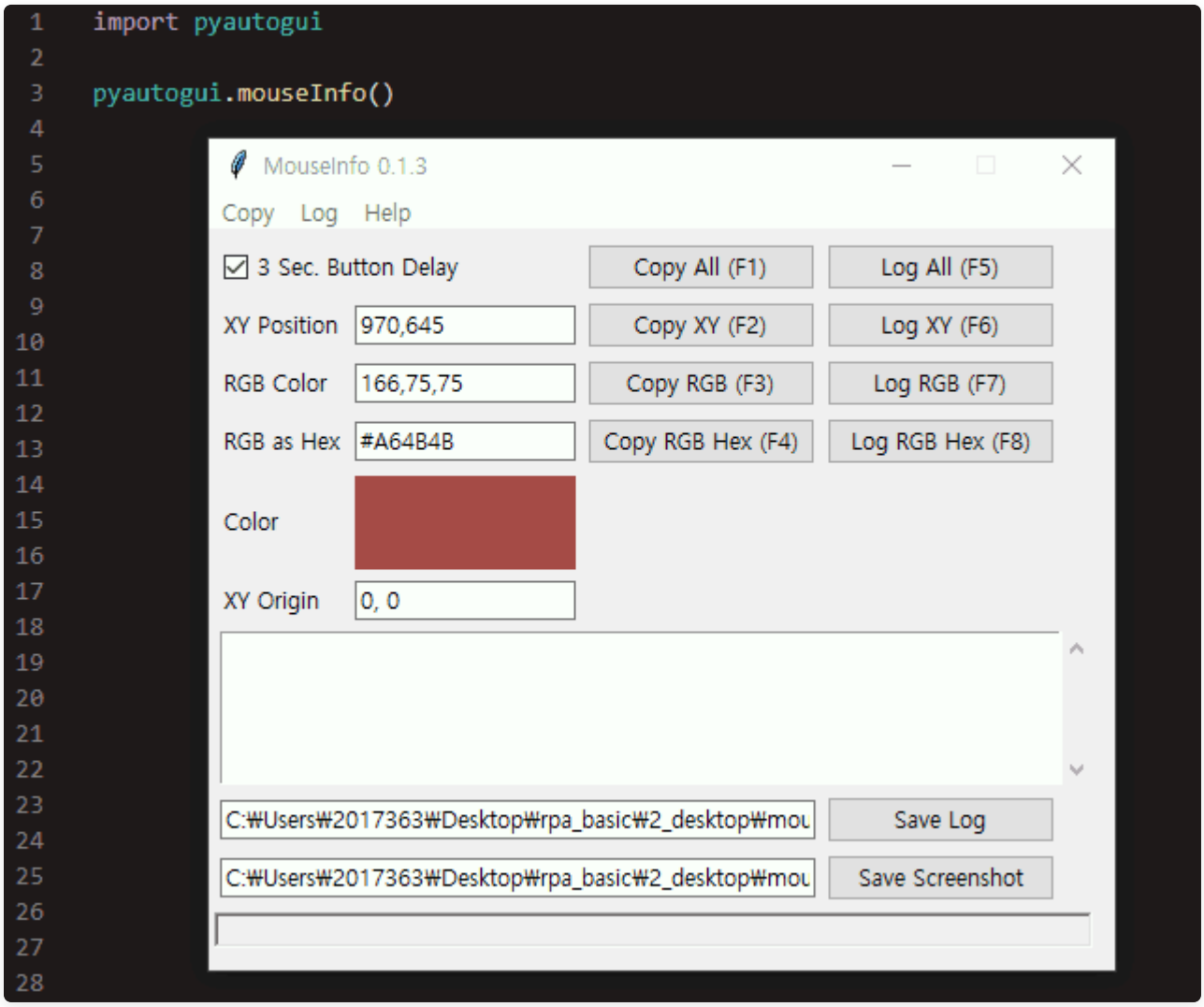


Tip. 마우스 현재 위치 & RGB 색상 실시간으로 얻기

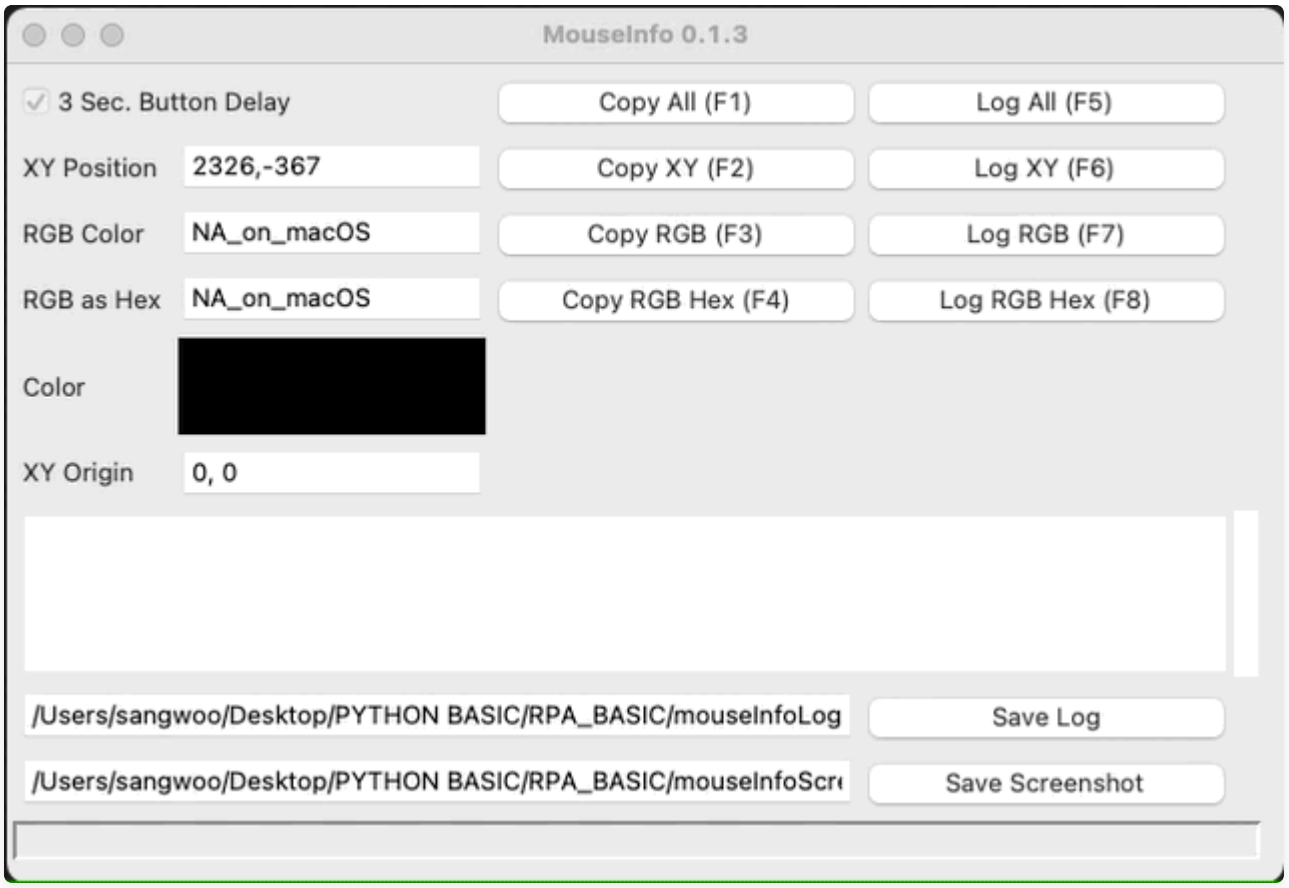
이 기능을 알기 전엔 제가 의도하는 위치의 x, y 값을 얻기위해서 위에서 배운 print(pyautogui.position( ))을 무한 반복했었습니다. 하지만 이 코드 한 줄이면 정말 편리하게 원하는 좌표를 얻을 수 있답니다.

```
pyautogui.mouseInfo( )
```

참고로, windows OS 에서는 RGB 값까지 출력되지만, mac OS에서는 RGB값을 못받아옵니다.



Windows에서 실행 시



macOS에서 실행 시

2. 마우스 이동

마우스를 이동하는 방법은 두 가지가 있습니다. 위에서 언급한 영점을 기준으로 좌표를 찍어 이동하는 절대 이동과 현재 커서의 위치를 기준으로 지정한 만큼 이동하는 상대 이동입니다. 코드를 보시면 바로 이해 가실거예요!

```
# 절대 좌표로 이동

pyautogui.moveTo(100, 100)                  # 100, 100 위치로 즉시 이동
pyautogui.moveTo(200, 200, duration=0.5)    # 200, 200 위치로 0.5초간 이동


# 상대 좌표로 이동

pyautogui.move(100, 100, duration=1)    # 현재 위치 기준으로 100, 100만큼 1초간 이동
```

3. 마우스 클릭

마우스를 이동하는 법을 배웠으면 클릭하는 법도 배워야겠죠?

가장 많이 사용하실 함수는 click( ) 함수입니다. 아래와 같이 많은 옵션이 있으니 하나하나 알아보겠습니다.

```
pyautogui.click(x=None, y=None, clicks=1, interval=0, button=PRIMARY,
                duration=0, tween=linear, logScreenshot=None, _pause=True)
```

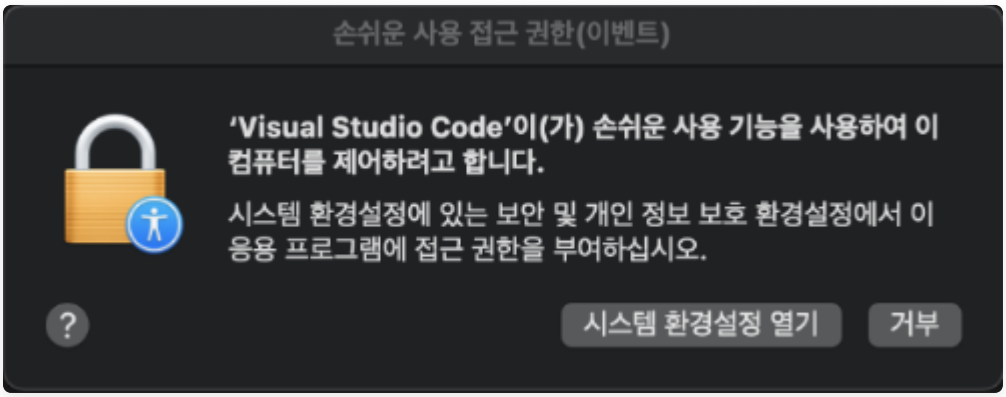
1) 단순 클릭

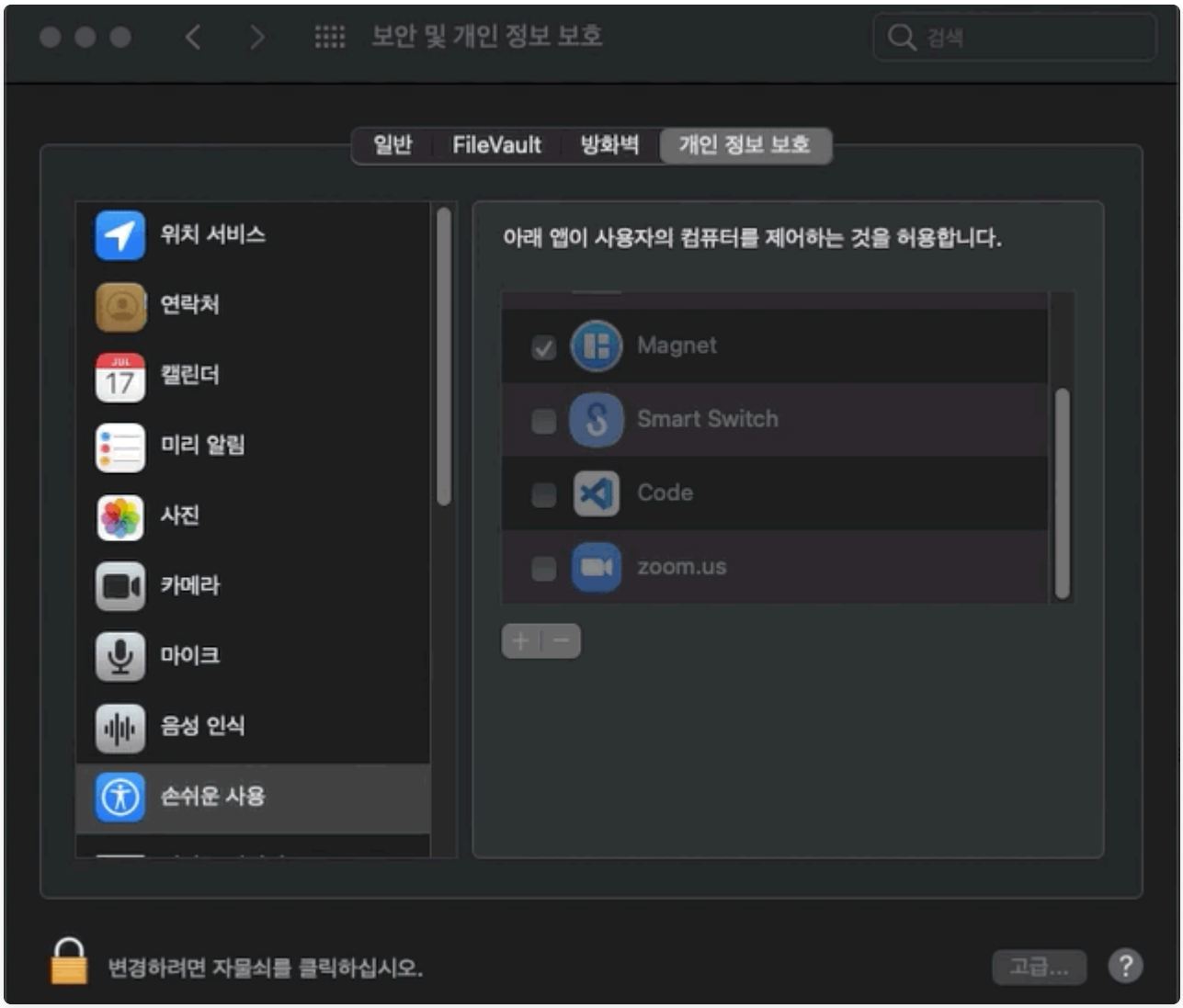
단순히 해당 위치에서 클릭을 수행합니다.

```
pyautogui.click()
```

<여기서 잠깐>

macOS의 경우엔 아래와 같이 권한 허용을 해주셔야지, 마우스 클릭 제어를 할 수 있습니다.





2) 원하는 위치 클릭

해당 위치로 가서 클릭을 수행합니다.

```
pyautogui.click(200, 200)
```

3) 클릭 횟수 & 좌/우클릭 설정

해당 위치에서 총 2번 클릭하며, 클릭간의 시간은 0.2초입니다. 마우스는 오른쪽 버튼을 클릭합니다.

```
pyautogui.click(clicks=2, interval=0.2, button='right')
```

참고로, button 옵션을 설정안하면 기본적으로 왼쪽 버튼으로 클릭하며, 'middle'과 'right', 'left'를 옵션으로 넣어줄 수 있습니다.

그 외의 옵션은 거의 사용하지 않으므로 생략하도록 하겠습니다.

4. 마우스 드래그

마우스를 드래그 하는 경우도 종종 있습니다. 사용법은 click 함수와 거의 같습니다.

```
# 1초간 400, 400 위치로 이동 후, 절대 좌표 500, 500으로 2초간 드래그
pyautogui.click(400, 400, duration=1)
pyautogui.dragTo(500, 500, 2, button='left')

# 현재 마우스 위치 기준으로 300, 300 범위만큼 왼쪽 버튼으로 드래그
pyautogui.dragRel(300, 300, 2, button='left')
```

5. 스크롤 내리기/올리기

우리는 마우스 휠을 조작해서 페이지의 스크롤을 내리고 올립니다. 이것도 아래의 코드로 가능합니다.

```
# 양수이면 위, 음수이면 아래

pyautogui.scroll(-100)

pyautogui.scroll(100)
```

2) 키보드 조작하기

이번 시간에는 지난 시간에 이어 PyAutoGUI 패키지로 키보드 조작하는 법을 공부해 보겠습니다.

키보드가 작동하는 것을 확인하려면 일단 메모장을 켜주세요. 메모장이 아니더라도 입력이 가능한 프로그램이면 뭐든지 가능합니다.

저의 경우엔 그냥 VS code 코딩창을 이용했습니다.

1. 문자 입력하기

가장 기본적인 write 함수입니다. 문자열이나 리스트의 요소를 입력하는 함수입니다.

참고로 write 함수와 typewrite는 이름만 다르고 그 기능은 같은 함수이며, 이왕이면 좀 더 짧은 write 함수를 추천드립니다.

```
import pyautogui

pyautogui.click(500, 500)

pyautogui.write('Hello world!', interval=0.2)

pyautogui.write(['H', 'e', 'l', 'l', 'o'], interval=0.2)
```

2. 개별 키 입력하기

이러한 개별 키 입력 함수는 위에서와 같이 내용을 입력하기 보다는 키 자체의 기능을 이용할 때 많이 사용할 것 같습니다.

입력 가능한 키는 보기 좋게 표로 정리해 놓았으니 참고해주세요.

```
pyautogui.press('원하는 키')
```

키	기능
'a', 'b', 'c', 'A', 'B', 'C', '1', '2', '3', '!', '@', '#' 등등	알파벳, 숫자, 특수문자 등등
'enter' (or 'return' or '\n')	ENTER
'esc'	ESC
'shift', 'shiftleft', 'shiftright'	SHIFT, 왼쪽/오른쪽 SHIFT
'alt', 'altleft', 'altright'	ALT, 왼쪽/오른쪽 ALT
'ctrl', 'ctrlleft', 'ctrlright'	CTRL, 왼쪽/오른쪽 CTRL
'tab' (or '\t')	TAB
'space'	SPACE
'backspace', 'delete', 'del'	BACKSPACE, DELETE
'pageup', 'pagedown', 'pgdn', 'pgup'	PAGE UP, PAGE DOWN
'home', 'end'	HOME, END
'up', 'down', 'left', 'right'	방향키
'f1', 'f2', 'f3', and so on	F1 ~ F12
'pause'	PAUSE
'capslock', 'numlock', 'scrolllock'	CAPS LOCK, NUM LOCK, SCROLL LOCK
'insert'	INSERT
'printscreen'	화면 캡처
'winleft', 'winright'	오른쪽/왼쪽 윈도우 키 (Windows 전용)
'command'	Command (macOS 전용)
'option'	OPTION (macOS 전용)
hanguel', 'hangul',	한/영 전환 (pyautogui에서 한글 지원 안해서 필요한지 모르겠음)

그 외에도 사용가능한 키 확인 해보시려면 아래의 코드로 확인 가능합니다.

```
print(pyautogui.KEYBOARD_KEYS)
```

3. 조합 키 입력

키를 하나가 아니라 동시에 눌러줘야하는 경우가 있습니다. 단축키 처럼 말이죠.

이 기능을 이용하면 여러분들이 아시는 단축키를 모두 사용할 수 있으니 참 유용하겠죠?!

```
# 붙여넣기 단축키
pyautogui.hotkey("ctrl", "c")

# 같은 기능 (비추천)
pyautogui.keyDown("ctrl")
pyautogui.keyDown("c")
pyautogui.keyUp("c")
pyautogui.keyUp("ctrl")
```

4. 한글 입력

위에서 배운 write 함수로는 한글을 입력할 수가 없답니다. 그래서 pyperclip이라는 패키지를 이용해서 클립보드에 잠시 한글을 저장해뒀다가 붙여넣는 방식을 사용해야 합니다.

먼저 아래의 코드를 커맨드 창에 입력하셔서 pyperclip 패키지를 설치합니다. 저는 왜인지 모르지만 이미 설치되어 있네요. 아마 Anaconda 패키지에 포함되어 있는 걸까요?

```
pip install pyperclip
```

설치를 완료하셨다면 pyperclip을 임포트 해주시고, 아래와 같이 원하는 한글 문자열을 저장했다가 Ctrl+V 단축키로 붙여넣어 주면 끝~!

```
import pyperclip

# "코딩유치원"을 클립보드에 저장
pyperclip.copy("코딩유치원")

# 클립보드에 있는 내용을 붙여넣기
pyautogui.hotkey("ctrl", "v")
```

3) 화면 이미지 인식하기

이번 시간에는 직접 좌표를 지정해서 마우스를 움직이는 방법이 아니라,

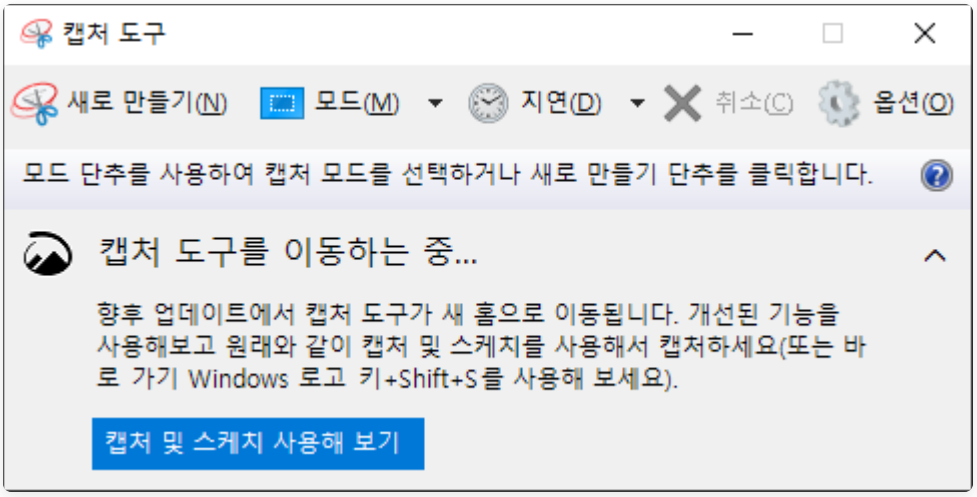
화면에서 찾고자 하는 이미지를 캡처&저장해놓고 그 이미지의 좌표를 찾아가는 방법에 대해서 배워보겠습니다.

1. 화면 원하는 부분 캡처하기

이를 위해서 먼저 화면을 캡처하는 방법을 간단히 알아보겠습니다.

1) Windows OS

- 캡처 도구 (모드는 사각형 캡처를 추천)
- 캡처 후 저장하면 자동으로 png 파일로 저장됨 (파일명은 영어로 할 것)



- Shift + 윈도우키 + S

2) mac OS

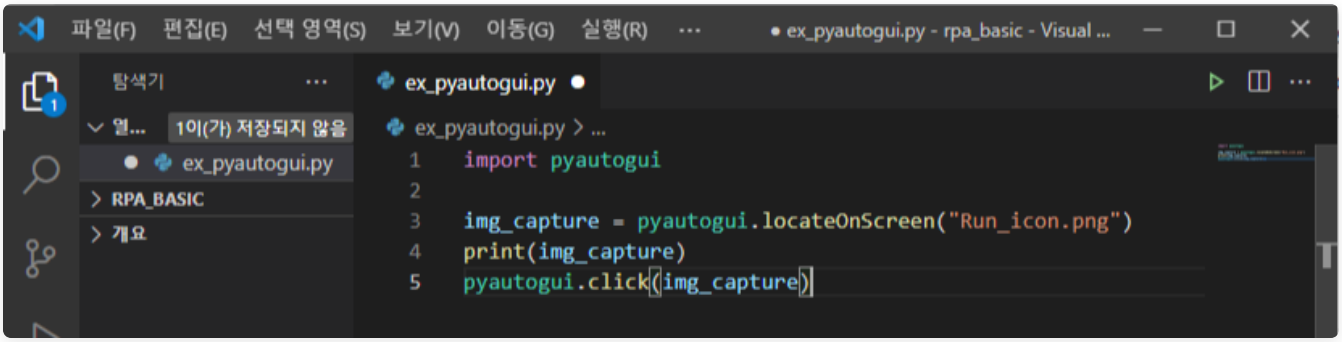
- Shift + Command + 4 (부분 캡처)
- Shift + Command + 3 (전체 캡처)
- 마찬가지로 저장 시 자동으로 png 파일로 저장됨 ((파일명은 영어로 할 것)

2. 캡처한 이미지로 좌표 정보 얻기

캡처한 이미지의 좌표를 얻으려면 locateOnScreen( ) 함수를 사용하시면 됩니다. 이때 괄호 안의 이미지는 같은 디렉토리에 위치해야 작동합니다. 굳이 이미지 파일을 다른 폴더에 위치시키고 싶다면 해당 경로를 입력해주어야합니다.

저의 경우는 화면 우측상단의 Run 버튼을 캡처해서 찾아보는 코드를 작성해보았습니다. img_capture 변수에 들어가 있는 값을 확인해보면 **Box(left=x 좌표, top=y 좌표, width = 이미지 너비, height=이미지 높이)**가 출력되는 것을 확인하실 수 있습니다.

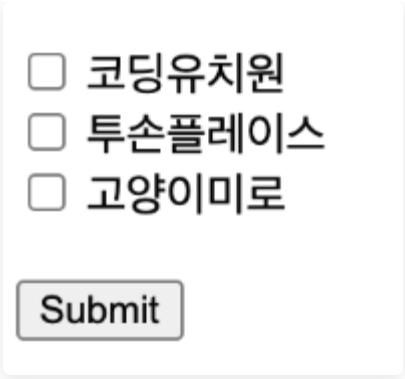
```
img_capture = pyautogui.locateOnScreen("Run_icon.png")

print(img_capture)

pyautogui.click(img_capture)
```



이미지의 경로가 제대로 설정되지 않은 경우에는 에러가 출력되며, 경로는 정확하지만 화면상에서 해당 이미지를 못 찾았을 때는 None(값이 없음을 의미)을 돌려줍니다.

3. 캡처한 이미지가 화면에 여러개 일 때

예를 들어서 아래와 같은 체크박스를 중복 선택해야한다면 어떻게 해야할까요?



그럴 때는 아래와 같이 locateAllOnScreen( ) 함수와 for 문으로 해결 가능합니다. 위에서 배운 locateOnScreen( ) 함수와 무엇이 다른지 보이시죠? 해당 함수를 사용하면 원하는 이미지의 위치들을 리스트에 저장해놓고 for문으로 하나씩 꺼내어 쓰는 개념입니다.

```
for i in pyautogui.locateAllOnScreen("checkbox.png"):

    pyautogui.click(i, duration=0.25)
```

4. 범위 안에서만 캡처한 이미지를 찾고 싶을 때

다른 영역에 있는 중복되는 이미지를 무시하고 싶거나, 이미지 검색 범위를 좁혀 속도를 개선하고 싶을 때 사용하면 좋을 것 같아요.

```
import pyautogui

img_capture = pyautogui.locateOnScreen("Run_icon.png", region=(1800, 0, 1920, 100))

pyautogui.moveTo(img_capture)
```

Tip. 코드 실행 속도 측정법

```
import pyautogui
import time

start = time.time()  # 시작 시간 저장

img_capture = pyautogui.locateOnScreen("Run_icon.png")

pyautogui.moveTo(img_capture)

# 현재시각 - 시작시각 = 실행 시간
print("코드 실행 시간 :", time.time() - start)
```

5. 이미지 인식이 됐다 안됐다 할 때

가끔 찾고자 하는 이미지와 화면의 이미지가 어떤 이유로 100% 일치하지 않아서 잘 되던 코드가 실행이 되지 않을 경우도 있습니다. 저는 아직 겪어보지 않았지만 혹시 이런 현상이 발생한다면 지금 알려드리는 옵션을 사용해보세요!

이런 경우에는 예를 들어 일치도를 70% 이상이면 같은 이미지로 인식하라고 설정할 수가 있습니다.

이 기능을 사용하려면 아래의 패키지를 설치해주셔야 합니다.

```
# 이미지 처리 패키지인 opencv-python 설치
pip install opencv-python
```

사용법은 간단합니다. 지금껏 배워왔던 locateOnScreen 함수안에 confidence 옵션을 설정해주시면 됩니다.

```
img_capture = pyautogui.locateOnScreen("Run_icon.png", confidence=0.7) # 70%

pyautogui.moveTo(img_capture)
```

4) 메시지 박스 띄우기

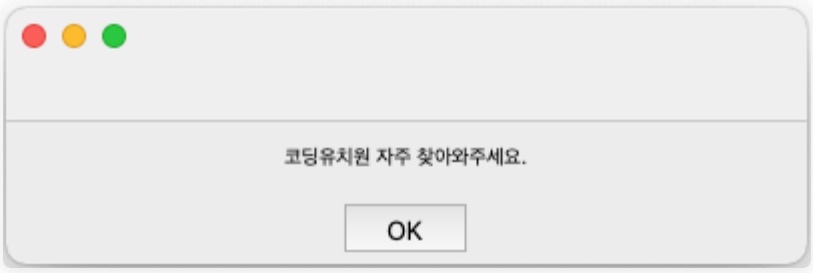
오늘은 지난 시간에 이어서 작동중에 사용자의 확인이 필요하거나 문자 혹은 패스워드를 입력 받아야 하는 경우에 대처할 수 있는 메세지 박스 기능에 대해 배워보겠습니다.

---

1. 알림창 - alert( )

가장 기본적인 알림을 띄워보겠습니다.

```
import pyautogui

pyautogui.alert('코딩유치원 자주 찾아와주세요.')
```



alert( ) 함수에는 어떤 파라미터가 있는지 한 번 보겠습니다.

파라미터	(자료형) 입력값	기본값
text	(str) 원하는 메시지	X
title	(str) 창 제목	X
button	(str) 원하는 버튼 이름 (bool) True/False	OK
timeout	(int) msecond (단위: 1/1000초)	None (시간제한 없음)

alert( ) 함수의 버튼을 클릭하거나 종료하면는 위에서 설정했던 버튼 이름을 문자열로 리턴해줍니다.

만약 timeout을 설정하셨다면 설정한 시간이 끝나고 timeout을 문자열로 리턴해줍니다.

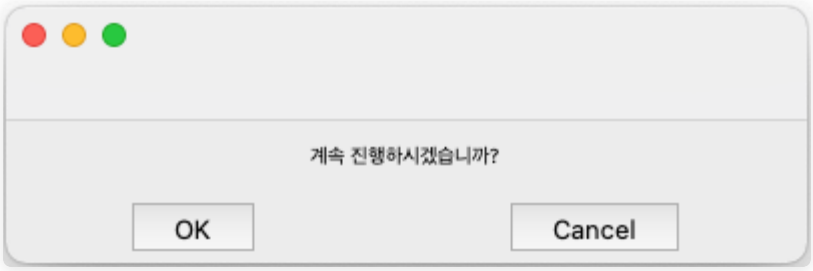
아래의 예제 코드를 실행하셔서 어떻게 작동하는지 확인해보세요.

```
alert = pyautogui.alert('코딩유치원 자주 찾아와주세요.', title='코딩유치원', button='확인', timeout = 3000)

print(alert)
```

2. 확인창 - confirm( )

위에서 배운 알림창과 다른 것은 버튼 2개 중 하나를 선택할 수 있다는 것입니다.

```
import pyautogui

pyautogui.confirm('계속 진행하시겠습니까?')
```



confirm( ) 함수도 어떤 파라미터가 있는지 보겠습니다.

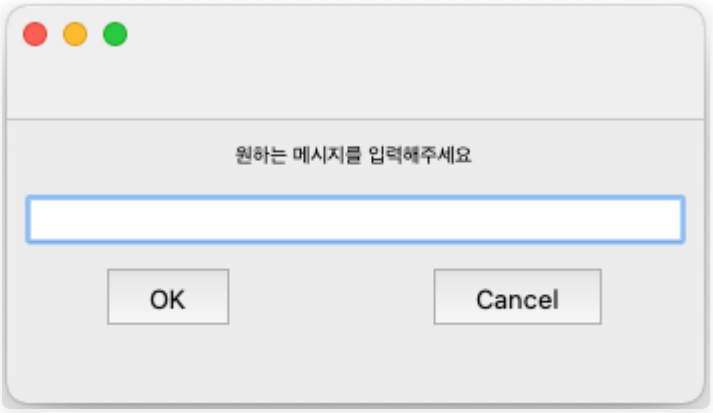
파라미터	(자료형) 입력값	기본값
text	(str) 원하는 메시지	X
title	(str) 창 제목	X
buttons	(tuple) ('버튼 이름1', '버튼 이름2')	OK & Cancel
timeout	(int) msecond (단위: 1/1000초)	None (시간제한 없음)

이 함수 역시 위에서 배운 alert( )와 같이 선택한 버튼의 버튼명을 문자열로 리턴해줍니다.

3. 텍스트 입력창 - prompt( )

이번에는 텍스트를 입력받는 것 까지 추가된 창을 띄워보겠습니다.

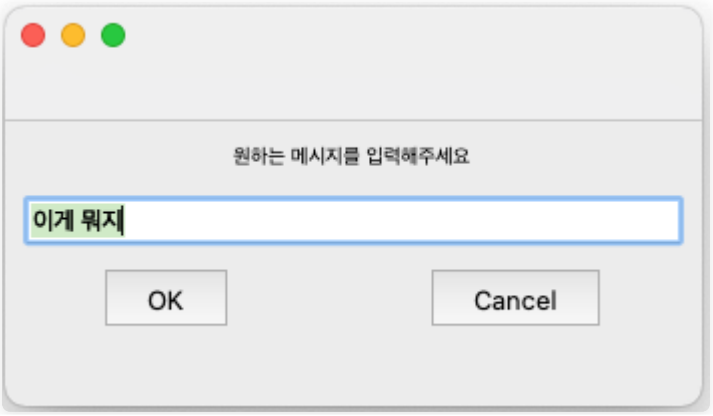
```
import pyautogui

text = pyautogui.prompt('원하는 메시지를 입력해주세요')
```



마찬가지로 설정 가능한 파라미터를 보겠습니다. default 말고는 이미 봤던 친구들이네요.

파라미터	(자료형) 입력값	기본값
text	(str) 원하는 메시지	X
title	(str) 창 제목	X
default	(str) 기본적으로 입력창에 입력된 문자열	X
timeout	(int) msecond (단위: 1/1000초)	None (시간제한 없음)

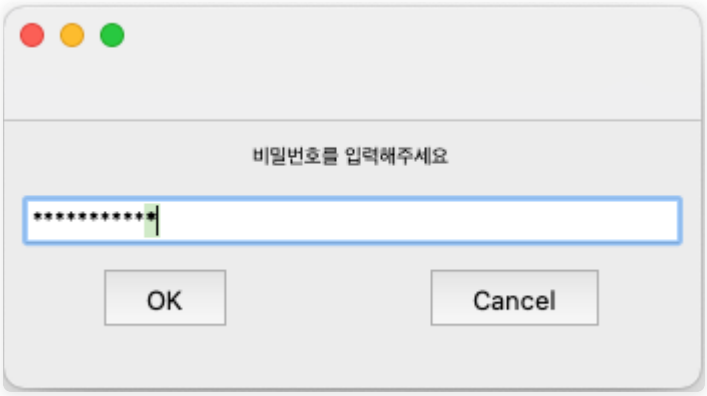
default 옵션이 뭔가 했더니 메시지 입력창에 처음부터 입력되어 있는 메시지였습니다.



prompt( ) 함수는 OK 버튼을 누르면 위에서 입력받은 텍스트를 문자열로 리턴해주며, Cancel 버튼을 누를 시엔 None을 리턴해줍니다.

4. 비밀번호 입력창 - password( )

이 함수는 위에서 배운 prompt( ) 함수와 같지만 입력 시에 *를 출력하여 보안을 위한 기능을 추가한 함수입니다.



위의 prompt( ) 함수와 다른 파라미터는 mask 하나입니다. 만약 mask를 '='로 해주면 * 대신에 =으로 출력됩니다.

파라미터	(자료형) 입력값	기본값
text	(str) 원하는 메시지	X
title	(str) 창 제목	X
default	(str) 기본적으로 입력창에 입력된 문자열	X
mask	(str) 원하는 마스크 기호	*
timeout	(int) msecond (단위: 1/1000초)	None (시간제한 없음)

password( ) 함수는 prompt( ) 함수와 마찬가지로 OK 버튼을 누르면 위에서 입력받은 텍스트를 문자열로 리턴해주며, Cancel 버튼을 누를 시엔 None을 리턴해줍니다.

3-8. 웹크롤링

1. 웹 (Web)

Web은 World Wide Web의 줄임말입니다. 우리가 인터넷 웹사이트의 주소를 칠 때 'www.google.com' 이런 식으로 입력하죠?

바로 이 www가 World Wide Web에서 유래된 것입니다.

흔히 인터넷과 웹을 자주 혼동해서 사용하는데요.

인터넷은 **컴퓨터 네트워크 통신망**을 의미하며 **웹**은 **인터넷 상에서 동작하는 하나의 서비스**입니다.

우리가 인터넷과 웹을 혼동해서 사용하는 이유는 그만큼 인터넷을 활용한 서비스에서 웹의 영향력이 압도적이기 때문입니다.

2. 웹 브라우저 (Web Browser)

Web은 HTML(Hypertext Markup Language)이란 언어로 작성됩니다.

[HTML 예시]

```
<!DOCTYPE html>
<html lang="ko">

<head>
    <meta charset="UTF-8">
    <title>HTML Intro</title>
</head>

<body>

    <h1>여기는 코딩유치원입니다</h1>

</body>


</html>
```

HTML로 작성된 웹을 보기 편하게 해주는 소프트웨어를 웹 브라우저(Web browser)라고 하는데요.

여러분들이 흔히 아시는 인터넷 익스플로러, 크롬, 사파리 등이 있습니다.

출처. pixabay

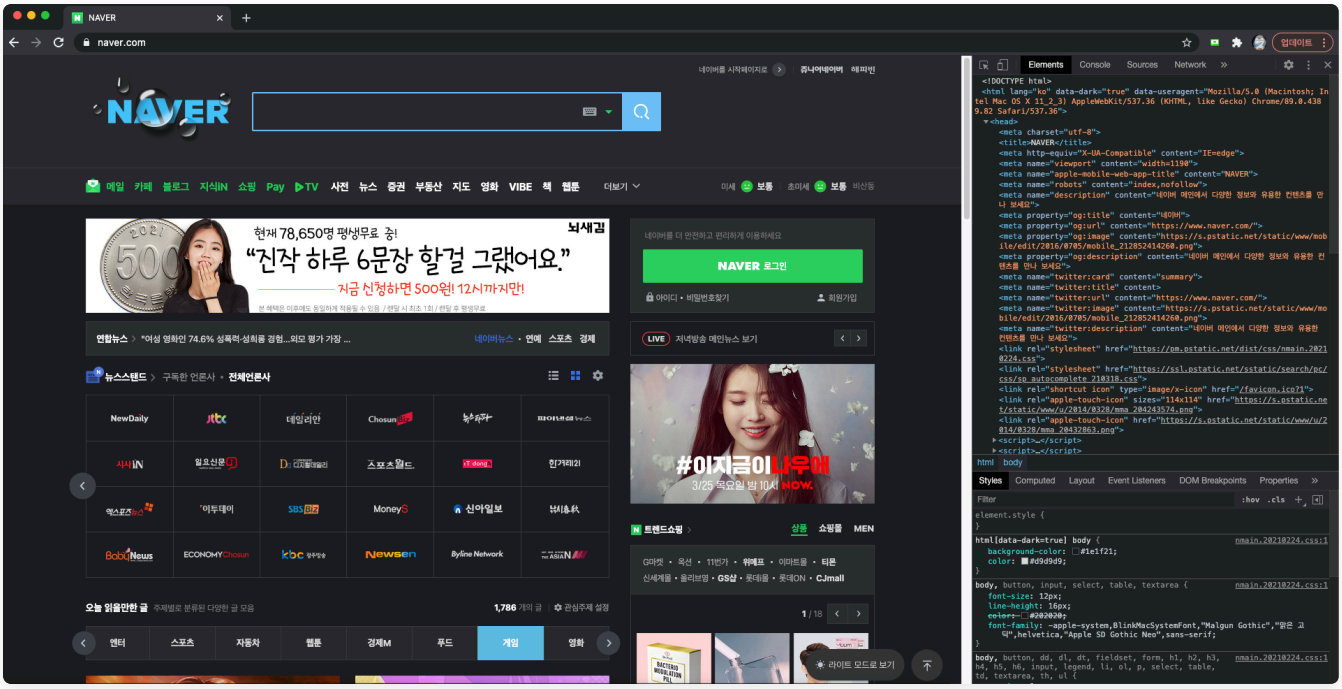
3. 웹 크롤링과 웹 스크래핑

우리는 앞으로 파이썬을 이용해 웹 페이지에서 HTML 문서를 분석해서 원하는 정보를 얻어오는 웹 크롤링을 해볼 예정입니다.

참고로 웹 크롤링이란 용어가 많이 사용되는데 사실 웹 크롤링은 싹 다 긁어 오는 것이고, 웹 스크래핑은 원하는 부분만 선택적으로 추려내는 것이니 우리가 할 것은 웹 스크래핑이라고 할 수 있겠네요.

앞으로는 우리는 웹 크롤링을 위해서 크롬(Chrome)을 사용 할 예정인데요.

크롬을 켜고 우클릭-검사 혹은 F12를 누르면 아래 사진과 같이 우측에 HTML 문서가 쭉 나옵니다.



1) 크롬과 크롬 드라이버

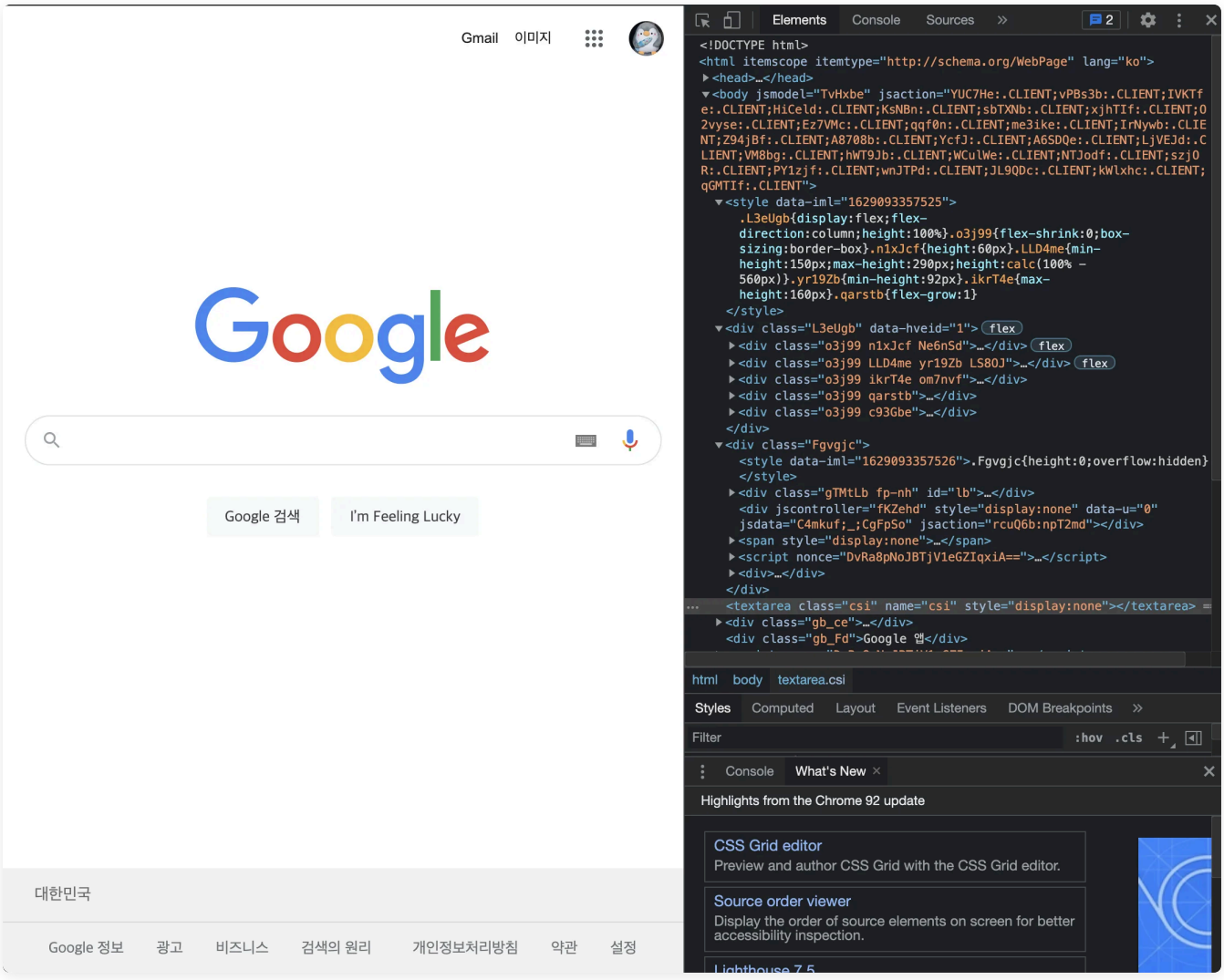
오늘은 웹 크롤링을 위한 준비 단계로 크롬과 크롬드라이버를 설치해보겠습니다.

1. 크롬을 사용하는 이유

웹크롤링을 할 때는 크롬 브라우저를 사용하는 것이 아주 편합니다.

그 이유는 아래와 같습니다.

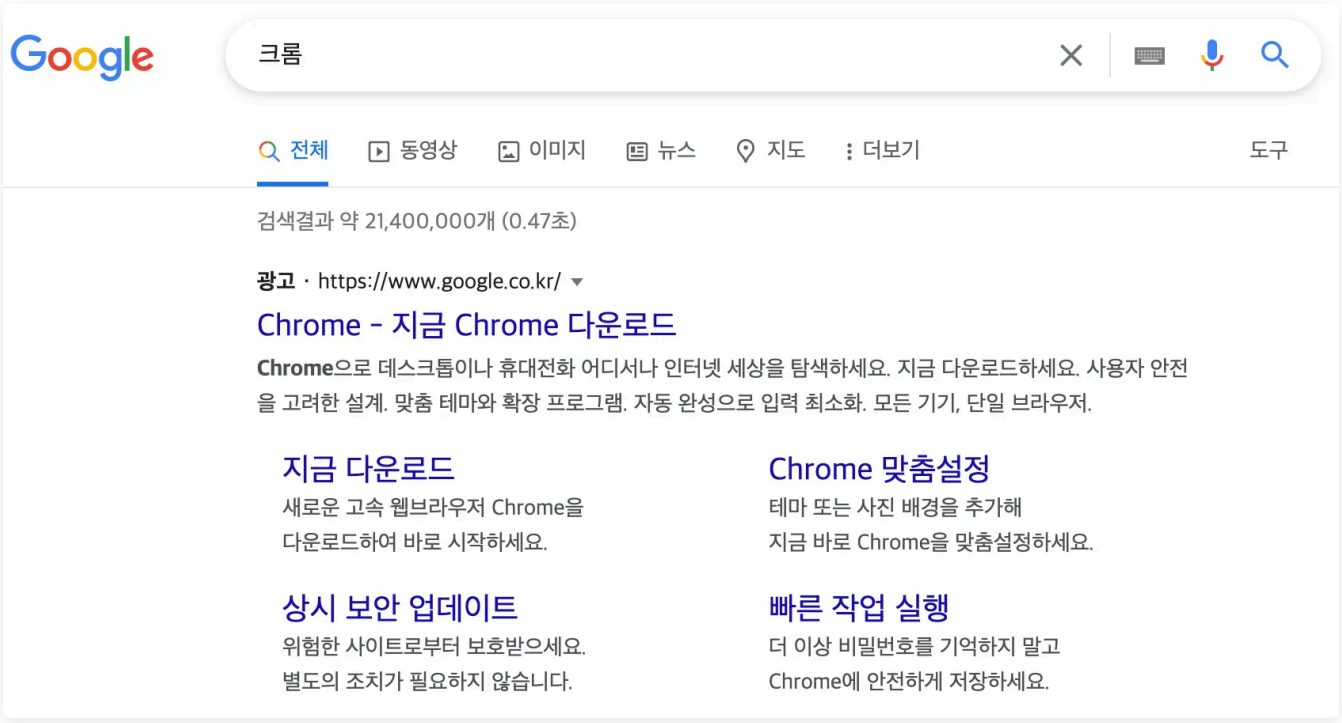
1) F12를 눌렀을 때 오른쪽에 출력되는 HTML 문서 (정식명칭: 개발자 도구)



2) 컴퓨터가 크롬 브라우저를 제어할 수 있게 해주는 크롬 드라이버

크롬의 설치방법은 정말 간단합니다.

구글이나 네이버에서 '크롬'을 검색하시고 가장 위에 나오는 결과를 클릭해주시면 자연스럽게 크롬 설치파일을 다운로드하는 링크로 연결됩니다.



다운로드 파일을 받고 설치를 진행해주시면 되겠습니다.

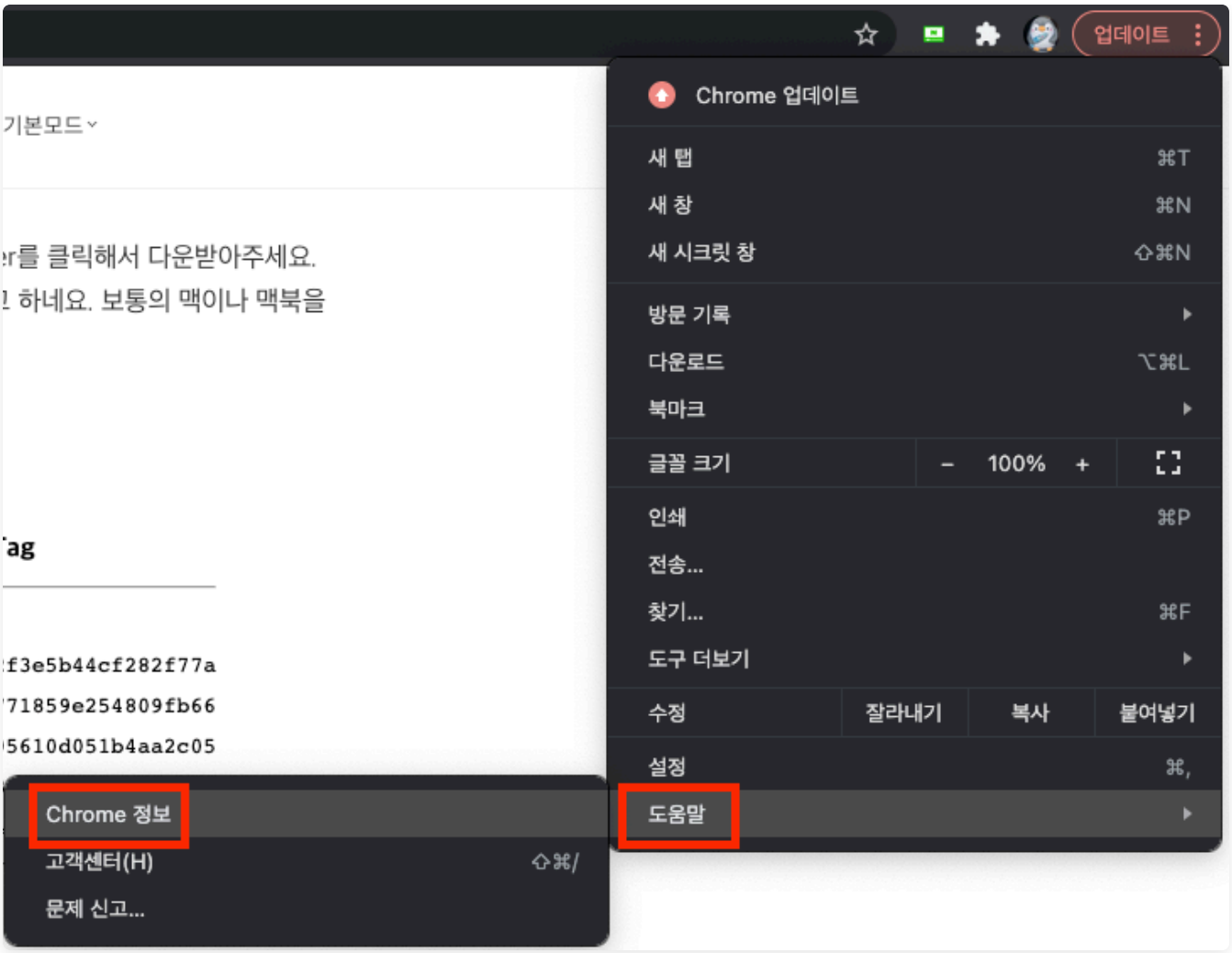
2. 크롬 버전 확인 방법

크롬을 설치하셨다면 버전을 확인할 필요가 있습니다. 크롬 드라이버를 다운로드 받을 때 크롬의 버전과 일치하는 것을 받아줘야 정상 작동하니까요.

크롬 창을 열면 우측상단에 (...) 표시가 보이실겁니다.

저는 글 쓰는 현재 크롬의 버전이 최신이 아니라서 업데이트라는 붉은색 글씨가 함께 나오네요.

클릭해주시면 아래와 같이 창이 나오는데 도움말 - Chrome 정보를 클릭해 주세요.



클릭하시면 새로운 창이 뜨면서 아래와 같이 나옵니다.

최신 버전이 아니었을 때

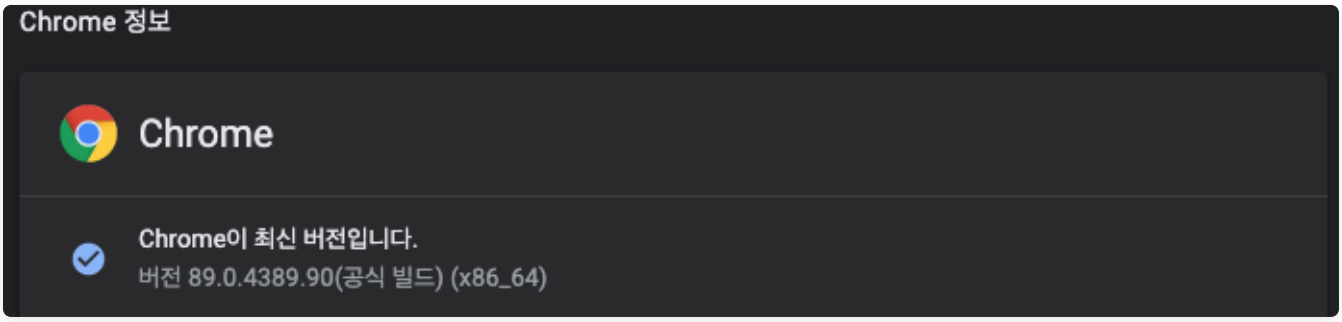
앞서 언급 하였 듯이 최신버전이 아니라서 아래와 같이 자동 업데이트된 후, 크롬을 다시 시작하라고 나왔습니다.

이 버전 (위의 사진에서는 89.0.4389.82)을 기억하고 다음 단계로 넘어가 보겠습니다.

최신 버전이었을 때

방금 크롬을 처음 설치하신분들은 최신버전일테니 분들은 아래의 두번째 사진처럼 최신버전이라고 나올거예요.

마찬가지로 버전을 기억하고 다음 단계로 넘어갑니다.



> *[Tip]*
>
> *한동안 웹크롤링을 하지 않다가 실행하시면 되던 코드가 실행되지 않을 때가 있습니다.*
>
> *그 때는 크롬 버전에 비해 크롬 드라이버가 예전 것일 확률이 높으니, 크롬 드라이버를 최신 것으로 다시 다운받아서 교체해주시면 됩니다.*

3. 크롬드라이버 설치 방법

뒤에서 배울 selenium이라는 패키지가 2022년 4월에 4.6 버전으로 업데이트 되기 전까지는 크롬 드라이버를 매번 크롬 버전에 맞춰서 설치해주어야 했지만, 더 이상 그런 번거로운 일을 하지 않아도 됩니다.

따라서 예전에 설명드렸던 크롬 드라이버 설치에 대한 내용은 더 이상 필요 없어 졌기 때문에 삭제하였습니다.

다만, 4.6 보다 더 오래된 버전을 사용하시는 경우에는 다음과 같은 에러가 출력될 수 있습니다.

```
selenium.common.exceptions.SessionNotCreatedException: Message: session not created: This vers
ion of ChromeDriver only supports Chrome version 106

Current browser version is 116.0.5845.111 with binary path C:\Program Files\Google\Chrome\Appl
ication\chrome.exe
```

이런 경우에는 selenium을 최신 버전으로 업그레이드 한 후, VSCODE를 종료했다가 실행하셔야 합니다. 최신 버전 업그레이드는 터미널 창이나 Jupyter의 셀에서 아래 명령어를 입력하시면 됩니다.

pip install --upgrade selenium

2) 속성 HTML/CSS 강의

웹 크롤링을 책이나 강의를 보고 독학으로 공부하다보면 여러 예제들이 나옵니다.

그 예제들을 그대로 따라해보면 자동으로 착착 움직이며 텍스트나 그림 등을 가져오는 것을 보면서 웹크롤링을 마스터한 듯한 기분이 듭니다.

하지만 웹페이지의 형태가 조금만 바뀌어도, 수집하고자 하는 데이터의 위치나 종류가 조금만 바뀌어도 그 생각은 바로 착각이었다는 것을 깨닫게 되죠.

우리가 어떤 웹사이트를 만나던 어떤 종류의 자료든 크롤링 할 수 있으려면, HTML과 CSS에 대한 개념을 이해할 필요가 있습니다.

두 언어만 공부하는데에도 각각 책이 한 권 씩일 정도로 내용이 많으므로, 최소한의 필요한 부분만 공부해 보도록 하겠습니다.

1. HTML 구조

html은 기본적으로 아래와 같은 구조를 가집니다.

한마디로 html은 태그로 감싸진 속성과 내용들의 모음입니다.

출처. TCP School

1) 태그 이름

태그는 그 종류가 다양하며, 각 태그마다 의미하는 바가 다릅니다. 우리가 웹크롤링을 하면서 자주 볼 수 있는 태그는 아래와 같습니다.

한 번에 다 이해하려고 하실 필요는 없습니다! 필요할 때마다 해당 표를 참고하시면 되니깐 여러가지의 태그가 있다라는 정도만 이해하고 넘어가시면 되겠습니다.

태그	설명	사용 예
h1~h6	제목을 입력할 때 사용 (h1이 가장 큰 제목, h6이 가장 작은 제목)	제일 큰 제목
p	하나의 문장을 입력할 때 사용	문장
div	박스형태의 구역 설정 (block 요소) (다른 태그들이 div 안에 모여있게 됨)	 제목 문장
span	줄 형태의 구역 설정 (inline 요소) (독립적으로 사용하지 않고 p태그 안에 span 태그가 들어감)	이렇게 span요소로 텍스트의 일부분 만 스타일을 적용할 수 있음
img	이미지와 관련된 태그 (속성명은 src, 속성값은 "이미지의 url 주소") 종료 태그(/img)가 없는 빈 태그(empty tag)	<img src="/img 주소.png" alt="이미지가 없을 때 이미지 대신 출력할 문장 입력">
a	하이퍼링크를 추가할 때 사용	HTML 링크
ul	unordered list : 기호로 된 리스트 li 태그가 하위 태그로 사용되어 내용을 채움	 - 기호 - 기호 - 기호
ol	ordered list : 순서가 있는 리스트 li 태그가 하위 태그로 사용되어 내용을 채움	 1번 2번 3번

2) 속성명

html은 수 많은 태그로 이루어져 있어서 **동일한 태그가 무수히 많이 사용**됩니다.

그래서 **각 태그들을 구분해주기 위해서 속성을 부여** 해줍니다.

각 태그별로 속성이 다양하지만 꼭 기억해야할 속성 2가지는 id와 class입니다.

ID

하나의 웹페이지에 **하나만 쓸 수 있는 고유한 이름**으로 <태그이름 id="속성값">와 같이 쓰임

CLASS

비슷한 형태를 가진 요소에 **여러번 사용할 수 있는 이름**으로 <태그이름 class="속성값">와 같이 쓰임

3) 속성값과 내용

프로그래머가 정해주기 나름

---

2. CSS 구조

css는 html로 만들어진 밋밋한 화면을 예쁘게 꾸며주는 역할을 합니다.

css는 html 특정 태그를 지목해서 속성값(글자색, 크기, 폰트, 배경색 등등)을 넣어 주는 것이라 생각하시면 돼요.

이때 **css가 특정 태그를 지목하는 방식, 이 규칙을 알게되면 우리가 원하는 데이터를 감싸고 있는 태그를 지목해서 그 안의 데이터를 가져올 수 있는 것입니다.**

출처. TCP School

CSS Selector

CSS 코드는 위와 같이 선택자(selector)와 선언부(declaratives)로 구성됩니다.

여기서, 선택자라는 것이 우리가 주목해야할 개념입니다.

간단히 정리하면 아래와 같습니다.

분류	설명	예
태그 선택	특정 태그를 선택	div --> 태그를 선택
아이디 선택	id='속성값'인 태그를 선택	#query --> id의 속성값이 query인 태그 선택
클래스 선택	class='속성값'인 태그를 선택	.title --> class의 속성값이 title인 태그 선택
태그+아이디 선택	특정 태그 중 id가 '속성값'인 태그를 선택	input#query --> input 태그 중, id의 속성값이 query인 태그 선택
태그+클래스 선택	특정 태그 중 class가 '속성값'인 태그를 선택	p.title --> p 태그 중, class의 속성값이 title인 태그 선택

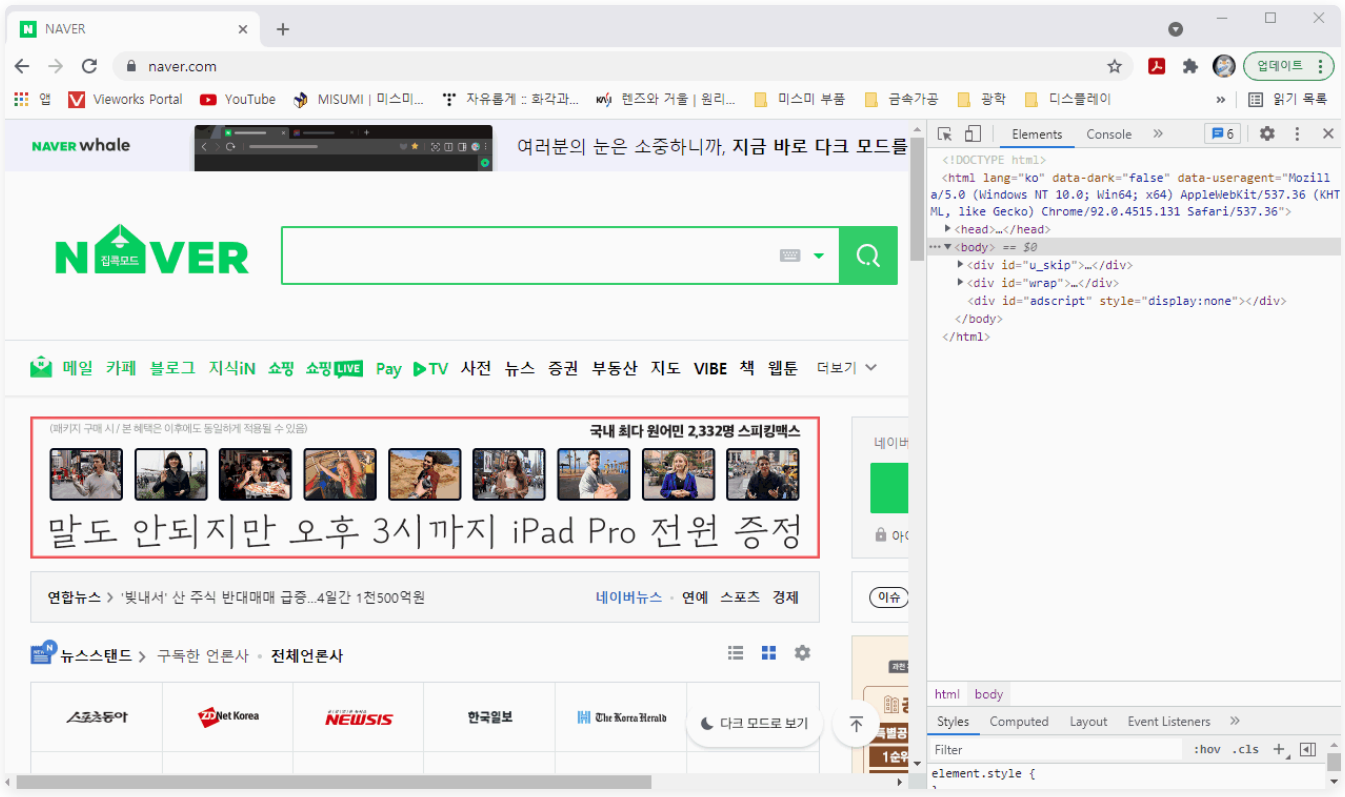
쉬운 이해를 위해서 CSS 선택자를 어떻게 활용하는지 예시를 보여드리겠습니다.

1) 크롬 브라우저로 네이버에 접속

2) 개발자 도구(단축키:F12)를 열어서, 검색창의 HTML을 확인

3) HTML을 우클릭해서, Copy - Copy selector를 클릭

4) 코딩창의 원하는 위치에 붙여넣기(Ctrl+V)



이렇게 하면 #query라는 css selector가 불러와지고, 이 선택자를 아래 코드들의 괄호 안에 넣으시면 원하시는 동작을 수행하거나 데이터를 가져올 수 있습니다.

```
# selenium으로 id가 query인 html 1개 선택(여러개 일 때 가장 위의 요소 선택)
driver.find_element_by_css_selector('#query')

# selenium으로 id가 query인 html 모두 선택
driver.find_elements_by_css_selector('#query')
```

개인적으로 CSS selector 개념이 직관적이고 이해하기 쉬워서 주로 이용할 예정입니다.

하지만 인터넷에 돌아다니는 웹 크롤링 관련 코드들은 CSS selector를 사용하지 않고 html 태그를 선택하는 경우도 있답니다.

이에 대한 내용은 추후에 다루어 보도록 하겠습니다.

3) 동적/정적 웹 페이지

1. 정적/동적 웹 페이지

웹 크롤링을 하면서 여러분들이 만나게 되는 페이지는 크게 두 가지로 분류할 수 있습니다.

정지해있다는 의미의 정적(static)과 움직인다는 의미의 동적(dynamic)으로 말이죠.

이 두 가지의 차이점에 대한 이해가 선행되지 않으면 여러분들의 코드는 제대로 작동되지 않을 것입니다.

1) 정적 웹 페이지

정적 웹 페이지란 서버(Web Server)에 미리 저장된 파일이 그대로 전달되는 웹 페이지를 말합니다.

즉, 제가 특정 웹페이지의 url 주소만 주소창에 입력하면 웹 브라우저로 HTML 정보를 마음대로 가져다 쓸 수 있는 것입니다.

동적 웹 페이지와 가장 큰 차이점은 **'url 주소 외에는 아무 것도 필요없다'** 는 점입니다. 아래에서도 설명 하겠지만 만약 마우스 휠을 스크롤 다운 했는데, url에 변화는 없고 페이지에 내용이 추가된다면 그 페이지는 동적 웹 페이지입니다.

말이 어려울 수 있으니 정적 웹 페이지의 예를 보여드리겠습니다.

네이버 검색 결과

1) 네이버를 열어서 검색창에 '손흥민'을 검색합니다.
2) 검색 결과의 url을 복사하시고 다시 주소창에 해당 url을 입력합니다.
3) 처음 검색결과와 동일한 페이지를 볼 수 있습니다.

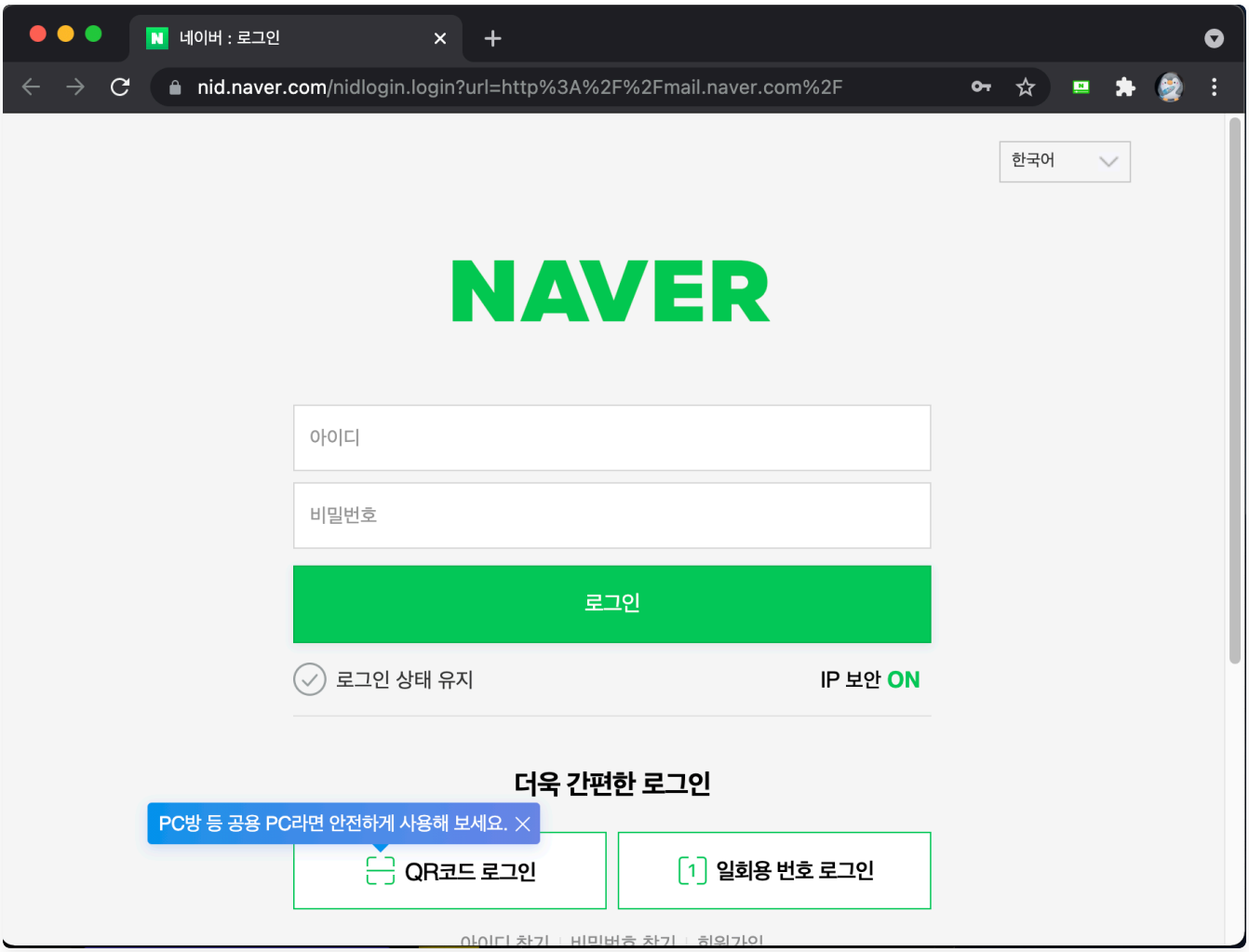
2) 동적 웹 페이지

동적 웹페이지란 url만으로는 들어갈 수 없는 웹페이지를 말합니다.

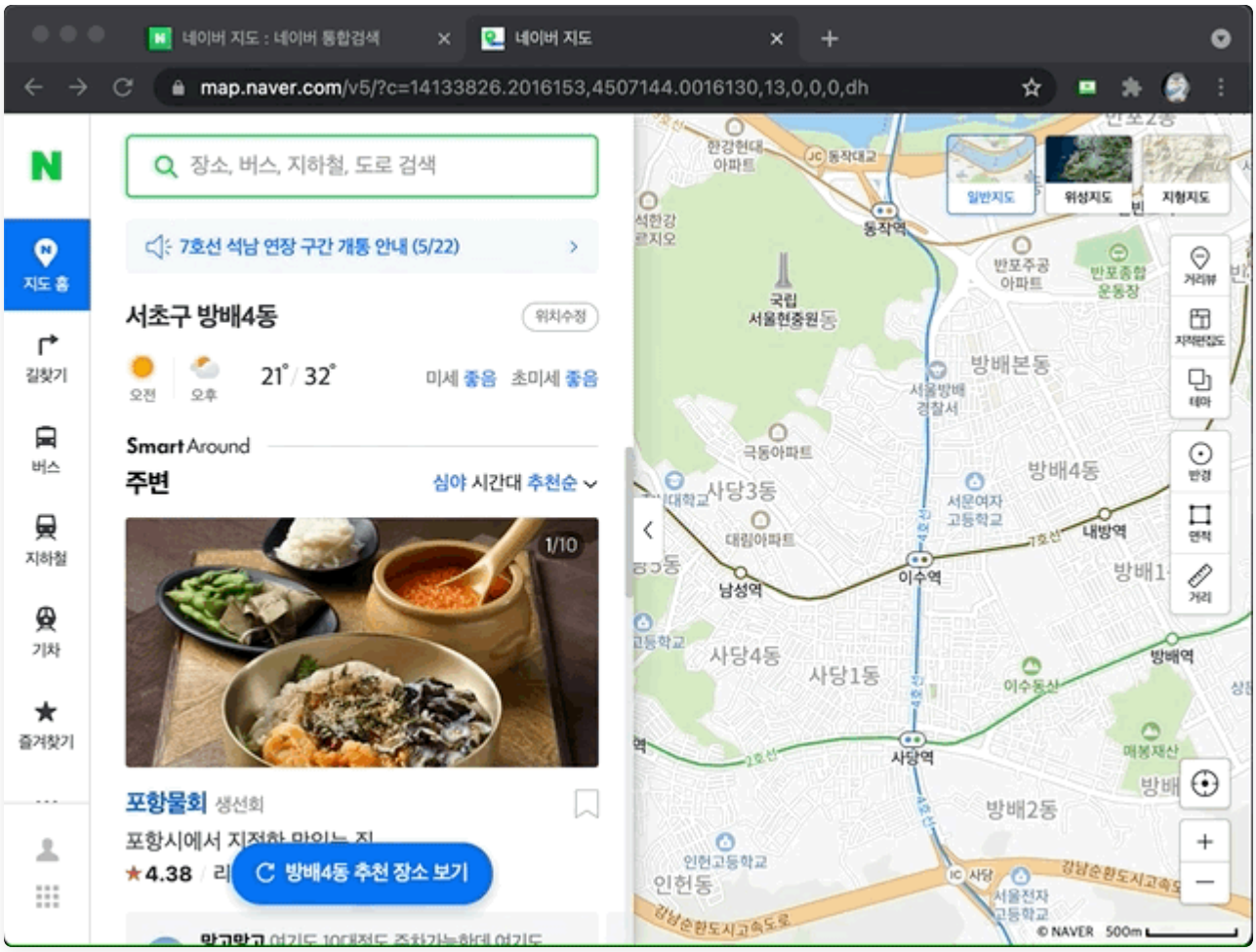
혹시 들어가지더라도 url의 변화가 없는데도 실시간으로 내용이 계속해서 추가되거나 수정된다면 동적 웹 페이지입니다.

여기서 무언가를 클릭해서 페이지가 변경되는 것은 다른 경우입니다.

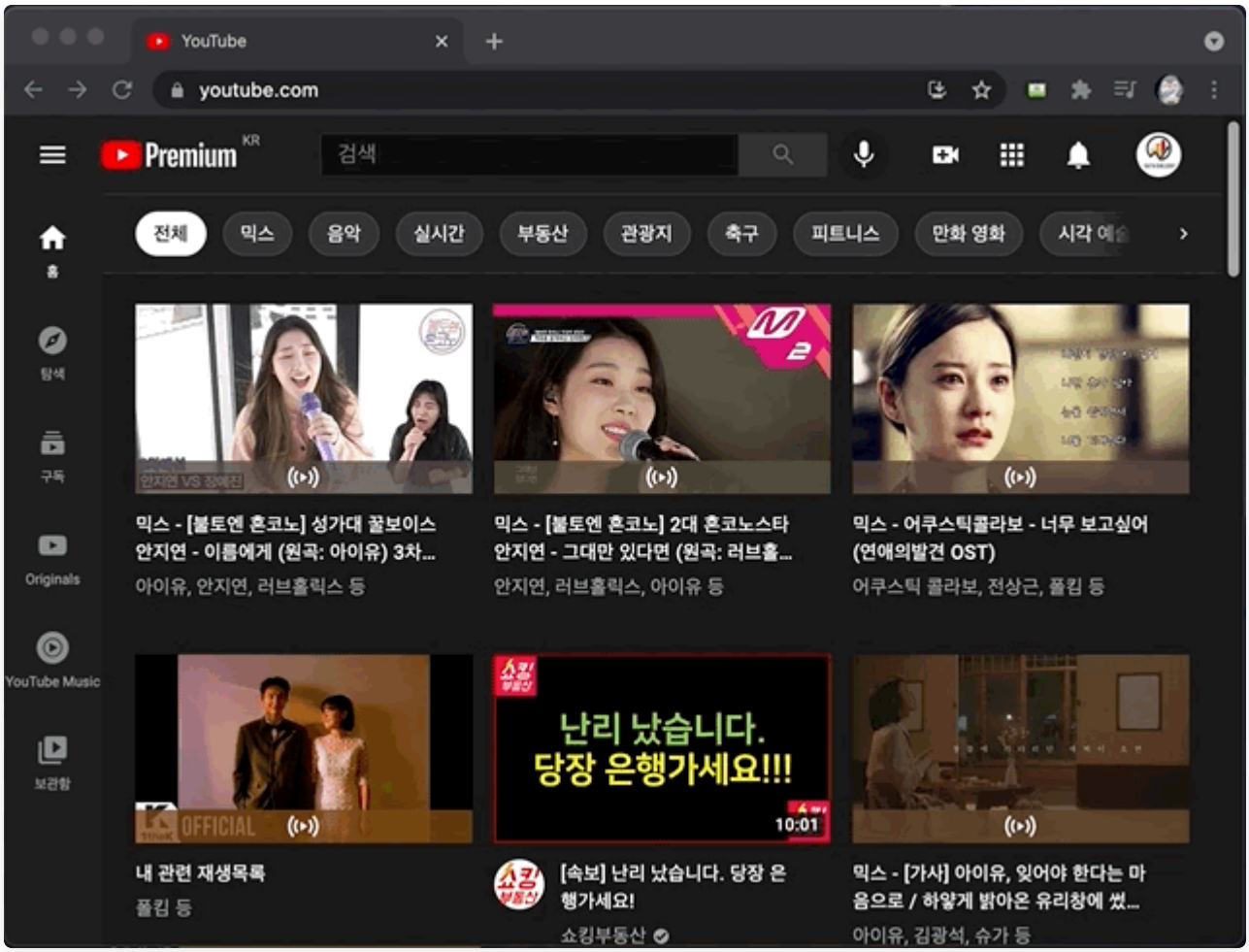
case 1. 로그인을 해야만 접속 가능한 네이버 메일



case2. 보고 있는 위치에 출력 결과와 url이 계속 변하는 네이버 지도



case 3. 드래그를 아래로 내리면 계속 새로운 사진과 영상이 나타나는 인스타그램과 유튜브



2. 정적/동적 수집

정적 페이지에서 정보를 수집 하느냐, 동적 페이지를 하느냐에 따라서 사용되는 파이썬 패키지는 달라집니다. 간단히 표로 정리해보면 아래와 같습니다.

	정적 수집	동적 수집
사용 패키지	requests / urllib	selenium
수집 커버리지	정적 웹 페이지	정적/동적 웹 페이지
수집 속도	빠름 (별도 페이지 조작 필요 X)	상대적으로 느림
파싱 패키지	beautifulsoup	beautifulsoup / selenium

자세한 내용은 목차에서 설명 드리겠습니다.

4) 정적 웹크롤링 - request / beautifulsoup

이번 시간에는 정적 웹페이지의 데이터를 가져오는 '정적 웹크롤링'에 대해 배워보겠습니다.

정적 웹 크롤링은 아주 간단합니다.

1단계. 원하는 웹 페이지의 html문서를 싹 긁어온다.

2단계. 긁어온 html 문서를 파싱(Parsing)한다.

3단계. 파싱한 html 문서에서 원하는 것을 골라서 사용한다.

위의 3단계를 수행하기 위해 사용되는 패키지를 하나씩 살펴보겠습니다.

1. 정적 웹크롤링 관련 패키지

1단계. requests

html 문서를 가져올 때 사용하는 패키지입니다. requests는 사용자 친화적인 문법을 사용하여 다루기 쉬우면서 안정성이 뛰어나다고 합니다.

그래서 파이썬 기본 라이브러리에 포함된 urllib 패키지보다 자주 사용됩니다.

설치 방법

VS code의 터미널창에서 아래와 같이 입력하면 됩니다.

만약 anaconda를 사용하신다면 pip 대신 conda를 입력해주셔도 됩니다.

```
pip install requests
```

사용 방법

```
# requests 패키지 가져오기
import requests

# 가져올 url 문자열로 입력
url = 'https://www.naver.com'

# requests의 get함수를 이용해 해당 url로 부터 html이 담긴 자료를 받아옴
response = requests.get(url)

# 우리가 얻고자 하는 html 문서가 여기에 담기게 됨
html_text = response.text
```

2단계. BeautifulSoup4

BeautifulSoup4 패키지는 매우 길고 정신없는 html 문서를 잘 정리되고 다루기 쉬운 형태로 만들어 원하는 것만 쏙쏙 가져올 때 사용합니다.

이 작업을 파싱(Parsing)이라고도 부릅니다.

파싱(Parsing) 전 파싱(Parsing) 후

설치 방법

마찬가지로 VS code의 터미널창에서 아래와 같이 입력하면 됩니다.

```
pip install beautifulsoup4
```

사용 방법

방금 전 Requests 패키지로 받아온 html 문서를 파싱해야하므로, 이전 코드 블록을 실행하셔야 합니다.

```
# BeautifulSoup 패키지 불러오기
# 주로 bs로 이름을 간단히 만들어서 사용함
from bs4 import BeautifulSoup as bs

# html을 잘 정리된 형태로 변환
html = bs(html_text, 'html.parser')
```

3단계. BeautifulSoup4 (find /select)

이제 마지막 3단계입니다.

사실 1, 2단계는 느끼셨겠지만 url만 수정하면 아주 쉽게 응용할 수 있을 정도로 쉽습니다.

즉, 정적 웹크롤링의 핵심은 필요한 정보의 위치와 구조를 파악해서 쏙쏙 가져오는 3단계입니다.

크롤링을 원하는 사이트가 생겼을 때에 능수능란하게 코딩하기 위해서는 꼭 이 스킬을 확실히 익혀두셔야 합니다.

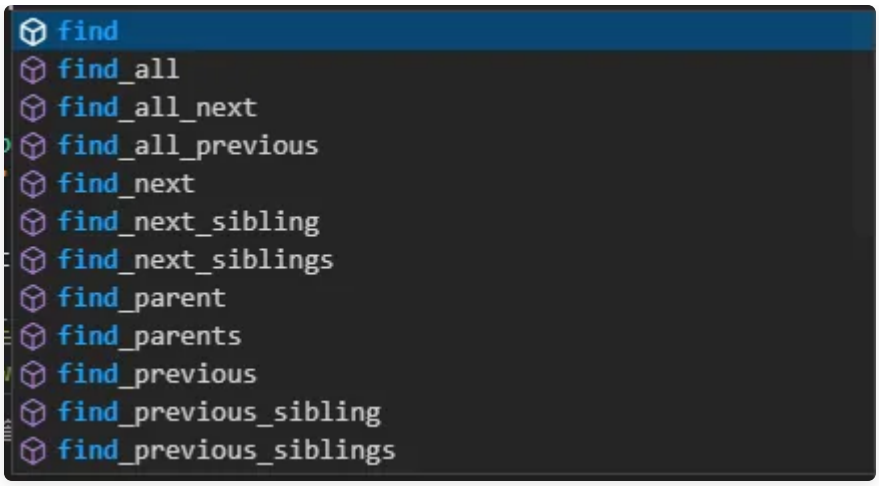
1) find( ), find_all( ) 함수

BeautifulSoup 패키지에서 find 함수는 아래와 같이 12개가 있습니다.

하지만 우리가 알면 되는 것은 딱 2개입니다.

- find( ): 하나만 찾는 것
- find_all( ): 모두 다 찾는 것

주의할 것은 find로 찾는 것이 여러개라면 가장 첫 번째 것만 가져온다는 것입니다.



괄호( ) 안에는 html의 태그(tag)나 속성(attribute)이 들어갑니다. 웹 크롤링 관련 코드를 보다보면 find 함수를 종종 사용한 경우를 보는데, 그 코드를 이해할 목적으로만 가볍게 알고 넘어가시는 것을 추천 드립니다. 왜냐하면 CSS 선택자 개념을 사용하는 select( ) 함수를 사용하는 것이 훨씬 직관적이거든요!

```
# 목표 태그 예)
<p class = "para">코딩유치원</p>
<div id = "zara">코딩유치원</p>

# 태그 이름으로 찾기
soup.find('p')

# 태그 속성(class)으로 찾기 - 2가지 형식
soup.find(class_='para') #이 형식을 사용할 때는 class 다음에 언더바_를 꼭 붙여주어야 한다
soup.find(attrs = {'class':'para'})

# 태그 속성(id)으로 찾기 - 2가지 형식
soup.find(id='zara')
soup.find(attrs = {'id':'zara'})

# 태그 이름과 속성으로 찾기
soup.find('p', class_='para')
soup.find('div', {'id' = 'zara'})
```

2) select( ), select_one( ) 함수

select( )는 find_all( )과 같은 개념이고, select_one( )은 find( )와 같은 개념입니다. 다만, select( ) 함수는 괄호( )안에 CSS 선택자라는 것을 넣어서 원하는 정보를 찾는 것이 특징입니다.

아주 직관적이고 쉬워서 이 방법을 추천드립니다.

```
# a태그의 class 속성명이 news_tit인 태그
soup.select_one('a.news_tit')

soup.select('a.news_tit')

for i in titles:
    title = i.get_text() print(title)
```

5) 정적 웹크롤링 - 텍스트, 하이퍼링크, 이미지 가져오기

이번 시간에는 정적 웹크롤링을 이용해 텍스트, 하이퍼링크, 이미지를 가져오는 방법에 대해서 알아보겠습니다.

지난 시간에 배웠듯이 정적 웹 크롤링은 아주 간단합니다.

1단계. 원하는 웹 페이지의 html문서를 싹 긁어온다.

2단계. 긁어온 html 문서를 파싱(Parsing)한다.

3단계. 파싱한 html 문서에서 원하는 것을 골라서 사용한다.

오늘은 3단계, 원하는 것을 골라서 사용하는 방법을 조금 더 자세히 다루어 볼 예정입니다.

1. 크롤링 할 페이지(url) HTML 가져오기

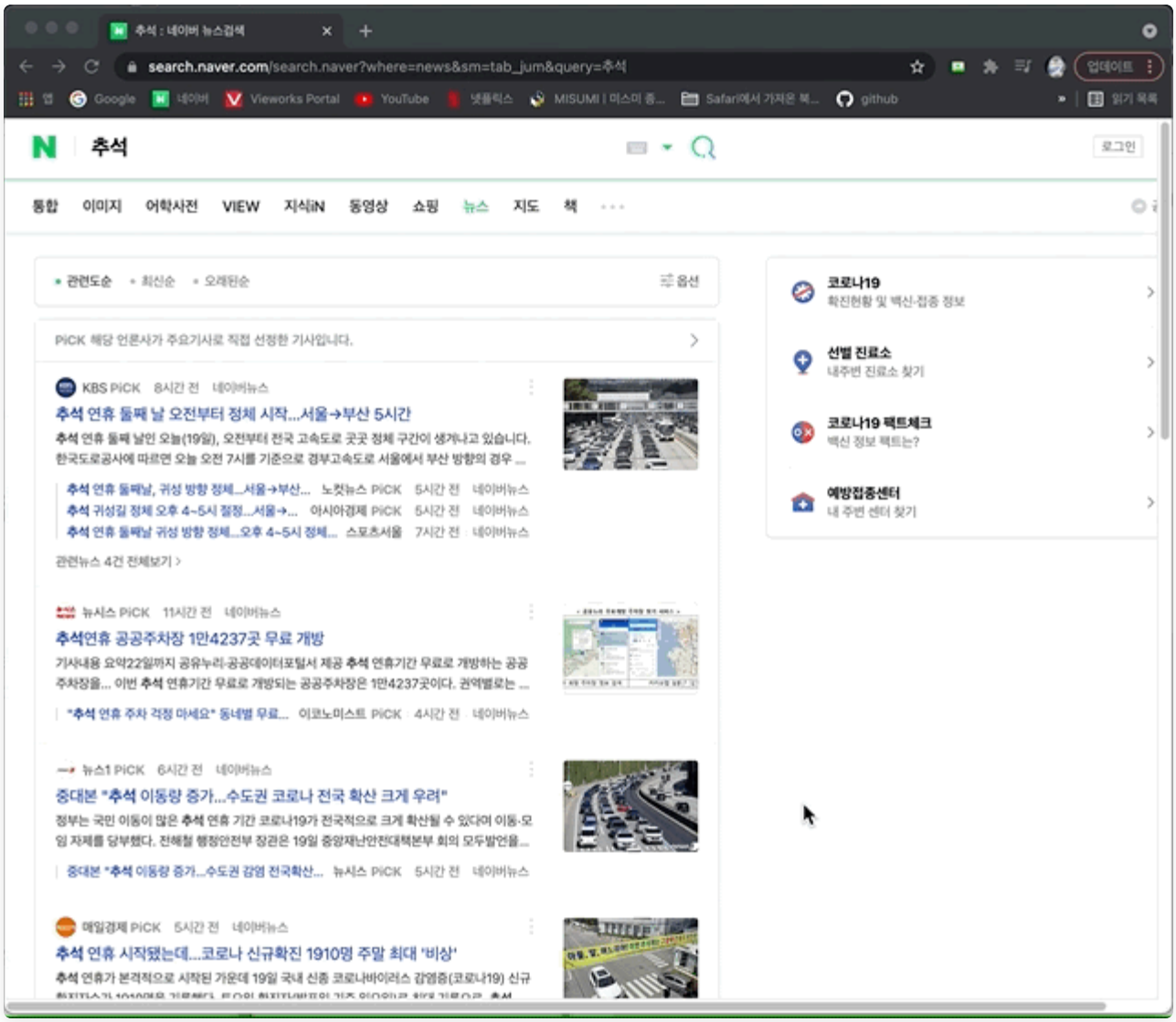
해당 코드는 1단계와 2단계에 해당하는 코드입니다. 자세한 내용은 코드의 주석을 참고해주세요.

```
# step1.프로젝트에 필요한 패키지 불러오기

from bs4 import BeautifulSoup as bs

import requests


# step2. 검색할 키워드 입력

query = input('검색할 키워드를 입력하세요: ')


# step3. 입력받은 query가 포함된 url 주소(네이버 뉴스 검색 결과 페이지) 저장

url = 'https://search.naver.com/search.naver?where=news&sm=tab_jum&query='+'%s'%query


# step4. requests 패키지를 이용해 'url'의 html 문서 가져오기

response = requests.get(url)

html_text = response.text


# step5. beautifulsoup 패키지로 파싱 후, 'soup' 변수에 저장

soup = bs(response.text, 'html.parser')
```

2. 크롤링 할 페이지 실제로 들어가서 추출할 HTML 확인하기

코드를 실행하시면 vs code의 터미널 창에 '검색할 키워드를 입력하세요: '라고 출력될텐데, 원하는 검색 키워드를 입력하시면 됩니다.

저의 경우엔 '추석'을 검색해보았습니다.

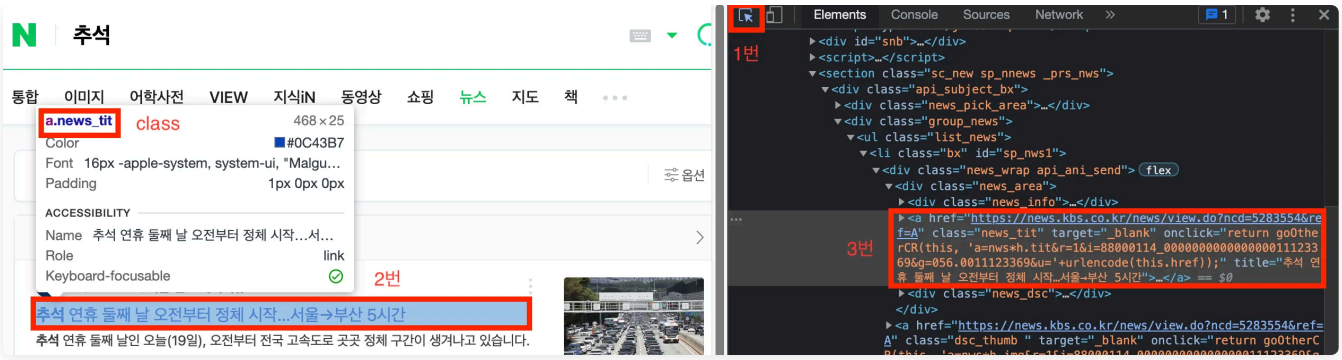


검색을 하셨다면 F12를 눌러서 HTML 문서가 출력되도록 해주시고, 이제부터 원하는 부분을 추출해보도록 하겠습니다.

1) 텍스트 추출 / .get_text( )

requests와 beautifulsoup를 이용한 텍스트 추출 방법은 아래와 같습니다.

1번. 우측 개발자 도구에서 커서 모양의 아이콘을 클릭 2번. 추출을 원하는 부분, 여기서는 뉴스 제목을 클릭 3번. 해당 부분의 HTML 태그를 분석하여 select( ) 함수로 추출



3번 단계를 조금 더 자세히 설명드려보겠습니다.

css selector, 여기서는 a.news_tit이 되겠네요. 이 선택자는 해당 페이지의 뉴스 기사 제목 10개가 공통으로 가지고 있는 class 이름입니다.

css selector를 알아냈다면 select 함수에 입력하여 원하는 부분의 html을 변수에 저장해준 후에, for 문과 .get_text( ) 함수를 이용해 주시면 됩니다.

```
#step6.뉴스 제목 텍스트 추출

news_titles = soup.select("a.news_tit")

for i in news_titles:
    title = i.get_text()
    print(title)
```

결과는 아래와 같이 출력됩니다.

```
검색할 키워드를 입력하세요: 추석
추석 연휴 둘째 날 오전부터 정체 시작…서울→부산 5시간
추석연휴 공공주차장 1만4237곳 무료 개방
추석에 더 서러운 임금체불…쿠팡이츠·배민도 체불신고
추석 연휴 시작됐는데…코로나 신규확진 1910명 주말 최대 '비상'
코로나 신규확진 1910명, 주말 최다 기록…추석 연휴 재확산 우려
추석 연휴 이틀째 답답한 고속도로…서울→부산 5시간 30분
추석 연휴 쇼핑 어디서? 유통업계 휴점일은 언제
추석 연휴 둘째날 귀성 방향 정체…서울→부산 5시간10분
여야 대권 주자, '추석 민심' 올인…대장동 의혹 공방
추석 이후 수도권 분양시장 활짝…연말까지 7만9000가구 공급
```

2) 링크 추출 / .attrs['href']

링크를 가져오고 싶을 때, 속성을 가져오는 .attrs['href'] 함수를 사용합니다. href 속성은 방금 전 가져온 news_titles 변수에 저장되어 있으므로, 따로 한 번더 가져와줄 필요 없이 바로 for문을 써줬습니다.

```
#step7.뉴스 하이퍼링크 추출

for i in news_titles:
    href = i.attrs['href']
    print(href)
```

결과는 아래와 같이 10개의 url이 출력되는 것을 확인하실 수 있을거예요.

```
https://news.kbs.co.kr/news/view.do?ncd=5283554&ref=A
http://www.newsis.com/view/?id=NISX20210916_0001586425&cID=10201&pID=10200
http://www.edaily.co.kr/news/newspath.asp?newsid=01371046629181760
http://news.mk.co.kr/newsRead.php?no=902371&year=2021
https://biz.chosun.com/it-science/bio-science/2021/09/19/O2I3WRVG7ZDFTFVVQZB
http://www.busan.com/view/busan/view.php?code=2021091910084961925
https://www.news1.kr/articles/?4438637
https://www.seoul.co.kr/news/newsView.php?id=20210919500014&wlog_tag3=naver
https://www.ytn.co.kr/_ln/0101_202109191031491091
https://hankookilbo.com/News/Read/A2021091915170002789?did=NA
```

3) 이미지 추출 .attrs['src']

다음으로는 이미지 추출입니다. 텍스트나 url 링크를 가져올 때와는 다르게 조금 길지만 어려워 하실 필요없어요.

앞서 배우신 것과 같이 이미지 주소(src)를 갖고 있는 html을 news_thumnail 변수에 저장해줍니다.

그 다음엔 이미지 다운로드를 위해서 해당 주소들을 리스트(link_thumnail)에 append 함수를 이용해 하나씩 담아주세요.

사진을 다운로드 받아서 내 PC에 저장하기 위해서는 저장할 폴더를 만들고, src 주소를 이용해 다운로드 해주면 됩니다.

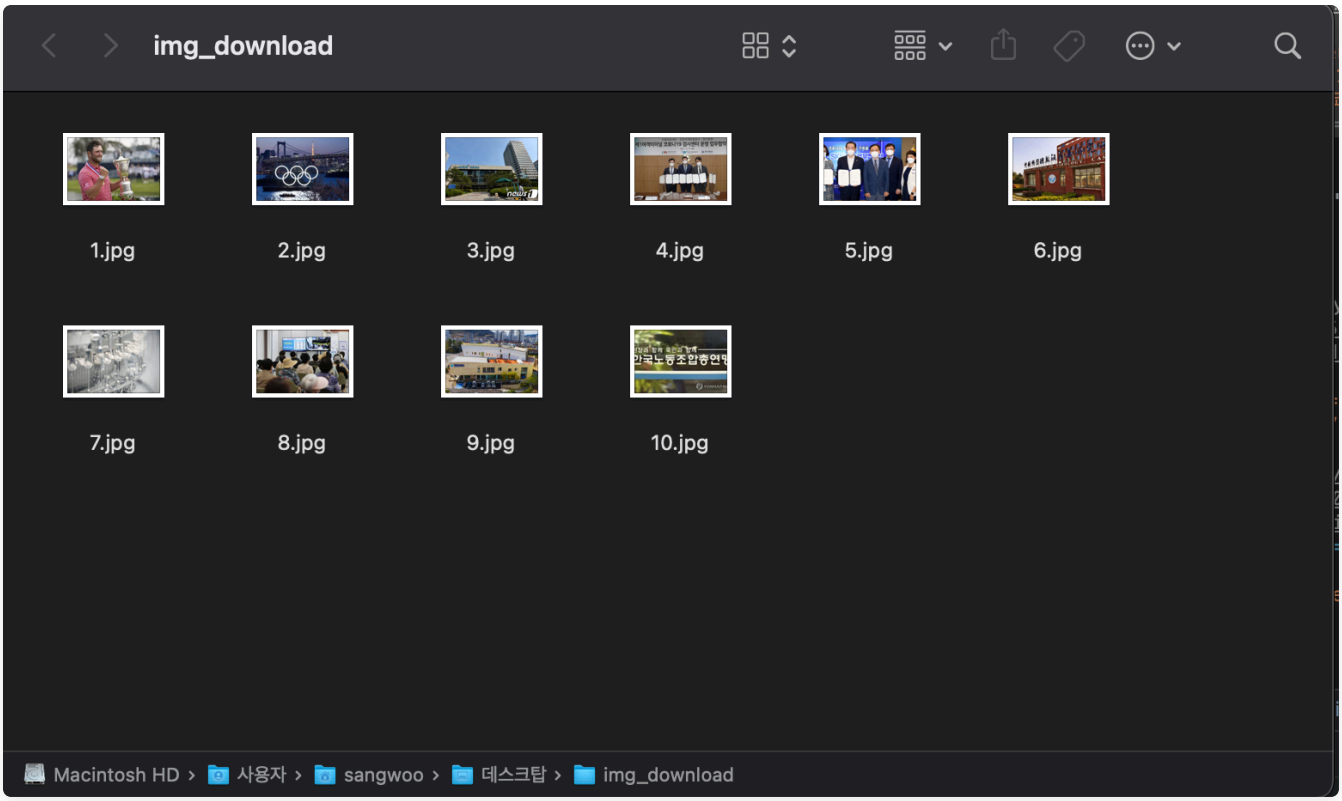
폴더 생성에는 os 모듈과 urllib.request 패키지의 urlretrieve 함수가 필요한데, 모두 다 파이썬 내장 라이브러리이므로 따로 설치해주실 필요 없이 import 해주시면 돼요.

os 모듈과 경로에 대한 개념을을 잘 모르신다면 아래 글을 참고해주세요.

2021.06.07 - [파이썬 패키지/사무자동화] - [Python / os모듈] 디렉토리(폴더)와 경로 정보 활용하기

```
#step8.뉴스 썸네일 이미지 추출

news_content_div = html.select(".news_contents")

news_thumbnail = [thumbnail.select_one(".thumb") for thumbnail in news_content_div]

link_thumbnail = []

for img in news_thumbnail:
    if img is  not  None  and  'data-lazysrc'  in img.attrs:
        link_thumbnail.append(img.attrs['data-lazysrc'])

# 이미지 저장할 폴더 생성
import os

# path_folder의 경로는 각자 저장할 폴더의 경로를 적어줄 것(ex.img_download)
path_folder = '/Users/sangwoo/Desktop/img_download/'

if not os.path.isdir(path_folder):
    os.mkdir(path_folder)

# 이미지 다운로드
from urllib.request import urlretrieve

i = 0

for link in link_thumbnail:
    i += 1
    urlretrieve(link, path_folder + f'{i}.jpg')
```

<실행 결과>


img_download
1.jpg
2.jpg
3.jpg
4.jpg
5.jpg
6.jpg
7.jpg
8.jpg
9.jpg
10.jpg
Macintosh HD › 사용자 › sangwoo › 데스크탑 › img_download

6) 동적 웹크롤링 - selenium 소개 및 기초사용법

1. Selenium 패키지란?

selenium 패키지는 chromedriver를 제어하거나 원하는 정보를 얻기 위해 사용합니다.

크롤링을 하다보면 무엇인가 입력하거나 특정 버튼을 눌러야 하는 상황이 발생합니다. 사람이 그러한 행동을 하는 대신 컴퓨터가 할 수 있도록 해주는 패키지가 selenium입니다.

selenium 패키지도 웹 정보를 크롤링하는 것이 가능하기 때문에 저는 웹 크롤링은 거의 selenium을 패키지를 사용하는 편입니다.

2. 설치 방법

vscode의 터미널창에서 아래와 같이 입력하면 됩니다. 만약 anaconda를 사용하신다면 pip 혹은 conda 명령어를 둘 다 사용 가능하시고, python 공식 홈페이지를 통해 python을 설치하셨다면 아래의 코드를 사용하시면 됩니다.

```
pip install selenium
```

3. 사용 방법

디테일한 사용 방법은 다음 시간부터 알아보도록 하고, 이번 시간에는 크롬 드라이버를 이용해서 원하는 사이트에 접속하는 것까지 해보겠습니다.

1) 관련 패키지 import

selenium 패키지를 사용하기 위해서는 아래의 두 가지 모듈을 import 해주어야 합니다.

추가적으로 페이지 로딩을 기다리는데에 사용할 time 모듈도 import 해줍니다. 이 모듈은 파이썬 내장 라이브러리에 포함되어 있어 별도 설치는 필요 없습니다.

```
# selenium의 webdriver를 사용하기 위한 import
from selenium import webdriver

# selenium으로 키를 조작하기 위한 import
from selenium.webdriver.common.keys import Keys

# 페이지 로딩을 기다리는데에 사용할 time 모듈 import
import time
```

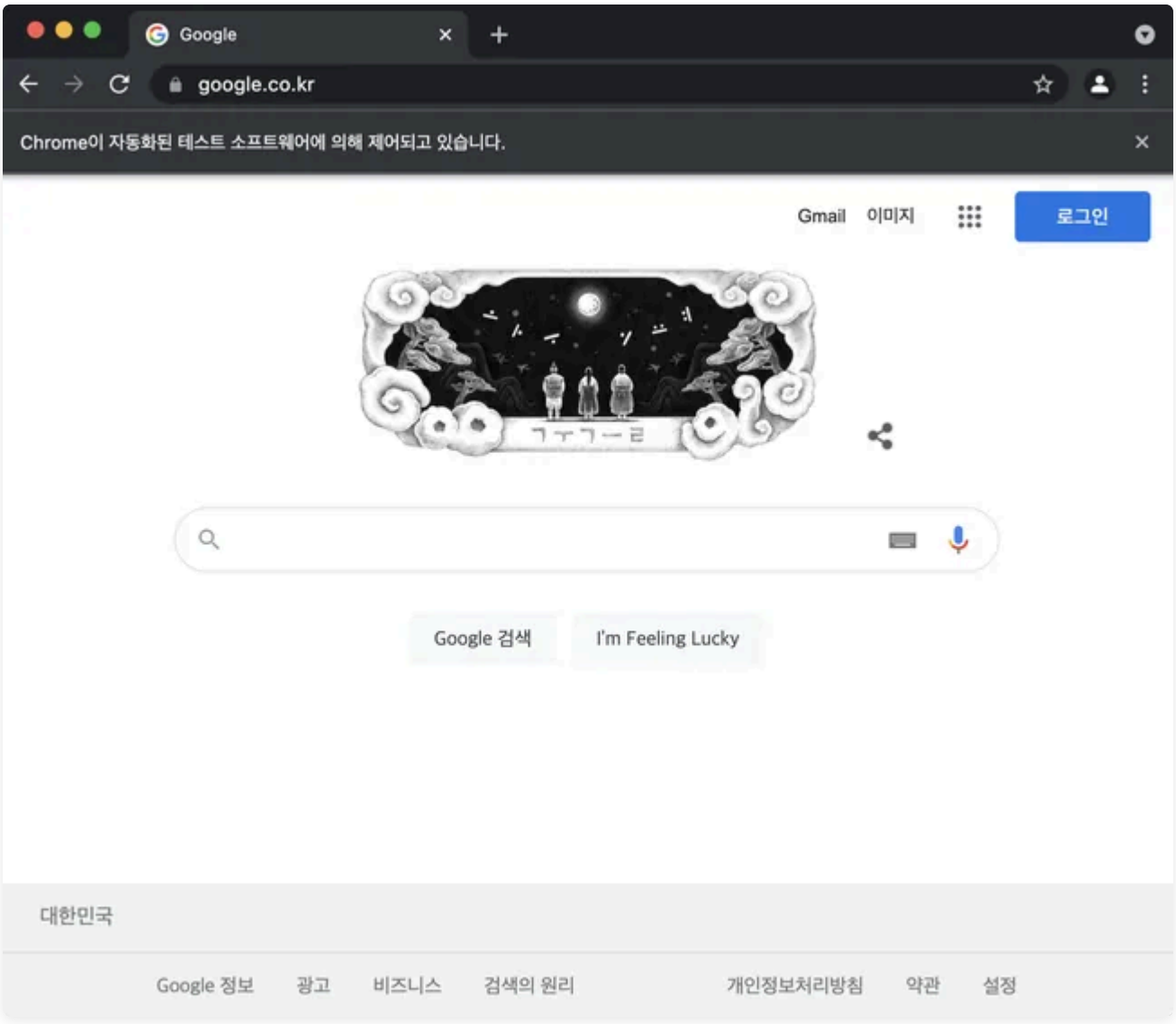
2) 크롬 드라이버 실행

관련 패키지를 불러왔으면 다음으로 해줄 일은 크롬 드라이버를 이용해서 크롬을 실행해주는 일입니다.

크롬 드라이버는 컴퓨터가 크롬 웹 브라우저를 다룰 수 있도록 해주는 프로그램으로 구글에서 제공 해줍니다. 크롬 드라이버는 코드를 실행 시키시면 자동으로 다운로드 되니 별도 설치가 필요 없습니다. (4.6 버전부터 크롬드라이버 별도 다운로드 및 경로 설정 불필요)

```
# 크롬드라이버 실행
driver = webdriver.Chrome()

#크롬 드라이버에 url 주소 넣고 실행
driver.get('https://www.google.co.kr/')

# 페이지가 완전히 로딩되도록 3초동안 기다림
time.sleep(3)
```

<크롬 드라이버로 구글 웹페이지에 접속한 상태>


Google
google.co.kr
Chrome이 자동화된 테스트 소프트웨어에 의해 제어되고 있습니다.
Gmail
이미지
로그인
Google 검색
I'm Feeling Lucky
대한민국
Google 정보
광고
비즈니스
검색의 원리
개인정보처리방침
약관
설정

7) 동적 웹크롤링 - selenium으로 화면 조작하기

Selenium으로 화면 조작하기

selenium은 원하는 화면 상태에 도달할 수 있도록 웹 브라우저를 조작 가능합니다.

예를 들어서, 네이버 메일을 확인하기 위해서 로그인 과정을 거쳐야 한다거나, 유튜브 댓글을 모두 크롤링하는데 스크롤을 내려야지 댓글이 추가적으로 업데이트 되는 상황이 있겠죠.

이런 상황에서 원하는 버튼을 클릭 하거나 문자를 입력하는 등의 컴퓨터가 해주어야 하는데 이를 Selenium 패키지가 대신 해줄 수 있습니다.

쉽게 말해서 웹 브라우저용 매크로랄까요?

Selenium으로 화면을 조작하는 전체적인 개념은 다음과 같습니다.

<개념>

1) 조작을 원하는 버튼이나 입력창의 html을 파악

2) 아래의 두 함수에 html 정보를 입력해서 객체(버튼/입력창 등) 선택

- find_element(By.ID)
- find_element(By.CLASS_NAME)
- find_element(By.XPATH)
- find_element(By.CSS_SELECTOR)

3) 기능 동작 관련 함수로 원하는 기능 조작

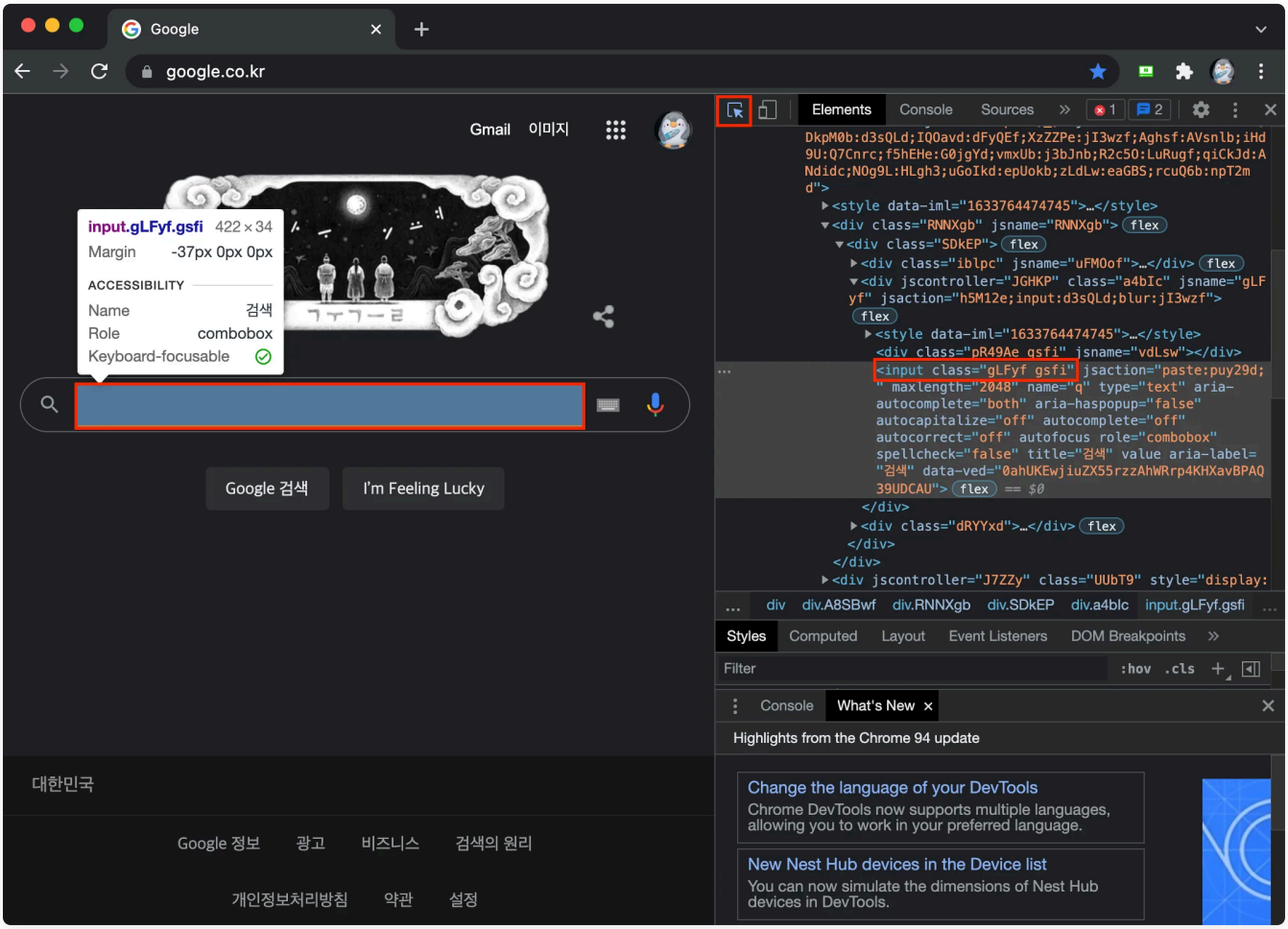
- 클릭 : .click( )
- 키 입력: .send_keys( )

원하는 키워드를 검색하는 아주 간단한 예제를 통해서 어떤 방식으로 사용하는지 보여드리겠습니다.

1단계. 원하는 버튼의 html 타겟팅

가장 먼저 해줄 일은 크롬을 실행해서 F12를 누르는 것 부터 시작합니다.

F12를 누르면 지난 강의에서 배우셨 듯이 웹 브라우저 우측에 '개발자 도구'가 나타날 거예요. 그러면 개발자 도구 상단의 화살표 버튼을 눌러서 조작을 원하는 부분을 클릭해주세요.



검색창에 원하는 키워드를 입력하고 앤터를 눌러주는 것이 이번 예제의 목적이므로 검색창을 타겟팅 할 수 있도록 **속성 HTML/CSS 강의** 시간에 공부 하셨던 'CSS Selector' 개념을 이용해보겠습니다.

방금 전까지 잘 따라오셨다면 아마 개발자 도구에 '검색창'의 html 부분이 나타나 있을텐데요. 우선 빨간색으로 표시된 class 속성값 부분을 더블클릭해서 복사(Ctrl+c)를 해주세요.

> *<여기서 잠깐>*
>
> *class명이 gLFyf gsfi와 같이 빈칸이 존재하는 경우는 빈칸을 점(.)으로 변경해주지 않으면 원하는 정보를 제대로 찾지 못합니다. 즉, 클래스명을 gLFyf.gsfi라고 입력해주어야 합니다.*

여기까지 따라오셨다면 이제 코딩창으로 넘어가서 원하는 조작을 수행해 보겠습니다.

2단계. Selenium으로 타겟팅한 html 찾기

그 다음으로 해줄 일은 Selenium의 **find_element( ) 함수** 에 방금 복사해놓은 클래스명 (**gLFyf.gsfi**)를 ( ) 안에 넣어서 **크롬 드라이버가 알아먹을 수 있게 변환**해주는 것입니다.

이때 클래스명 앞에 붙여주어야 하는 코드가 있는데 바로 로케이터(Locator)라는 것입니다.

로케이터(Locator)	설명
By.TAG_NAME	<p>, 등과 같은 태그명으로 검색
By.ID	태그의 id 속성값을 이용하여 검색 (id 속성값은 페이지 내에서 유일하므로 find_element와 함께 사용)
By.CLASS_NAME	태그의 class 속성값을 이용하여 검색 (class 속성값은 페이지 내에서 여러개가 있으므로 주로 find_elements와 함께 사용)
By.CSS_SELECTOR	css선택자로 검색 (위의 3가지 로케이터를 합쳐놓은 개념)
By.XPATH	태그의 경로로 검색 (사다리타기처럼 html의 가장 시작부터 목표한 요소까지 태그를 타고타고 내려온 경로를 xpath라고 함)
By.NAME	태그의 name값으로 검색
By.LINK_TEXT	링크 텍스트값으로 검색
By.PARTIAL_LINK_TEXT	링크 텍스트의 자식 텍스트 값을 추출

제가 주로 초보자분들께 추천드리는 로케이터는 ID, CLASS_NAME, XPATH 총 3가지 입니다. 이것만 잘 쓰셔도 웬만한 사이트는 모두 자동화 가능하거든요.

그 전에 먼저 관련 모듈을 import 해주고, 구글 드라이버를 이용해 'Google'까지 접속해줍시다. 아래 코드에 주석을 달아두었으니 그리 어렵진 않으실거예요.

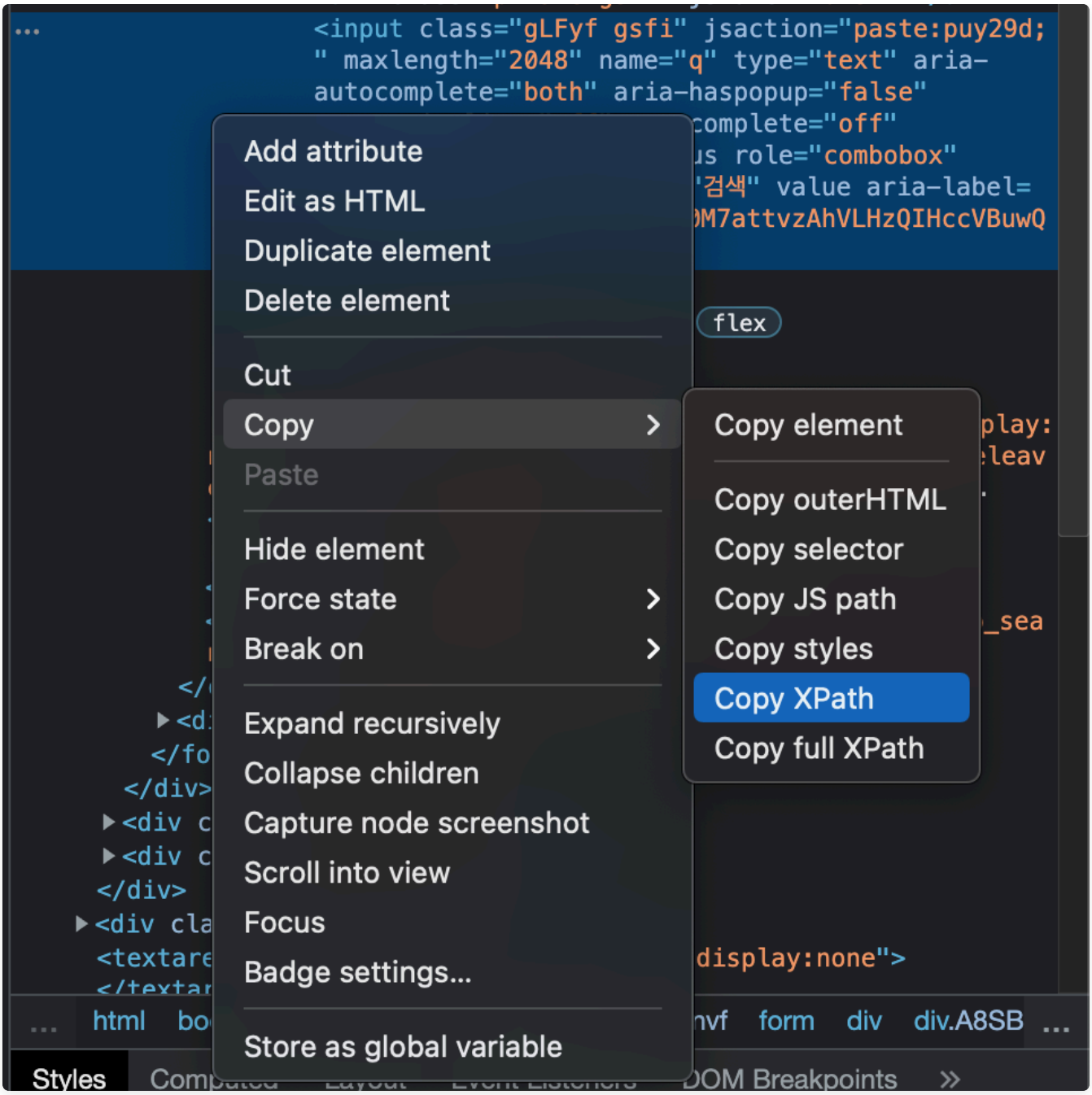
```
# selenium의 webdriver를 사용하기 위한 import
from selenium import webdriver

# selenium으로 무엇인가 입력하기 위한 import
from selenium.webdriver.common.keys import Keys

# 페이지 로딩을 기다리는데에 사용할 time 모듈 import
import time

# 크롬드라이버 실행
driver = webdriver.Chrome()

#크롬 드라이버에 url 주소 넣고 실행
driver.get('https://www.google.co.kr/')

# 페이지가 완전히 로딩되도록 3초동안 기다림
time.sleep(3)
```

자, 이제 **find_element( ) 함수**로 chromedriver가 검색창을 찾을 수 있게 해봅시다.

```
# 검색어 창을 찾아 search 변수에 저장 (By.CLASS_NAME 방식)
search_box = driver.find_element(By.CLASS_NAME, 'gLFyf.gsfi')

# 검색어 창을 찾아 search 변수에 저장 (By.XPATH 방식)
search_box = driver.find_element_by_xpath('//*[@id="google_search"]')
```

> ***<여기서 잠깐>***
>
> *갑자기 xpath가 나와서 당황스러우시겠지만 둘 다 같은 결과를 수행하는 코드라는 것을 알아두세요.*
>
> *참고로 xpath는 아까 찾으셨던 html 코드 위에서 '우클릭'을 하신 후, 아래 그림과 같이 'Copy Xpath'를 클릭하시면 클립보드에 //*[@id="google_search"] 라는 xpath가 저장되어 있을거예요.*
>
> *이걸 위의 코드와 같이 넣어주시면 된답니다.*

어렵지 않죠? html에 class나 id가 없어 인덱싱 하기 어려운 경우에 자주 사용하니 꼭 알아두시면 좋을 것 같아요!



3단계. 원하는 조작 수행

여기까지 하셨으면 Chromedriver가 Selenium 패키지의 find_element 함수를 이용해서 원하는 html을 찾은 상태입니다.

이제 남은 일은 원하는 조작을 수행하도록 명령 내리는 일입니다.

명령은 간단합니다. 클릭하거나 원하는 키를 입력해주는 것 2가지 입니다.

- **클릭 : .click( )**
- **키 입력: .send_keys( )**

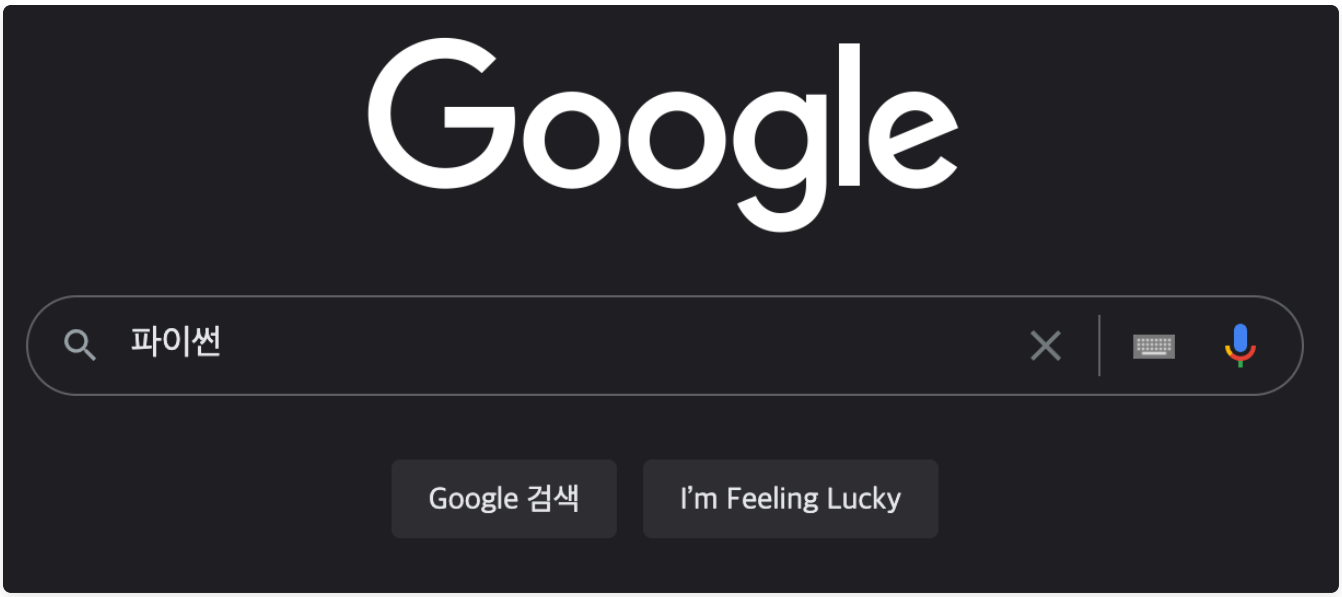
명령어	기능
Kyes.ENTER Keys.RETURN	엔터
Keys.SPACE	스페이스
Keys.ARROW_UP Keys.ARROW_DOWN Keys.ARROW_LEFT Keys.ARROW_RIGHT	방향키(상하좌우)
Keys.BACK_SPACE Keys.DELETE	지우기 (벡스페이스) 지우기 (딜리트)
Keys.CONTROL Keys.ALT Keys.SHIFT Keys.TAB	자주 사용하는 기능키(Ctrl, Alt, Shift, Tab)
Keys.PAGE_UP Keys.PAGE_DOWN	스크롤 업 스크롤 다운
Keys.F1~9	F1 부터 F9 (F+숫자)
Keys.EQUALS, Keys.ESCAPE, Keys.HOME, Keys.INSERT	기타 등등

그럼 간단히 어떻게 코드로 적용할 수 있는지 알아보겠습니다.

```
search_box.send_keys('파이썬')
search_box.send_keys(Keys.RETURN)
time.sleep(1)
```

사용법 역시 정말 간단합니다.

아까 찾았던 검색창(search_box) 변수 다음에 .send_keys('검색어')를 넣어주면 아래와 같은 상태가 되겠죠?



그 다음에 search_box.send_keys(Keys.RETURN)을 입력해주면 우리가 검색어를 치고 '엔터키'를 입력해주는 것과 동일합니다.

참고로 그림에서 Google 검색 버튼을 우리가 했던 방식으로 타겟팅 해주고 .click( ) 함수로 '마우스 클릭' 해주어도 엔터키를 입력한 것과 동일한 결과를 얻을 수 있습니다.

```
# click 함수는 ()안에 아무것도 넣지 않으면 좌클릭을 수행
search_button.click()
```

이 부분은 여러분들의 실습 과제로 남겨두겠습니다.

8) 동적 웹크롤링 - 텍스트, 하이퍼링크, 이미지 가져오기

이번 시간에는 Selenium 패키지를 이용한 동적 웹크롤링을 할 때, 텍스트와 속성(하이퍼링크, 이미지)을 가져오는 방법에 대해서 알아보겠습니다.

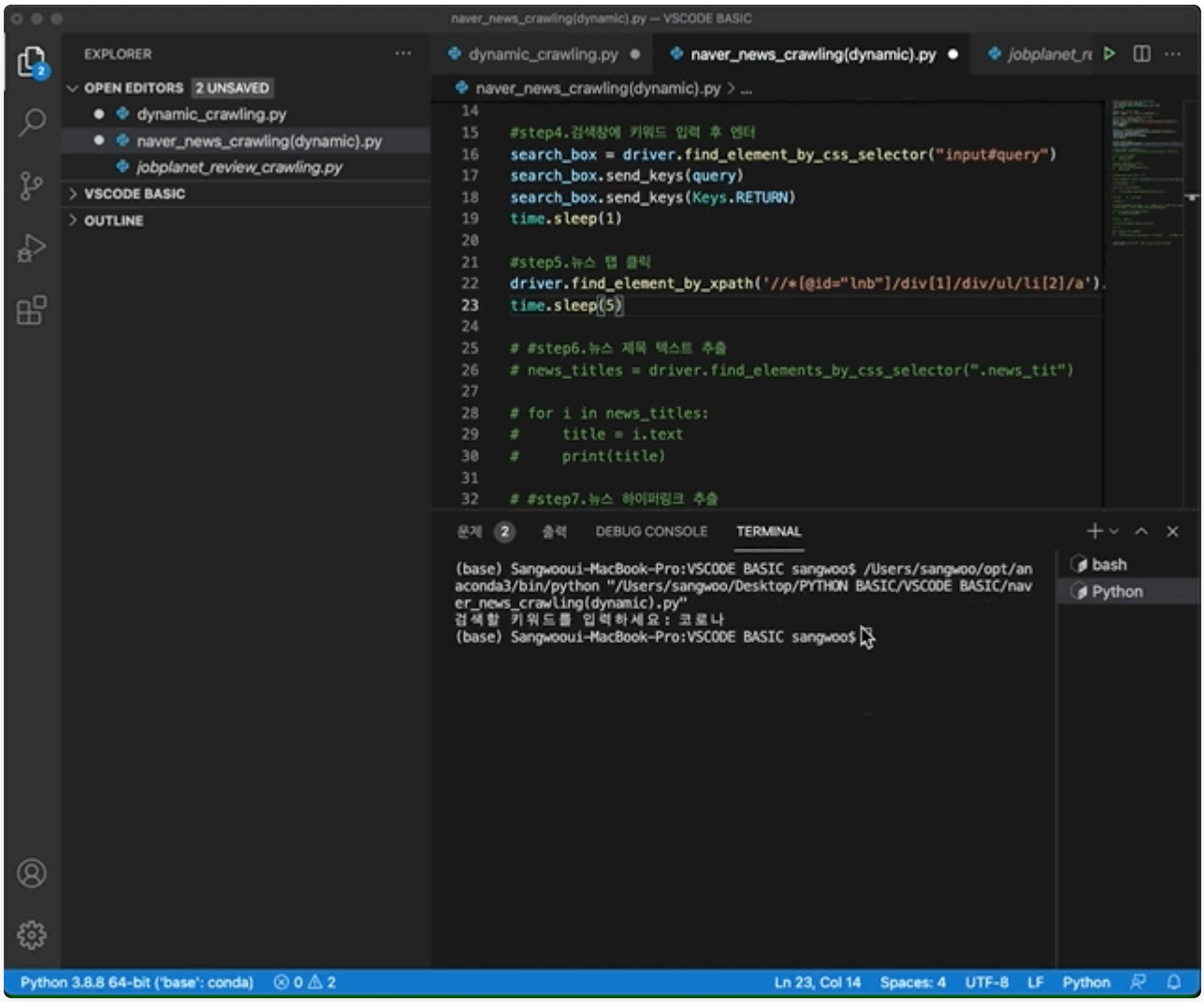
실습을 통해서 하나씩 알아볼텐데요. 대략적인 순서는 이렇습니다.

1. 검색할 키워드 입력
2. 크롬 드라이버로 원하는 url 접속
3. 뉴스 제목 텍스트 추출
4. 뉴스 url 링크 추출
5. 뉴스 썸네일 이미지 추출
    - 이미지 src 리스트에 저장
    - 이미지 저장할 폴더 생성
    - src를 이용해 이미지 다운로드

우선 지난 강의에서 배웠던 1번 2번 단계는 설명 없이 코드만 공유드리고 넘어가도록 하겠습니다.

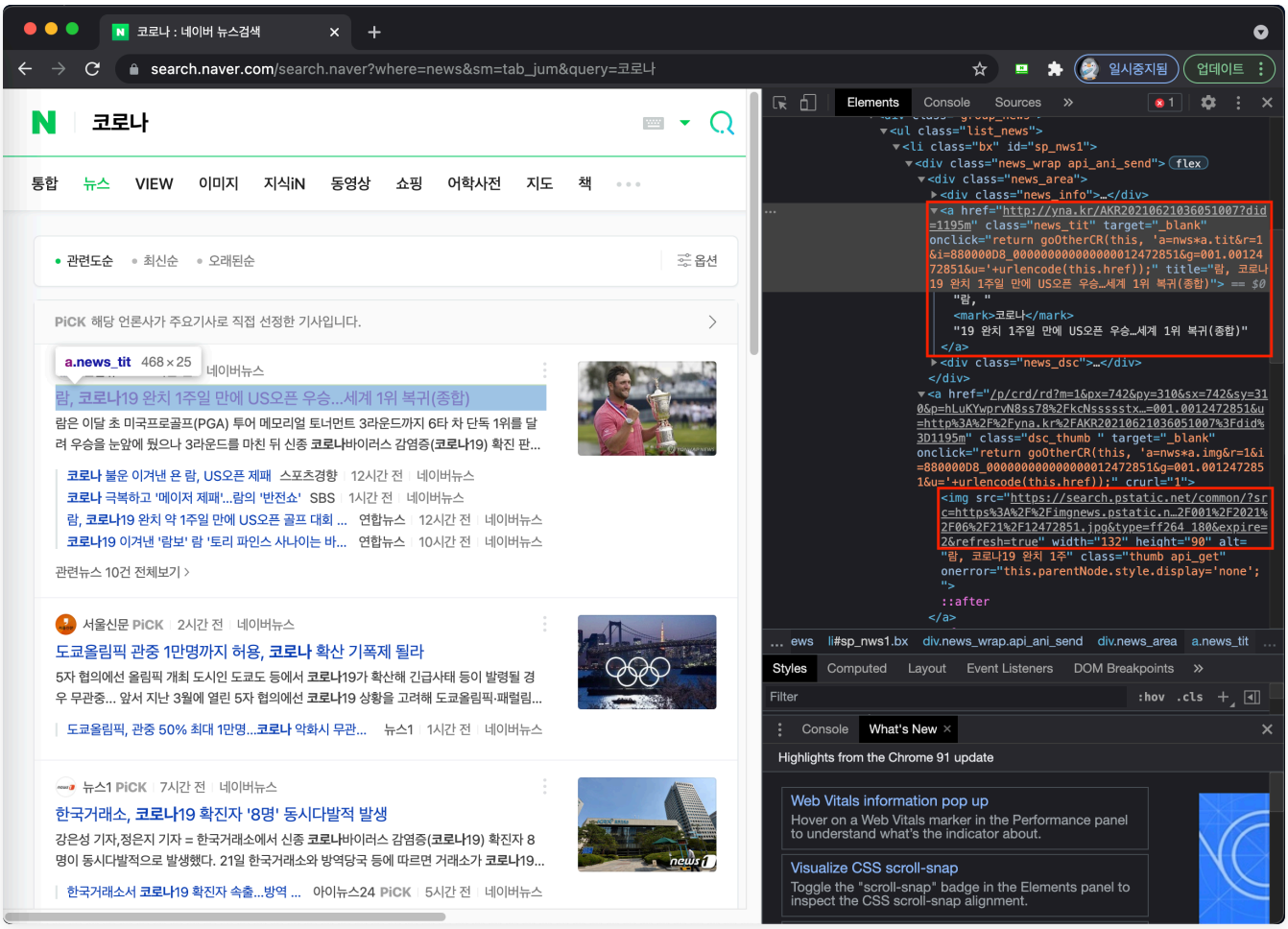
```
#step1.selenium 패키지와 time 모듈 import
from selenium import webdriver
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.common.by import By
import time

#step2.검색할 키워드 입력
query = input('검색할 키워드를 입력하세요: ')

#step3.크롬드라이버로 원하는 url로 접속
url = 'https://www.naver.com/'
driver = webdriver.Chrome('/Users/sangwoo/Desktop/chromedriver')
driver.get(url)
time.sleep(3)

#step4.검색창에 키워드 입력 후 엔터
search_box = driver.find_element(By.ID, "query")
search_box.send_keys(query)
search_box.send_keys(Keys.RETURN)
time.sleep(2)

#step5.뉴스 탭 클릭
driver.find_element_by_xpath('//*[@id="lnb"]/div[1]/div/ul/li[2]/a').click()
time.sleep(2)
```

참고로 위의 코드를 실행하면 아래와 같이 입력한 키워드의 뉴스 검색 결과가 출력됩니다. 참고로 한 페이지에 뉴스는 10개가 출력됩니다.



해당 화면이 출력되셨다면 F12 버튼을 누르시고 크롤링을 원하는 부분을 selenium 패키지로 가져와야합니다.

오늘 사용할 html은 아래의 빨간색 사각형으로 표시한 부분입니다. 위 영역은 기사제목과 기사링크(href)를 담고있으며, 아래 영역은 이미지 주소(src)를 담고 있습니다.



텍스트 추출 .text

selenium의 텍스트 추출은 아주 쉽습니다. 아래와 같이 선택자(css selector)를 이용해 원하는 부분의 html을 변수에 저장해준 후에, 아래와 같이 for 문과 .text 함수를 이용해 주시면 됩니다.

```
#step6.뉴스 제목 텍스트 추출

news_titles = driver.find_elements(By.CLASS_NAME, "news_tit")

for i in news_titles:
    title = i.text
    print(title)
```

결과는 아래와 같이 출력됩니다.

```
검색할 키워드를 입력하세요: 코로나
람, 코로나19 완치 1주일 만에 US오픈 우승…세계 1위 복귀(종합)
도쿄올림픽 관중 1만명까지 허용, 코로나 확산 기폭제 될라
한국거래소, 코로나19 확진자 '8명' 동시다발적 발생
인천공항 1터미널에 코로나19 검사센터 신설···출국여객 증가 대비
광주시, SKT '누구 케어콜' 코로나19 활용
'코로나19 유출설' 우한연구소, 中 최고과학상 후보 올라
삼바 "내년 상반기 코로나 백신 GMP 시설 가동"
코로나 신규 확진자, 9시까지 324명… 22일 300명대 중반 예상
[단독] 영덕 코로나 백신 1차 접종, 60대·70대 부부 확진
한국노총 "택배기사 등에 '코로나19 위험수당' 지급해야"(종합)
```

링크 추출 .get_attribute('href')

링크를 가져오고 싶을 때, 속성을 가져오는 .get_attribute('href')를 사용합니다. href 속성은 방금 전 가져온 news_titles 변수에 저장되어 있으므로, 따로 한 번더 가져와줄 필요 없이 바로 for문을 써줬습니다.

```
#step7.뉴스 하이퍼링크 추출

for i in news_titles:
    href = i.get_attribute('href')
    print(href)
```

결과는 아래와 같이 10개의 url이 출력되는 것을 확인하실 수 있을거예요.

```
http://yna.kr/AKR20210621036051007?did=1195m
https://www.seoul.co.kr/news/newsView.php?id=20210621500228&wlog_tag3=naver
https://www.news1.kr/articles/?4345970
https://www.sedaily.com/NewsView/22NQ2B0UKI
http://www.newsis.com/view/?id=NISX20210621_0001484005&cID=10406&pID=13100
https://www.seoul.co.kr/news/newsView.php?id=20210621500089&wlog_tag3=naver
http://www.edaily.co.kr/news/newspath.asp?newsid=02932326629084016
https://biz.chosun.com/topics/topics_social/2021/06/21/BS2QYMWLOVBNFCOSY4PRG6GJIY/?utm_sou
rce=naver&utm_medium=original&utm_campaign=biz
https://news.imaeil.com/Society/2021062118221171280
http://yna.kr/AKR20210621120351530?did=1195m
```

이미지 추출 .get_attribute('src')

다음으로는 이미지 추출입니다. 텍스트나 url 링크를 가져올 때와는 다르게 조금 길지만 어려워 하실 필요없어요.

앞서 배우신 것과 같이 이미지 주소(src)를 갖고 있는 html을 news_thumnail 변수에 저장해줍니다.

그 다음엔 이미지 다운로드를 위해서 해당 주소들을 리스트(link_thumnail)에 append 함수를 이용해 하나씩 담아주세요.

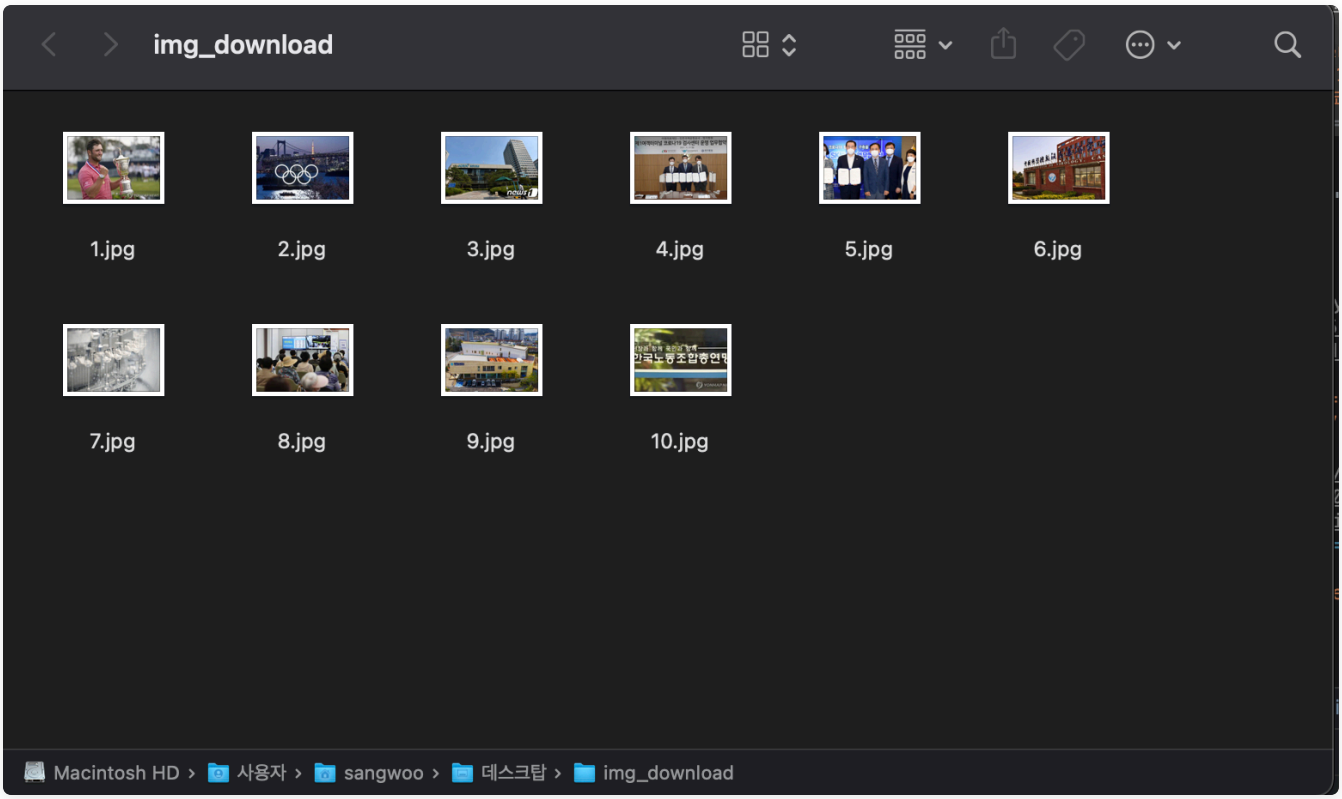
사진을 다운로드 받아서 내 PC에 저장하기 위해서는 저장할 폴더를 만들고, src 주소를 이용해 다운로드 해주면 됩니다.

폴더 생성에는 os 모듈과 urllib.request 패키지의 urlretrieve 함수가 필요한데, 모두 다 파이썬 내장 라이브러리이므로 따로 설치해주실 필요 없이 import 해주시면 돼요.

```
#step8.뉴스 썸네일 이미지 추출

# 스크롤 내리기 (모든 썸네일 이미지 로딩을 위함)
driver.execute_script("window.scrollTo(0, document.body.scrollHeight);")

time.sleep(2)

#뉴스 썸네일 이미지 다운로드
news_content_div = driver.find_elements(By.CLASS_NAME, 'news_contents')

news_thumbnail = []

for i in news_content_div:

    try:

        thumbnail = i.find_element(By.CLASS_NAME, "thumb")
        news_thumbnail.append(thumbnail)

    except:
        pass

link_thumbnail = [img.get_attribute('src') for img in news_thumbnail]

# 이미지 저장할 폴더 생성
import os

# path_folder의 경로는 각자 저장할 폴더의 경로를 적어줄 것
path_folder = r'/Users/sangwoo/Desktop/img_download/'

if  not os.path.isdir(path_folder):
    os.mkdir(path_folder)

# 이미지 다운로드
from urllib.request import urlretrieve

i = 0

for link in link_thumbnail:
    i += 1
    urlretrieve(link, path_folder + f'{i}.jpg')
```

```
    time.sleep(0.3)
```

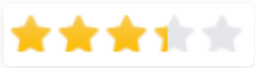
<실행 결과>



<참고>

별점 추출 get_attribute('style')

이건 자주 사용하진 않는데 가끔 속성을 가져와야할 때가 있습니다. 예를 들면 제가 잡플래닛 평점을 크롤링 할 때, 별점을 아래와 같이 style 속성으로 정의하더군요. 여기에 width: 68%를 속성 추출을 이용해서 받아와서 5점 만점의 별점으로 재가공 했던 적이 있습니다.



```
i.get_attribute('style')
```

3-9. 기타 유용한 패키지

기타 유용한 패키지들을 소개드립니다.

1) zipfile 압축/해제

오늘은 파이썬을 이용해 zip 파일을 만드는 방법에 대해 알아보겠습니다.

사무자동화를 할 때에 압축된 폴더를 다운로드 받거나, 압축해서 업로드 해야할 때를 위한 패키지를 소개드릴게요.

---

1. 관련 모듈 불러오기

zipfile은 파이썬 표준 라이브로리에 포함된 모듈입니다.

따로 설치할 필요는 없고 import로 불러와주시면 됩니다.

```
import zipfile
import os
```

파이썬 표준 라이브러리 zipfile 공식 문서

2. 압축하기

바로 압축하는 방법에 대해서 알아보겠습니다. 원리는 간단합니다.

1단계. 압축하고자 하는 zip 파일 생성

2단계. 파일에 넣고 싶은 파일 write

3단계. zip 파일 닫기

이 3단계를 하나의 파일을 압축하는 경우와 여러개의 파일을 압축하는 경우 2가지로 나눠서 설명드려 보겠습니다.

1) 하나의 파일 압축하기

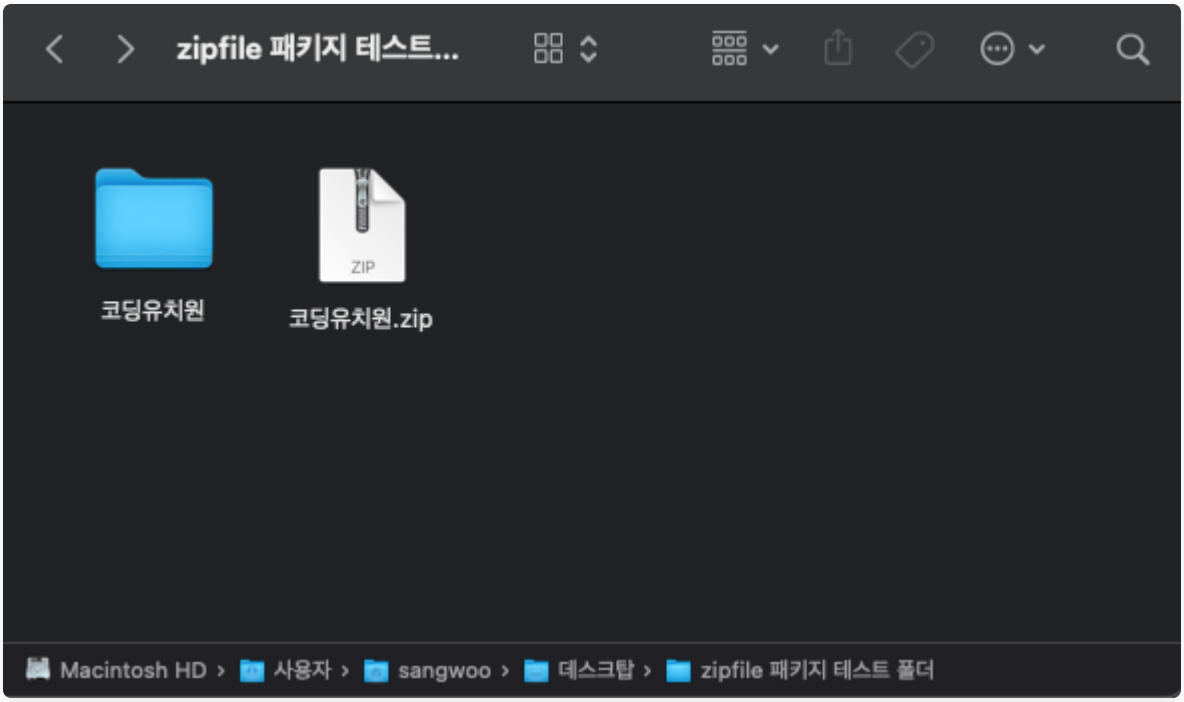
1단계. 압축하고자 하는 zip 파일 생성 (꼭 os.chdir로 현재 경로를 바꾸어 줄 것)

먼저 os.chdir( )로 현재 작업할 경로를 다시 설정해줄 필요가 있습니다. 그러지 않고 절대 경로로 할 시, 의도와 다르게 가장 상위 디렉토리(폴더)부터 모든 디렉토리가 압축됩니다.

또한 zip 파일 생성을 위해 폴더를 따로 만들어 줄 필요가 없습니다. 곧 바로 zip 파일이 생성되는 것이니깐요.

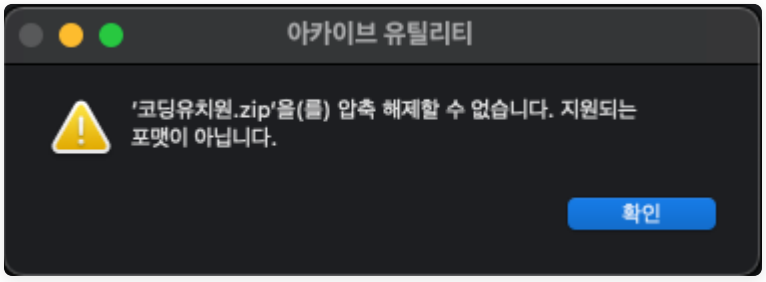
```
os.chdir('/Users/username/Desktop/zipfile 패키지 테스트 폴더')

my_zip = zipfile.ZipFile('코딩유치원.zip', 'w')
```

<실행 결과> 코딩유치원.zip 생성됨



코딩유치원 폴더는 따로 만들어 줄 필요 없음

현재의 상태(zip 파일 close 안 해줌)에서는 아래와 같이 zip 파일을 열 수가 없다는 점 참고바랍니다.



2단계. 생성한 zip 파일에 넣고자 하는 파일 입력하기

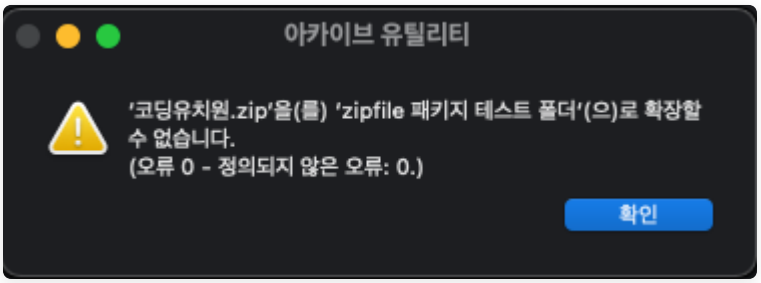
아까 설정해준 현재 작업 경로에 test1.txt를 생성해두고 그것을 write 해줍니다.

참고로 현재 작업 폴더가 아닌 곳의 test1 파일을 write('절대경로')로 입력할 시, 이상하게 작동하니 주의하세요.

```
# 현재 작업 경로에 test1.txt 파일이 있어야 함
my_zip.write('test1.txt')
```

<실행 결과> 코딩유치원.zip 안에 test1.txt 들어감

현재 상태에서도 zip 파일을 더블클릭해서 열어보려 해도 아래와 같은 경고창이 뜨면서 확인할 수가 없습니다.



3단계. zip 파일 닫기

zip 파일 안에 원하는 파일을 모두 넣으셨다면 close( ) 메소드를 통해 zip를 닫아주어야합니다.

```
my_zip.close()
```

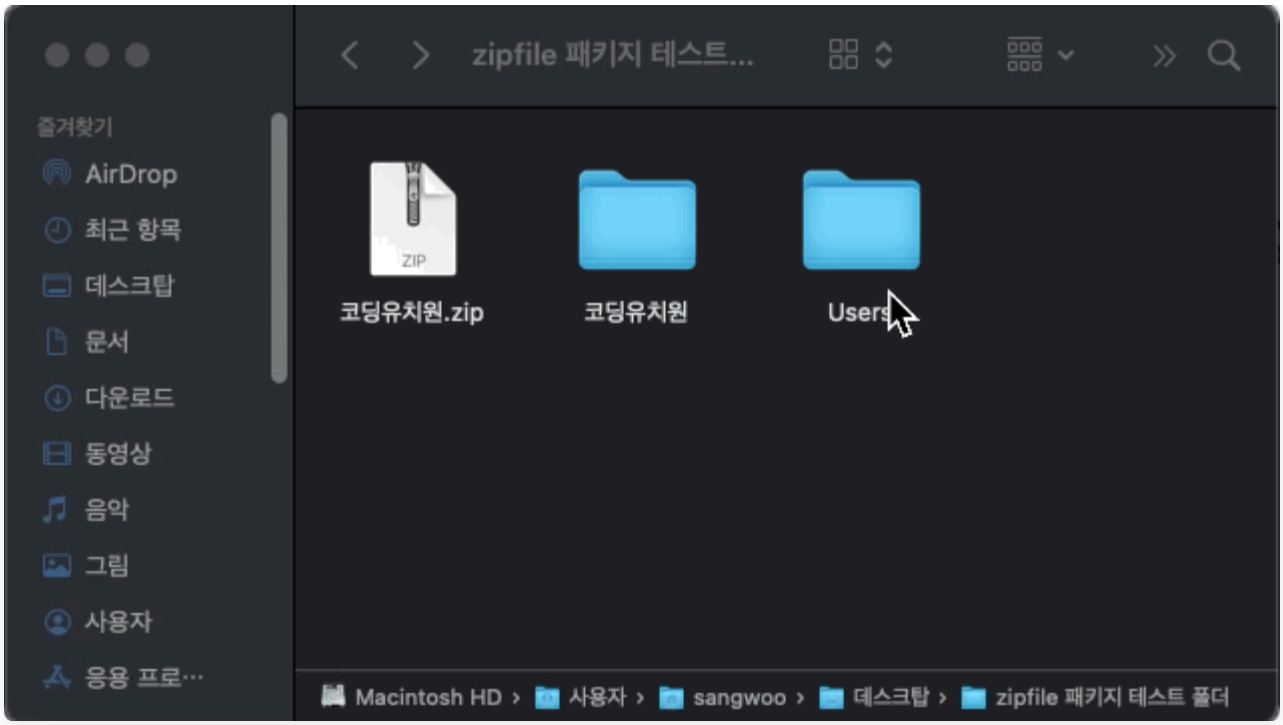
<실행 결과> 코딩유치원.zip을 더블 클릭하여 압축해제 시

의도와 다르게 가장 상위 디렉토리(폴더)부터 모든 디렉토리가 압축된 것을 확인 할 수 있습니다. 다른 방법이 필요하겠습니다.

<주의!>

만약 경로를 chdir( )로 바꾸어 주지 않고 절대 경로를 사용하여 압축할 시 아래와 같이 의도치 않은 압축 파일이 생성됩니다.

```
my_zip = zipfile.ZipFile('/Users/username/Desktop/zipfile 패키지 테스트 폴더/코딩유치원.zip',
'w')

my_zip.write('/Users/sangwoo/Desktop/zipfile 패키지 테스트 폴더/코딩유치원/test1.txt')

my_zip.close()
```



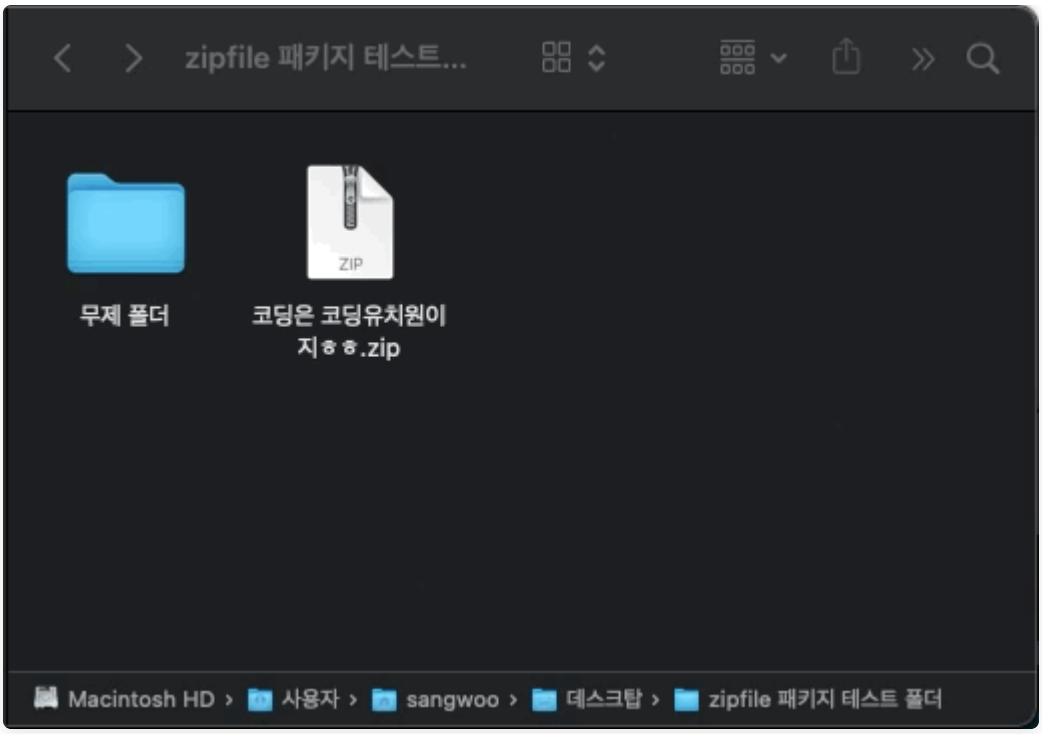
2) 여러개의 파일 압축하기

여러개의 파일을 한꺼번에 압축하려면 with 문과 for문을 함께 사용해 주시면 됩니다.

아래의 코드를 보시고 직접 한 번 해보시기 바랍니다. 그 외 개념은 위에서 했던 것과 같으므로 자세한 설명은 생략하도록 하겠습니다.

```
file_list = ['코딩유치원.txt', '투손플레이스.txt', '고양이미로.xlsx']

with zipfile.ZipFile("코딩은 코딩유치원이지ㅎㅎ.zip", 'w') as my_zip:

    for i in file_list:

        my_zip.write(i)

    my_zip.close()
```

<실행 결과>



3. 압축해제

압축을 해제 하는 방법은 크게 2가지가 있습니다. 여러 파일 중 1개만 골라서 압축해제하는 방법과 전체를 다 압축해제하는 방법입니다.

1) 개별파일 압축해제

ZipFile.extract(member, path=None, pwd=None)

path에는 추출할 zip 폴더의 경로를 넣어줄 수 있으며, 넣어주지 않으면 현재 작업 폴더로 지정한 파일을 추출합니다. member는 파일명.확장자명이며, pwd는 암호화된 파일에 사용되는 비밀번호입니다.

```
zipfile.ZipFile('코딩은 코딩유치원이지ㅎㅎ.zip').extract('코딩유치원.txt')
```

<실행 결과>

2) 모든파일 압축해제

ZipFile.extractall(path=None, members=None, pwd=None)

path에는 추출할 zip 폴더의 경로를 넣어줄 수 있으며, 넣어주지 않으면 현재 작업 폴더로 지정한 파일을 추출합니다.
members에는 리스트 형식의 '파일명'을 넣어주시면 선택적으로 여러개의 파일을 압축해제 가능하며, pwd는 암호화된 파일에 사용되는 비밀번호입니다.

```
# 전체를 압축해제 하고 싶을 때
zipfile.ZipFile('코딩은 코딩유치원이지ㅎㅎ.zip').extractall()

# 전체가 아닌 몇 개만 압축해제 하고 싶을 때
zipfile.ZipFile('코딩은 코딩유치원이지ㅎㅎ.zip').extractall(members = ['투손플레이스.txt','고양이미로.xlsx'])
```

2) exe 파일 배포

1.PyInstaller란?

PyInstaller란 코딩한 파이썬 프로그램을 파이썬에 대해 전혀 모르는 분들도 사용하실 수 있도록 실행 파일(.exe)로 만들어주는 파이썬 패키지입니다.

1) Python 3.6이상만 사용 가능

2) Windows, Mac OS X 및 GNU / Linux에서 사용가능함

3) Windows에서 컴파일 된 실행파일은 Windows에서만 사용가능 (다른 OS의 경우도 마찬가지)

4) Windows 8 이상만 지원 / Mac OS X 10.7(Lion) 이상만 지원

https://pyinstaller.readthedocs.io/en/stable/

2.설치 방법

지금 껏 다른 패키지를 설치하신 분들은 아시겠지만, 아래의 코드는 터미널창에서 실행해 주셔야합니다.

```
# 설치
pip install pyinstaller

# 최신 버전으로 업그레이드 (첫 설치 시에는 필요없음)
pip install --upgrade pyinstaller

# 설치되어 있는지 확인
pyinstaller --version
```

3.초간단 사용 방법

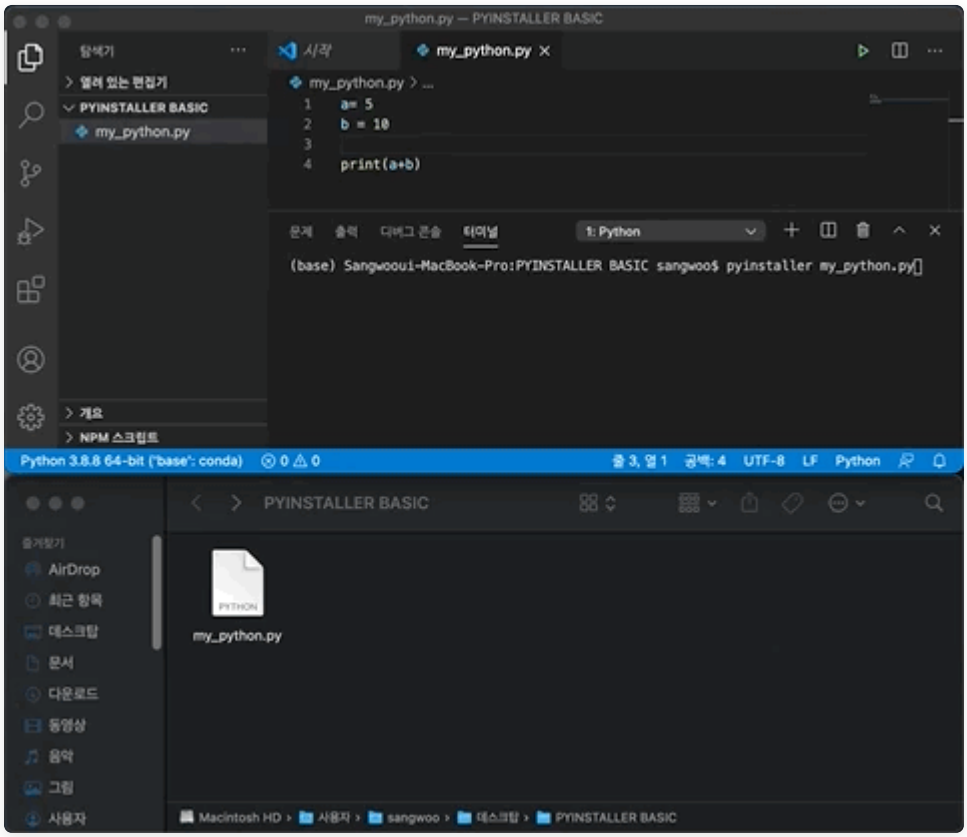
PyInstaller를 사용하기 위해서는 별도의 import가 필요 없습니다.

사용 시에도 설치했을 때와 마찬가지로 터미널 창에서 입력해주시면 됩니다.

```
# 가장 기본적인 사용법
pyinstaller 파이썬 파일명.py
```

아주 간단한 코드를 작성해서 pyinstaller로 실행파일을 만들어 보았습니다. build 폴더와 dist 폴더가 생기는데 우리가 사용할 exe 파일은 dist 폴더에 생성됩니다.

여기서 주목하실 점은 너무나도 많은 파일들이 지저분하게(?) 생성된다는 것입니다. 다른 사람에게 배포할 때 이렇게 전달하기엔 좀 별로겠죠?



그래서 아래의 코드를 사용하시길 추천드립니다. PyInstaller에는 여러가지 옵션들이 있지만 딱 알아야하는 것만 설명드리도록 하겠습니다.

```
pyinstaller -F -w 파이썬 파일명.py
```

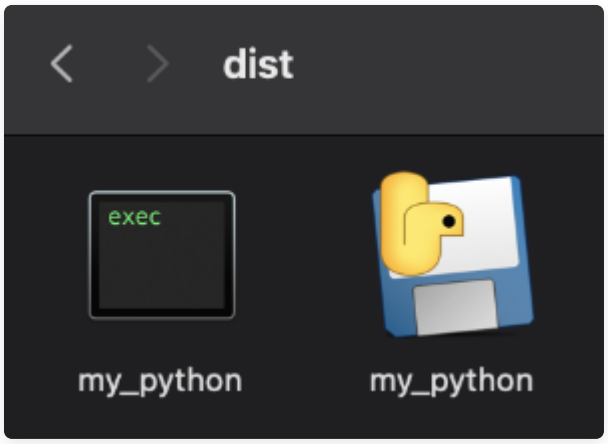
1) -F, --onefile

방금 전과 같이 지저분하게 파일이 생성되지 않고, 실행파일 1개만 생성되도록 해주는 설정입니다.

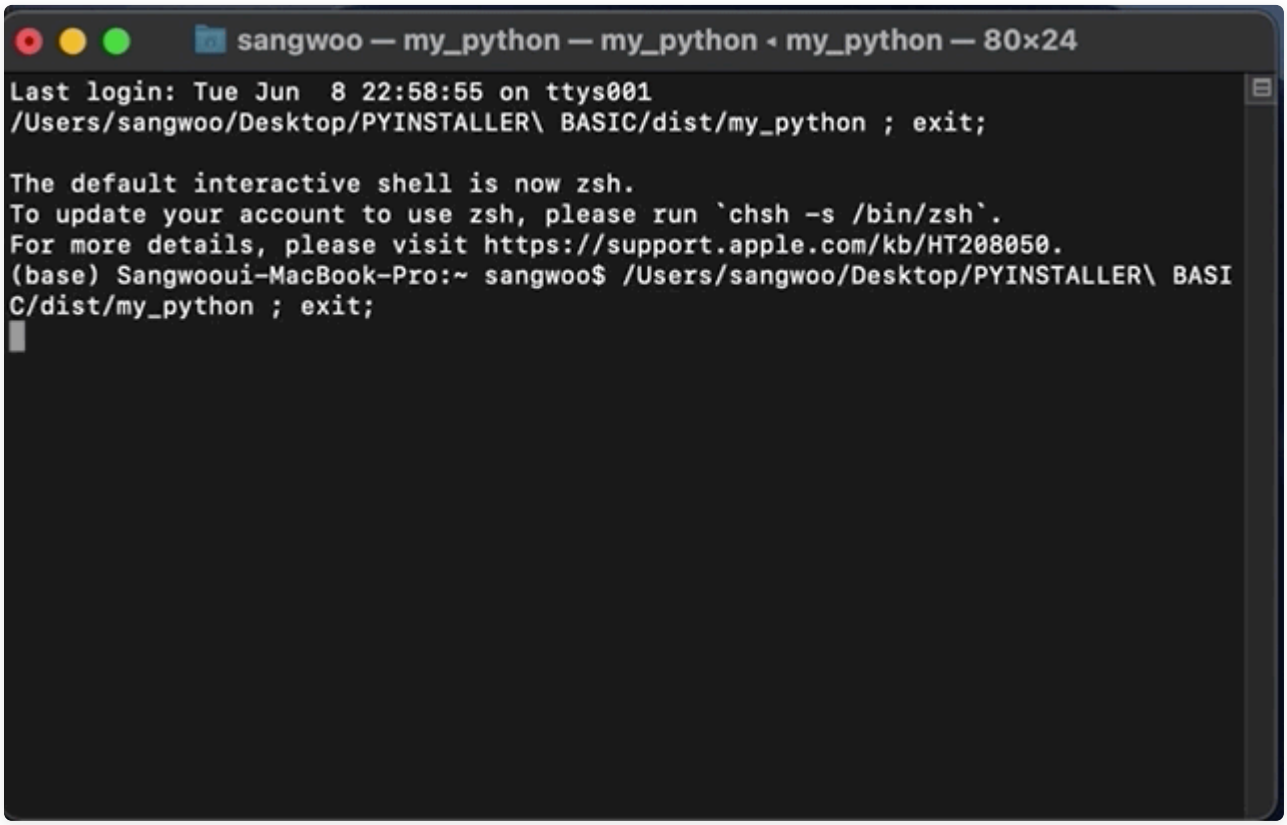
2) -w, --windowed, --noconsole

기본적으로 실행 파일을 실행하면 표준 I/O용 콘솔 창(까만 창)을 열도록 되어있는데 이걸 뜨지 않도록 하는 옵션입니다.

<깔끔하게 실행파일로 출력됨>



생성된 실행 파일을 더블클릭해서 실행하면 아래와 같이 print(a+b)의 결과인 15를 출력해주는 것을 보실 수 있습니다.



<참고> 실행파일 아이콘 변경하기

출처. 블로그

실행파일의 아이콘을 바꾸고 싶다면, '--icon = ico 파일의 경로'을 추가해주시면 됩니다. 이미지의 포맷은 꼭 ico여야 합니다.

```
# ico 파일의 경로에서 ./의 의미는 현재 폴더라는 뜻, 절대경로를 넣어줘도 됨

pyinstaller --icon=./test.ico --onefile 파이썬 파일명.py
```

https://icon-icons.com/ko/

4.용량 줄이는 방법 (추후 반영 예정)

3) 원하는 시간마다 파이썬 자동 실행

시작하기에 앞서, 오늘은 VS Code에서 파이썬 파일(.py)를 개발환경으로 사용하는 점 참고바랍니다.

schedule 패키지를 쥬피터 노트북(.ipynb)으로 다루니 프로그램이 종료되지 않고 계속 함수를 실행하는 문제가 있더라구요.

1. schedule 설치

우리가 오늘 배워볼 패키지는 앞서 언급했 듯이 schedule 입니다. 자세한 설명 없이 바로 설치하고 코드로 보여 드리겠습니다.

터미널 창에 아래의 코드를 입력하시고 엔터를 누르시면 설치가 완료됩니다.

```
pip install schedule
```

```
Collecting schedule
  Downloading schedule-1.1.0-py2.py3-none-any.whl (10 kB)
Installing collected packages: schedule
Successfully installed schedule-1.1.0
Note: you may need to restart the kernel to use updated packages.
```

설치 완료 시 출력되는 문구

2. schedule & time 모듈 불러오기

schedule 패키지는 파이썬 내장 라이브러리에 포함되어 있는 time 모듈과 함께 사용됩니다.

```
import schedule
import time
```

3. schedule 패키지 주기 설정법

잠시 schedule 패키지로 다양한 주기를 설정하는 방법을 살펴보겠습니다.

1) 시, 분, 초 단위로 실행

```
# 5초에 한번씩 함수 실행
schedule.every(5).seconds.do(함수)

# 10분에 한번씩 함수 실행
schedule.every(10).minutes.do(함수)

# 2시간에 한번씩 함수 실행
schedule.every(2).hour.do(함수)
```

2) 일, 주 단위로 실행

```
# 3일에 한번씩 함수 실행
schedule.every(3).days.do(함수)

# 2주에 한번씩 함수 실행
schedule.every(2).weeks.do(함수)
```

3) 매일 정해진 시각에 실행

위에서 언급한 시, 분, 초, 일, 주 단위와 조합하여 **더 디테일하게 주기를 설정**할 수도 있습니다.

```
# 매일 13시 30분에 함수 실행
schedule.every().day.at("13:30").do(함수)

# 매일 "11:11:11"에 함수 실행
schedule.every().day.at("11:11:11").do(함수)
```

4) 매주 정해진 요일의 특정 시각에 실행

요일도 설정 가능합니다. 모두 소문자로 입력하는 점 주의해주세요.

(monday, tuesday, wednesday, thursday, friday, saturday, sunday)

```
# 매주 월요일 13시 30분에 함수 실행
schedule.every().monday.at("13:30").do(함수)
```

4. schedule 패키지 실행

실행 방법은 아주 간단합니다. 이 부분은 전체적인 코드로 보여드리겠습니다.

가장 step4의 무한루프 코드를 넣어주지 않으면 파이썬 파일이 종료되면서 바로 스케쥴링도 작동하지 않습니다.

```
# step1.관련 패키지 및 모듈 import
import schedule
import time

# step2.실행할 함수 선언
def message():
    print("스케쥴 실행중...")

# step3.실행 주기 설정
schedule.every(1).seconds.do(message1)

# step4.스캐쥴 시작
while True:
    schedule.run_pending()
    time.sleep(1)
```

<실행 결과>

5. 인자를 받는 함수 사용법

위에서는 아주 단순한 print 함수를 사용한 함수를 사용해서 문제 없었지만, 우리는 분명 인자를 받는 함수를 사용할 일이 있습니다.

이럴 때는 기본적인 함수가 인자를 받는 방법과 다르니 알아두시면 도움이 되실거예요.

```
# text라는 파라미터가 존재하여, 실제 사용 시엔 인자를 받아야함

def message2(text):
    print(text)

# 사용방법
schedule.every(2).seconds.do(message2,'2초마다 알려줄게요')
```

보시다시피 message2(인자)가 아닌 점에 주의해주세요. 작동하는 것은 아래에서 보여드리겠습니다.

6. schedule 중지하는 법

제가 아는한 schedule을 중지하는 법은 크게 3가지 입니다.

1. while문에 조건을 걸어서 무한루프를 종료시키고 파이썬 파일을 종료한다.
2. sys 모듈의 exit( ) 함수로 파이썬 파일을 강제 종료

3. schedule 패키지의 cancel_job( ) 함수 사용

제가 이번 시간에 알려드릴 방법은 세번째 방법인 cancel_job( ) 함수입니다.

추가적인 모듈을 import할 필요도 없고, 주기 설정한 함수 별로 종료할 수 있기 때문입니다.

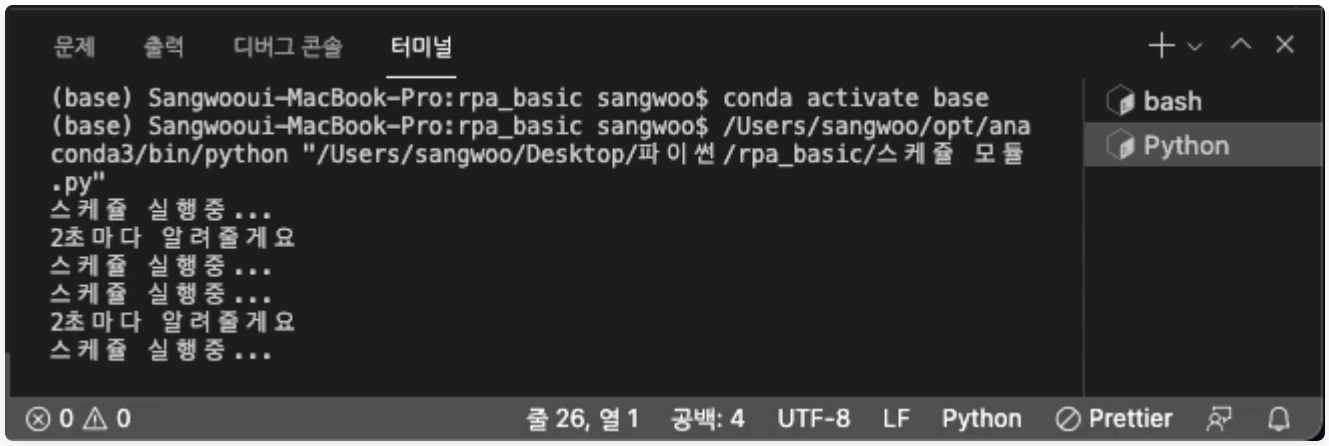
코드는 아래와 같습니다.

여기서 주목할 점은 설정한 주기를 변수(job1, job2)에 넣어주었고, 이 job들을 cancel_job( )의 인자로 넣어 준다는 것입니다.

바로 실행 시켜보겠습니다.

```
import schedule
import time

def message1():
    print("스케쥴 실행중...")

def message2(text):
    print(text)

# 주기 설정
job1 = schedule.every(1).seconds.do(message1)
job2 = schedule.every(2).seconds.do(message2,'2초마다 알려줄게요')


count = 0

while True:

    schedule.run_pending()
    time.sleep(1)

    count = count + 1

    if count > 5:
        schedule.cancel_job(job1)
```

<출력 결과>


문제
출력
디버그 콘솔
터미널
(base) Sangwooui-MacBook-Pro:rpa_basic sangwoo$ conda activate base
(base) Sangwooui-MacBook-Pro:rpa_basic sangwoo$ /Users/sangwoo/opt/ana
conda3/bin/python "/Users/sangwoo/Desktop/파이썬/rpa_basic/스케줄 모듈
.py"
스케줄 실행중...
2초마다 알려줄게요
스케줄 실행중...
스케줄 실행중...
2초마다 알려줄게요
스케줄 실행중...
bash
Python
0 0
줄 26, 열 1
공백: 4
UTF-8
LF
Python
Prettier

4) PDF 다루기

1. PyPDF2란?

PyPDF2는 이름에서 직관적으로 알 수 있듯이 Python으로 PDF 문서를 다룰 수 있게 해주는 라이브러리입니다.

참고로 설치는 아래의 코드 한 줄을 커맨드 창에 입력해주시면 됩니다. 대소문자에 주의해주세요!

```
pip install PyPDF2
```

아래의 링크는 PyPDF2 공식 문서이며, 그 안에는 여러 기능들이 있지만 이번 시간에는 코딩유치원 컨셉에 맞게 최대한 쉽고 가장 자주 사용할 만한 기능 위주로 설명드리겠습니다.

[PyPDF2 공식문서]

PyPDF2는 크게 3가지 클래스)로 나눌 수 있습니다.

1) 원하는 작업을 하기 위해 pdf 파일을 불러올 때 사용하는 **PdfFileReader**

2) 회전, 추출, 분리 등의 페이지 단위 작업을 할 때 사용하는 **PdfFileWriter**

(참고로, 회전은 한번에 하는 것이 아니라, 페이지 하나씩 돌려서 새로운 pdf에 붙여넣는 방식입니다)

3) 여러 pdf 파일을 하나의 pdf로 병합할 때 사용하는 **PdfFileMerger**

<2023.02.11 - PyPDF2 버전 3.0.1 기준 변경 사항>

PdfFileReader → **PdfReader**

PdfFileWriter → **PdfWriter**

PdfFileMerger → **PdfMerger**

2. 페이지 추출

가장 처음으로 원하는 페이지를 추출해보겠습니다.

저는 쉬운 이해를 위해서 테스트용 PDF를 만들어서 사용하였으나, 여러분들은 각자의 PDF 파일을 사용하시면 되는 점 참고바랍니다.

테스트 PDF 파일 1.pdf에서 2번째 페이지만 추출해서 따로 저장해보겠습니다.

테스트 PDF 파일 1.pdf

1

2

<코드> PyPDF2 (ver 3.0.0 이상)

*** 변경점**

addPage(page_number) → add_page(page_number)

.getPage(page_number) → .pages[page_number]

```
# step1.관련 모듈 import
from PyPDF2 import PdfFileReader, PdfFileWriter

# step2.기존 pdf 불러오기
pdfReader = PdfReader("테스트 PDF 파일 1.pdf", "rb")

# step3.새로 만들 pdf 객체 생성
pdfWriter = PdfWriter()

# step4.기존의 1번 페이지를 가져와서 새로만든 pdf에 붙여넣기
pdfWriter.add_page(pdfReader.pages[1])

# step5.1번 페이지가 붙여진 새로운 pdf 파일을 현재 경로('./')에 원하는 이름으로 저장
pdfWriter.write(open("./추출한 테스트 PDF 파일.pdf", "wb"))
```

<변경 전 코드_주로 ChatGPT가 주로 추천해줘서 오류가 남.>

```
# step1.관련 모듈 import
from PyPDF2 import PdfFileReader, PdfFileWriter

# step2.기존 pdf 불러오기
pdfReader = PdfFileReader("테스트 PDF 파일 1.pdf", "rb")

# step3.새로 만들 pdf 객체 생성
pdfWriter = PdfFileWriter()

# step4.기존의 1번 페이지를 가져와서 새로만든 pdf에 붙여넣기
pdfWriter.addPage(pdfReader.getPage(1))

# step5.1번 페이지가 붙여진 새로운 pdf 파일을 현재 경로('./')에 원하는 이름으로 저장
pdfWriter.write(open("./추출한 테스트 PDF 파일.pdf", "wb"))
```

<결과물>

3. PDF 분할

다음으로 구현해 볼 기능은 페이지를 회전하는 것입니다.

거의 비슷하지만 2가지가 다릅니다.

1) 기존 PDF의 페이지 수를 파악해서 for문으로 한 페이지씩 처리

- numPages 함수로 기존 PDF 파일의 전체 페이지 수 파악
- getPage( ) 함수로 지정한 PDF 페이지 한 장 가져오기

2) 마지막 저장 시, f 문자열 포매팅 개념을 이용해서 pageNum 변수를 파일명에 반영

<코드> PyPDF2 (ver 3.0.0 이상)

*** 변경점**

pdfReader.numPages → len(pdfReader.pages)

addPage(page_number) → add_page(page_number)

.getPage(page_number) → .pages[page_number]

```
# step1.관련 모듈 import
from PyPDF2 import PdfReader, PdfWriter

# step2.기존 PDF 불러오기
pdfReader = PdfReader("테스트 PDF 파일 1.pdf", "rb")

# 페이지 하나씩 받아와서 저장하는 반복문
for pageNum in range(len(pdfReader.pages)):

    # step3.새로 만들 pdf 객체 생성 (계속 누적되지 않기 위해 for문 안으로 넣음)
    pdfWriter = PdfWriter()

    # step4.기존 PDF에서 한 페이지씩 가져오기
    page = pdfReader.pages[pageNum]

    # step5.위에서 가져온 페이지를 새로 만든 PDF에 붙여넣기
    pdfWriter.add_page(page)

    # step6.새로운 PDF 파일을 해당 경로('./')에 원하는 이름으로 저장
    # (이름을 계속 다르게 해주기 위해서 f 문자열 포매팅 개념을 이용)
    pdfWriter.write(open(f"./분할한 PDF 파일 {pageNum+1}.pdf", "wb"))
```

<변경 전 코드_주로 ChatGPT가 주로 추천해줘서 오류가 남.>

```
# step1.관련 모듈 import
from PyPDF2 import PdfFileReader, PdfFileWriter

# step2.기존 PDF 불러오기
pdfReader = PdfFileReader("테스트 PDF 파일 1.pdf", "rb")

# 페이지 하나씩 받아와서 저장하는 반복문
for pageNum in range(pdfReader.numPages):

    # step3.새로 만들 pdf 객체 생성 (계속 누적되지 않기 위해 for문 안으로 넣음)
    pdfWriter = PdfFileWriter()

    # step4.기존 PDF에서 한 페이지씩 가져오기
    page = pdfReader.getPage(pageNum)

    # step5.위에서 가져온 페이지를 새로 만든 PDF에 붙여넣기
    pdfWriter.addPage(page)

    # step6.새로운 PDF 파일을 해당 경로('./')에 원하는 이름으로 저장
    # (이름을 계속 다르게 해주기 위해서 f 문자열 포매팅 개념을 이용)
    pdfWriter.write(open(f"./분할한 PDF 파일 {pageNum+1}.pdf", "wb"))
```

<결과물>

4. PDF 회전

위에서 배운 PDF 분할에서 딱 2가지만 달라졌습니다.

1) rotate( ) 함수로 회전

2) pdfWriter가 for문 밖으로 나감 (기존의 pdf를 회전시키고 새로운 PDF에 계속 붙여넣어 주어야하기 때문)

이 부분에 유의하시면서 아래의 코드를 보시면 좋을 것 같습니다.

<코드> PyPDF2 (ver 3.0.0 이상)

*** 변경점**

rotateClockwise( ) → rotate( )

pdfReader.numPages → len(pdfReader.pages)

addPage(page_number) → add_page(page_number)

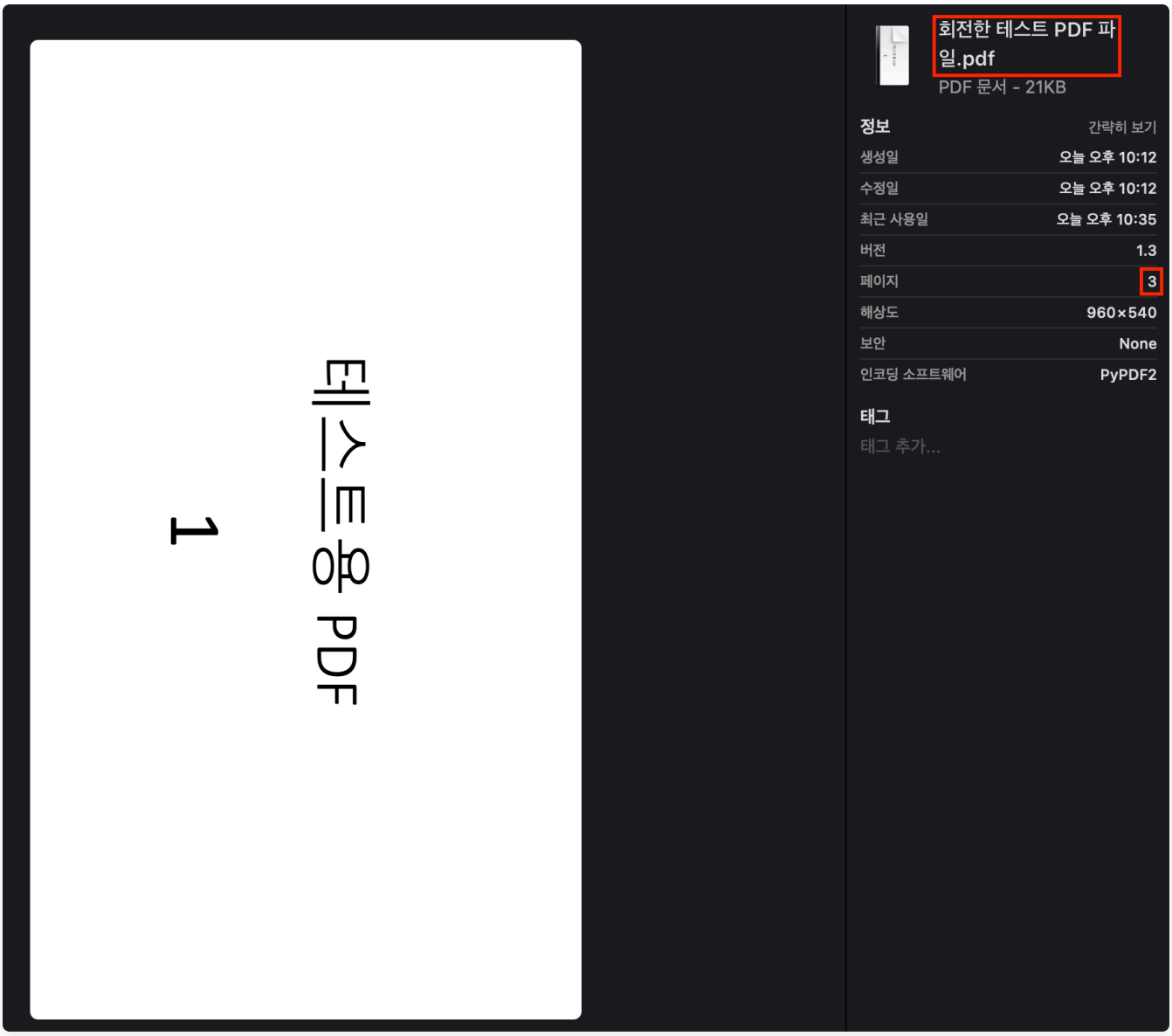
.getPage(page_number) → .pages[page_number]

```
# step1.관련 모듈 import
from PyPDF2 import PdfReader, PdfWriter

# step2.기존 pdf 불러오기
pdfReader = PdfReader("데이터 자격시험 일정_2022.pdf", "rb")

# step3.새로 만들 pdf 객체 생성
pdfWriter = PdfWriter()

# 페이지 하나씩 받아와서 돌린 후, 붙여넣는 반복문
for pageNum in range(len(pdfReader.pages)):

    # step4.기존 PDF에서 한 페이지씩 가져오기
    page = pdfReader.pages[pageNum]

    # step5.시계 방향으로 90도 회전 (반시계로 90도 회전은 270 입력하면 됨)
    page.rotate(90)

    # step6.회전된 페이지 새로운 PDF에 붙여넣기
    pdfWriter.add_page(page)

# step7.새로운 pdf 파일을 해당 경로('./')에 원하는 이름으로 저장
pdfWriter.write(open('./회전한 테스트 PDF 파일.pdf', 'wb'))
```

<변경 전 코드_주로 ChatGPT가 주로 추천해줘서 오류가 남.>

```
# step1.관련 모듈 import
from PyPDF2 import PdfFileReader,PdfFileWriter

# step2.기존 pdf 불러오기
pdfReader = PdfFileReader(open("테스트 PDF 파일 1.pdf","rb"))

# step3.새로 만들 pdf 객체 생성
pdfWriter = PdfFileWriter()

# 페이지 하나씩 받아와서 돌린 후, 붙여넣는 반복문
for pageNum in range(pdfReader.numPages):

    # step4.기존 PDF에서 한 페이지씩 가져오기
    page = pdfReader.getPage(pageNum)

    # step5.시계 방향으로 90도 회전 (반시계로 90도 회전은 270 입력하면 됨)
    page.rotateClockwise(90)

    # step6.회전된 페이지 새로운 PDF에 붙여넣기
    pdfWriter.addPage(page)

# step7.새로운 pdf 파일을 해당 경로('./')에 원하는 이름으로 저장
pdfWriter.write(open('./회전한 테스트 PDF 파일.pdf', 'wb'))
```

<결과물>



4. PDF 병합

병합은 앞서 배운 분할과 회전에 비해서 매우 간단합니다.

PdfFileWriter가 아니라 PdfFileMerger를 import 해준 점만 주의 하시면 아래 코드 이해하시는데에 어려움 없으시리라 생각합니다.

<코드> PyPDF2 (ver 3.0.0 이상)

*** 변경점**

PdfReader와 PdfMerger 클래스 외에는 따로 변경점 없음

<코드>

```
# step1.관련 모듈 import (Writer 대신 Merger를 사용하는 것에 주의!)
from PyPDF2 import PdfReader, PdfMerger

# step2.기존 pdf 불러오기
pdfReader1 = PdfReader(open("테스트 PDF 파일 1.pdf","rb")) # 3장
pdfReader2 = PdfReader(open("테스트 PDF 파일 2.pdf","rb")) # 2장

# step3.새로 만들 pdf 객체 생성 (병합용)
pdfMerger = PdfMerger()

# step4.PDF 하나씩 가져와서 차례대로 병합
pdfMerger.append(pdfReader1)
pdfMerger.append(pdfReader2)

# step5.새로운 pdf 파일을 해당 경로('./')에 원하는 이름으로 저장
pdfMerger.write("./병합한 테스트 PDF 파일.pdf")
```

<변경 전 코드_주로 ChatGPT가 주로 추천해줘서 오류가 남.>

```
# step1.관련 모듈 import (Writer 대신 Merger를 사용하는 것에 주의!)
from PyPDF2 import PdfFileReader, PdfFileMerger

# step2.기존 pdf 불러오기
pdfReader1 = PdfFileReader(open("테스트 PDF 파일 1.pdf","rb")) # 3장
pdfReader2 = PdfFileReader(open("테스트 PDF 파일 2.pdf","rb")) # 2장

# step3.새로 만들 pdf 객체 생성 (병합용)
pdfMerger = PdfFileMerger()

# step4.PDF 하나씩 가져와서 차례대로 병합
pdfMerger.append(pdfReader1)
pdfMerger.append(pdfReader2)

# step5.새로운 pdf 파일을 해당 경로('./')에 원하는 이름으로 저장
pdfMerger.write("./병합한 테스트 PDF 파일.pdf")
```

<결과물>

테스트용 PDF
5
병합한 테스트 PDF 파일.pdf
PDF 문서 - 41KB
정보
간략히 보기
생성일
오늘 오후 11:16
수정일
오늘 오후 11:16
버전
1.3
페이지
5
해상도
960×540
보안
None
인코딩 소프트웨어
PyPDF2
태그
태그 추가...